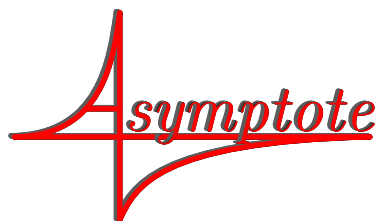


Asymptote: 矢量图形语言

适用于版本 3.14git



本文档介绍 Asymptote, 版本 3.14git。

<https://asymptote.sourceforge.io>

Copyright © 2004-2026 Andy Hammerlindl, John Bowman, and Tom Prince.

在 GNU Lesser General Public License 的条款之下, 允许复制、分发和/或修改本文档 (参见顶层源代码目录中的 LICENSE 文件)。

目录

1	描述	1
2	安装	3
2.1	UNIX 二进制发行版	3
2.2	macOS X 二进制发行版	3
2.3	Microsoft Windows	3
2.4	配置	4
2.5	搜索路径	5
2.6	从 UNIX 源码编译	6
2.7	编辑模式	6
2.8	Git	7
2.9	构建文档	7
2.10	卸载	8
3	教程	9
3.1	批处理模式绘图	9
3.2	交互模式绘图	9
3.3	图形尺寸	10
3.4	标签	11
3.5	路径	11
4	绘图命令	14
4.1	draw	14
4.2	fill	16
4.3	clip	18
4.4	label	18
5	贝塞尔曲线	22
6	编程	24
6.1	数据类型	24
6.2	路径与引导	30
6.3	画笔	37
6.4	变换	46
6.5	框与图片	47
6.6	延迟绘制	52
6.7	文件	54
6.8	变量初始化器	56
6.9	结构	57
6.10	运算符	60
6.10.1	算术与逻辑运算符	60

6.10.2	自运算符与前缀运算符	61
6.10.3	用户自定义运算符	62
6.10.4	方括号运算符	62
6.10.5	转换运算符	63
6.11	隐式缩放	63
6.12	函数	64
6.12.1	默认参数	66
6.12.2	命名参数	66
6.12.3	剩余参数	67
6.12.4	数学函数	68
6.13	数组	70
6.13.1	切片	76
6.14	类型转换	77
6.15	导入	78
6.15.1	模板化导入	81
6.16	迭代器	81
6.16.1	range	83
6.17	Static 静态	83
6.18	Autounravel 自动展开	85
6.18.1	字段自动展开的位置	86
6.18.2	autounravel 合法的位置	86
6.19	哈希函数	86
6.19.1	为用户定义结构体计算哈希	86
6.20	集合容器(Collections)	87
6.20.1	数对(Pairs)	88
6.20.2	迭代器与工具	88
6.20.2.1	collections.iter(T)	88
6.20.2.2	合并(zip)迭代器	89
6.20.2.3	为迭代器附加计数器	90
6.20.3	映射(Map)	91
6.20.3.1	初始化 Map	92
6.20.3.2	Map 的 API	92
6.20.4	将映射用作集合	93
6.20.5	集合	94
6.20.5.1	初始化 Set	94
6.20.5.2	Set API	95
6.20.5.3	SortedSet API	97
6.20.6	将集合用作映射	98
6.20.7	队列	100
6.20.8	包装数组	100
7	LaTeX 用法	102
8	用户模块	107
8.1	graph	107
8.2	three	136

8.3	graph3.....	148
8.4	palette.....	153
8.5	colormap.....	158
8.6	texcolors.....	158
8.7	x11colors.....	158
8.8	fontsize.....	159
8.9	contour.....	159
8.10	geometry.....	165
8.11	grid3.....	165
8.12	interpolate.....	166
8.13	markers.....	166
8.14	math.....	168
8.15	patterns.....	169
8.16	roundedpath.....	169
8.17	slopefield.....	169
8.18	smoothcontour3.....	170
8.19	solids.....	170
8.20	stats.....	171
8.21	tube.....	171
9	专用模块.....	173
9.1	animation.....	173
9.2	animate.....	173
9.3	annotate.....	173
9.4	babel.....	173
9.5	binarytree.....	174
9.6	bsp.....	175
9.7	CAD.....	175
9.8	contour3.....	175
9.9	drawtree.....	175
9.10	embed.....	175
9.11	external.....	175
9.12	feynman.....	175
9.13	flowchart.....	175
9.14	labelpath.....	178
9.15	labelpath3.....	178
9.16	lmfit.....	178
9.17	MetaPost.....	178
9.18	obj.....	179
9.19	ode.....	179
9.20	pstoedit.....	179
9.21	rational.....	179
9.22	rationalSimplex.....	179
9.23	simplex.....	179
9.24	size10.....	179
9.25	size11.....	179

9.26	slide	179
9.27	syzygy	180
9.28	tree	180
9.29	trembling	180
10	开发者模块	181
10.1	plain	181
10.2	bezulate	181
10.3	simplex2	181
10.4	v3d	181
11	命令行选项	182
12	交互模式	187
13	图形用户界面	189
13.1	GUI 安装	189
13.2	GUI 使用	189
14	命令行界面	190
15	语言服务器协议	191
16	PostScript 转 Asymptote	192
17	帮助	193
18	调试器	194
19	致谢	195
	总索引	196

1 描述

Asymptote 是一种功能强大的描述性矢量图形语言，为技术制图提供了基于数学坐标的框架。标签与公式均使用 $\text{\LaTeX}$ 排版，以保证整个文档的一致性，从而获得与 $\text{\LaTeX}$ 为科学文本所提供的同等高水准的排版质量。默认情况下它生成 PostScript 输出，但也可以生成 OpenGL、PDF、SVG、WebGL、V3D 以及旧式的 PRC 矢量图形，此外还能生成 ImageMagick 软件包所能产生的任何格式。你甚至无需安装即可在 Web 浏览器中试用它，只需使用 **Asymptote Web Application**：

<https://asymptote.ualberta.ca>

还可以通过 curl 工具向该服务器发送远程命令(见 第 14 章 [Command-Line Interface], 190 页)。

Asymptote 相对于其他图形软件包的一大优势在于：它是一门高级编程语言，而不仅仅是一个图形应用程序。因此，它可以同时利用脚本（命令驱动）方式与图形用户界面(GUI)方式各自的优点来绘制图形。软件包自带的初级 GUI 程序 **xasy** 允许用户移动由脚本生成的对象。为了让普通用户也能使用 **Asymptote**，这一 GUI 目前正被开发为能够直接生成对象的成熟界面。不过，该语言的脚本部分现在已经可以供愿意学习几条简单 **Asymptote** 绘图命令(见 第 4 章 [Drawing commands], 14 页)的用户日常使用了。

Asymptote 面向数学（可以用复数乘法来旋转向量），并使用 $\text{\LaTeX}$ 来完成标签的排版。这对科学应用而言是一项重要特性。它的灵感来自一个更早的绘图程序（语法和能力都较弱），名为 **MetaPost**。

Asymptote 矢量图形语言提供以下特性：

- 一个数学图形的排版标准，正如 $\text{\TeX}/\text{\LaTeX}$ 是公式排版事实上的标准一样。
- 标签采用 $\text{\LaTeX}$ 排版，保证整个文档的一致性；
- 能够在 HTML 文件中生成并嵌入三维矢量 WebGL 图形；
- 能够在 PDF 文件中生成并嵌入三维矢量 PRC 图形；
- 一个自然的、基于坐标的技术制图框架，灵感来自 **MetaPost**，并拥有干净得多、强大得多、类似 C++ 的编程语法；
- 将图形编译为虚拟机代码以获得速度，同时不牺牲可移植性；
- 脚本语言的强大能力与 GUI 的便利性相结合；
- 使用其自有的、类似 C++ 的图形编程语言进行定制；
- 图形特性均有合理的默认值，且可以覆盖；
- 面向 PostScript 矢量图形语言的高级数学化接口，包括仿射变换和复变量；
- 能够创建新（匿名）函数的函数；
- 延迟绘制(deferred drawing)：使用单纯形法来解决固定尺寸对象（标签和箭头）与应随图形尺寸缩放的对象之间的整体尺寸约束问题；

Asymptote 的许多特性本身就是用 **Asymptote** 语言编写的。虽然 **Asymptote** 是为数学排版需求而设计的，但可以编写 **Asymptote** 模块，使其适应特定的应用。例如，提供了一个科学制图模块(见 8.1 节 [graph], 107 页)。**Asymptote** 的代码与输出示例（包括动画）可在以下地址查看：

<https://asymptote.sourceforge.io/gallery/>

在本手册中点击示例文件名（如 **Pythagoras**）会显示 PDF 输出，而点击其 **.asy** 扩展名则会在单独的窗口中显示相应的 **Asymptote** 代码。

许多外部资源的链接（包括一份出色的、由用户撰写的 `Asymptote` 教程）可在以下地址找到：

<https://asymptote.sourceforge.io/links.html>

`Asymptote` 的快速参考卡可在以下地址获取：

<https://asymptote.sourceforge.io/asyRefCard.pdf>

2 安装

在按照你所使用的具体发行版的说明完成安装之后，还请一并参阅 2.4 节 [Configuring], 4 页。我们建议在以下站点订阅新版本发布通告

<https://sourceforge.net/projects/asymptote>

用户也可以关注 Asymptote 论坛：

<https://sourceforge.net/p/asymptote/discussion/409349>

2.1 UNIX 二进制发行版

我们同时发布 Asymptote 的 tgz 和 RPM 两种二进制发行版。root 用户可以用以下命令安装 x.xx 版 Asymptote 的 Linux x86_64 tgz 发行版：

```
tar -C / -zxf asymptote-x.xx.x86_64.tgz
texhash
```

用于安装 LaTeX 样式文件的 texhash 命令是可选的。可执行文件将是 /usr/local/bin/asy，而示例代码默认会安装在 /usr/local/share/doc/examples 中。

Fedora 用户可以用下面这条命令轻松安装较新版本的 Asymptote

```
dnf -enablerepo=rawhide install asymptote
```

要在基于 Debian 的发行版（如 Ubuntu）上安装最新版本的 Asymptote，请按照从 UNIX 源码编译的说明操作（见 2.6 节 [Compiling from UNIX source], 6 页）。此外，Debian 用户也可以安装 Hubert Chathi 预编译的 Asymptote 二进制文件，下载地址为

<https://ftp.debian.org/debian/pool/main/a/asymptote>

2.2 macOS X 二进制发行版

macOS X 用户既可以选择编译 UNIX 源代码（见 2.6 节 [Compiling from UNIX source], 6 页），也可以安装可从以下地址获取的 Asymptote 二进制文件

<https://www.macports.org/>

或从

<https://brew.sh/>

请注意，许多 macOS X（以及 FreeBSD）系统缺少 GNU readline 库。要获得完整的交互功能，必须安装 4.3 或更高版本的 GNU readline。

2.3 Microsoft Windows

Microsoft Windows 操作系统的用户可以安装自解压(self-extracting)的 Asymptote 可执行文件 asymptote-x.xx-setup.exe，其中 x.xx 表示最新版本号。

排版标签需要有一个可用的 $\TeX$ 实现（我们推荐 <https://www.tug.org/texlive> 或 <https://miktex.org>）。你还需要从 <https://www.ghostscript.com/> 安装 9.56 或更高版本的 GPL Ghostscript。

要查看 PostScript 输出，你可以安装可从 <https://www.sumatrapdfreader.org/> 获取的 Sumatra PDF 程序。

来自 <https://www.imagemagick.org/script/binary-releases.php> 的 ImageMagick 软件包

是支持 HTML、PDF、SVG 和 PNG (见 [magick], 185 页) 之外的其他输出格式所必需的。来自 <https://www.python.org> 的 Python 3 解释器仅在你想要试用图形用户界面(见 第 13 章 [GUI], 189 页) 时才需要。

示例代码默认将安装在安装目录的 `examples` 子目录中 (默认安装目录为 `C:\Program Files\Asymptote`)。

2.4 配置

在交互模式下, 或在给定 `-v` 选项时 (在 MSDOS 下对单个文件运行 `Asymptote` 时该选项为默认值), `Asymptote` 会自动调用你的 PostScript 查看器 (UNIX 下为 `evince`) 来显示图形输出。PostScript 查看器应当能够在输出文件每次更新时自动重绘。UNIX 下的 PostScript 查看器 `gv` 支持这一功能 (通过 `SIGHUP` 信号)。`ggv` 用户需要在 `Edit/PostScript Viewer Preferences` 下启用 `Watch file`。

配置变量最容易的设置方式, 是将其作为 `Asymptote` 变量写入可选的配置文件 `config.asy` (见 [configuration file], 185 页)。例如, 设置 `pdfviewer` 指定 PDF 查看器的位置。以下是 UNIX 下若干重要配置变量的默认值:

```
import settings;
pdfviewer="acroread";
htmlviewer="google-chrome";
psviewer="evince";
display="display";
animate="animate";
gs="gs";
libgs="";
```

在 MSDOS 下, 查看器设置 `htmlviewer`、`pdfviewer`、`psviewer`、`display` 和 `animate` 默认为字符串 `cmd`, 表示请求与各文件类型常规关联的应用程序。`gs` 和 `libgs` 的默认值 (因安装方式而异) 由 Microsoft Windows 注册表自动确定。`gs` 设置指定 PostScript 处理器 Ghostscript 的位置, 可从 <https://www.ghostscript.com/> 获取。

配置变量 `htmlviewer` 指定用于显示 3D WebGL 输出的浏览器。默认设置在 UNIX 下为 `google-chrome`, 在 Microsoft Windows 下为 `cmd`。注意 Internet Explorer 不支持 WebGL; Microsoft Windows 用户应将其默认 html 浏览器设置为 `chrome` 或 `microsoft-edge`。默认情况下, 2D 和 3D HTML 图像会扩展到容纳它的画布(canvas); 将配置变量 `absolute` 设为 `true` 即可禁用这一行为。

在 UNIX 系统上, 为了支持 Adobe Reader 中 PDF 文件的自动文档重载, 我们建议把 `Asymptote` 系统目录 (默认在 UNIX 下为 `/usr/local/share/asymptote`) 中的文件 `reload.js` 复制到 `~/.adobe/Acrobat/x.x/JavaScripts/`, 其中 `x.x` 表示相应的 Adobe Reader 版本号。随后必须把以下内容放入 `Asymptote` 配置文件, 以显式启用自动文档重载功能:

```
import settings;
pdfreload=true;
pdfreloadOptions="-tempFile";
```

这一重载功能在 MSDOS 下没有用处, 因为在该操作系统上, 文档在被 Adobe Reader 首次关闭之前本来就无法更新。

配置变量 `dir` 可用于调整搜索路径(见 2.5 节 [Search paths], 5 页)。

默认情况下, Asymptote 会假定纸张类型为 `letter`, 尝试将图形居中放在页面上。默认纸张类型可通过配置变量 `papertype` 改为 `a4`。要按其他纸张尺寸对齐, 可以设置配置变量 `paperwidth` 和 `paperheight`。

这些额外的配置变量通常无需调整:

```
config
texpath
texcommand
dvips
dvisvgm
convert
asygl
```

警告 (例如 `"unbounded"` 和 `"offaxis"`) 可以通过以下函数来启用或禁用:

```
warn(string s);
nowarn(string s);
```

也可以直接修改字符串数组 `settings.suppress`, 其中列出了所有被禁用的警告。

配置变量也可以通过命令行选项来设置或覆盖:

```
asy -psviewer=evince -V venn
```

或者, 也可以按常规方式设置上述配置变量对应的系统环境变量。相应环境变量名称由配置变量名转换为大写并加上前缀 `ASYMPTOTE_` 而得到: 例如, 要设置环境变量

```
ASYMPTOTE_PAPERTYPE="a4";
```

在 Microsoft Windows XP 下可按以下步骤操作:

1. 单击 Start 按钮;
2. 右键单击 My Computer;
3. 选择 View system information;
4. 单击 Advanced 选项卡;
5. 单击 Environment Variables 按钮。

2.5 搜索路径

在查找 Asymptote 文件时, `asy` 会按下面列出的顺序搜索以下路径:

1. 当前目录;
2. 由配置变量 `dir` 或环境变量 `ASYMPTOTE_DIR` 指定的一个或多个目录组成的列表 (在 UNIX 下以 `:` 分隔, 在 MSDOS 下以 `;` 分隔);
3. 由环境变量 `ASYMPTOTE_HOME` 指定的目录; 如果未设置该变量, 则使用用户主目录中的 `.asy` 目录 (在 MSDOS 下为 `%USERPROFILE%\asy`);
4. Asymptote 系统目录 (默认在 UNIX 下为 `/usr/local/share/asymptote`, 在 MSDOS 下为 `C:\Program Files\Asymptote`)。
5. Asymptote 示例目录 (默认在 UNIX 下为 `/usr/local/share/doc/examples`, 在 MSDOS 下为 `C:\Program Files\Asymptote\examples`)。

2.6 从 UNIX 源码编译

要从位于以下地址之下、子目录 `x.xx` 中的源代码发行版 `asymptote-x.xx.src.tgz` 编译并安装 UNIX 可执行文件，请执行以下命令：

```
gunzip asymptote-x.xx.src.tgz
tar -xf asymptote-x.xx.src.tar
cd asymptote-x.xx
```

然后用以下命令编译 Asymptote

```
./configure
make all
make install
```

你可能需要在先以非 root 权限运行 `make all` 之后（以避免生成权限不正确的文件），把最后一行换成 `sudo make install`。请务必使用 GNU make（在非 GNU 系统上，这一命令可能叫作 `gmake`）。要构建文档，你可能需要安装 `texinfo-tex` 软件包。如果因 `texinfo` 或 `pdftex` 安装损坏而报错，只需把

<https://asymptote.sourceforge.io/asymptote.pdf>

放入 `doc` 目录并重复执行 `make all` 命令即可。

对于（默认的）系统级安装，最后一条命令应以 root 用户身份执行。要在无 root 权限的情况下安装，请把 `./configure` 命令改为

```
./configure --prefix=$HOME/asymptote
```

可以通过配置选项 `./configure --disable-gc` 来禁用 Boehm 垃圾回收器(garbage collector)。要查看其他配置选项的列表，可执行 `./configure --help`。例如，在 macOS X 下，可以让 `configure` 使用 `clang` 编译器，并在非标准位置查找头文件和库：

```
./configure CC=clang CXX=clang++ CPPFLAGS=-I/opt/local/include LDFLAGS=-L/opt/local/lib
```

如果你用 `gcc` 编译 Asymptote，需要使用支持 C++17 的版本。要获得完整的交互功能，需要 4.3 或更高版本的 GNU readline 库。`patches` 目录中的文件 `gcc3.3.2curses.patch` 可用来修补某些 AIX 和 IRIX 系统上损坏的 `curses.h` 头文件（或其在当前目录中的本地副本）。

只有当你希望 Asymptote 能够对数据进行傅里叶变换(Fourier transform)（例如计算音频功率谱）时，才需要 FFTW 库。只有当你需要 GSL 所支持的特殊函数(special function)时，才需要 GSL 库。

如果你不想在系统范围内安装 Asymptote，只需确保编译好的二进制文件 `asy` 和 GUI 脚本 `xasy` 位于你的路径中，并把配置变量 `dir` 指向 `base` 目录（位于 Asymptote 源代码的顶层目录中）。

2.7 编辑模式

emacs 用户可以用 `asy-mode` 模式编辑 Asymptote 代码，启用方法是在其 `.emacs` 初始化文件中加入以下几行，并把 `ASYDIR` 替换为 Asymptote 系统目录的位置（默认为 `/usr/local/share/asymptote`，在 MSDOS 下为 `C:\Program Files\Asymptote`）：

```
(add-to-list 'load-path "ASYDIR")
(autoload 'asy-mode "asy-mode.el" "Asymptote major mode." t)
(autoload 'lasy-mode "asy-mode.el" "hybrid Asymptote/Latex major mode." t)
(autoload 'asy-insinuate-latex "asy-mode.el" "Asymptote insinuate LaTeX." t)
(add-to-list 'auto-mode-alist '("\\.asy$" . asy-mode))
```

此模式中特别有用的键绑定是 `C-c C-c`，它编译并显示当前缓冲区；还有键绑定 `C-c ?`，它显示光标处命令可用的函数原型。为了获得完整功能，你还应当安装 Apache Software Foundation 软件包 `two-mode-mode`：

```
https://github.com/erlyaws/yaws/blob/master/two-mode-mode.el
```

混合模式 `lasy-mode` 可用来编辑内嵌 Asymptote 代码的 LaTeX 文件(见 第 7 章 [LaTeX usage], 102 页)。这一模式可以在 `latex-mode` 中用按键序列 `M-x lasy-mode <RET>` 启用。在 UNIX 系统上，系统会从环境变量 `ASYMPTOTE_SITEDIR` 指定的以空格分隔的目录列表中的所有 `asy` 文件生成额外的关键字。关于 `asy-mode` 的更多文档可在 `emacs` 中按按键序列 `C-h f asy-mode <RET>` 查看。

`vim` 爱好者可以用以下命令为 Asymptote 定制 `vim`

```
cp /usr/local/share/asymptote/asy.vim ~/.vim/syntax/asy.vim
```

并在其 `~/.vimrc` 文件中加入以下内容：

```
augroup filetypedetect
au BufNewFile,BufRead *.asy      setf asy
augroup END
filetype plugin on
```

如果这些目录或文件不存在，直接创建即可。要让 `vim` 能用 `:make` 运行当前的 `asymptote` 脚本，只需在 `~/.vim/ftplugin/asy.vim` 中加入：

```
setlocal makeprg=asy\ %
setlocal errorformat=%f:\ %l.%c:\ %m
```

要为 KDE 编辑器 Kate 启用语法高亮(syntax highlighting)支持，可以在 `/usr/local/share/asymptote` 目录中运行 `asy-kate.sh`，并把生成的 `asymptote.xml` 文件放入 `~/.local/share/org.kde.syntax-highlighting/syntax/`。

2.8 Git

要使用 `git` 安装最新开发版本的 Asymptote，需要执行以下命令：

```
git clone https://github.com/vectorgraphics/asymptote
```

```
cd asymptote
./autogen.sh
./configure
make all
make install
```

若要在不进行优化的情况下编译，在 Ubuntu 系统上，你可能需要先安装所需的依赖：

```
apt-get build-dep asymptote
```

2.9 构建文档

要构建 PDF 文档 `doc/asymptote.pdf`：

```
make man
```

要构建 HTML 文档 `doc/png/index.html`：

```
make html
```

编译文档中的图形需要 `asy` 可执行文件。

2.10 卸载

要卸载 Linux x86_64 二进制发行版，执行

```
tar -zxvf asymptote-x.xx.x86_64.tgz | xargs -replace=% rm %/  
texhash
```

要卸载从源代码发行版安装的所有 Asymptote 文件，请使用命令

```
make uninstall
```

3 教程

这里给出 Asymptote 的简要介绍。若需要更全面的入门材料，请参阅 Charles Staats 撰写的优秀 Asymptote 教程：

https://asymptote.sourceforge.io/asymptote_tutorial.pdf

另一份 Asymptote 教程以 wiki 的形式提供，其中的图像由在线 Asymptote 引擎渲染：

[https://www.artofproblemsolving.com/wiki/?title=Asymptote_\(Vector_Graphics_Language\)](https://www.artofproblemsolving.com/wiki/?title=Asymptote_(Vector_Graphics_Language))

3.1 批处理模式绘图

要从坐标(0,0)到坐标(100,100)画一条直线，可创建一个内容如下的文本文件 test.asy：

```
draw((0,0)-(100,100));
```

然后执行命令

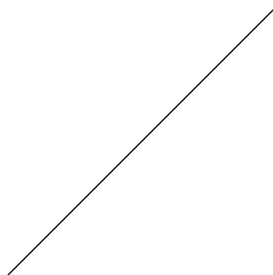
```
asy -V test
```

另外，MSDOS 用户也可以将 test.asy 拖放到桌面上的 asy 图标（或者将 Asymptote 设为扩展名 asy 的默认应用程序）。

这种方法称为批处理模式(batch mode)，它输出 PostScript 文件 test.eps。如果你更想要 PDF 输出，请使用命令行

```
asy -V -f pdf test
```

无论采用哪种方式，-v 选项都会打开一个查看器窗口，让你可以立即查看结果：



这里，-- 连接符用一条线段将 (0,0) 和 (100,100) 两点连接起来。

3.2 交互模式绘图

另一种方法是交互模式(interactive mode)：Asymptote 在用户输入命令的同时逐条读取它们。要试用一下，可点击 Asymptote 图标或键入命令 asy 来进入 Asymptote 的交互模式。然后键入

```
draw((0,0)-(100,100));
```

再按下 Enter，即可得到上面的图像。

此时你可以继续键入更多 draw 命令，它们会被添加到所显示的图形中；键入 erase 可清空画布；键入

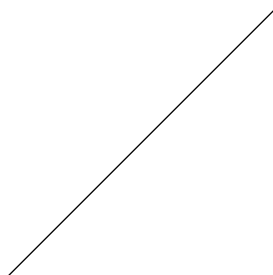
```
input test;
```

以执行文件 test.asy 中包含的全部命令，或者键入 quit 退出交互模式。在交互模式中，你可以使用方向键来编辑先前输入的行。tab 键会自动补全没有歧义的单词；否则，再按一次 tab 会显示可能的选择。交互模式特有的更多命令在第 12 章 [Interactive mode], 187 页, 中介绍。

3.3 图形尺寸

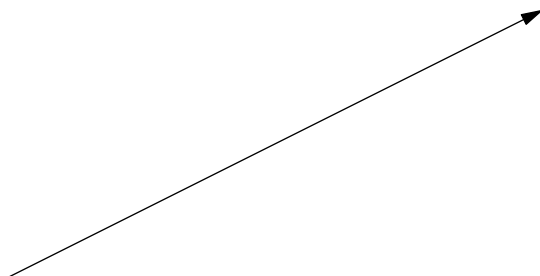
在 Asymptote 中, 形如 $(0,0)$ 和 $(100,100)$ 的坐标称为数对 (pair), 以 PostScript 的 "big points" 为单位 ($1 \text{ bp} = 1/72 \text{ inch}$), 而默认线宽为 0.5bp 。不过, 直接使用 PostScript 坐标往往并不方便。下一个例子通过把直线 $(0,0) \rightarrow (1,1)$ 缩放到恰好放入一个宽 100.5bp 、高 100.5bp 的矩形 (多出的 0.5bp 用于补偿线宽), 产生与上一个例子完全相同的输出:

```
size(100.5,100.5);
draw((0,0)-(1,1));
```



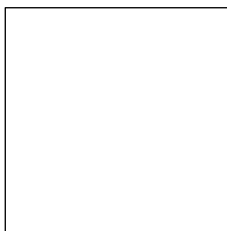
也可以用 pt ($1 \text{ pt} = 1/72.27 \text{ inch}$)、 cm 、 mm 或 inches 来指定尺寸。两个非零的尺寸参数 (或单个尺寸参数) 会同时限制两个方向上的尺寸, 并保持宽高比。如果某个尺寸参数取 0 , 则该方向不受任何限制; 整体缩放将由另一个方向决定 (见 [size], 48 页):

```
size(0,100.5);
draw((0,0)-(2,1),Arrow);
```



要连接若干个点并创建一条循环路径(cyclic path), 可使用 `cycle` 关键字:

```
size(3cm);
draw((0,0)-(1,0)-(1,1)-(0,1)-cycle);
```



为方便起见, 路径 $(0,0) \text{--}(1,0) \text{--}(1,1) \text{--}(0,1) \text{--cycle}$ 可以用预定义变量 `unitsquare` 来代替, 或者等价地写为 `box((0,0),(1,1))`。

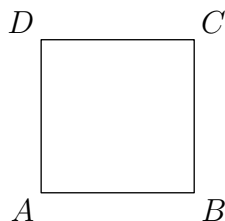
要让用户坐标恰好表示 1cm 的倍数:

```
unitsize(1cm);
draw(unitsquare);
```

3.4 标签

在 Asymptote 中添加标签很容易: 只需将标签指定为一个双引号括起的 $\text{\LaTeX}$ 字符串、一个坐标, 以及一个可选的对齐方向:

```
size(3cm);
draw(unitsquare);
label("$A$", (0,0), SW);
label("$B$", (1,0), SE);
label("$C$", (1,1), NE);
label("$D$", (0,1), NW);
```

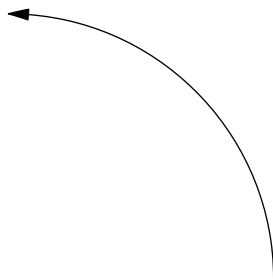


Asymptote 使用标准的罗盘方向(compass directions) $E=(1,0)$ 、 $N=(0,1)$ 、 $NE=\text{unit}(N+E)$ 和 $ENE=\text{unit}(E+NE)$ 等; 这些方向与 `up`、`down`、`right` 和 `left` 一起, 在 Asymptote 的基础模块 `plain` 中定义为数对 (若某个用户恰好有一个名为 `E` 的局部变量, 可以在罗盘方向 `E` 前面加上定义它的模块名来访问它: `plain.E`)。

3.5 路径

下面的例子绘制一条近似四分之一圆的路径, 并以一个箭头结尾:

```
size(100,0);
draw((1,0){up}..{left}(0,1),Arrow);
```



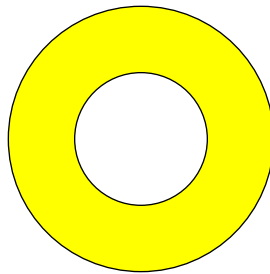
这里，花括号中的方向 `up` 和 `left` 分别指定了点 $(1,0)$ 处的出射方向和点 $(0,1)$ 处的入射方向。

一般而言，路径被指定为一个由若干个点（或其他路径）组成的列表，相邻元素之间用 `--`（表示一条直线段）或 `..`（表示三次样条(cubic spline)）来连接(见 第 5 章 [Bezier curves], 22 页)。在末尾指定 `..cycle` 会创建一条平滑连回起始节点的循环路径，例如下面这个单位圆的近似（精度在 0.06% 以内）：

```
path unitcircle=E..N..W..S..cycle;
```

一条 Asymptote 路径是连通的，因此等价于一个 PostScript subpath。二元运算符 `^^` 要求将画笔从左侧路径的终点移动到右侧路径的起点（既不绘制，也不影响端点曲率）；可以用它把若干条 Asymptote 路径组合成一个 `path[]` 数组（等价于一条 PostScript path）：

```
size(0,100);
path unitcircle=E..N..W..S..cycle;
path g=scale(2)*unitcircle;
filldraw(unitcircle^^g,evenodd+yellow,black);
```



这里的 PostScript 奇偶(even-odd)填充规则指定只填充两个单位圆之间所围的区域(见 [fillrule], 40 页)。在本例中，如果留意让各条路径的取向交替变化，也可以用默认为零环绕数(zero winding number)填充规则达到相同效果：

```
filldraw(unitcircle^^reverse(g),yellow,black);
```

模块 `three` 中的 `box(triple, triple)` 函数使用了 `^^` 运算符，以便在构造立方体 `unitbox` 的棱边时不重复描线(见 8.2 节 [three], 136 页)：

```
import three;

currentprojection=orthographic(5,4,2,center=true);

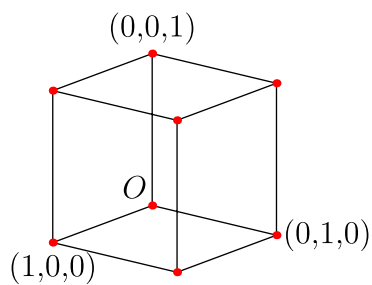
size(5cm);
size3(3cm,5cm,8cm);

draw(unitbox);

dot(unitbox,red);

label("$O$", (0,0,0),NW);
label("(1,0,0)", (1,0,0),S);
label("(0,1,0)", (0,1,0),E);
```

```
label("(0,0,1)",(0,0,1),Z);
```



更多示例请参见 8.1 节 [graph], 107 页, 一节 (或在线 Asymptote 画廊 以及 <https://asymptote.sourceforge.io> 上公布的外部链接), 其中包括二维和交互式三维科学图形。Philippe Ivaldi 也在 <https://blog.piprime.fr/asymptote/> 上发布了更多示例。

4 绘图命令

Asymptote 的所有图形功能都建立在四个基本命令之上。三个 PostScript 绘图命令 `draw`、`fill` 与 `clip` 按其执行顺序将对象添加到图片中，最近绘制的对象显示在最上方。标签命令 `label` 可用于添加文本标签和外部 EPS 图像，它们会出现在 PostScript 对象之上（因为通常这正是用户所期望的），但同样遵循它们被执行的相对顺序。在图片上绘制对象后，可以使用 `shipout` 函数输出该图片（见 [shipout], 48 页）。

如果你希望在标签（或原样 `tex` 命令；见 [tex], 52 页）之上绘制 PostScript 对象，可以使用 `layer` 命令来开启一个新的 PostScript/LaTeX 图层(layer)：

```
void layer(picture pic=currentpicture);
```

`layer` 函数使用户可以完全控制对象的绘制顺序。图层是按顺序绘制的，最近的图层显示在最上方。在每个图层内部，标签、图像和原样 `tex` 命令总是会在该图层的 PostScript 对象之后绘制。

可以使用以下命令生成分页：

```
void newpage(picture pic=currentpicture);
```

虽然这些绘图命令中的一部分接受许多选项，但它们都具有合理的默认值（例如，图片参数默认值为 `currentpicture`）。

4.1 draw

```
void draw(picture pic=currentpicture, Label L="", path g,
          align align=NoAlign, pen p=currentpen,
          arrowbar arrow=None, arrowbar bar=None, margin margin=NoMargin,
          Label legend="", marker marker=nomarker);
```

使用画笔 `p` 在图片 `pic` 上绘制路径 `g`，并附带可选的绘制属性（标签 `L`、显式标签对齐 `align`、箭头与端线(bar) `arrow` 和 `bar`、边距 `margin`、图例 `legend` 以及标记 `marker`）。只有路径这一个参数是必需的。为方便起见，参数 `arrow` 与 `bar` 可以按任意顺序指定。参数 `legend` 是用于构造可选图例条目的 `Label`。

端线 `bar` 常用于标示尺寸。`bar` 的可选值为 `None`、`BeginBar`、`EndBar`（或等价的 `Bar`）以及 `Bars`（在路径的两端都绘制端线）。除 `None` 外，这些端线指定符都可以接受一个可选的实数参数，用以表示以 PostScript 坐标为单位的端线长度。默认端线长度为 `barsize(pen)`。

`arrow` 的可选值为 `None`、`Blank`（不绘制箭头或路径）、`BeginArrow`、`MidArrow`、`EndArrow`（或等价的 `Arrow`）以及 `Arrows`（在路径的两端都绘制箭头）。

还有一些箭头版本具有稍作修改的 `size` 和 `angle` 默认值，适用于弧形箭头：`BeginArcArrow`、`EndArcArrow`（或等价的 `ArcArrow`）、`MidArcArrow` 以及 `ArcArrows`。

例如：

```
draw((0,0)-(1,1),arrow=Arrows);
```

除 `None` 和 `Blank` 之外的所有箭头指定符都可以赋予可选参数，例如：

```
draw((0,0)-(1,1),arrow=Arrow(
    arrowhead=HookHead,size=3mm,angle=20,filltype=Draw,position=0.9));
```

函数 `Arrow` 的签名如下：

```
arrowbar Arrow(arrowhead arrowhead=DefaultHead,
               real size=0, real angle=arrowangle,
```

`filltype filltype=null, position position=EndPoint)`

调用 `Arrow()` 会返回一个 `arrowbar` 对象 `Arrow`。各参数含义如下：

- `arrowhead` 可以是预定义的箭头样式 `DefaultHead`、`SimpleHead`、`HookHead`、`TeXHead` 之一。
- 实数 `size` 是以 `PostScript` 坐标表示的箭头大小。
使用画笔 `p` 绘制时的默认箭头大小为 `arrowsize(p)`。
- 实数 `angle` 是以度为单位的箭头角度。
- `filltype filltype` (见 [filltype], 49 页),
- (除 `MidArrow` 和 `Arrows` 外) 实数 `position` (沿着路径的 `point(path p, real t)` 意义下的位置), 用于指定箭头尖端放置的位置。

边距 `margin` 可用于将路径的可见部分缩短 `labelmargin(p)`, 以避免与其他绘制的对象重叠。

`margin` 的典型值有：

`NoMargin`

`BeginMargin`

`EndMargin`

(等价于 `Margin`)

`Margins` 在路径的两端都保留边距。

`Margin(real begin, real end=begin)`

分别以用于对齐标签的单位 `labelmargin(p)` 的倍数来指定起始和终止边距的大小。

`BeginPenMargin`

`EndPenMargin`

(等价于 `PenMargin`)

`PenMargins`

`PenMargin(real begin, real end=begin)`

以画笔线宽为单位指定边距, 在绘制路径或箭头时会将画笔线宽纳入考虑。

`DotMargin`

`PenMargin(-0.5*dotfactor, 0.5*dotfactor)` 的缩写, 用于从通常的起始点一直绘制到宽度为 `dotfactor*linewidth(p)` 的终点圆点边界处。

`BeginDotMargin`

`DotMargins`

其工作原理类似。

`TrueMargin(real begin, real end=begin)`

直接以 `PostScript` 单位指定边距, 与画笔线宽无关。

箭头、端线和边距的使用可通过示例 `Pythagoras.asy` 与 `sqrtx01.asy` 说明。由于端线旨在标示精确的尺寸, 因此它们应当仅在指定 `NoMargin` (默认值) 的情况下绘制。

图片 `pic` 的图例可以使用以下例程适应并对齐到框(frame):

`frame legend(picture pic=currentpicture, int perline=1,`

`real xmargin=legendmargin, real ymargin=xmargin,`

```

    real linelength=legendlinelength,
    real hskip=legendhskip, real vskip=legendvskip,
    real maxwidth=0, real maxheight=0,
    bool hstretch=false, bool vstretch=false, pen p=currentpen);

```

这里 `xmargin` 和 `ymargin` 指定四周的 x 和 y 边距, `perline` 指定每行的条目数量 (默认为 1; 0 表示自动选择此数量), `linelength` 指定路径线条的长度, `hskip` 和 `vskip` 指定行间距 (作为图例条目大小的倍数), `maxwidth` 和 `maxheight` 指定生成的图例宽度和高度的可选上限 (0 表示无限制), `hstretch` 和 `vstretch` 允许图例水平或垂直拉伸, `p` 指定用于绘制边框的画笔。然后可以使用 `add` 或 `attach` (见 [add about], 50 页) 将图例框添加并围绕目标图片 `dest` 上的某一点对齐。

要绘制点(dot), 只需绘制包含单个点的路径即可。模块 `plain` 中定义的 `dot` 命令用于绘制一个直径等于显式画笔线宽或默认线宽放大 `dotfactor` (默认为 6) 倍的点, 使用指定的 `filltype` (见 [filltype], 49 页) 或 `dotfilltype` (默认为 `Fill`):

```

void dot(frame f, pair z, pen p=currentpen, filltype filltype=dotfilltype);
void dot(picture pic=currentpicture, pair z, pen p=currentpen,
        filltype filltype=dotfilltype);
void dot(picture pic=currentpicture, Label L, pair z, align align=NoAlign,
        string format=defaultformat, pen p=currentpen, filltype filltype=dotfilltype);
void dot(picture pic=currentpicture, Label[] L=new Label[], pair[] z,
        align align=NoAlign, string format=defaultformat, pen p=currentpen,
        filltype filltype=dotfilltype);
void dot(picture pic=currentpicture, path[] g, pen p=currentpen,
        filltype filltype=dotfilltype);
void dot(picture pic=currentpicture, Label L, pen p=currentpen,
        filltype filltype=dotfilltype);

```

如果变量 `Label` 作为第三个例程的 `Label` 参数给出, 则 `format` 参数将用于根据点的位置格式化字符串 (这里的 `defaultformat` 为 `"$%.4g$"`)。第四个例程会在数对(pair)数组 `z` 的每个点上绘制一个点。用户也可以在路径的每个节点上绘制点:

```

void dot(picture pic=currentpicture, Label[] L=new Label[],
        explicit path g, align align=RightSide, string format=defaultformat,
        pen p=currentpen, filltype filltype=dotfilltype);

```

关于标记路径节点的更通用方法, 请参见 [pathmarkers], 116 页, 和 8.13 节 [markers], 166 页。

要围绕用户坐标 `origin` 绘制 (以 PostScript 坐标为单位的) 固定大小对象, 请使用例程:

```

void draw(pair origin, picture pic=currentpicture, Label L="", path g,
        align align=NoAlign, pen p=currentpen, arrowbar arrow=None,
        arrowbar bar=None, margin margin=NoMargin, Label legend="",
        marker marker=nomarker);

```

4.2 fill

```

void fill(picture pic=currentpicture, path g, pen p=currentpen);

```

在图片 `pic` 上, 使用画笔 `p` 填充由闭合路径 `g` 围成的内部区域。

还有一个方便的 `filldraw` 命令，它先填充路径，然后再绘制边界。可以为每个操作指定单独的画笔：

```
void filldraw(picture pic=currentpicture, path g, pen fillpen=currentpen,
              pen drawpen=currentpen);
```

`fill` 的这种固定大小版本允许用户围绕用户坐标 `origin` 填充以 PostScript 坐标描述的对象：

```
void fill(pair origin, picture pic=currentpicture, path g, pen p=currentpen);
```

这只是以下命令的简便缩写：

```
picture opic;
fill(opic,g,p);
add(pic,opic,origin);
```

例程

```
void filloutside(picture pic=currentpicture, path g, pen p=currentpen);
```

填充路径 `g` 外部、一直延伸到图片 `pic` 当前边界的区域。

使用填充规则 `fillrule`，在画笔二维数组 `p` 上平滑变化的栅格渐变着色，可以通过以下函数生成：

```
void latticeshade(picture pic=currentpicture, path g, bool stroke=false,
                  pen fillrule=currentpen, pen[][] p)
```

如果 `stroke=true`，则填充区域与使用 `draw(pic,g,zerowinding)` 绘制的区域相同；在这种情况下，路径 `g` 无需是闭合的。`p` 中的画笔必须属于相同的颜色空间。可以使用函数 `rgb(pen)` 或 `cmyk(pen)` 将画笔提升到更高的颜色空间，如示例文件 `latticeshading.asy` 中所示。

沿线段 `a--b` 方向从 `pena` 平滑过渡到 `penb` 的轴向渐变着色可以通过以下函数实现：

```
void axialshade(picture pic=currentpicture, path g, bool stroke=false,
                pen pena, pair a, bool extenda=true,
                pen penb, pair b, bool extendb=true);
```

布尔参数 `extenda` 和 `extendb` 指明着色是否应当延伸到轴端点 `a` 和 `b` 之外。示例文件 `axialshade.asy` 中提供了轴向着色的示例。

在以 `a` 为圆心、`ra` 为半径的圆上的 `pena` 与以 `b` 为圆心、`rb` 为半径的圆上的 `penb` 之间平滑变化的径向渐变着色与之类似：

```
void radialshade(picture pic=currentpicture, path g, bool stroke=false,
                 pen pena, pair a, real ra, bool extenda=true,
                 pen penb, pair b, real rb, bool extendb=true);
```

布尔参数 `extenda` 和 `extendb` 指明着色是否应当延伸到半径 `a` 和 `b` 之外。示例文件 `shade.asy`、`ring.asy` 以及 `shadestroke.asy` 中提供了径向着色的插图。

在由顶点 `z` 和边标志 `edges` 定义的三角网格上，使用填充规则 `fillrule` 以及画笔数组 `p` 中的顶点颜色实现的 Gouraud 着色可通过以下函数实现：

```
void gouraudshade(picture pic=currentpicture, path g, bool stroke=false,
                  pen fillrule=currentpen, pen[] p, pair[] z,
                  int[] edges);
void gouraudshade(picture pic=currentpicture, path g, bool stroke=false,
                  pen fillrule=currentpen, pen[] p, int[] edges);
```

在第二种形式中, z 的元素被取为路径 g 的连续节点。 p 中的画笔必须属于相同的颜色空间。示例文件 `Gouraud.asy` 中提供了 Gouraud 着色的插图。Gouraud 着色中使用的边标志说明参见 PostScript Language Reference (第 3 版) 第 270–274 页:

<https://www.adobe.com/jp/print/postscript/pdfs/PLRM.pdf>

使用剪切路径 g 、填充规则 `fillrule`, 在由路径数组 b 中的 n 个长度为 4 的闭合路径所限定的网格片(patch)上, 使用 $n \times 4$ 画笔数组 p 中指定的顶点颜色以及 $n \times 4$ 数组 z 中的内部控制点实现的张量积(tensor product)着色, 可以通过以下函数实现:

```
void tensorshade(picture pic=currentpicture, path[] g, bool stroke=false,
                pen fillrule=currentpen, pen[] p, path[] b=g,
                pair[] z=new pair[]);
```

如果数组 z 为空, 则使用自动计算颜色控制点的 Coons 着色。 p 中的画笔必须属于相同的颜色空间。对于单个网格片 ($n = 1$) 的情况, 还提供了更简单的接口:

```
void tensorshade(picture pic=currentpicture, path g, bool stroke=false,
                pen fillrule=currentpen, pen[] p, path b=g,
                pair[] z=new pair[]);
```

还可以使用给定的画笔数组在一序列路径中相邻路径之间的区域上平滑着色:

```
void draw(picture pic=currentpicture, pen fillrule=currentpen, path[] g,
          pen[] p);
```

示例文件 `tensor.asy`、`Coons.asy`、`BezierPatch.asy` 以及 `rainbow.asy` 中提供了张量积着色和 Coons 着色的示例。

使用可生成 PDF 输出的 $\text{T}_\text{E}\text{X}$ 引擎 (见 [texengines], 185 页) 可以获得更通用的着色可能性: 例程

```
void functionshade(picture pic=currentpicture, path[] g, bool stroke=false,
                  pen fillrule=currentpen, string shader);
```

根据填充规则 `fillrule`, 使用由字符串 `shader` 指定的 PostScript 计算器例程在图片 `pic` 上对路径 g 的内部着色; 该例程接收 2 个参数 (均在 $[0,1]$ 范围内), 并返回 `colors(fillrule).length` 个颜色分量。函数着色在示例 `functionshading.asy` 中进行了展示。

以下例程结合使用 `evenodd` 剪切与 `^^` 运算符来取消填充(unfill)某个区域:

```
void unfill(picture pic=currentpicture, path g);
```

4.3 clip

```
void clip(picture pic=currentpicture, path g, bool stroke=false,
          pen fillrule=currentpen);
```

使用填充规则 `fillrule` (见 [fillrule], 40 页), 将图片 `pic` 的当前内容剪切到由路径 g 围成的区域。如果 `stroke=true`, 被剪切的部分与使用 `draw(pic,g,zerowinding)` 绘制的区域相同; 在这种情况下, 路径 g 无需是闭合的。虽然剪切没有深度概念 (它穿透图层乃至页面), 但可以将剪切局部化到一个临时图片, 然后再将该图片添加到 `pic` 中。关于图片剪切的示意, 请参阅第 7 章 [LaTeX usage], 102 页, 中的第一个示例。

4.4 label

```
void label(picture pic=currentpicture, Label L, pair position,
           align align=NoAlign, pen p=currentpen, filltype filltype=NoFill)
```

使用画笔 `p` 在图片 `pic` 上绘制标签 `L`。如果 `align` 为 `NoAlign`，标签将居中于用户坐标 `position` 处；否则它将沿 `align` 的方向对齐，并从 `position` 偏移 PostScript 偏移量 `align*labelmargin(p)`。

这里，`real labelmargin(pen p=currentpen)` 是用于对齐标签的量。在下面的代码中，`label("abcdefg",(0,0),align=up,basealign);`

标签的基线将恰好在中心上方 `labelmargin(currentpen)` 个 PostScript 单位处。

常量 `Align` 可用于将标签的左下角对齐到 `position` 处。标签 `L` 既可以是字符串，也可以通过调用以下例程之一获得的结构：

```
Label Label(string s="", pair position, align align=NoAlign,
            pen p=nullpen, embed embed=Rotate, filltype filltype=NoFill);
Label Label(string s="", align align=NoAlign,
            pen p=nullpen, embed embed=Rotate, filltype filltype=NoFill);
Label Label(Label L, pair position, align align=NoAlign,
            pen p=nullpen, embed embed=L.embed, filltype filltype=NoFill);
Label Label(Label L, align align=NoAlign,
            pen p=nullpen, embed embed=L.embed, filltype filltype=NoFill);
```

`Label` 的文本可以通过在左侧乘以仿射变换（见 6.4 节 [Transforms], 46 页）来进行缩放、倾斜、旋转或平移。例如，`rotate(45)*xscale(2)*L` 首先沿 x 方向缩放 `L`，然后将其逆时针旋转 45 度。`Label` 的最终位置也可以通过 PostScript 坐标平移来进行偏移：`shift(10,0)*L`。在 `Label` 内部指定的显式画笔会覆盖其他画笔参数。`embed` 参数决定了 `Label` 应当如何随嵌入图片进行变换：

`Shift` 仅随嵌入图片平移；
`Rotate` 仅随嵌入图片平移和旋转（默认）；
`Rotate(pair z)`
 随（经图片变换的）向量 z 旋转。
`Slant` 仅随嵌入图片平移、旋转、倾斜和镜像；
`Scale` 随嵌入图片平移、旋转、倾斜、镜像和缩放。

要为路径添加标签，请使用：

```
void label(picture pic=currentpicture, Label L, path g, align align=NoAlign,
          pen p=currentpen, filltype filltype=NoFill);
```

默认情况下，标签将放置在路径的中点。在构造 `Label` 时，可以将替代的标签位置（在 `point(path p, real t)` 的意义下）指定为 `position` 的实数值。位置 `Relative(real)` 指定相对于路径总弧长的位置。以下便捷缩写已预定义：

```
position BeginPoint=Relative(0);
position MidPoint=Relative(0.5);
position EndPoint=Relative(1);
```

路径标签沿 `align` 方向对齐，该方向可以指定为绝对罗盘方向（数对 `pair`），或者相对于路径局部方向的北轴测量的方向 `Relative(pair)`。为了方便起见，`LeftSide`、`Center` 和 `RightSide` 分别定义为 `Relative(W)`、`Relative((0,0))` 和 `Relative(E)`。在左侧将 `LeftSide` 和 `RightSide` 乘以实数缩放因子将使标签离路径更远或更近。

带有长度为 `arrowlength` 的固定大小箭头（从方向 `dir` 指向 `b`）的标签可以通过以下例程生成：

```
void arrow(picture pic=currentpicture, Label L="", pair b, pair dir,
           real length=arrowlength, align align=NoAlign,
           pen p=currentpen, arrowbar arrow=Arrow, margin margin=EndMargin);
```

如果没有指定对齐方式（无论是在 `Label` 中还是作为显式参数），可选的 `Label` 将使用边距 `margin` 沿方向 `dir` 对齐。

函数 `string graphic(string name, string options="")` 返回一个可用于包含封装的 PostScript (EPS) 文件的字符串。这里，`name` 是要包含的文件名，`options` 是包含以逗号分隔的可选边界框 (`bb=llx lly urx ury`)、宽度 (`width=value`)、高度 (`height=value`)、旋转 (`angle=value`)、缩放 (`scale=factor`)、剪切 (`clip=bool`) 和草稿模式 (`draft=bool`) 参数列表的字符串。可以使用 `layer()` 函数来强制后续对象绘制在包含图像的上方：

```
label(graphic("file.eps","width=1cm"),(0,0),NE);
layer();
```

函数 `string baseline(string s, string template="\strut")` 可用于扩大标签的边界框以匹配给定的模板，从而使它们的基线在水平线上排版。有关示例，请参见 `Pythagoras.asy`。或者，可以使用画笔 `basealign` 来强制标签遵循 TeX 基线（见 `[basealign]`, 41 页）。

可以使用 `overwrite` 画笔属性（见 `[overwrite]`, 46 页）防止标签相互覆盖。

`plain_Label.asy` 中定义的结构 `object` 允许将 `Label` 和框(`frame`)以统一的方式对待。可以使用例程将一组对象打包(pack)进单个框中：

```
frame pack(pair align=2S, ... object inset[]);
```

要围绕 `Label` 绘制或填充框（或椭圆或其他路径）并返回边界对象，请使用以下例程之一：

```
object draw(picture pic=currentpicture, Label L, envelope e,
            real xmargin=0, real ymargin=xmargin, pen p=currentpen,
            filltype filltype=NoFill, bool above=true);
object draw(picture pic=currentpicture, Label L, envelope e, pair position,
            real xmargin=0, real ymargin=xmargin, pen p=currentpen,
            filltype filltype=NoFill, bool above=true);
```

这里 `envelope` 是边界绘制例程，例如 `plain_boxes.asy` 中定义的 `box`、`roundbox` 或 `ellipse`。

函数 `path[] texpath(Label L)` 返回 TeX 绘制 `Label L` 时会填充的路径数组。

函数 `string minipage(string s, width=100pt)` 可用于将字符串 `s` 排版为宽度为 `width` 的段落。本示例使用 `minipage`、`clip` 和 `graphic` 生成光盘(CD)标签：



```
size(11.7cm,11.7cm);

asy(nativeformat(),"logo");

fill(unitcircle^(scale(2/11.7)*unitcircle),
      evenodd+rgb(124/255,205/255,124/255));
label(scale(1.1)*minipage(
      "\centering\scriptsize \textbf{\LARGE {\tt Asymptote}}\\
\smallskip
\small The Vector Graphics Language}\\
\smallskip
\textsc{Andy Hammerlindl, John Bowman, and Tom Prince}
https://asymptote.sourceforge.io\\
",8cm),(0,0.6));
label(graphic("logo","height=7cm"),(0,-0.22));
clip(unitcircle^(scale(2/11.7)*unitcircle),evenodd);
```

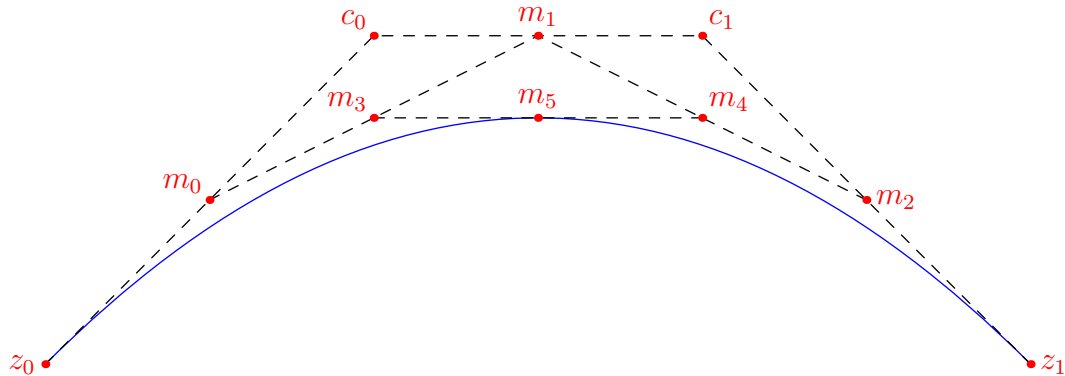
5 贝塞尔曲线

三次样条(cubic spline)的每个内部节点都可以带有方向前缀或后缀 {dir}: 数对 dir 的方向分别指定曲线在该节点处的入切线或出切线方向。外部节点只能在其内侧带有方向指定符(direction specifier)。

节点 z_0 带有后控制点(postcontrol point) c_0 , 节点 z_1 带有前控制点(precontrol point) c_1 , 它们之间的三次样条按下述贝塞尔曲线计算:

$$(1-t)^3 z_0 + 3t(1-t)^2 c_0 + 3t^2(1-t) c_1 + t^3 z_1 \quad 0 \leq t \leq 1.$$

如下面的示意图所示, 由两个端点 z_0 和 z_1 以及两个控制点 c_0 和 c_1 构造出的三阶中点(m_5), 是四元组(z_0, c_0, c_1, z_1) 所构成的贝塞尔曲线上对应 $t = 1/2$ 的点。这样就可以递归地构造所需的曲线: 把新提取出的三阶中点作为一个端点, 把相应的二阶与一阶中点作为控制点:



这里 m_0 、 m_1 和 m_2 是一阶中点, m_3 和 m_4 是二阶中点, m_5 是三阶中点。然后对(z_0, m_0, m_3, m_5) 和(m_5, m_4, m_2, z_1) 递归地应用该算法, 即可构造出曲线。

事实上, 对于位于每段上任意比例 t (t 取自 $[0, 1]$) 处的点, 类似的性质同样成立, 而不仅限于中点 ($t = 1/2$)。

这样构造出的贝塞尔曲线具有以下性质:

- 它完全包含在给定四点的凸包(convex hull)之内。
- 它在起点处的行进方向是从第一个端点指向第一个控制点, 在终点处的行进方向是从第二个控制点指向第二个端点。

用户可以像这样在两个节点之间显式地指定控制点:

```
draw((0,0)..controls (0,100) and (100,100)..(100,0));
```

不过, 通常更方便的做法是直接使用 .. 运算符, 它让 Asymptote 按照 Donald Knuth 的专著《The MetaFontbook》第 14 章中描述的算法自行选择控制点。用户仍然可以通过指定方向(direction)、张力(tension)和卷曲(curl)值来自定义引导(guide) (或路径)。

张力越大, 曲线越直, 也就越接近一条直线。可以把样条(spline)的张力从默认值 1 改为任何大于或等于 0.75 的实数值[参见 John D. Hobby, Discrete and Computational Geometry 1, 123-140 (1986)]:

```
draw((100,0)..tension 2 ..(100,100)..(0,100));
```

```
draw((100,0)..tension 3 and 2 ..(100,100)..(0,100));
draw((100,0)..tension atleast 2 ..(100,100)..(0,100));
```

在这些例子中，2 与 .. 之间有一个空格。这是必需的，因为 2. 会被解释为一个数值常量。

curl 参数指定路径端点处的曲率 (o 表示笔直；默认值 1 表示近似圆形)：

```
draw((100,0){curl 0}..(100,100)..{curl 0}(0,100));
```

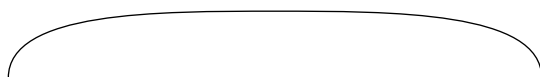
MetaPost ... 路径连接符(path connector)在可能的情况下请求一条无拐点、且被限制在由端点和方向所定义的三角形内的曲线，Asymptote 把它实现为 ..tension atleast 1 .. 的便捷缩写 :: (省略号 ... 在 Asymptote 中用来表示数目可变的参数；见 6.12.3 节 [Rest arguments], 67 页)。例如，请比较

```
draw((0,0){up}..(100,25){right}..(200,0){down});
```



与

```
draw((0,0){up}::(100,25){right}::(200,0){down});
```



--- 连接符是 ..tension atleast infinity.. 的缩写，而 & 连接符会先把第一条路径的最后一个节点去掉（它通常应与第二条路径的第一个节点重合），然后再把两条路径拼接起来。

6 编程

下面是一个 Asymptote 编程语言的简短入门示例，它着重体现了其控制结构与 C、C++ 以及 Java 的控制结构之间的相似之处：

```
// This is a comment.

// Declaration: Declare x to be a real variable;
real x;

// Assignment: Assign the real variable x the value 1.
x=1.0;

// Conditional: Test if x equals 1 or not.
if(x == 1.0) {
    write("x equals 1.0");
} else {
    write("x is not equal to 1.0");
}

// Loop: iterate 10 times
for(int i=0; i < 10; ++i) {
    write(i);
}
```

Asymptote 与 C/C++ 中一样支持 while、do、break 和 continue 语句。它还支持基于范围的 for 循环，这种循环便于遍历数组的所有元素：

```
// Iterate over an array
int[] array={1,1,2,3,5};
for(int k : array) {
    write(k);
}
```

此外，它还支持许多这些语言所不具备的特性。

6.1 数据类型

Asymptote 支持下列数据类型（此外还有用户自定义类型）：

- | | |
|-------|---|
| void | void 类型仅用于不接受参数或不返回任何值的函数。 |
| bool | 一种布尔类型，只能取值 true 或 false。例如：
<pre>bool b=true;</pre> 定义了一个布尔变量 b 并将其初始化为值 true。如果没有给出初始化器：
<pre>bool b;</pre> 则假定其值为 false。 |
| bool3 | 一种扩展的布尔类型，可以取值 true、default 或 false。bool3 类型可以与 bool 类型相互强制转换。bool3 的默认初始化器是 default。 |

<code>int</code>	一种整数类型；如果没有给出初始化器，则隐式取值 0。整数允许取的最小值是 <code>intMin</code> ，最大值是 <code>intMax</code> 。
<code>real</code>	一种实数；它应当被设置为当前体系结构上精度最高的原生浮点类型。实数的隐式初始化器是 0.0。实数具有精度 <code>realEpsilon</code> ，以及 <code>realDigits</code> 位有效数字。最小的正实数是 <code>realMin</code> ，最大的正实数是 <code>realMax</code> 。变量 <code>inf</code> 和 <code>nan</code> 连同函数 <code>bool isnan(real x)</code> 在浮点异常被 <code>-mask</code> 命令行选项屏蔽（这是交互模式下的默认设置）时非常有用。
<code>pair</code>	复数，即由实数分量组成的有序数对 (x,y)。数对 z 的实部和虚部分别可以通过 <code>z.x</code> 和 <code>z.y</code> 读取。我们称 x 和 y 为 <code>pair</code> 这一数据元素的虚成员；不过它们不能被直接修改。数对的隐式初始化器是 $(0.0,0.0)$。 求数对的复共轭有多种方法： <pre>pair z=(3,4); z=(z.x,-z.y); z=z.x-I*z.y; z=conj(z);</pre> 这里 I 是数对 $(0,1)$。为数对定义了许多内置函数： <pre>pair conj(pair z) 返回 z 的共轭； real length(pair z) 返回其参数 z 的复模 z。例如， pair z=(3,4); length(z); 返回结果 5。length(pair) 的同义词是 abs(pair)。函数 abs2(pair z) 返回 $z ^2$； real angle(pair z, bool warn=true) 以弧度为单位返回 z 在区间 $[-\pi,\pi]$ 内的角度；若 <code>warn</code> 为 <code>false</code> 且 $z=(0,0)$，则返回 0（而不是产生错误）； real degrees(pair z, bool warn=true) 以度为单位返回 z 在区间 $[0,360)$ 内的角度，若 <code>warn</code> 为 <code>false</code> 且 $z=(0,0)$，则返回 0（而不是产生错误）； pair unit(pair z) 返回数对 z 方向上的单位向量； pair expi(real angle) 返回以弧度度量的 <code>angle</code> 方向上的单位向量； pair dir(real degrees) 返回以度度量的 <code>degrees</code> 方向上的单位向量； real xpart(pair z) 返回 $z.x$； real ypart(pair z) 返回 $z.y$； </pre>

```

pair realmult(pair z, pair w)
    返回逐分量的乘积 (z.x*w.x, z.y*w.y);

real dot(explicit pair z, explicit pair w)
    返回点积 z.x*w.x+z.y*w.y;

real cross(explicit pair z, explicit pair w)
    返回二维标量积 z.x*w.y-z.y*w.x;

real orient(pair a, pair b, pair c);
    若 a--b--c--cycle 的定向为逆时针 (顺时针), 则返回正值 (负值); 若三点共线则返回零。等价地说, 若 c 位于经过 a 和 b 的直线的左侧 (右侧), 则返回正值 (负值); 若 c 位于该直线上则返回零。返回值可以用二维标量叉积表示为 cross(a-c, b-c), 它就是行列式
    |a.x a.y 1|
    |b.x b.y 1|
    |c.x c.y 1|

real incircle(pair a, pair b, pair c, pair d);
    若 d 位于按逆时针定向的点 a, b, c 所在圆的内部 (外部), 则返回正值 (负值); 若 d 位于该圆上则返回零。返回值是行列式
    |a.x a.y a.x^2+a.y^2 1|
    |b.x b.y b.x^2+b.y^2 1|
    |c.x c.y c.x^2+c.y^2 1|
    |d.x d.y d.x^2+d.y^2 1|

pair minbound(pair z, pair w)
    返回 (min(z.x, w.x), min(z.y, w.y));

pair maxbound(pair z, pair w)
    返回 (max(z.x, w.x), max(z.y, w.y))。

triple 由实数分量组成的有序三元组 (x, y, z), 用于三维绘图。三元组 v 的各个分量可以通过 v.x、v.y 和 v.z 读取。三元组的隐式初始化器是 (0.0, 0.0, 0.0)。
    以下是针对三元组的内置函数:

real length(triple v)
    返回其参数 v 的长度 |v|。length(triple) 的同义词是 abs(triple)。函数 abs2(triple v) 返回 |v|^2;

real polar(triple v, bool warn=true)
    以弧度为单位返回 v 从 z 轴量起的余纬度(colatitude), 若 warn 为 false 且 v=0, 则返回 0 (而不是产生错误);

real azimuth(triple v, bool warn=true)
    以弧度为单位返回 v 从 x 轴量起的经度, 若 warn 为 false 且 v.x=v.y=0, 则返回 0 (而不是产生错误);

real colatitude(triple v, bool warn=true)
    以度为单位返回 v 从 z 轴量起的余纬度, 若 warn 为 false 且 v=0, 则返回 0 (而不是产生错误);

```

```

real latitude(triple v, bool warn=true)
    以度为单位返回 v 从  $xy$  平面量起的纬度, 若 warn 为 false 且  $v=0$ ,
    则返回 0 (而不是产生错误);

real longitude(triple v, bool warn=true)
    以度为单位返回 v 从  $x$  轴量起的经度, 若 warn 为 false 且  $v.x=v.y=0$ ,
    则返回 0 (而不是产生错误);

triple unit(triple v)
    返回三元组 v 方向上的单位三元组;

triple expi(real polar, real azimuth)
    返回以弧度度量的 (polar, azimuth) 方向上的单位三元组;

triple dir(real colatitude, real longitude)
    返回以度度量的 (colatitude, longitude) 方向上的单位三元组;

real xpart(triple v)
    返回  $v.x$ ;

real ypart(triple v)
    返回  $v.y$ ;

real zpart(triple v)
    返回  $v.z$ ;

real dot(triple u, triple v)
    返回点积  $u.x*v.x+u.y*v.y+u.z*v.z$ ;

triple cross(triple u, triple v)
    返回叉积
     $(u.y*v.z-u.z*v.y, u.z*v.x-u.x*v.z, u.x*v.y-v.x*u.y)$ ;

triple minbound(triple u, triple v)
    返回  $(\min(u.x, v.x), \min(u.y, v.y), \min(u.z, v.z))$ ;

triple maxbound(triple u, triple v)
    返回  $(\max(u.x, v.x), \max(u.y, v.y), \max(u.z, v.z))$ .

```

string 一种字符串, 用 STL 的 string 类实现。

由双引号(")界定的字符串要经过下列映射, 以便在 $\text{T}_{\text{E}}\text{X}$ 中也能使用双引号 (用于 babel 宏包, 参见见 9.4 节 [babel], 173 页):

- \" 映射为 "
- \\ 映射为 \

由单引号(')界定的字符串具有与 ANSI C 中字符串相同的映射:

- \' 映射为 '
- \" 映射为 "
- \? 映射为 ?
- \\ 映射为反斜杠
- \a 映射为响铃字符
- \b 映射为退格字符

- \f 映射为换页字符
- \n 映射为换行字符
- \r 映射为回车字符
- \t 映射为制表符
- \v 映射为垂直制表符
- \0-\377 映射为相应的八进制字节
- \x0-\xFF 映射为相应的十六进制字节

字符串的隐式初始化器是空字符串 ""。字符串可以用 + 运算符连接。

对于整数索引 `pos` 位于区间 `[-n, n-1]` 内的情形（其中 `n` 为字符串 `s` 的长度），可以通过 `s[pos]` 访问字符串 `s` 的单字符子串。负索引从字符串末尾开始计数，因此 `s[-1]` 是 `s` 的最后一个字符。字符串的开头是 `s[0]`。如果 `pos` 超出范围，则返回空字符串 ""。

在下面的字符串函数中，位置 0 表示字符串的开头：

```
int length(string s)
    返回字符串 s 的长度；

int find(string s, string t, int pos=0)
    返回字符串 t 在字符串 s 中从位置 pos 起或其之后第一次出现的位置；若 t 不是 s 的子字符串，则返回 -1；

int rfind(string s, string t, int pos=-1)
    返回字符串 t 在字符串 s 中从位置 pos 起或其之前最后一次出现的位置（若 pos=-1，则在字符串 s 的末尾）；若 t 不是 s 的子字符串，则返回 -1；

string insert(string s, int pos, string t)
    返回将字符串 t 插入到 s 中位置 pos 处所形成的字符串；

string erase(string s, int pos, int n)
    返回将 s 中位置 pos 处长度为 n 的字符串（若 n=-1，则到字符串 s 的末尾）删除后所形成的字符串；

string substr(string s, int pos, int n=-1)
    返回 s 中从位置 pos 开始、长度为 n 的子字符串（若 n=-1，则一直到字符串 s 的末尾）；

string reverse(string s)
    返回将字符串 s 反转后形成的字符串；

string replace(string s, string before, string after)
    返回一个字符串，其中字符串 s 里的所有字符串 before 都被替换成了字符串 after；

string replace(string s, string[] table)
    返回一个字符串，它是把字符串 s 中所有出现的字符串 before（取自由字符串对 {before,after} 组成的数组 table）都替换为相应的字符串 after 后得到的；
```

```
string[] split(string s, string delimiter="")
    返回由 s 按 delimiter 分隔出的子字符串组成的字符串数组（空分
    隔符表示空格，但会丢弃重复的分隔符）；

string[] array(string s)
    返回把 s 分割成单个字符得到的字符串数组。其逆运算由 operator
    +(...string[] a) 提供。

string format(string s, int n, string locale="")
    返回一个字符串，其中 n 已按 C 风格的格式字符串 s、使用区域设置
    locale（若指定的是空字符串，则使用当前区域设置）格式化，其行
    为遵循 C 函数 fprintf，但只允许有一个数据字段。

string format(string s=defaultformat, bool forcemath=false, string
s=defaultseparator, real x, string locale="")
    返回一个字符串，其中 x 已按 C 风格的格式字符串 s、使用区域
    设置 locale（若指定的是空字符串，则使用当前区域设置）格式
    化，其行为遵循 C 函数 fprintf，但只允许有一个数据字段，默认
    会去掉末尾的零（除非指定了 #），并且如果 s 指定了数学模式或
    forcemath=true，则会用 TeX 排版科学计数法，使用的分隔符为
    defaultseparator="\!\times\!";

int hex(string s);
    将十六进制字符串 s 强制转换为整数；

int ascii(string s);
    返回字符串 s 第一个字符的 ASCII 码；

string string(real x, int digits=realDigits)
    使用精度 digits 和 C 区域设置将 x 强制转换为字符串；

string locale(string s="")
    若给定的字符串非空，则将区域设置设为该字符串，并返回当前的区
    域设置；

string time(string format="%a %b %d %T %Z %Y")
    返回当前时间，该时间由 ANSI C 例程 strftime 依照字符串 format、
    使用当前区域设置格式化。因此，
    time();
    time("%a %b %d %H:%M:%S %Z %Y");
    都是以 UNIX date 命令使用的默认格式返回当前时间的等价写法；

int seconds(string t="", string format="")
    返回自纪元(Epoch)（1970 年 1 月 1 日星期四 00:00:00 UTC）以来以秒
    计量的时间，该时间由 ANSI C 例程 strtptime 依照字符串 format、
    使用当前区域设置确定；若 t 为空字符串，则返回当前时间。注意，
    当前的 GNU C 库会忽略 POSIX strtptime 规范的 "%Z" 扩展。若发生
    错误，则返回值 -1。下面是一些例子：
    seconds("Mar 02 11:12:36 AM PST 2007", "%b %d %r PST %Y");
    seconds(time("%b %d %r %Z %Y"), "%b %d %r %Z %Y");
    seconds(time("%b %d %r %Z %Y"), "%b %d %r "+time("%Z")+ " %Y");
```

```
1+(seconds()-seconds("Jan 1", "%b %d"))/(24*60*60);
```

最后一个例子返回今天的年积日(ordinal date), 即从年初起算的天数。

```
string time(int seconds, string format="%a %b %d %T %Z %Y")
```

返回纪元(Epoch) (1970 年 1 月 1 日星期四 00:00:00 UTC) 之后 seconds 秒所对应的时间, 该时间由 ANSI C 例程 `strftime` 依照字符串 `format`、使用当前区域设置格式化。例如, 要返回 24 小时前对应的日期:

```
time(seconds()-24*60*60);
```

```
int system(string s)
```

```
int system(string[] s)
```

若设置 `safe` 为 `false`, 则调用任意的系统命令 `s`;

```
void asy(string format, bool overwrite=false, ... string[] s)
```

在新环境中按格式 `format` 有条件地处理数组 `s` 中的每个文件名, 仅当 `overwrite` 为 `true` 时才覆盖输出文件;

```
void abort(string s="")
```

中止执行 (在批处理模式下返回码非零); 如果字符串 `s` 非空, 则打印一条由源文件、行号和 `s` 构造的诊断消息;

```
void assert(bool b, string s="")
```

若 `b=false`, 则以一条由 `s` 构造的错误消息中止执行;

```
void exit()
```

退出 (在批处理模式下以零错误返回码退出);

```
void sleep(int seconds)
```

暂停给定的秒数;

```
void usleep(int microseconds)
```

暂停给定的微秒数;

```
void beep()
```

在控制台上发出一声蜂鸣;

与 C/C++ 中一样, 复杂的类型可以用 `typedef` 或 `using` 缩写。例如, 下面这一行

```
using multipath = path[];
```

将使 `multipath` 成为等价于 `path[]` 的类型, 同样, 与之等价的这一行

```
typedef path[] multipath;
```

也能做到这一点。这种类型别名多半只是一种便利。然而, 在声明返回类型为函数的函数时, 它是必需的 (参见 6.12 节 [Functions], 64 页, 中的例子)。

6.2 路径与引导

`path` 解算为固定路径的三次样条(spline)。路径的隐式初始化对象为 `nullpath`。

例如, 例程 `circle(pair c, real r)` 返回近似表示圆心为 `c`、半径为 `r` 的圆的贝塞尔曲线, 它基于 `unitcircle` (见 [unitcircle], 12 页):

```
path circle(pair c, real r)
```

```
{
```

```
    return shift(c)*scale(r)*unitcircle;
}
```

如果需要高精度, 可以使用模块 `graph` 中定义的例程 `Circle` 来生成真正的圆:

```
import graph;
path Circle(pair c, real r, int n=nCircle);
```

一个与 `circle` 一致的圆弧, 圆心为 `c`、半径为 `r`, 角度从 `angle1` 到 `angle2` 度 (若 `angle2 >= angle1` 则按逆时针方向绘制), 可以通过以下方式构造:

```
path arc(pair c, real r, real angle1, real angle2);
```

也可以显式指定方向:

```
path arc(pair c, real r, real angle1, real angle2, bool direction);
```

此处方向可指定为 CCW (逆时针) 或 CW (顺时针)。为方便起见, 圆心位于 `c`、从数对 `z1` 到 `z2` (假设 $|z2-c|=|z1-c|$) 的圆弧也可以通过以下方式构造:

```
path arc(pair c, explicit pair z1, explicit pair z2,
        bool direction=CCW)
```

如果需要高精度, 可以使用模块 `graph` 中通过 `n` 个控制点生成贝塞尔曲线的例程来生成真正的圆弧:

```
import graph;
path Arc(pair c, real r, real angle1, real angle2, bool direction,
        int n=nCircle);
path Arc(pair c, real r, real angle1, real angle2, int n=nCircle);
path Arc(pair c, explicit pair z1, explicit pair z2,
        bool direction=CCW, int n=nCircle);
```

椭圆可以通过以下例程绘制:

```
path ellipse(pair c, real a, real b)
{
    return shift(c)*scale(a,b)*unitcircle;
}
```

数对 `a` 与 `b` 之间的大括号(`brace`)可以通过以下方式构造:

```
path brace(pair a, pair b, real amplitude=bracedefaultratio*length(b-a));
```

本例演示了第 3 章 [Tutorial], 9 页, 和第 5 章 [Bezier curves], 22 页, 中讨论的所有五种引导(`guide`)连接符的使用:

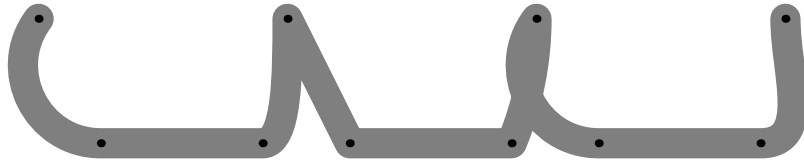
```
size(300,0);
pair[] z=new pair[10];

z[0]=(0,100); z[1]=(50,0); z[2]=(180,0);

for(int n=3; n <= 9; ++n)
    z[n]=z[n-3]+(200,0);

path p=z[0]..z[1]-z[2>::{up}z[3]
&z[3]..z[4]-z[5>::{up}z[6]
&z[6>::z[7]-z[8]..{up}z[9];
```

```
draw(p, grey+linewidth(4mm));
dot(z);
```



以下是一些用于路径的实用函数：

```
int length(path p);
    这是路径 p 中（线性或三次）段的数量。如果 p 是循环的，这与 p
    中的节点数相同。

int size(path p);
    这是路径 p 中的节点数。如果 p 是循环的，这与 length(p) 相同。

bool cyclic(path p);
    当且仅当路径 p 是循环的时返回 true。

bool straight(path p, int i);
    当且仅当路径 p 在节点 i 与节点 i+1 之间的段为直线时返回 true。

bool piecewisestraight(path p)
    当且仅当路径 p 为分段直线时返回 true。

pair point(path p, int t);
    如果 p 是循环的，返回节点 t 模 length(p) 的坐标。否则返回节点
    t 的坐标，除非 t < 0（此时返回 point(0)）或 t > length(p)（此时
    返回 point(length(p))）。

pair point(path p, real t);
    这会返回节点 floor(t) 与 floor(t)+1 之间对应于三次样条(spline)参
    数 t-floor(t) 的点的坐标（见 第 5 章 [Bezier curves], 22 页）。如
    果 t 位于范围[0,length(p)] 之外，在 p 为循环的情况下首先将其模
    length(p) 化简，否则转换为 p 的对应端点。

pair dir(path p, int t, int sign=0, bool normalize=true);
    如果 sign < 0，返回路径 p 在节点 t 处的入切线方向（作为数对）；
    如果 sign > 0，返回出切线方向。如果 sign=0，返回这两个方向的
    平均值。

pair dir(path p, real t, bool normalize=true);
    返回路径 p 在节点 floor(t) 与 floor(t)+1 之间的点处的切线方
    向，该点对应于三次样条(spline)参数 t-floor(t)（见 第 5 章 [Bezier
    curves], 22 页）。

pair dir(path p)
    返回 dir(p,length(p))。

pair dir(path p, path q)
    返回 unit(dir(p)+dir(q))。
```

```

pair accel(path p, int t, int sign=0);
    如果 sign < 0, 返回入路径 p 在节点 t 处的加速度; 如果 sign > 0,
    返回出路径的加速度。如果 sign=0, 返回这两者加速度的平均值。

pair accel(path p, real t);
    返回路径 p 在点 t 处的加速度。

real radius(path p, real t);
    返回路径 p 在点 t 处的曲率半径。

pair precontrol(path p, int t);
    返回 p 在节点 t 处的前控制点。

pair precontrol(path p, real t);
    返回 p 在参数 t 处的有效前控制点。

pair postcontrol(path p, int t);
    返回 p 在节点 t 处的后控制点。

pair postcontrol(path p, real t);
    返回 p 在参数 t 处的有效后控制点。

real arclength(path p);
    返回路径 p 所代表的分段线性或三次曲线的长度 (以用户坐标表示)。

real arctime(path p, real L);
    返回路径 \时间" (一个介于 o 和 point(path p, real t) 意义下的路径长度之间的实数), 在此时间处累计弧长 (从路径起点开始测量) 等于 L。

pair arcpoint(path p, real L);
    返回 point(p, arctime(p, L))。

real dirtime(path p, pair z);
    返回第一个路径 \时间" (一个介于 o 和 point(path, real) 意义下的路径长度之间的实数), 在此时间处路径切线具有数对 z 的方向; 如果从未发生则返回 -1。

real reltime(path p, real l);
    返回路径 p 上处于其弧长相对比例 l 处的时间。

pair relpoint(path p, real l);
    返回路径 p 上处于其弧长相对比例 l 处的点。

pair midpoint(path p);
    返回路径 p 上处于其弧长一半处的点。

path reverse(path p);
    返回沿 p 反向运行的路径。

path subpath(path p, int a, int b);
    返回 p 从节点 a 到节点 b 的子路径。如果 a > b, 则子路径的方向反转。

```

```

path subpath(path p, real a, real b);
    返回 p 从路径时间 a 到路径时间 b (在 point(path, real) 意义下)
    的子路径。如果 a > b, 则子路径的方向反转。

real[] intersect(path p, path q, real fuzz=-1);
    如果 p 与 q 至少有一个交点, 返回一个长度为 2 的实数数组, 包含表
    示沿 p 与 q 相应路径时间 (在 point(path, real) 意义下) 的时间
    值, 对应其中一个交点 (按《The MetaFontbook》第 137 页描述的算
    法选择)。计算按照 fuzz 指定的绝对误差进行, 若 fuzz < 0 则达到
    机器精度。如果路径不相交, 返回一个长度为 0 的实数数组。

real[] [] intersections(path p, path q, real fuzz=-1);
    将路径 p 与 q 的所有 (除非有无限多个) 交点时间作为一个由长度
    为 2 的实数数组组成的已排序数组返回 (见 [sort], 73 页)。计算按
    照 fuzz 指定的绝对误差进行, 若 fuzz < 0 则达到机器精度。

real[] intersections(path p, explicit pair a, explicit pair b, real
fuzz=-1);
    将路径 p 与穿过点 a 和 b 的 (无限长) 直线的求交时间 (除非有无限
    多个) 作为一个已排序数组返回。保证返回的交点在 fuzz 指定的
    绝对误差范围内正确, 若 fuzz < 0 则达到机器精度。

real[] times(path p, real x)
    返回路径 p 与穿过 (x,0) 的垂直线的所有交点时间。

real[] times(path p, explicit pair z)
    返回路径 p 与穿过 (0,z.y) 的水平线的所有交点时间。

real[] mintimes(path p)
    返回一个长度为 2 的数组, 分别包含路径 p 达到其最小水平和垂直范
    围的时间。

real[] maxtimes(path p)
    返回一个长度为 2 的数组, 分别包含路径 p 达到其最大水平和垂直范
    围的时间。

pair intersectionpoint(path p, path q, real fuzz=-1);
    返回交点 point(p, intersect(p,q,fuzz)[0])。

pair[] intersectionpoints(path p, path q, real fuzz=-1);
    返回一个包含路径 p 与 q 的所有交点的数组。

pair extension(pair P, pair Q, pair p, pair q);
    返回线段 P--Q 与 p--q 的延长线的交点; 若两直线平行, 则返回
    (infinity,infinity)。

slice cut(path p, path knife, int n);
    以结构体 slice 形式返回路径 p 在与路径 knife 的第 n 个交点之前
    和之后的部分 (如果未找到交点, 整条路径被视为在交点之前):
    struct slice {
        path before,after;
    }
    参数 n 按交点总数取模处理。

```

`slice firstcut(path p, path knife);`
 等价于 `cut(p,knife,0);`。注意 `firstcut.after` 起到了 `MetaPost cutbefore` 命令的作用。

`slice lastcut(path p, path knife);`
 等价于 `cut(p,knife,-1);`。注意 `lastcut.before` 起到了 `MetaPost cutafter` 命令的作用。

`path buildcycle(...path[] p);`
 遵循 `MetaPost buildcycle` 命令的行为，返回环绕由两个或多个连续相交路径所围成区域的路径。

`pair min(path p);`
 返回路径 `p` 的路径边界框的数对(left,bottom)。

`pair max(path p);`
 返回路径 `p` 的路径边界框的数对(right,top)。

`int windingnumber(path p, pair z);`
 返回循环路径 `p` 相对于点 `z` 的环绕数(winding number)。若路径按逆时针方向包围 `z`，则环绕数为正。如果 `z` 位于 `p` 上，则返回常量 `undefined` (定义为最大奇整数)。

`bool interior(int windingnumber, pen fillrule)`
 如果根据 `fillrule`，`windingnumber` 对应于内部点，则返回 `true`。

`bool inside(path p, pair z, pen fillrule=currentpen);`
 当且仅当根据填充规则 `fillrule` (见 [fillrule], 40 页)，点 `z` 位于循环路径 `p` 所围区域的内部或边缘上时返回 `true`。

`int inside(path p, path q, pen fillrule=currentpen);`
 如果根据填充规则 `fillrule` (见 [fillrule], 40 页)，循环路径 `p` 严格包含 `q`，则返回 1；若循环路径 `q` 严格包含 `p`，则返回 -1；否则返回 0。

`pair inside(path p, pen fillrule=currentpen);`
 根据填充规则 `fillrule` (见 [fillrule], 40 页)，返回非退化循环路径 `p` 严格内部的任意一点。

`path[] strokepath(path g, pen p=currentpen);`
 返回用画笔 `p` 绘制路径 `g` 时 `PostScript` 会填充的路径数组。

guide 未解算的三次样条(spline) (三次样条节点与控制点列表)。引导(guide)的隐式初始化对象为 `nullpath`；这对于在循环中构建 `guide` 很有用。

引导(guide)与路径类似，不同之处在于三次样条(spline)的计算延迟到绘制时 (此时它被解算为路径)；这使得两个具有自由端点条件的 `guide` 能够光滑地连接在一起。在以下示例中，实线曲线是作为 `guide` 逐步构建的，但仅在绘制时才进行解算；虚线曲线则在每次迭代时逐步解算，此时全部节点集 (以红色显示) 尚未完全确定：

`size(200);`

`real mexican(real x) {return (1-8x^2)*exp(-(4x^2));}`

```

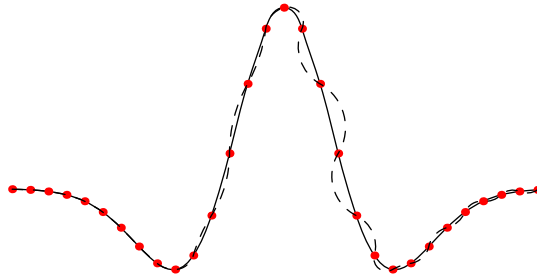
int n=30;
real a=1.5;
real width=2a/n;

guide hat;
path solved;

for(int i=0; i < n; ++i) {
    real t=-a+i*width;
    pair z=(t,mexican(t));
    hat=hat..z;
    solved=solved..z;
}

draw(hat);
dot(hat,red);
draw(solved,dashed);

```



我们指出在引导(guide)与路径的使用中存在效率差异:

```

guide g;
for(int i=0; i < 10; ++i)
    g=g-(i,i);
path p=g;
在线性时间内运行, 而

path p;
for(int i=0; i < 10; ++i)
    p=p-(i,i);

```

在二次时间内运行, 因为在迭代的每一步中, 到该点为止的整个路径都会被复制。

以下例程可用于检查引导(guide)的各个元素, 而无需实际将 guide 解算为固定路径 (内部循环除外, 它们会被解算):

```

int size(guide g);
    类似于 size(path p)。

int length(guide g);
    类似于 length(path p)。

```

```

bool cyclic(guide g);
    类似于 cyclic(path p)。

pair point(guide g, int t);
    类似于 point(path p, int t)。

guide reverse(guide g);
    类似于 reverse(path p)。如果 g 是循环的, 并且还包含次级循环,
    则首先将其解算为路径, 然后反转。如果 g 不是循环的但包含内部循
    环, 在反转前仅解算内部循环。如果没有内部循环, 则将 guide 反转
    但不解算为路径。

pair[] dirSpecifier(guide g, int i);
    返回一个长度为 2 的数对数组, 包含引导(guide) g 在节点 i 与节点
    i+1 之间的段的出方向指定符 (位于元素 0) 和入方向指定符 (位于
    元素 1) (若未指定则为 (0,0))。

pair[] controlSpecifier(guide g, int i);
    如果引导(guide) g 在节点 i 与节点 i+1 之间的段具有显式的出控制
    点和入控制点, 它们将分别作为双元素数组的元素 0 和 1 返回。否则
    返回一个空数组。

tensionSpecifier tensionSpecifier(guide g, int i);
    返回引导(guide) g 在节点 i 与节点 i+1 之间的段的张力指定
    符。tensionSpecifier 类型的各个组件可以通过虚成员 in、out
    和 atLeast 访问。

real[] curlSpecifier(guide g);
    返回一个数组, 包含引导(guide) g 的初始卷曲(curl)指定符 (位于元
    素 0) 和终止卷曲(curl)指定符 (位于元素 1)。

```

作为技术细节, 我们指出赋予 nullpath 的方向指定符会修改另一侧的节点: 引导(guide)

```

a..{up}nullpath..b;
c..nullpath{up}..d;
e..{up}nullpath{down}..f;

```

分别等价于

```

a..nullpath..{up}b;
c{up}..nullpath..d;
e{down}..nullpath..{up}f;

```

6.3 画笔

在 Asymptote 中, 画笔(pen)为四条基本绘图命令(见 第 4 章 [Drawing commands], 14 页)提供上下文。它们用于指定以下绘图属性: 颜色、线型、线宽、线帽(line cap)、线接样式(line join)、填充规则、文本对齐、字体、字体大小、图案、覆盖模式以及对笔尖的书法变换。绘图过程默认使用的画笔称为 currentpen。这与 MetaPost 命令 pickup 具有相同的功能。画笔的隐式初始化器为 defaultpen。

画笔可以通过非结合二元运算符 + 加在一起。这将把两只画笔的颜色相加。最右侧画笔的所有其他非默认属性将覆盖最左侧画笔的属性。因此, 可以通过写

dashed+red+green、red+green+dashed 或 red+dashed+green 来获得黄色虚线画笔。二元运算符 * 可用于将画笔颜色缩放一个实数倍，直到它在一个或多个颜色分量等于 1 时达到饱和。

- 颜色可以使用以下色彩空间之一指定：

```
pen gray(real g);
```

这会生成灰度颜色，其中强度 g 位于区间 $[0,1]$ 内，0.0 表示黑色，1.0 表示白色。

```
pen rgb(real r, real g, real b);
```

这会生成 RGB 颜色，其中红、绿、蓝每个强度分量 r 、 g 、 b 均位于区间 $[0,1]$ 内。

```
pen RGB(int r, int g, int b);
```

这会生成 RGB 颜色，其中红、绿、蓝每个强度分量 r 、 g 、 b 均位于区间 $[0,255]$ 内。

```
pen cmyk(real c, real m, real y, real k);
```

这会生成 CMYK 颜色，其中青、品红、黄和黑每个分量强度 c 、 m 、 y 、 k 均位于区间 $[0,1]$ 内。

```
pen invisible;
```

这种特殊的画笔以不可见墨水书写，但会调整边界框(bounding box)，就像绘制了某些内容一样（类似于 $\text{\TeX}$ 中的 `\phantom` 命令）。函数 `bool invisible(pen)` 可用于测试画笔是否不可见。

默认颜色为 black；这可以通过例程 `defaultpen(pen)` 来更改。函数 `colorspace(pen p)` 以字符串形式返回画笔 p 的色彩空间（"gray"、"rgb"、"cmyk" 或 ""）。

函数 `real[] colors(pen)` 返回画笔的各个颜色分量。函数 `pen gray(pen)`、`pen rgb(pen)` 和 `pen cmyk(pen)` 返回将参数转换为对应色彩空间后获得的新画笔。函数 `colorless(pen=currentpen)` 返回剥离了颜色属性的参数副本（以避免颜色混合）。

可以使用例程将 6 个字符的 RGB 十六进制字符串转换为画笔

```
pen rgb(string s);
```

- 画笔可以通过 `string hex(pen p)`；转换为十六进制字符串。

灰度原色 black 和 white、RGB 原色 red、green 和 blue，以及 RGB 二次色 cyan、magenta 和 yellow 的各种深浅与混合，与 CMYK 原色 Cyan、Magenta、Yellow 和 Black，一起在模块 plain 中被定义为命名颜色：

 palered	 palecyan	 black
 lightred	 lightcyan	 white
 mediumred	 mediumcyan	 orange
 red	 cyan	 fuchsia
 heavyred	 heavycyan	 chartreuse
 brown	 deepcyan	 springgreen
 darkbrown	 darkcyan	 purple
 palegreen	 pink	 royalblue
 lightgreen	 lightmagenta	 Cyan
 mediumgreen	 mediummagenta	 Magenta
 green	 magenta	 Yellow
 heavygreen	 heavymagenta	 Black
 deepgreen	 deepmagenta	 cmyk(red)
 darkgreen	 darkmagenta	 cmyk(blue)
 paleblue	 paleyellow	 cmyk(green)
 lightblue	 lightyellow	
 mediumblue	 mediumpyellow	
 blue	 yellow	
 heavyblue	 lightolive	
 deepblue	 olive	
 darkblue	 darkolive	
	 palegray	
	 lightgray	
	 mediumgray	
	 gray	
	 heavygray	
	 deepgray	
	 darkgray	

标准的 140 种 RGB X11 颜色可以通过命令导入

```
import x11colors;
```

标准的 68 种 CMYK $\text{\TeX}$ 颜色可以通过命令导入

```
import texcolors;
```

请注意，这两个标准之间存在一些重叠，某些颜色（例如 Green）的定义实际上是不一致的。

Asymptote 还附带一个 `asycolors.sty` $\text{\TeX}$ 宏包，它为 $\text{\TeX}$ 定义了 Asymptote 预定义色彩的 CMYK 版本，以便直接在 $\text{\TeX}$ 字符串中使用它们。通常，此类颜色是通过画笔参数传递给 $\text{\TeX}$ 的；然而，为了仅更改字符串的一部分颜色（例如在幻灯片演示中，见 9.26 节 [slide, 179 页]），可能希望直接向 $\text{\TeX}$ 指定颜色。可以使用 Asymptote 命令将该文件传递给 $\text{\TeX}$

```
usepackage("asycolors");
```

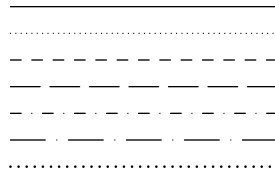
`plain_pens.asy` 中定义的结构体 `hsv` 可用于在 HSV 与 RGB 空间之间进行转换，其中色调 `h` 是 $[0, 360)$ 内的角度，而饱和度 `s` 与明度 `v` 位于 $[0, 1]$ 内：

```
pen p=hsv(180,0.5,0.75);
write(p);           // ([default], red=0.375, green=0.75, blue=0.75)
hsv q=p;
write(q.h,q.s,q.v); // 180      0.5      0.75
```

- 线型使用函数 `pen linetype(real[] a, real offset=0, bool scale=true, bool adjust=true)` 指定，其中 `a` 是实数构成的数组。可选参数 `offset` 指定从模式的何处开始。第一个数字指定落笔绘制的距离（如果 `scale` 为 `true`，以画笔线宽为单位；否则以

PostScript 单位计算), 第二个数字指定提笔不画的距离, 依此类推。如果 `adjust` 为 `true`, 这些间隔会被 `Asymptote` 自动调整以契合路径的弧长。以下是预定义的线型:

```
pen solid=linetype(new real[]);
pen dotted=linetype(new real[] {0,4});
pen dashed=linetype(new real[] {8,8});
pen longdashed=linetype(new real[] {24,8});
pen dashdotted=linetype(new real[] {8,8,0,8});
pen longdashdotted=linetype(new real[] {24,8,0,8});
pen Dotted(pen p=currentpen) {return linetype(new real[] {0,3})+2*linewidth(p);}
pen Dotted=Dotted();
```



默认线型为 `solid`; 这可以通过 `defaultpen(pen)` 来更改。画笔的线型可以通过函数 `real[] linetype(pen p=currentpen)`、`real offset(pen p)`、`bool scale(pen p)` 以及 `bool adjust(pen p)` 来确定。

- 画笔线宽以 PostScript 单位指定, 使用 `pen linewidth(real)`。默认线宽为 0.5 bp; 该值可以通过 `defaultpen(pen)` 更改。画笔的线宽由 `real linewidth(pen p=currentpen)` 返回。为了方便起见, 在模块 `plain_pens` 中定义了

```
void defaultpen(real w) {defaultpen(linewidth(w));}
pen operator +(pen p, real w) {return p+linewidth(w);}
pen operator +(real w, pen p) {return linewidth(w)+p;}
```

以便可以像这样设置线宽:

```
defaultpen(2);
pen p=red+0.5;
```

- 以整数参数调用 `linecap` 会返回具有特定 PostScript 线帽(line cap)的画笔:

```
pen squarecap=linecap(0);
pen roundcap=linecap(1);
pen extendcap=linecap(2);
```

默认的线帽 `roundcap` 可以通过 `defaultpen(pen)` 更改。画笔的线帽由 `int linecap(pen p=currentpen)` 返回。

- 以整数参数调用 `linejoin` 会返回具有特定 PostScript 线接样式(join style)的画笔:

```
pen miterjoin=linejoin(0);
pen roundjoin=linejoin(1);
pen beveljoin=linejoin(2);
```

默认的线接样式 `roundjoin` 可以通过 `defaultpen(pen)` 更改。画笔的线接样式由 `int linejoin(pen p=currentpen)` 返回。

- 调用 `miterlimit(real)` 会返回具有特定 PostScript 斜接限制(miter limit)的画笔。默认斜接限制 10.0 可以通过 `defaultpen(pen)` 更改。画笔的斜接限制由 `real miterlimit(pen p=currentpen)` 返回。

- 以整数参数调用 `fillrule` 会返回具有特定 PostScript 填充规则(fill rule)的画笔:

```
pen zerowinding=fillrule(0);
pen evenodd=fillrule(1);
```

填充规则标识用于确定路径或路径数组内部区域的算法, 它仅影响 `clip`、`fill` 以及 `inside` 函数。对于 `zerowinding` (零环绕数) 填充规则, 如果路径与水平射线 $z \rightarrow z + \text{infinity}$ 的向上交点数减去向下交点数为零, 则点 z 位于路径界定的区域之外。对于 `evenodd` (奇偶) 填充规则, 如果此类交点总数为偶数, 则认为 z 位于该区域之外。默认填充规则 `zerowinding` 可以通过 `defaultpen(pen)` 更改。画笔的填充规则由 `int fillrule(pen p=currentpen)` 返回。

- 以整数参数调用 `basealign` 会返回具有特定文本对齐设置的画笔:

```
pen nobasealign=basealign(0);
pen basealign=basealign(1);
```

默认设置 `nobasealign` (可通过 `defaultpen(pen)` 更改) 使标签对齐例程使用完整的标签边界框(bounding box)进行对齐。相比之下, `basealign` 要求遵从 $\text{T}_{\text{E}}\text{X}$ 基线(baseline)。画笔的基线对齐设置由 `int basealign(pen p=currentpen)` 返回。

例如, 在下图中, 绿色 π 与 γ 的基线是对齐的, 而红色 $-\pi$ 与 $-\gamma$ 的底边是对齐的。

```
import fontsize;
import three;

settings.autobillboard=false;
settings.embed=false;
currentprojection=orthographic(Z);

defaultpen(fontsize(100pt));

dot(0);

label("acg",0,align=N,basealign);
label("ace",0,align=N,red);
label("acg",0,align=S,basealign);
label("ace",0,align=S,red);
label("acg",0,align=E,basealign);
label("ace",0,align=E,red);
label("acg",0,align=W,basealign);
label("ace",0,align=W,red);

picture pic;
dot(pic,(labelmargin(),0,0),blue);
dot(pic,(-labelmargin(),0,0),blue);
dot(pic,(0,labelmargin(),0),blue);
dot(pic,(0,-labelmargin(),0),blue);
add(pic,0);

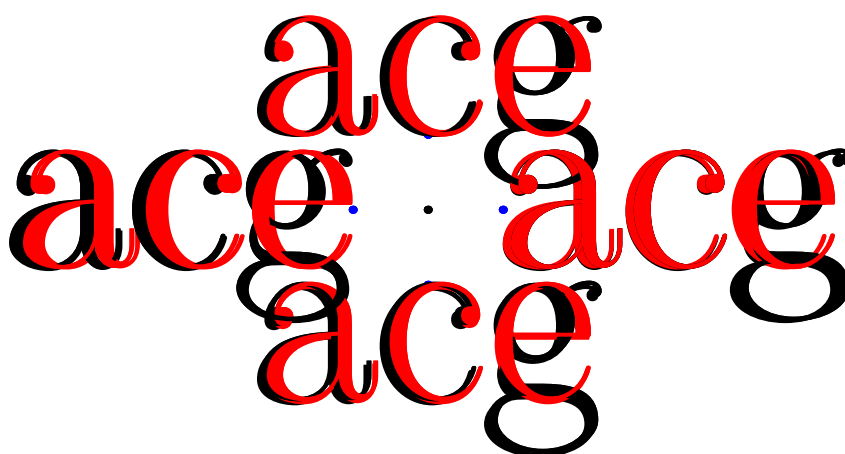
dot((0,0));
```

```

label("acg", (0,0), align=N, basealign);
label("ace", (0,0), align=N, red);
label("acg", (0,0), align=S, basealign);
label("ace", (0,0), align=S, red);
label("acg", (0,0), align=E, basealign);
label("ace", (0,0), align=E, red);
label("acg", (0,0), align=W, basealign);
label("ace", (0,0), align=W, red);

picture pic;
dot(pic, (labelmargin(), 0), blue);
dot(pic, (-labelmargin(), 0), blue);
dot(pic, (0, labelmargin()), blue);
dot(pic, (0, -labelmargin()), blue);
add(pic, (0,0));

```



对齐基线的另一种方法由 `baseline` 函数提供(见 [baseline], 20 页)。

- 字体大小以 $\text{T}_{\text{E}}\text{X}$ 磅(1 pt = 1/72.27 英寸)为单位, 使用函数 `pen fontsize(real size, real lineskip=1.2*size)` 指定。默认字号 12pt 可以通过 `defaultpen(pen)` 更改。非标准字号可能需要在文件开头插入

```
import fontsize;
```

(这需要来自

<http://mirror.ctan.org/macros/latex/contrib/type1cm/>

的 `type1cm` 宏包, 近期的 $\text{E}_{\text{T}}\text{X}$ 发行版已包含该包)。画笔的字号和行距可分别使用例程 `real fontsize(pen p=currentpen)` 和 `real lineskip(pen p=currentpen)` 来检查。

- 调用函数 `pen font(string encoding, string family, string series, string shape)` 会返回使用特定 $\text{E}_{\text{T}}\text{X}$ NFSS 字体的画笔。默认设置 `font("OT1", "cmr", "m", "n")` 对应 12pt Computer Modern Roman; 这可以通过 `defaultpen(pen)` 更改。画笔的字体设置由 `string font(pen p=currentpen)` 返回。

或者, 也可以使用函数 `pen font(string name)` 选择固定大小的 $\text{\TeX}$ 字体 (在此字体上 `fontsize` 不起作用), 例如 `"cmr12"` (12pt Computer Modern Roman) 或 `"pcrr"` (Courier)。也可以提供可选的尺寸参数, 将字体缩放到请求的大小: `pen font(string name, real size)`。

非标准字体命令可以通过 `pen fontcommand(string)` 生成。

还提供了通往以下标准 PostScript 字体的便捷接口:

```
pen AvantGarde(string series="m", string shape="n");
pen Bookman(string series="m", string shape="n");
pen Courier(string series="m", string shape="n");
pen Helvetica(string series="m", string shape="n");
pen NewCenturySchoolBook(string series="m", string shape="n");
pen Palatino(string series="m", string shape="n");
pen TimesRoman(string series="m", string shape="n");
pen ZapfChancery(string series="m", string shape="n");
pen Symbol(string series="m", string shape="n");
pen ZapfDingbats(string series="m", string shape="n");
```

- 自 2018/04/01 发布版本起, $\text{\TeX}$ 采用 UTF-8 作为新的默认输入编码。不过, 你仍然可以设置不同的输入编码 (字体、字体编码甚至语言上下文亦然)。以下是西里尔字母的 cp1251 编码和俄语语言的示例 (字体编码 T2A):

```
tex preamble("\usepackage[math]{anttor}");
tex preamble("\usepackage[T2A]{fontenc}");
tex preamble("\usepackage[cp1251]{inputenc}");
tex preamble("\usepackage[russian]{babel}");
```

对中文、日文和韩文字体的支持由 CJK 宏包提供:

<https://ctan.org/pkg/cjk>

以下命令启用 CJK 宋体(song)字体族 (在标签内部, 你也可以通过在标签字符串前附加 `"\CJKfamily{kai}"` 临时切换到另一个字体族, 例如楷体):

```
tex preamble("\usepackage{CJK}
\AtBeginDocument{\begin{CJK*}{GBK}{song}}
\AtEndDocument{\clearpage\end{CJK*}}");
```

- 画笔的透明度可以使用命令更改:

```
pen opacity(real opacity=1, string blend="Compatible");
```

不透明度(opacity)可以在 0 (完全透明) 到默认值 1 (不透明) 之间变化, 且 `blend` 指定以下前景-背景混合操作之一:

```
"Compatible", "Normal", "Multiply", "Screen", "Overlay", "SoftLight",
"HardLight", "ColorDodge", "ColorBurn", "Darken", "Lighten", "Difference",
"Exclusion", "Hue", "Saturation", "Color", "Luminosity",
```

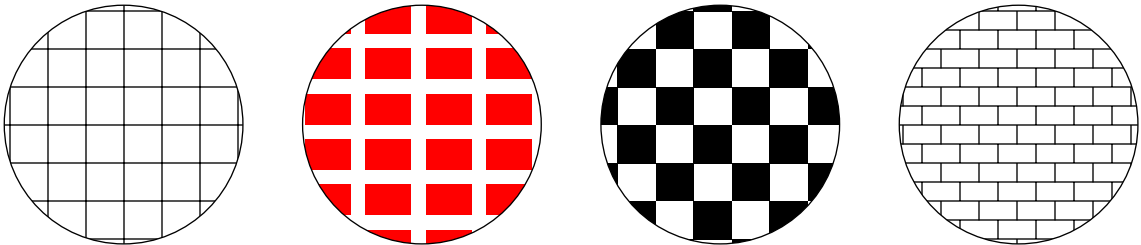
如 https://opensource.adobe.com/dc-acrobat-sdk-docs/standards/pdfstandards/pdf/PDF32000_2008.pdf 的表 136 和 137 中所述。由于 PostScript 不支持透明度, 此特性仅对 `-f pdf` 输出格式选项有效; 其他格式可以通过 ImageMagick 的 `magick` 程序由生成的 PDF 文件产生。绘制标签时始终使用 1 的 `opacity`。示例文件 `transparency.asy` 中提供了一个透明填充的简单示例。

- 包含在 `picture` 中的 `PostScript` 命令可通过添加到全局 `PostScript` 框(frame) `currentpatterns` 中(带有可选的左下边距 `lb` 和右上边距 `rt`)，来创建用于 `fill` 与 `draw` 操作、由字符串 `name` 标识的平铺图案(tiling pattern)。

```
import patterns;
void add(string name, picture pic, pair lb=0, pair rt=0);
要使用图案 name 进行 fill 或 draw, 使用画笔 pattern("name")。例如, 矩形平
铺可以使用模块 patterns 中定义的例程 picture tile(real Hx=5mm, real Hy=0,
pen p=currentpen, filltype filltype=NoFill)、picture checker(real Hx=5mm,
real Hy=0, pen p=currentpen) 以及 picture brick(real Hx=5mm, real Hy=0, pen
p=currentpen) 来构造:
size(0,90);
import patterns;

add("tile",tile());
add("filledtilewithmargin",tile(6mm,4mm,red,Fill),(1mm,1mm),(1mm,1mm));
add("checker",checker());
add("brick",brick());

real s=2.5;
filldraw(unitcircle,pattern("tile"));
filldraw(shift(s,0)*unitcircle,pattern("filledtilewithmargin"));
filldraw(shift(2s,0)*unitcircle,pattern("checker"));
filldraw(shift(3s,0)*unitcircle,pattern("brick"));
```

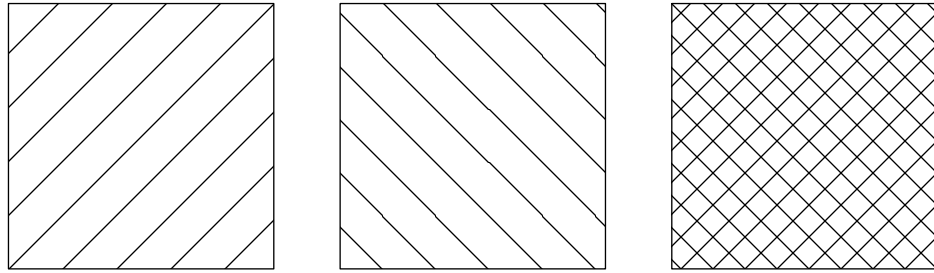


阴影线(Hatch)图案可以使用以下例程生成: `picture hatch(real H=5mm, pair dir=NE, pen p=currentpen)`、`picture crosshatch(real H=5mm, pen p=currentpen)`:

```
size(0,100);
import patterns;

add("hatch",hatch());
add("hatchback",hatch(NW));
add("crosshatch",crosshatch(3mm));

real s=1.25;
filldraw(unitsquare,pattern("hatch"));
filldraw(shift(s,0)*unitsquare,pattern("hatchback"));
filldraw(shift(2s,0)*unitsquare,pattern("crosshatch"));
```



你可能需要在 PostScript 查看器中关闭抗锯齿(anti-aliasing)功能, 以便图案能正确显示。跟随模块 `patterns` 中的示例, 可以轻松构造自定义图案。平铺图案甚至可以包含渐变阴影/渐变填充(见 `[gradient shading]`, 17 页), 如此示例所示(未包含在手册中, 因为并非所有打印机都支持 PostScript 3):

```
size(0,100);
import patterns;

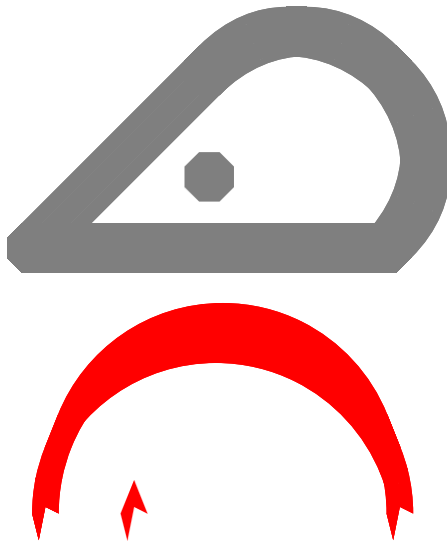
real d=4mm;
picture tiling;
path square=scale(d)*unitsquare;
axialshade(tiling,square,white,(0,0),black,(d,d));
fill(tiling,shift(d,d)*square,blue);
add("shadedtiling",tiling);

filldraw(unitcircle,pattern("shadedtiling"));
```

- 可以使用 `pen makepen(path)` 将自定义笔尖(pen nib)指定为任意多边形路径; 该路径代表包含单个点的路径所要绘制的标记。该笔尖路径可以通过 `path nib(pen)` 从画笔中恢复。与 MetaPost 中不同, 该路径不必是凸的:

```
size(200);
pen convex=makepen(scale(10)*polygon(8))+grey;
draw((1,0.4),convex);
draw((0,0)-(1,1)..(2,0)-cycle,convex);

pen nonconvex=scale(10)*
  makepen((0,0)-(0.25,-1)-(0.5,0.25)-(1,0)-(0.5,1.25)-cycle)+red;
draw((0.5,-1.5),nonconvex);
draw((0,-1.5)..(1,-0.5)..(2,-1.5),nonconvex);
```



值 `nullpath` 表示圆形笔尖（默认）；只需将画笔乘以一个变换即可获得椭圆形画笔：`yscale(2)*currentpen`。

- 可以通过使用画笔属性 `overwrite` 来防止标签相互覆盖，它接受单个参数：

Allow 允许标签相互覆盖。这是默认行为（除非使用 `defaultpen(pen)` 覆盖）。

Suppress 抑制（并发出警告）每一个会覆盖其他标签的标签。

SuppressQuiet

静默抑制（不发出警告）每一个会覆盖其他标签的标签。

Move

将一个会覆盖其他标签的标签移开并发出警告。由于该调整是在最终输出阶段（以 PostScript 坐标计）进行的，这可能导致图形比要求的更大。

MoveQuiet

将一个会覆盖其他标签的标签移开，不发出警告。由于该调整是在最终输出阶段（以 PostScript 坐标计）进行的，这可能导致图形比要求的更大。

例程 `defaultpen()` 返回当前的默认画笔属性。调用例程 `resetdefaultpen()` 会将所有画笔默认属性重置为其初始值。

6.4 变换

Asymptote 广泛使用仿射变换(affine transform)。数对 (x, y) 经过变换 $t=(t.x, t.y, t.xx, t.xy, t.yx, t.yy)$ 作用后变为 (x', y') ，其中

$$x' = t.x + t.xx * x + t.xy * y$$

$$y' = t.y + t.yx * x + t.yy * y$$

这等价于 PostScript 的变换 $[t.xx \ t.yx \ t.xy \ t.yy \ t.x \ t.y]$ 。

通过在左侧乘上变换（使用二元运算符 $*$ ），可以将变换应用于数对、引导(guide)、路径(path)、画笔(pen)、字符串、变换、框(frame)以及图片(picture)（示例参见见 `[circle]`, 30 页）。变换之间可以相互复合，并可使用函数 `transform inverse(transform t)` 求逆；还可以使用 $\sim$ 运算符将其提升到任意整数次幂。

内置的变换有：

```
transform identity;
    恒等变换;
```

```

transform shift(pair z);
    按数对 z 平移;

transform shift(real x, real y);
    按数对 (x,y) 平移;

transform xscale(real x);
    在 x 方向缩放 x 倍;

transform yscale(real y);
    在 y 方向缩放 y 倍;

transform scale(real s);
    在 x 和 y 两个方向均缩放 s 倍;

transform scale(real x, real y);
    在 x 方向缩放 x 倍, 在 y 方向缩放 y 倍;

transform slant(real s);
    映射 (x,y) -> (x+s*y,y);

transform rotate(real angle, pair z=(0,0));
    绕点 z 旋转 angle 度;

transform reflect(pair a, pair b);
    关于直线 a--b 反射;

transform zeroTransform;
    零变换;

```

变换的隐式初始化为 `identity()`。例程 `shift(transform t)` 与 `shiftless(transform t)` 分别返回变换 $(t.x, t.y, 0, 0, 0, 0)$ 和 $(0, 0, t.xx, t.xy, t.yx, t.yy)$ 。函数 `bool isometry(transform t)` 可用于测试 `t` 是否为等距变换（保持距离不变）。

6.5 框与图片

frame 框(frame)是在 PostScript 坐标系中绘图的画布。在构造延迟绘制(deferred drawing)例程时, 偶尔需要直接使用框。框的隐式初始化为 `newframe`。函数 `bool empty(frame f)` 仅在框 `f` 为空时返回 `true`。可以使用 `erase(frame)` 例程擦除框。函数 `pair min(frame)` 与 `pair max(frame)` 分别返回框边界框(bounding box)的(左, 下)和(右, 上)坐标。框 `src` 的内容可以通过以下命令附加到框 `dest` 的末尾:

```
void add(frame dest, frame src);
```

或使用以下命令附加到开头:

```
void prepend(frame dest, frame src);
```

通过类似于 `label` 的 `align` 参数(参见见 4.4 节 [`label`], 18 页)的方式, 将框 `f` 在 `align` 方向对齐所得到的框, 由以下函数返回:

```
frame align(frame f, pair align);
```

picture 图片(picture)是定义在模块 `plain` 中的高级结构(参见见 6.9 节 [`Structures`], 57 页), 提供了在用户坐标系中绘图的画布。默认的图片称为 `currentpicture`。可以像这样创建一个新图片:

```
picture pic;
```

匿名图片可以通过表达式 `new picture` 创建。

`size` 例程用于指定所需图片的尺寸：

```
void size(picture pic=currentpicture, real x, real y=x,
          bool keepAspect=pic.keepAspect);
```

如果 `x` 和 `y` 的尺寸均为 0，用户坐标将被解释为 PostScript 坐标。在这种情况下，将 `pic` 映射到最终输出框的变换为 `identity()`。

如果 `x` 或 `y` 中恰好有一个为 0，则该方向不受尺寸限制；它将按与另一个方向相同的比例进行缩放。

如果 `keepAspect` 设置为 `Aspect` 或 `true`（默认值），图片缩放时将保持其宽高比，使最终宽度不超过 `x`，最终高度不超过 `y`。

如果 `keepAspect` 设置为 `IgnoreAspect` 或 `false`，图片将在两个方向上分别缩放，使最终宽度为 `x`，高度为 `y`。

要使图片 `pic` 的用户坐标在 `x` 方向上表示 `x` 个单位的倍数，在 `y` 方向上表示 `y` 个单位的倍数，请使用

```
void unitsize(picture pic=currentpicture, real x, real y=x);
```

当这些 `x` 与 `y` 值非零时，它们会覆盖图片 `pic` 相应的尺寸参数。

例程

```
void size(picture pic=currentpicture, real xsize, real ysize,
          pair min, pair max);
```

强制最终的图片缩放将用户坐标 `box(min,max)` 映射到宽度为 `xsize` 且高度为 `ysize` 的区域（当这些参数非零时）。

或者，调用例程

```
transform fixedscaling(picture pic=currentpicture, pair min,
                        pair max, pen p=nullpen, bool warn=false);
```

将使图片 `pic` 使用固定缩放将 `box(min,max)` 中的用户坐标映射到（已指定的）图片尺寸，同时考虑画笔 `p` 的宽度。如果最终图片超出指定尺寸，将发出警告。

通过调用 `shipout` 函数，可以将图片 `pic` 适应(fit)到框中，并使用图像格式 `format` 输出到文件 `prefix.format`：

```
void shipout(string prefix=defaultfilename, picture pic=currentpicture,
             orientation orientation=orientation,
             string format="", bool wait=false, bool view=true,
             string options="", string script="",
             light light=currentlight, projection P=currentprojection)
```

默认输出格式为 PostScript，可以通过命令行选项 `-f` 或 `-tex` 更改。`options`、`script` 和 `projection` 参数仅与 3D 图片相关。如果 `defaultfilename` 是空字符串，将使用前缀 `outprefix()`。

在文件退出时会自动隐式添加一条 `shipout()` 命令。如果显式的 `shipout()` 命令输出到与最终隐式 `shipout` 相同的文件，该命令将被忽略。默认页面方向为 Portrait（纵向）；可以通过更改变量 `orientation` 来修改。要以横向模式输出，只需设置变量 `orientation=Landscape` 或执行命令

```
shipout(Landscape);
```

要将页面旋转 -90 度，使用方向 `Seascape`。方向 `UpsideDown` 将页面旋转 180 度。

可以通过调用以下函数显式将图片 `pic` 适应(`fit`)到框:

```
frame pic.fit(real xsize=pic.xsize, real ysize=pic.ysize,
              bool keepAspect=pic.keepAspect);
```

默认的尺寸和宽高比设置是传给 `size` 命令的值 (默认分别为 0、0 和 `true`)。当前用于将图片 `pic` 适应到框的变换由成员函数 `pic.calculateTransform()` 返回。

在某些情况下 (例如二维图表), 仅能获得 `pic` 的近似尺寸估计, 此时图片适应例程

```
frame pic.scale(real xsize=this.xsize, real ysize=this.ysize,
                bool keepAspect=this.keepAspect);
```

(该例程会缩放生成的框, 包括标签和固定尺寸的对象) 将严格符合请求的尺寸规范, 但通常不需要使用。

要在图片四周绘制带有边距的边界框(`bounding box`), 可以使用以下函数将图片适应到框:

```
frame bbox(picture pic=currentpicture, real xmargin=0,
           real ymargin=xmargin, pen p=currentpen,
           filltype filltype=NoFill);
```

这里的 `filltype` 指定以下填充类型之一:

FillDraw 填充内部并绘制边界。

```
FillDraw(real xmargin=0, real ymargin=xmargin, pen fillpen=nullpen,
          pen drawpen=nullpen)
```

如果 `fillpen` 为 `nullpen`, 则使用当前绘图笔填充; 否则使用画笔 `fillpen` 填充。如果 `drawpen` 为 `nullpen`, 则使用 `fillpen` 绘制边界; 否则使用 `drawpen` 绘制。可以指定可选的边距 `xmargin` 和 `ymargin`。

Fill 填充内部。

```
Fill(real xmargin=0, real ymargin=xmargin, pen p=nullpen)
```

如果 `p` 为 `nullpen`, 则使用当前绘图笔填充; 否则使用画笔 `p` 填充。可以指定可选的边距 `xmargin` 和 `ymargin`。

NoFill 不填充。

Draw 仅绘制边界。

```
Draw(real xmargin=0, real ymargin=xmargin, pen p=nullpen)
```

如果 `p` 为 `nullpen`, 则使用当前绘图笔绘制边界; 否则使用画笔 `p` 绘制。可以指定可选的边距 `xmargin` 和 `ymargin`。

UnFill 裁剪该区域。

```
UnFill(real xmargin=0, real ymargin=xmargin)
```

裁剪该区域及周围边距 `xmargin` 和 `ymargin`。

```
RadialShade(pen penc, pen penr)
```

呈径向渐变填充, 从边界框中心的 `penc` 渐变至边缘的 `penr`。

```
RadialShadeDraw(real xmargin=0, real ymargin=xmargin, pen penc,
                 pen penr, pen drawpen=nullpen)
```

使用 `RadialShade` 填充并绘制边界。

例如, 要在图片周围绘制带 0.25 cm 边距的边界框并输出生成的框, 可使用命令:

```
shipout(bbox(0.25cm));
```

使用函数 `bbox(p, Fill)` 可以将 `picture` 适应到背景颜色画笔为 `p` 的框中。

要在两个方向上将图片填充(pad)至精确尺寸, 可使用以下函数将图片适应到框:

```
frame pad(picture pic=currentpicture, real xsize=pic.xsize,
          real ysize=pic.ysize, filltype filltype=NoFill);
```

函数

```
pair min(picture pic, user=false);
pair max(picture pic, user=false);
pair size(picture pic, user=false);
```

计算图片 `pic` 在按其默认尺寸规范适应到框时所具有的边界。如果 `user` 为 `false`, 返回值使用 PostScript 坐标; 否则使用用户坐标。

函数

```
pair point(picture pic=currentpicture, pair dir, bool user=true);
```

是一种便捷的方法, 用于确定 `pic` 的边界框上相对于其中心在 `dir` 方向上的点, 忽略固定尺寸对象 (如标签和箭头) 的贡献。如果 `user` 为 `true`, 返回值使用用户坐标; 否则使用 PostScript 坐标。

函数

```
pair truepoint(picture pic=currentpicture, pair dir, bool user=true);
```

与 `point` 相同, 不同之处在于它还计入了固定尺寸对象, 使用的是图片 `pic` 在其默认尺寸规范适应到框时所具有的缩放变换。如果 `user` 为 `true`, 返回值使用用户坐标; 否则使用 PostScript 坐标。

有时在独立的图片上绘制对象, 并使用 `add` 函数将一个图片添加到另一个图片中会很有用:

```
void add(picture src, bool group=true,
         filltype filltype=NoFill, bool above=true);
void add(picture dest, picture src, bool group=true,
         filltype filltype=NoFill, bool above=true);
```

第一个示例将 `src` 添加到 `currentpicture`; 第二个示例将 `src` 添加到 `dest`。 `group` 选项指定图形用户界面(GUI)是否应将 `src` 的所有元素视为单一实体 (参见见 第 13 章 [GUI], 189 页); `filltype` 用于请求可选的背景填充或裁剪; `above` 指定是将 `src` 添加在现有对象之上还是之下。

还有一些例程可用于围绕用户坐标 `position`, 将固定尺寸的图片或框 `src` 添加到另一个图片 `dest` (或 `currentpicture`):

```
void add(picture src, pair position, bool group=true,
         filltype filltype=NoFill, bool above=true);
void add(picture dest, picture src, pair position,
         bool group=true, filltype filltype=NoFill, bool above=true);
void add(picture dest=currentpicture, frame src, pair position=0,
         bool group=true, filltype filltype=NoFill, bool above=true);
void add(picture dest=currentpicture, frame src, pair position,
         pair align, bool group=true, filltype filltype=NoFill,
         bool above=true);
```

最后一种形式中可选的 `align` 参数指定用于对齐框的方向，其方式类似于 `label` 的 `align` 参数（参见 4.4 节 [label], 18 页）。但是，一个关键区别在于：未指定 `align` 时，标签默认居中，而框和图片则对齐为使其原点位于 `position` 处。框对齐的示例可以在示例 [errorbars], 117 页, 和 [image], 154 页, 中找到。如果您想对齐三个或更多子图片(subpicture)，请每次将它们两两组合：

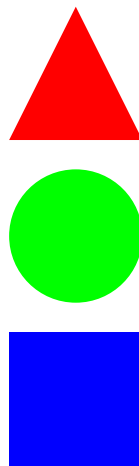
```
picture pic1;
real size=50;
size(pic1,size);
fill(pic1,(0,0)-(50,100)-(100,0)-cycle,red);

picture pic2;
size(pic2,size);
fill(pic2,unitcircle,green);

picture pic3;
size(pic3,size);
fill(pic3,unitsquare,blue);

picture pic;
add(pic,pic1.fit(),(0,0),N);
add(pic,pic2.fit(),(0,0),10S);

add(pic.fit(),(0,0),N);
add(pic3.fit(),(0,0),10S);
```



或者，可以使用 `attach` 自动增大图片 `dest` 的尺寸，以容纳围绕用户坐标 `position` 添加框 `src`：

```
void attach(picture dest=currentpicture, frame src,
            pair position=0, bool group=true,
            filltype filltype=NoFill, bool above=true);
void attach(picture dest=currentpicture, frame src,
```

```
pair position, pair align, bool group=true,
filltype filltype=NoFill, bool above=true);
```

要擦除图片的内容（但保留尺寸规范），使用函数

```
void erase(picture pic=currentpicture);
```

要保存 `currentpicture`、`currentpen` 和 `currentprojection` 的快照，使用函数 `save()`。

要恢复 `currentpicture`、`currentpen` 和 `currentprojection` 的快照，使用函数 `restore()`。

基础模块 `plain` 中提供了许多图片和框操作的更多示例。

可以使用以下例程之一在图片中插入原样(verbatim) PostScript 命令：

```
void postscript(picture pic=currentpicture, string s);
void postscript(picture pic=currentpicture, string s, pair min,
pair max)
```

这里的 `min` 和 `max` 可用于指定与生成的 PostScript 代码相关的显式边界。

可以使用以下函数之一在中间 $\text{\LaTeX}$ 输出文件中插入原样 $\text{\TeX}$ 命令：

```
void tex(picture pic=currentpicture, string s);
void tex(picture pic=currentpicture, string s, pair min, pair max)
```

这里的 `min` 和 `max` 可用于指定与生成的 $\text{\TeX}$ 代码相关的显式边界。

要在 $\text{\TeX}$ 导言区(preamble)中发出一条全局 $\text{\TeX}$ 命令（例如 $\text{\TeX}$ 宏定义，在顶层模块的其余部分均有效），请使用：

```
void texpreamble(string s);
```

可以使用函数

```
void texreset();
```

将 $\text{\TeX}$ 环境重置为初始状态，清除所有宏定义。

例程

```
void usepackage(string s, string options="");
```

提供了以下写法的便捷缩写

```
texpreamble("\usepackage["+options+"]{"+s+"}");
```

可用于导入 $\text{\LaTeX}$ 宏包。

6.6 延迟绘制

有时我们希望某些元素具有固定的绝对尺寸，不受图片从用户坐标到 PostScript 坐标缩放的影响。例如，通常人们希望使用 `dot` 创建的点的大小、使用 `arrow`（见 [arrows], 14 页）创建的箭头的大小以及标签在绘制时不受缩放影响。

然而，由于 `Asymptote` 的自动缩放特性（见 3.3 节 [Figure size], 10 页），用户坐标与 PostScript 坐标之间的转换直到输出(shipout)时刻才能确定：

```
size(1cm);
dot((0,0));
dot((1,1));
shipout("x"); // at this point, 1 unit coordinate = 1cm
dot((2,2));
```

```
shipout("y"); // at this point, 1 unit coordinate = 0.5cm
```

因此，有必要将这些元素的绘制延迟到输出时刻进行。

例如，可以通过延迟绘制例程 `add(picture dest=currentpicture, frame src, pair position)` 在图片的指定用户坐标处添加一个框(frame)：

```
frame f;
fill(f,circle((0,0),1.5pt));
add(f,position=(1,1),align=(0,0));
```

延迟绘制例程是类型为 `drawer` 的对象，它是一个函数签名为 `void(frame f, transform t)` 的函数。例如，若将绘制例程

```
void d(frame f, transform t){
    fill(f,shift(3cm,0)*circle(t*(1,1),2pt));
}
```

通过

```
currentpicture.add(d);
```

添加到 `currentpicture` 中，则会在 `currentpicture` 上绘制一个半径为 `2pt` 的填充圆，其圆心位于用户坐标 `(1,1)` 右侧 `3cm` 处。参数 `t` 是从用户坐标到 PostScript 坐标的仿射变换(affine transformation)。

尽管实际的绘制被延迟了，你仍需要向 `Asymptote` 缩放例程指定最终将在用户坐标 `(1,1)` 右侧 `3cm` 处绘制一个半径为 `2pt` 的固定尺寸圆，方法是围绕 `(1,1)` 添加一个从 `(3cm-2pt,-2pt)+min(currentpen)` 延伸超出 `(1,1)` 到 `(3cm+2pt,2pt)+max(currentpen)` 的框(frame)，其中甚至考虑到了画笔(pen)的线宽。下面的示例将生成一个宽度为 `10 cm` 的 PDF 文件：

```
settings.outformat="pdf";
size(10cm);
dot((0,0));
dot((1,1),red);
add(new void(frame f, transform t) {
    fill(f,shift(3cm,0)*circle(t*(1,1),2pt));
});
pair trueMin=(3cm-2pt,-2pt)+min(currentpen);
pair trueMax=(3cm+2pt,2pt)+max(currentpen);
currentpicture.addPoint((1,1),trueMin);
currentpicture.addPoint((1,1),trueMax);
```

这里我们通过 `picture` 例程

```
void addPoint(pair user, pair truesize);
```

指定了绘图器的最小和最大用户坐标及真实尺寸(truesize，即固定)坐标。另一种方式是使用

```
void addBox(pair userMin, pair userMax, pair trueMin=0, pair trueMax=0) {
```

来指定一个左下角为 `t*(1,1)+trueMin` 且右上角为 `t*(1,1)+trueMax` 的边界框(bounding box)，其中 `t` 是将用户坐标转换为 PostScript 坐标的变换。

有关延迟绘制的更多细节，参见 `\Asymptote: A vector graphics language`," John C. Bowman and Andy Hammerlindl, TUGBOAT: The Communications of the TeX Users Group, 29:2, 288-294 (2008),

<https://www.math.ualberta.ca/~bowman/publications/asyTUG.pdf>。

6.7 文件

Asymptote 可以读写文本文件（包括逗号分隔值，即 CSV）以及可移植的 XDR（External Data Representation，外部数据表示）二进制文件。

输入文件可以通过以下方式打开：

```
input(string name="")
```

随后通过赋值来进行读取：

```
file fin=input("test.txt");
real a=fin;
```

如果可选的布尔型参数 `check` 为 `false`，则不会检查文件是否存在。如果文件不存在或不可读，函数 `bool error(file)` 将返回 `true`。字符串 `comment` 的第一个字符指定注释字符。如果在数据文件中遇到该字符，该行余下的内容将被忽略。在读取字符串时，紧跟在另一个注释字符后面的注释字符会被当作单个字面注释字符处理。如果编译 Asymptote 时启用了 `libcurl` 支持，则 `name` 可以是一个 URL。

除非指定了 `-noglobalread` 命令行选项，否则可以使用函数 `string cd(string s)` 将读取操作的当前工作目录更改为字符串 `s` 的内容，该函数返回新的工作目录。如果 `strings` 为空，则路径会重置为程序启动时的值。

读取数对(pair)时，外层的括号是可选的。字符串也是通过赋值读取的，读取字符直到（但不包含）换行符。此外，Asymptote 提供了函数 `string getc(file)` 来读取下一个字符（将注释字符当作普通字符处理）并将其作为字符串返回。

名为 `name` 的文件可以通过以下方式打开以用于输出：

```
file output(string name="", bool update=false, string comment="#", string mode="");
```

如果 `update=false`，文件中任何现有数据将被擦除，且该文件仅能进行写入操作。如果 `update=true`，任何现有数据都将被保留，位置将设置为文件末尾，且同时启用读取和写入操作。出于安全原因，只有在指定了 `-globalwrite`（或 `-nosafe`）命令行选项时，才允许向当前目录以外的目录中的文件写入。除非指定了 `-noglobalread` 命令行选项，否则允许从其他目录中的文件进行读取。函数 `string mktemp(string s)` 可用于根据字符串 `s` 在当前目录中创建并返回一个唯一的临时文件名。

有两个特殊文件：`stdin`（从键盘读取）和 `stdout`（写入终端）。文件的隐式初始化器为 `null`。

内建类型 `T` 的数据可以通过调用以下函数之一写入输出文件：

```
write(string s="", T x, suffix suffix=endl, ... T[]);
write(file file, string s="", T x, suffix suffix=none, ... T[]);
write(file file=stdout, string s="", explicit T[] x, ... T[] []);
write(file file=stdout, T[] []);
write(file file=stdout, T[] [] []);
write(suffix suffix=endl);
write(file file, suffix suffix=none);
```

如果未指定 `file`，则使用 `stdout` 并且默认以换行符结尾。如果指定了可选的标识字符串 `s`，它将在数据 `x` 之前写入。在写入标量或一维数组时，可以列出任意数量的数据值。`suffix` 可以是以下之一：`none`（什么都不做）、`flush`（输出缓冲区数据）、`endl`（以换行符结尾并刷新）、`newl`（以换行符结尾）、`DOSendl`（以 DOS 换行符结尾并刷新）、`DOSnewl`（以 DOS 换行符结尾）、`tab`（以制表符结尾）或 `comma`（以逗号结尾）。下面是一些数据输出的简单示例：

```
file fout=output("test.txt");
```

```

write(fout,1);           // Writes "1"
write(fout, endl);       // Writes a new line
write(fout,"List: ",1,2,3); // Writes "List: 1      2      3"

```

打开文件时可以使用 `mode="xdr"`，以 Sun Microsystem 的 XDR (External Data Representation, 外部数据表示) 可移植二进制格式读写双精度 (64 位) 实数和单精度 (32 位) 整数。文件也可以使用 `mode="binary"` 打开，以原生 (不可移植) 机器二进制格式读写双精度实数和单精度整数。虚成员函数 `file singlereal(bool b=true)` 和 `file singleint(bool b=true)` 分别用于更改 XDR 或二进制文件中实数和整数 I/O 操作的精度。类似地，函数 `file signedint(bool b=true)` 可用于修改 XDR 或二进制文件中整数读写的有无符号性。由于这些格式中没有行尾终止符，因此从 XDR 或二进制文件读取字符串时，默认会将文件的剩余部分作为字符串读入。虚成员函数 `file word(bool b=true)` 可用于更改此行为：首先读取或写入一个指定字符串长度的双精度整数，随后是相应数量的字符。

虚字段 `name`、`mode`、`singlereal`、`singleint` 和 `signedint` 可用于查询文件的这些相应状态。

可以使用布尔函数 `eof(file)` 测试文件是否到达文件末尾，使用 `eol(file)` 测试是否到达行尾，使用 `error(file)` 测试是否存在 I/O 错误。可以使用 `flush(file)` 刷新输出缓冲区，使用 `clear(file)` 清除先前的 I/O 错误，并使用 `close(file)` 关闭文件。函数 `int precision(file file=stdout, int digits=0)` 在 `digits` 非零时将 `file` 的输出精度位数设置为 `digits`，并返回先前的精度设置。函数 `int tell(file)` 返回文件中相对于起始位置的当前位置。例程 `seek(file file, int pos)` 可用于更改此位置，其中位置 `pos` 的负值被解释为相对于文件末尾。例如，可以使用命令 `seek(file,0)` 重置文件 `file` 的位置，并使用 `seek(file,-1)` 定位到文件中的最后一个字符。命令 `seekeof(file)` 将位置设置为文件末尾。

将 `settings.scroll=n` 赋值为正整数 `n`，要求每向 `stdout` 输出 `n` 行后暂停一次。然后可以按 `Enter` 继续输出接下来的 `n` 行，按 `s` 后跟 `Enter` 进行无中断滚动，或者按 `q` 后跟 `Enter` 退出当前的输出操作。如果 `n` 为负数，输出将一次滚动一页（比当前显示行数少一行）。默认值 `settings.scroll=0` 指定连续滚动。

模块 `plain` 中定义的例程

```

string getstring(string name="", string default="", string prompt="",
                 bool store=true);
int getint(string name="", int default=0, string prompt="",
           bool store=true);
real getreal(string name="", real default=0, string prompt="",
             bool store=true);
pair getpair(string name="", pair default=0, string prompt="",
             bool store=true);
triple gettriple(string name="", triple default=(0,0,0), string prompt="",
                 bool store=true);

```

可用于使用 GNU `readline` 库从 `stdin` 提示输入一个值。如果 `store=true`，`name` 的历史值将存储在文件 `".asy_history-"+name` 中（见 [history], 187 页）。历史记录中的最新值将用于为后续运行提供默认值。默认值（初始为 `default`）显示在 `prompt` 之后。这些函数基于内部例程

```

string readline(string prompt="", string name="", bool tabcompletion=false);
void saveline(string name, string value, bool store=true);

```

在此, `readline` 按照 `prompt` 格式化的默认值提示用户, 而 `saveline` 用于将字符串 `value` 保存到名为 `name` 的本地历史记录中, 并可选地将本地历史记录存储在文件 `".asy_history-"+name` 中。

例程 `history(string name, int n=1)` 可用于查找为字符串 `name` 输入的最近 `n` 个值 (若 `n=0` 则为最多 `historylines` 个的所有值)。例程 `history(int n=0)` 返回交互历史记录。例如,

```
write(output("transcript.asy"),history());
```

将交互历史记录输出到文件 `transcript.asy`。

函数 `int delete(string s)` 删除由字符串 `s` 命名的文件。除非启用了 `-globalwrite` (或 `-nosafe`) 选项, 否则该文件必须位于当前目录中。函数 `int rename(string from, string to)` 可用于将文件 `from` 重命名为文件 `to`。除非启用了 `-globalwrite` (或 `-nosafe`) 选项, 否则此操作仅限于当前目录。函数

```
int convert(string args="", string file="", string format="");
```

```
int animate(string args="", string file="", string format="");
```

分别使用参数 `args` 以及从字符串 `file` 和 `format` 构造的文件名调用 `ImageMagick` 命令 `magick` 和 `animate`。

6.8 变量初始化器

变量可以在声明时被赋值, 例如 `int x=3;` 中变量 `x` 被赋值为 3。除了字面常量 (如 3) 之外, 任意表达式都可以用作初始化器, 例如在 `real x=2*sin(pi/2);` 中。

变量只有在初始化器求值之后才会被添加到命名空间中, 因此例如在

```
int x=2;
```

```
int x=5*x;
```

中, 第二行初始化器中的 `x` 指的是第一行声明的变量 `x`。于是, 第二行声明了一个遮蔽(shadowing)原始 `x` 的变量 `x`, 并将其初始化为值 10。

大多数类型的变量都可以在不指定显式初始化器的情况下进行声明, 它们将由该类型的默认初始化器进行初始化:

- 数值类型 `int`、`real` 和 `pair` 的变量都会被初始化为零; 类型为 `triple` 的变量会被初始化为 `0=(0,0,0)`。
- `boolean` 变量会被初始化为 `false`。
- `string` 变量会被初始化为空字符串。
- `transform` 变量会被初始化为恒等变换。
- `path` 和 `guide` 变量会被初始化为 `nullpath`。
- `pen` 变量会被初始化为默认画笔(`pen`)。
- `frame` 和 `picture` 变量会分别被初始化为空框(`frame`)和空图片(`picture`)。
- `file` 变量会被初始化为 `null`。

用户自定义的数组、结构和函数类型的默认初始化器将在它们各自的章节中进行说明。某些类型 (如 `code`) 没有默认初始化器。当引入这种类型的变量时, 用户必须通过显式赋予其一个值来进行初始化。

任何类型 `T` 的默认初始化器都可以通过定义函数 `T operator init()` 来重新声明。例如, `int` 变量通常初始化为零, 但在

```
int operator init() {
```

```
    return 3;
}
```

```
int y;
```

中，变量 `y` 被初始化为 3。给出这个示例是为了说明原理；不推荐重新声明内建类型的初始化器。通常，`operator init` 用于为用户自定义类型定义合理的默认值。

特殊类型 `var` 可用于从变量的初始化器推导其类型。如果初始化器是单一类型的表达式，则该变量将被定义为该类型。例如，

```
var x=5;
var y=4.3;
var reddash=red+dashed;
```

等价于

```
int x=5;
real y=4.3;
pen reddash=red+dashed;
```

`var` 也可以与扩展的 `for` 循环语法一起使用。

```
int[] a = {1,2,3};
for (var x : a)
    write(x);
```

6.9 结构

用户还可以将自己的数据类型定义为结构(structure)，并附带用户自定义的运算符，这与 C++ 非常相似。默认情况下，结构成员是 `public` 的（可以在代码中的任何位置读取和修改），但也可以选择声明为 `restricted`（在任何地方均可读，但仅能在定义它们的结构内部写入）或 `private`（仅在结构内部可读写）。在结构定义中，关键字 `this` 可以用作引用包围它的结构的表达式。结构内部顶层作用域中的任何代码都会在初始化时执行。

变量持有对结构的引用。也就是说，在示例中：

```
struct T {
    int x;
}
```

```
T foo;
T bar=foo;
bar.x=5;
```

变量 `foo` 持有对结构 `T` 的一个实例的引用。当 `bar` 被赋值为 `foo` 的值时，它现在也持有对与 `foo` 相同的实例的引用。赋值 `bar.x=5` 更改了该实例中字段 `x` 的值，因此 `foo.x` 也将等于 5。

表达式 `new T` 创建结构 `T` 的一个新实例并返回对该实例的引用。在创建新实例时，结构定义体中的任何代码都会被执行。例如：

```
int Tcount=0;
struct T {
    int x;
    ++Tcount;
}
```

```
T foo=new T;
T foo;
```

在这里，`new T` 生成了结构体的一个新实例，导致 `Tcount` 递增，从而追踪所生成的实例数量。声明 `T foo=new T` 和 `T foo` 是等价的：第二种形式隐式地创建 `T` 的一个新实例。也就是说，在结构 `T` 定义之后，类型为 `T` 的变量默认会被初始化为一个新实例（`new T`）。然而，在结构定义期间，类型为 `T` 的变量默认初始化为 `null`。这种特殊行为是为了避免在如下代码中创建新实例时发生无限递归：

```
struct tree {
    int value;
    tree left;
    tree right;
}
```

表达式 `null` 可以转换(cast)为任何结构类型以产生空引用（即实际上不指向结构的任何实例的引用）。尝试使用空引用的字段将导致错误。

函数 `bool alias(T,T)` 用于检查两个结构引用是否指向结构的同一个实例（或两者均指向 `null`）。在本节开头的示例中，`alias(foo,bar)` 将返回 `true`，但 `alias(foo,new T)` 将返回 `false`，因为 `new T` 会创建结构 `T` 的一个新实例。布尔运算符 `==` 和 `!=` 默认分别等价于 `alias` 和 `!alias`，但可以针对特定类型进行重写（例如进行深层比较(deep comparison)）。

这里是一个说明结构使用的简单示例：

```
struct S {
    real a=1;
    real f(real a) {return a+this.a;}
}

S s;                                // Initializes s with new S;

write(s.f(2));                      // Outputs 3

S operator + (S s1, S s2)
{
    S result;
    result.a=s1.a+s2.a;
    return result;
}

write((s+s).f(0));                  // Outputs 2
```

具备能够构造结构新实例的函数通常会很方便。假设我们有一个 `Person` 结构：

```
struct Person {
    string firstname;
    string lastname;
}

Person joe;
```

```
joe.firstname="Joe";
joe.lastname="Jones";
```

创建一个新的 Person 是件麻烦事；需要三行代码来创建一个新实例并初始化其字段（不过，这仍然比在现实生活中创造一个新人省力得多）。

我们可以通过定义 operator init 来简化这项工作：

```
struct Person {
    string firstname;
    string lastname;

    void operator init(string firstname, string lastname) {
        this.firstname=firstname;
        this.lastname=lastname;
    }
}
```

```
Person joe=Person("Joe", "Jones");
```

使用 operator init 隐式定义构造函数(constructor)不应与其用于为变量定义默认值混淆（见 6.8 节 [Variable initializers], 56 页）。实际上，在第一种情况下，operator init 的返回类型必须为 void；而在第二种情况下，必须是变量的（非 void）类型。

在内部，operator init 隐式定义了一个构造函数 Person(string,string)，如下所示，在本例中 args 为 string firstname, string lastname：

```
struct Person {
    string firstname;
    string lastname;

    static Person Person(args) {
        Person p=new Person;
        p.operator init(args);
        return p;
    }
}
```

然后它可以这样使用：

```
Person joe=Person.Person("Joe", "Jones");
```

在结构定义结束后，以下代码也会在包围作用域中隐式生成：

```
from Person unravel Person;
```

它允许我们在没有限定符的情况下使用构造函数，否则我们必须通过限定名称 Person.Person 来引用构造函数。

就像在 C++ 中一样，类型转换(casting)（见 6.14 节 [Casts], 77 页）为结构继承(inheritance)提供了优雅的实现，包括虚函数(virtual function) v：

```
struct parent {
    real x;
    void operator init(int x) {this.x=x;}
    void v(int) {write(0);}
    void f() {v(1);}
```

```

}

void write(parent p) {write(p.x);}

struct child {
    parent parent;
    real y=3;
    void operator init(int x) {parent.operator init(x);}
    void v(int x) {write(x);}
    parent.v=v;
    void f()=parent.f;
}

parent operator cast(child child) {return child.parent;}

parent p=parent(1);
child c=child(2);

write(c);                                // Outputs 2;

p.f();                                  // Outputs 0;
c.f();                                  // Outputs 1;

write(c.parent.x);                       // Outputs 2;
write(c.y);                             // Outputs 3;

```

有关结构的更多示例，参见 `Asymptote` 基础模块 `plain` 中的 `Legend` 和 `picture`。

6.10 运算符

6.10.1 算术与逻辑运算符

`Asymptote` 使用标准的二元算术运算符。然而，当一个整数除以另一个整数时，两个参数在相除前都会转换为 `real` 值，并返回一个 `real` 商（因为这通常是预期的行为；否则可以使用函数 `int quotient(int x, int y)`，它返回小于或等于 x/y 的最大整数）。在所有其他情况下，两个操作数都会提升为相同的类型，该类型也即结果的类型：

+	加法
-	减法
*	乘法
/	除法
#	整数除法；等价于 <code>quotient(x,y)</code> 。注意到 <code>Python3</code> 社区采纳了我们的注释符号 (<code>//</code>) 用于整数除法，我们决定投桃报李，在 <code>Asymptote</code> 中使用他们的注释符号来表示整数除法！
%	取模；结果始终与除数具有相同的符号。特别地，对于所有整数 p 和非零整数 q ，这使得 $q*(p \# q)+p \% q == p$ 恒成立。

~ 乘方；若指数（第二个参数）为 `int`，则使用递归乘法；否则，使用对数与指数运算（** 是 ~ 的同义词）。

常用的布尔运算符也有定义：

<code>==</code>	等于
<code>!=</code>	不等于
<code><</code>	小于
<code><=</code>	小于或等于
<code>>=</code>	大于或等于
<code>></code>	大于
<code>&&</code>	与（带有右侧参数的条件求值）
<code>&</code>	与
<code> </code>	或（带有右侧参数的条件求值）
<code> </code>	或
<code>^</code>	异或
<code>!</code>	非

Asymptote 还支持类似 C 的条件表达式语法：

```
bool positive=(pi > 0) ? true : false;
```

对于非整数内建算术类型 `T`，函数 `T interp(T a, T b, real t)` 返回 $(1-t)*a+t*b$ 。如果 `a` 与 `b` 为画笔(`pen`)，它们首先会被提升到相同的颜色空间。

Asymptote 还定义了按位操作函数 `int AND(int,int)`、`int OR(int,int)`、`int XOR(int,int)`、`int NOT(int)`、`int CLZ(int)`（计算前导零个数）、`int CTZ(int)`（计算后缀零个数）、`int popcount(int)`（计算值为 1 的二进制位数），以及 `int bitreverse(int a, int bits)`（在长度为 `bits` 的字内反转比特位）。

6.10.2 自运算符与前缀运算符

与 C 语言类似，每个算术运算符 `+`、`-`、`*`、`/`、`#`、`%` 和 `^` 都可以用作自运算符。前缀运算符 `++`（自增 1）和 `--`（自减 1）也有定义。例如，

```
int i=1;
i += 2;
int j=++i;
```

等价于如下代码

```
int i=1;
i=i+2;
int j=i+1;
```

然而，后缀运算符如 `i++` 与 `i--` 未定义（因为这会与路径连接运算符 `--` 产生固有的歧义）。在极少数确实需要 `i++` 和 `i--` 的情况下，可以分别用表达式 `(++i-1)` 和 `(--i+1)` 来替代。

6.10.3 用户自定义运算符

以下符号可以与 `operator` 一起使用，以在结构体和内建类型上定义或重定义运算符：

```
- + * / % ^ ! < > == != <= >= & | ^^ .. :: - - ++
« » $ $$ @ @@ <>
```

第二行的运算符优先级比布尔运算符 `<`、`>`、`<=` 和 `>=` 高一级。

引导(`guide`)运算符 (例如 `..`) 可以被重载，比方说，用来编写一个从给定的 `guide` 生成新 `guide` 的用户函数：

```
guide dots(...guide[] g)=operator ..;
```

```
guide operator ..(...guide[] g) {
    guide G;
    if(g.length > 0) {
        write(g[0]);
        G=g[0];
    }
    for(int i=1; i < g.length; ++i) {
        write(g[i]);
        write();
        G=dots(G,g[i]);
    }
    return G;
}
```

```
guide g=(0,0){up}..{SW}(100,100){NE}..{curl 3}(50,50)..(10,10);
write("g=",g);
```

重载 `+` 运算符的示例见 6.18 节 [Autounravel], 85 页。

6.10.4 方括号运算符

如果 `a` 不是数组，则记号 `a[i]` 等价于 `a.operator[](i)`。因此，可以定义 `[]` 运算符，以便为用户自定义的结构体提供这种记号。类似地，记号 `a[i]=b` 等价于 `a.operator[](i,b)` (除了前者总是返回 `b` 之外)。

内置类型 `string` 具有虚成员函数 `string operator[](int pos)`。由于基本类型是不可变的，因此没有任何基本类型具有虚成员函数 `operator []=`。

对于如何定义这些运算符有一些限制：

- 这两个运算符都不能在单个结构体内部被重载 (尽管不同的结构体可以对这些运算符有不同的定义)。
- `operator []` 必须接受单个参数，并且具有非 `void` 的返回类型。
- 只有在同样定义了 `operator []` 时，才能定义 `operator []=`。
- `operator []=` 必须接受两个参数，且返回类型为 `void`。此外，若 `K` 和 `V` 分别为 `operator []=` 的第一个和第二个参数的类型，则 `K` 必须为 `operator []` 的参数类型，而 `V` 必须为 `operator []` 的返回类型。

这里是一个简单示例。(类似于 `slowMap` 但更灵活、更强大且更快的数据结构见 6.20.3 节 [Maps], 91 页。)

```
struct slowMap {
```

```

string[] keys;
real[] values;
real operator [](string key) {
    for(int i=0; i < keys.length; ++i) {
        if(keys[i] == key) return values[i];
    }
    return -inf;
}
void operator []=(string key, real value) {
    assert(all(keys != key), 'duplicate key');
    keys.push(key);
    values.push(value);
}
}

```

```

slowMap m;
m['pi']=3.14159;
m['e']=2.71828;
assert(m['pi']==3.14159);
assert(m['e']==2.71828);
assert(m['sqrt(2)'] == -inf);
write("m['pi']=" , m['pi']);

```

6.10.5 转换运算符

关于 operator cast 和 operator ecast 的信息见 6.14 节 [Casts], 77 页。

6.11 隐式缩放

如果数值字面量位于某些类型的表达式前面，则两者相乘：

```

int x=2;
real y=2.0;
real cm=72/2.540005;

```

```

write(3x);
write(2.5x);
write(3y);
write(-1.602e-19 y);
write(0.5(x,y));
write(2x^2);
write(3x+2y);
write(3(x+2y));
write(3sin(x));
write(3(sin(x))^2);
write(10cm);

```

这将产生如下输出

```

5
6
-3.204e-19
(1,1)
8
10
18
2.72789228047704
2.48046543129542
283.464008929116

```

6.12 函数

`Asymptote` 函数被视为带有签名(signature)的变量（非函数变量的签名为 `null`）。只要变量具有不同的签名，就允许使用相同的名称。

函数参数按值传递。要按引用传递参数，只需将其封装在一个结构体中（见 6.9 节 [Structures], 57 页）。

以下是 `Asymptote` 函数的一些重要特性：

1. 带有签名的变量（函数）与没有签名的变量（非函数变量）是不同的：

```

int x, x();
x=5;
x=new int() {return 17;};
x=x();           // calls x() and puts the result, 17, in the scalar x

```

2. 允许使用传统的函数定义：

```

int sqr(int x)
{
    return x*x;
}
sqr=null;           // but the function is still just a variable.

```

3. 可以使用类型转换来消除歧义：

```

int a, a(), b, b(); // Valid: creates four variables.
a=b;                // Invalid: assignment is ambiguous.
a=(int) b;           // Valid: resolves ambiguity.
(int) (a=b);         // Valid: resolves ambiguity.
(int) a=b;           // Invalid: cast expressions cannot be L-values.

```

```

int c();
c=a;                // Valid: only one possible assignment.

```

4. 允许“高阶”函数，换言之即返回函数的函数。必须使用 `using` 或 `typedef` 为返回类型赋予一个无签名的别名：

```

using intop = int(int);
// typedef int intop(int); // Equivalent to previous line
intop adder(int m)
{
    return new int(int n) {return m+n;};
}

```

```

}
intop addby7=adder(7);
write(addby7(1));      // Writes 8.

```

5. 可以通过将另一个（匿名或具名）函数赋值给函数 `f`，来重新定义 `f`，即便是在先前声明的函数中对 `f` 的调用也是如此。然而，如果 `f` 被一个新的函数定义所重载，先前的调用仍将访问 `f` 的原始版本，正如本例所示：

```

void f() {
    write("hi");
}

void g() {
    f();
}

g(); // writes "hi"

f=new void() {write("bye");};

g(); // writes "bye"

void f() {write("overloaded");};

f(); // writes "overloaded"
g(); // writes "bye"

```

6. 匿名函数可用于重新定义一个已经声明（且隐式初始化为 `null` 函数）但尚未显式定义的函数变量：

```

void f(bool b);

void g(bool b) {
    if(b) f(b);
    else write(b);
}

f=new void(bool b) {
    write(b);
    g(false);
};

g(true); // Writes true, then writes false.

```

据我们所知，Asymptote 是唯一一种将函数视为变量，但又允许通过基于签名区分变量来进行重载的语言。

函数允许递归调用自身。与 C++ 一样，无限嵌套递归将产生栈溢出（表现为段错误或非法指令，除非在配置时安装了完全正常工作的 GNU 库 `libsigsegv`）。

6.12.1 默认参数

Asymptote 支持比 C++ 更灵活的函数默认参数机制：它们可以出现在函数原型的任何位置。因为某些数据类型会被隐式转换为更复杂的类型（见 6.14 节 [Casts], 77 页），所以通常可以通过按由简到繁的顺序排列函数参数来避免歧义。例如，给定

```
real f(int a=1, real b=0) {return a+b;}
```

则 `f(1)` 返回 1.0，但 `f(1.0)` 返回 2.0。

默认参数的值是通过在被调用函数定义的作用域中求值给定的 Asymptote 表达式来决定的。

6.12.2 命名参数

有时很难记住函数声明中参数出现的顺序。命名（关键字）参数使得调用带有多个参数的函数更加容易。与 C 和 C++ 语言不同，函数参数中的赋值被解释为在函数签名中对同名形式参数的赋值，而不是在局部作用域内。命令行选项 `-d` 可用于检查 Asymptote 代码中可能将命名参数误认为是局部赋值的情况。

在将实际参数与签名进行匹配时，首先匹配所有关键字，然后没有名称的实参将如常与未匹配形参进行匹配。例如，

```
int f(int x, int y) {
    return 10x+y;
}
```

```
write(f(4,x=3));
```

输出 34，因为当我们试图匹配未命名实参 4 时，`x` 已经匹配了，所以它被匹配到下一个形参 `y`。

在极少数希望在函数参数中为局部变量赋值的情况下（通常不是良好的编程习惯），只需将赋值括在括号中。例如，给定上例中 `f` 的定义，

```
int x;
write(f(4,(x=3)));
```

等价于语句

```
int x;
x=3;
write(f(4,3));
```

并且输出 43。

通过在参数名紧前面加上 `keyword`，可以将参数指定为“仅限关键字(keyword-only)”，如 `int f(int keyword x)` 或 `int f(int keyword x=77)`。这会强制函数的调用者必须使用命名参数来为该参数赋值。也就是说，`f(x=42)` 是合法的，但 `f(25)` 则不是。在函数定义中，仅限关键字参数必须列在普通参数之后。

作为一个技术细节，我们需要指出的是，由于在同一作用域内允许同名但不同签名的变量，代码

```
int f(int x, int x()) {
    return x+x();
}
int seven() {return 7;}
```

在 Asymptote 中是合法的，其中 `f(2,seven)` 返回 9。命名参数与同名的第一个未匹配形参进行匹配，因此 `f(x=2,x=seven)` 是等价调用，但 `f(x=seven,2)` 则不是，因为第一个参数

匹配到了第一个形参，而 `int()` 不能隐式转换为 `int`。默认参数不影响命名参数匹配哪个形参，因此若 `f` 定义为

```
int f(int x=3, int x()) {
    return x+x();
}
```

则 `f(x=seven)` 将是非法的，即使 `f(seven)` 显然是被允许的。

6.12.3 剩余参数

剩余参数(rest arguments)允许编写接受可变数量参数的函数：

```
// This function sums its arguments.
int sum(...int[] nums) {
    int total=0;
    for(int i=0; i < nums.length; ++i)
        total += nums[i];
    return total;
}
```

```
sum(1,2,3,4);           // returns 10
sum();                   // returns 0
```

```
// This function subtracts subsequent arguments from the first.
int subtract(int start, ... int[] subs) {
    for(int i=0; i < subs.length; ++i)
        start -= subs[i];
    return start;
}
```

```
subtract(10,1,2);        // returns 7
subtract(10);             // returns 10
subtract();               // illegal
```

省略号前面的逗号是可选的，因此以下两个声明是等价的：

```
int subtract(int start, ... int[] subs);
int subtract(int start ... int[] subs);
```

但请注意，逗号不能直接紧跟在左括号后面：`int sum(, ... int[] nums)` 是一个语法错误。

将实参放进剩余数组中被称为 打包(*packing*)。可以为剩余参数指定显式的实参列表，因此 `subtract` 也可以替代实现为

```
int subtract(int start, ... int[] subs) {
    return start - sum(... subs);
}
```

甚至可以将普通实参与剩余参数结合起来：

```
sum(1,2,3, ... new int[] {4,5,6}); // returns 21
```

这会构建一个新的六元素数组，并将其作为 `nums` 传递给 `sum`。相反的操作，解包(*unpacking*)，是不允许的：

```
subtract(...new int[] {10, 1, 2});
```

是非法的，因为 `start` 形参未被匹配。

如果没有打包任何实参，则一个长度为零的数组（而非 `null`）会被绑定到剩余形参上。注意，对于剩余形参，默认参数会被忽略，且剩余参数不会绑定到关键字。

在某些情况下，仅限关键字参数有助于避免预定给剩余参数的实参被分配给其他形参。例如，这里使用 `keyword` 是为了避免 `pnorm(1.0,2.0,0.3)` 将 1.0 匹配给 `p`。

```
real pnorm(real keyword p=2.0, ... real[] v)
{
    return sum(v^p)^(1/p);
}
```

Asymptote 中的重载决议(overloading resolution)类似于 C++ 中使用的函数匹配规则。每个参数匹配都会被赋予一个得分。精确匹配的得分高于带类型转换的匹配，而与形参的匹配（无论是否有类型转换）得分高于将实参打包进剩余数组。如果在一个候选者中所有参数的得分都与其他任何候选者一样好，则该候选者是极大的。如果存在唯一的极大候选者，则选择它；否则，发生歧义错误。

```
int f(path g);
int f(guide g);
f((0,0)-(100,100)); // matches the second; the argument is a guide

int g(int x, real y);
int g(real x, int x);

g(3,4); // ambiguous; the first candidate is better for the first argument,
        // but the second candidate is better for the second argument

int h(... int[] rest);
int h(real x, ... int[] rest);

h(1,2); // the second definition matches, even though there is a cast,
        // because casting is preferred over packing

int i(int x, ... int[] rest);
int i(real x, real y, ... int[] rest);

i(3,4); // ambiguous; the first candidate is better for the first argument,
        // but the second candidate is better for the second one
```

6.12.4 数学函数

Asymptote 内建了标准 `libm` 数学 `real(real)` 函数 `sin`、`cos`、`tan`、`asin`、`acos`、`atan`、`exp`、`log`、`pow10`、`log10` 以及恒等函数 `identity`。Asymptote 还定义了 n 阶第一类贝塞尔(Bessel)函数 `Jn(int n, real)` 和 第二类贝塞尔函数 `Yn(int n, real)`，以及伽玛(gamma)函数 `gamma`、误差函数 `erf` 和互补误差函数 `erfc`。标准的 `real(real, real)` 函数 `atan2`、`hypot`、`fmod`、`remainder` 也包含在内。

函数 `degrees(real radians)` 和 `radians(real degrees)` 可用于弧度与角度之间的转换。函数 `Degrees(real radians)` 返回区间 $[0,360)$ 内的角度（以度为单位）。为简便起见，Asymptote 定义了标准三角函数的变体 `Sin`、`Cos`、`Tan`、`aSin`、`aCos` 和 `aTan`，它们使用角度而非弧度。我们还定义了 `sqrt`、`sin`、`cos`、`exp`、`log` 和 `gamma` 函数的复数版本。

函数 `floor`、`ceil` 和 `round` 与它们通常的定义不同，它们都返回一个 `int` 值而不是 `real`（因为这通常是人们想要的）。函数 `Floor`、`Ceil` 和 `Round` 分别与其类似，不同之处在于，如果结果不能转换为有效的 `int`，它们将为正参数返回 `intMax`，为负参数返回 `intMin`，而不是产生整数溢出。我们还定义了一个函数 `sgn`，它将其 `real` 参数的符号作为整数（-1、0 或 1）返回。

存在 `abs(int)` 函数，以及 `abs(real)` 函数（等价于 `fabs(real)`）、`abs(pair)` 函数（等价于 `length(pair)`）。

`Asymptote` 的随机数生成器可以使用 `srand(int)` 设置种子。如果向 `srand` 提供负参数，则使用内部熵源作为种子。随机数最初使用 `srand(-1)` 设定种子。函数 `int rand(int a=0, int b=randMax)` 返回区间 `[a,b]` 内的一个随机整数。`unitrand()` 函数返回均匀分布在区间 `[0,1)` 内的一个随机数。`randString(int len, string chars)` 函数返回长度为 `len`、由字符串 `chars` 中的字符组成的随机字符串。如果 `chars` 为空字符串，则使用字符串 `'abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789'`。在输出的每个位置，字符出现的概率与其在 `chars` 中出现的次数成正比。

模块 `stats` 中提供了高斯随机数生成器 `Gaussrand` 和一组统计例程（包括 `histogram`）。

还定义了函数 `factorial(int n)`（返回 $n!$ ）和 `choose(int n, int k)`（返回 $n!/(k!(n-k)!)$ ）。

当配置了 GNU 科学库 (GSL, 可从 <https://www.gnu.org/software/gsl/> 获取) 时, `Asymptote` 包含一个内部模块 `gsl`, 该模块定义了 Airy 函数 `Ai(real)`、`Bi(real)`、`Ai_deriv(real)`、`Bi_deriv(real)`、`zero_Ai(int)`、`zero_Bi(int)`、`zero_Ai_deriv`、贝塞尔(Bessel)函数 `I(int, real)`、`K(int, real)`、`j(int, real)`、`y(int, real)`、`i_scaled(int, real)`、`k_scaled(int, real)`、`J(real, real)`、`Y(real, real)`、`I(real, real)`、`K(real, real)`、`zero_J(real, int)`; 椭圆函数 `F(real, real)`、`E(real, real)` 和 `P(real, real)`; 雅可比(Jacobi)椭圆函数 `real[] snrndn(real, real)`; 指数/三角积分 `Ei`、`Si` 和 `Ci`; 勒让德(Legendre)多项式 `Pl(int, real)`; 以及黎曼(Riemann) zeta 函数 `zeta(real)`。例如, 计算 1.0 的正弦积分 `Si`:

```
import gsl;
write(Si(1.0));
```

`Asymptote` 还提供了一些通用数值例程:

```
real newton(int iterations=100, real f(real), real fprime(real), real x,
bool verbose=false);
```

使用牛顿-拉夫逊(Newton-Raphson)迭代法, 给定实值可微函数 `f` 的导数 `fprime` 和初始猜测值 `x`, 求解其根。若 `verbose=true`, 则打印每次迭代的诊断信息。如果迭代在达到最大允许循环次数 (`iterations`) 后仍然失败, 则返回 `realMax`。

```
real newton(int iterations=100, real f(real), real fprime(real), real x1,
real x2, bool verbose=false);
```

使用有界牛顿-拉夫逊二分法, 在区间 `[x1,x2]` 内 (在该区间上 `f` 的端点值异号), 给定其导数 `fprime`, 求解实值可微函数 `f` 的根。若 `verbose=true`, 则打印每次迭代的诊断信息。如果迭代在达到最大允许循环次数 (`iterations`) 后仍然失败, 则返回 `realMax`。

```
real simpson(real f(real), real a, real b, real acc=realEpsilon, real
dxmax=b-a)
```

使用自适应辛普森(Simpson)积分, 返回 `f` 从 `a` 到 `b` 的积分。

`cputime cputime()`

返回一个 `cputime` 结构体, 其中包含累积 CPU 时间, 细分为字段 `parent.user`、`parent.system`、`child.user` 和 `child.system`, 以及 `parent.clock` 中的累积挂钟时间, 单位均为秒。为方便起见, 增量字段 `change.user`、`change.system` 和 `change.clock` 表示自上次调用 `cputime()` 以来相应字段的变化。函数

```
void write(file file=stdout, string s="", cputime c,
           string format=cputimeformat, suffix suffix=none);
```

显示增量用户 `cputime`, 后跟 `\u`"; 增量系统 `cputime`, 后跟 `\s`"; 总用户 `cputime`, 后跟 `\U`"; 以及总系统 `cputime`, 后跟 `\S`。

6.13 数组

在内置类型或自定义类型后附加 `[]` 即可生成数组。数组 `A` 的数组元素 `i` 可以通过 `A[i]` 来访问。默认情况下, 尝试使用负索引访问或赋值数组元素会产生错误。读取超出数组长度的数组元素也会产生错误; 然而, 给超出数组长度的元素赋值会导致数组自动调整大小, 以容纳新元素。也可以使用整数数组 `B` 来对数组 `A` 建立索引: 数组 `A[B]` 是通过用数组 `B` 的连续元素索引数组 `A` 来形成的。存在一种用于遍历数组所有元素的便捷简写; 参见 `[array iteration]`, 24 页。

声明

```
real[] A;
```

将 `A` 初始化为一个空 (长度为零) 数组。空数组应与 `null` 数组区分开来。如果我们写

```
real[] A=null;
```

则 `A` 完全无法被解引用 (`null` 数组没有长度, 且不能被读取或赋值)。

数组可以像这样显式初始化:

```
real[] A={0,1,2};
```

显式数组初始化也可以以另一个数组的副本结尾:

```
real[] A={0,1};
```

```
real[] B={...A}; // Now B contains {0,1}.
```

```
real[] C={4 ...A}; // Now C contains {4,0,1}.
```

(附带说明: 如果没有空格, `4...A` 会被解析为 `4.0 ... A`, 从而导致语法错误。)

`Asymptote` 中的数组赋值进行浅拷贝: 仅复制指针 (如果修改了其中一份副本, 另一份也会随之改变)。

```
real[] A = {0,1,2};
```

```
real[] B = A;
```

```
B[0] = 3;
```

```
write(A[0]); // Outputs 3.
```

下面列出的 `copy` 函数提供了数组的深拷贝。

类型为 `T[]` 的每个数组 `A` 都拥有虚成员:

- `int length,`
- `bool cyclic,`
- `int[] keys,`
- `T push(T x),`
- `void append(T[] a),`

- `T pop()`,
- `void insert(int i, ... T[] x)`,
- `void delete(int i, int j=i)`,
- `void delete()`, 以及
- `bool initialized(int n)`。

成员 `A.length` 求值为数组的长度。设置 `A.cyclic=true` 表示数组索引应当取模当前的数组长度。从非空循环数组读取或向其写入绝不会引发越界错误或数组大小调整。

成员 `A.keys` 求值为一个整数数组, 按升序包含数组中已初始化条目的索引。因此, 对于所有条目都已初始化的长度为 `n` 的数组, `A.keys` 求值为 $\{0, 1, \dots, n-1\}$ 。每次对 `A.keys` 求值时, 都会生成一个新的键数组。

函数 `A.push` 与 `A.append` 将其参数附加到数组末尾, 而 `A.insert(int i, ... T[] x)` 则在索引 `i` 处将 `x` 插入数组。为了方便, `A.push` 返回推入的元素。函数 `A.pop()` 弹出并返回最后一个元素, 而 `A.delete(int i, int j=i)` 删除索引在范围 $[i, j]$ 内的元素, 并将所有更高索引元素的位置向下移动。如果不提供参数, `A.delete()` 提供了删除 `A` 中所有元素的便捷方式。例程 `A.initialized(int n)` 可用于检查索引 `n` 处的元素是否已初始化。与所有 `Asymptote` 函数一样, `push`、`append`、`pop`、`insert`、`delete` 和 `initialized` 都可以从数组中“剥离(pulled off)”出来独立使用。例如,

```
int[] A={1};
A.push(2);           // A now contains {1,2}.
A.append(A);         // A now contains {1,2,1,2}.
int f(int)=A.push;
f(3);                // A now contains {1,2,1,2,3}.
int g()=A.pop;
write(g());          // Outputs 3.
A.delete(0);         // A now contains {2,1,2}.
A.delete(0,1);       // A now contains {2}.
A.insert(1,3);       // A now contains {2,3}.
A.insert(1, ... A);  // A now contains {2,2,3,3}
A.insert(2,4,5);     // A now contains {2,2,4,5,3,3}.
```

`[]` 后缀也可以出现在变量名之后; 这在声明相同类型的变量和数组列表时有时很方便:

```
real a,A[];
```

这声明 `a` 的类型为 `real`, 并隐式声明 `A` 的类型为 `real[]`。

在以下内置数组函数列表中, `T` 代表泛型类型。注意, 依赖于类型 `T[]` 的内部函数 `alias`、`array`、`copy`、`concat`、`sequence`、`map` 和 `transpose`, 仅在首次声明类型为 `T[]` 的变量之后才会被定义。

`new T[]` 返回类型为 `T[]` 的一个新空数组;

`new T[] {list}`
 返回类型为 `T[]` 的新数组, 用 `list` (以逗号分隔的元素列表) 初始化;

`new T[n]` 返回类型为 `T[]`、包含 `n` 个元素的新数组。除非这 `n` 个数组元素本身就是数组 (在这种情况下, 它们各自被初始化为空数组), 否则它们不会被初始化;

`T[] array(int n, T value, int depth=intMax)`
 返回由 `n` 个 `value` 副本组成的数组。如果 `value` 本身是数组, 则对每个条目都进行 `value` 的深拷贝。如果指定了 `depth`, 则此深拷贝仅递归到指定层数;

```

int[] sequence(int n)
    如果  $n \geq 1$ , 返回数组  $\{0, 1, \dots, n-1\}$  (否则返回 null 数组);

int[] sequence(int n, int m)
    如果  $m \geq n$ , 返回数组  $\{n, n+1, \dots, m\}$  (否则返回 null 数组); 亦参见 6.16.1 节 [range], 83 页;

int[] sequence(int n, int m, int skip)
    如果  $(m-n)/skip \geq 0$ , 返回以步长 skip 跳跃的数组  $\{n, n+skip, \dots, m\}$  (否则返回 null 数组); 亦参见 6.16.1 节 [range], 83 页;

T[] sequence(T f(int), int n)
    如果  $n \geq 1$ , 根据给定函数 T f(int) 和整数 int n, 返回序列  $\{f_i : i=0, 1, \dots, n-1\}$  (否则返回 null 数组);

T[] map(T f(T), T[] a)
    返回将函数 f 应用于数组 a 的每个元素后获得的数组。这等价于 sequence(new T(int i) {return f(a[i]);}, a.length);

T2[] map(T2 f(T1), T1[] a)
    通过运行 from mapArray(Src=T1, Dst=T2) access map; 构造, 返回将函数 f 应用于数组 a 的每个元素后获得的数组;

int[] reverse(int n)
    如果  $n \geq 1$ , 返回数组  $\{n-1, n-2, \dots, 0\}$  (否则返回 null 数组);

int[] complement(int[] a, int n)
    返回整数数组 a 在  $\{0, 1, 2, \dots, n-1\}$  中的补集, 使得 b[complement(a, b.length)] 产生 b[a] 的补集;

real[] uniform(real a, real b, int n)
    如果  $n \geq 1$ , 返回 [a, b] 划分为 n 个子区间的均匀分割 (否则返回 null 数组);

int find(bool[] a, int n=1)
    返回布尔数组 a 中第 n 个 true 值的索引, 若未找到则返回 -1。如果 n 为负数, 则从数组末尾向前搜索第 -n 个值;

int[] findall(bool[] a)
    返回布尔数组 a 中所有 true 值的索引;

int search(T[] a, T key)
    对于内置有序类型 T, 在包含 n 个元素的已排序数组 a 中搜索 key: 若 a[i] <= key < a[i+1], 则返回索引 i; 若 key 小于 a 的所有元素, 则返回 -1; 若 key 大于或等于 a 的最后一个元素, 则返回 n-1;

int search(T[] a, T key, bool less(T i, T j))
    在按升序排序的数组 a 中搜索 key, 使得当 less(i, j) 为真时元素 i 排在元素 j 前面;

T[] copy(T[] a)
    返回数组 a 的深拷贝;

T[] concat(...T[] a)
    返回通过连接作为参数给出的给定一维数组而形成的新数组;

```

```
bool alias(T[] a, T[] b)
    如果数组 a 和 b 是同一数组, 则返回 true;

T[] sort(T[] a)
    对于内置有序类型 T, 返回按升序排序的 a 的副本;

T[][] sort(T[][] a)
    对于内置有序类型 T, 返回 a 的副本, 其行按第一列排序, 如果第一列相同则依次使用后面更高的列打破平局。例如:
    string[][] a={{ "bob", "9"}, {"alice", "5"}, {"pete", "7"},
                  {"alice", "4"};
    // Row sort (by column 0, using column 1 to break ties):
    write(sort(a));
    产生
    alice    4
    alice    5
    bob      9
    pete     7

T[] sort(T[] a, bool less(T i, T j), bool stable=true)
    返回按升序排序的 a 的副本, 使得当 less(i, j) 为真时元素 i 排在元素 j 前面;
    受限于稳定性约束 (如果 stable 为 true), 当 less(i, j) 与 less(j, i) 均为 false 时保留元素 i 和 j 的原始顺序;

T[][] transpose(T[][] a)
    返回 a 的转置;

T[][][] transpose(T[][][] a, int[] perm)
    返回对每个条目的索引应用 new int[] {0, 1, 2} 的置换 perm 后获得的 a 的三维转置;

T sum(T[] a)
    对于算术类型 T, 返回 a 的和。在 T 为 bool 的情况下, 返回 a 中 true 元素的数量;

T min(T[] a)
T min(T[][] a)
T min(T[][][] a)
    对于内置有序类型 T, 返回 a 的最小元素;

T max(T[] a)
T max(T[][] a)
T max(T[][][] a)
    对于内置有序类型 T, 返回 a 的最大元素;

T[] min(T[] a, T[] b)
    对于内置有序类型 T, 以及相同长度的数组 a 和 b, 返回由 a 和 b 对应元素的最小值组成的数组;

T[] max(T[] a, T[] b)
    对于内置有序类型 T, 以及相同长度的数组 a 和 b, 返回由 a 和 b 对应元素的最大值组成的数组;
```

```

pair[] pairs(real[] x, real[] y);
    对于相同长度的数组 x 和 y, 返回 pair 数组 sequence(new pair(int i) {return
    (x[i],y[i]);},x.length);

pair[] fft(pair[] a, int sign=1)
    使用给定的 sign 返回 a 的非归一化快速傅里叶变换 (Fast Fourier Transform, 前
    提是安装了可选的 FFTW 软件包)。下面是一个简单的示例:
    int n=4;
    pair[] f=sequence(n);
    write(f);
    pair[] g=fft(f,-1);
    write();
    write(g);
    f=fft(g,1);
    write();
    write(f/n);

pair[][] fft(pair[][] a, int sign=1)
    使用给定的 sign 返回 a 的非归一化二维傅里叶变换;

pair[][][] fft(pair[][][] a, int sign=1)
    使用给定的 sign 返回 a 的非归一化三维傅里叶变换;

realschur schur(real[][] a)
    返回一个包含酉矩阵 U 与准三角矩阵 T 的结构 realschur, 使得
    a=U*T*transpose(U);

schur schur(pair[][] a)
    返回一个包含酉矩阵 U 与三角矩阵 T 的结构 schur, 使得
    a=U*T*conj(transpose(U));

real dot(real[] a, real[] b)
    返回向量 a 和 b 的点积;

pair dot(pair[] a, pair[] b)
    返回向量 a 和 b 的复数点积 sum(a*conj(b));

real[] tridiagonal(real[] a, real[] b, real[] c, real[] f);
    求解周期性三对角问题  $Lx = f$  并返回解 x, 其中 f 是 n 维向量, L 是  $n \times n$  矩阵
    [ b[0] c[0]          a[0]   ]
    [ a[1] b[1] c[1]          ]
    [      a[2] b[2] c[2]      ]
    [      ...              ]
    [ c[n-1]      a[n-1] b[n-1] ]
    对于狄利克雷(Dirichlet)边界条件 (在此用 u[-1] 与 u[n] 表示), 将 f[0]
    替换为 f[0]-a[0]u[-1], 将 f[n-1] 替换为 f[n-1]-c[n-1]u[n]; 然后设置
    a[0]=c[n-1]=0;

real[] solve(real[][] a, real[] b, bool warn=true)
    通过 LU 分解求解线性方程  $ax = b$  并返回解 x, 其中 a 为  $n \times n$  矩阵, b 为长度
    为 n 的数组。例如:
    import math;

```

```

real[] [] a={{1,-2,3,0},{4,-5,6,2},{-7,-8,10,5},{1,50,1,-2}};
real[] b={7,19,33,3};
real[] x=solve(a,b);
write(a); write();
write(b); write();
write(x); write();
write(a*x);

```

如果 a 是奇异矩阵且 `warn` 为 `false`, 则返回一个空数组。如果矩阵 a 是三对角的, 例程 `tridiagonal` 提供了更高效的算法 (见 `[tridiagonal]`, 74 页);

```

real[] [] solve(real[] [] a, real[] [] b, bool warn=true)
    求解线性方程  $ax = b$  并返回解  $x$ , 其中  $a$  为  $n \times n$  矩阵,  $b$  为  $n \times m$  矩阵。如果
     $a$  是奇异矩阵且 warn 为 false, 则返回一个空矩阵;

```

```

real[] [] identity(int n);
    返回  $n \times n$  单位矩阵;

```

```

real[] [] diagonal(...real[] a)
    返回以  $a$  为对角元素的对角矩阵;

```

```

real[] [] inverse(real[] [] a)
    返回方阵  $a$  的逆矩阵;

```

```

real[] quadraticroots(real a, real b, real c);
    此数值健壮的求解器按升序返回二次方程  $ax^2 + bx + c = 0$  的实根。重根会被分别
    列出;

```

```

pair[] quadraticroots(explicit pair a, explicit pair b, explicit pair c);
    此数值健壮的求解器返回二次方程  $ax^2 + bx + c = 0$  的复根;

```

```

real[] cubicroots(real a, real b, real c, real d);
    此数值健壮的求解器返回三次方程  $ax^3 + bx^2 + cx + d = 0$  的实根。重根会被分别列
    出。

```

`Asymptote` 包含用于算术 (包括自运算) 和逻辑运算的一整套向量化数组指令。这些逐元素指令在 C++ 代码中实现以提高速度。给定

```

real[] a={1,2};
real[] b={3,2};

```

则 `a == b` 与 `a >= 2` 均求值为向量 `{false, true}`。要测试 a 和 b 的所有分量是否一致, 请使用布尔函数 `all(a == b)`。也可以使用诸如 `(a >= 2) ? a : b` 的条件表达式, 它返回数组 `{3,2}`, 或者 `write((a >= 2) ? a : null)`, 它返回数组 `{2}`。

所有签名为 `real(real)` 的标准内置 `libm` 函数也可以接受实数数组作为参数, 效果就像是对 `map` 的隐式调用。

与其他内置类型一样, 基本数据类型的数组可以通过赋值读入。在这个示例中, 代码

```

file fin=input("test.txt");
real[] A=fin;

```

读取实数值到 A 中, 直到到达文件末尾 (或发生 I/O 错误)。

文件的虚成员函数 `line()`、`word()`、`csv()`、`dimension()` 和 `read()` 用于限定数组读取, 而虚字段 `line`、`word`、`csv` 和 `dimension` 可用于查询这些相应的状态。

如果行模式通过 `file line(bool b=true)` 设置, 则一旦到达行末就会停止读取:

```
file fin=input("test.txt");
real[] A=fin.line();
```

由于字符串读取默认本身就会读取到行末, 因此行模式通常对字符串数组读取没有影响。然而, 有一个用于读取字符串的空白字符分隔符模式 `file word(bool b=true)`, 它使字符串读取遵循空白字符分隔符, 而不是默认的行末分隔符:

```
file fin=input("test.txt").line().word();
real[] A=fin;
```

另一种有用的模式是逗号分隔值模式 `file csv(bool b=true)`, 它使读取遵循逗号分隔符:

```
file fin=input("test.txt").csv();
real[] A=fin;
```

要限制读取值的数量, 可以使用虚成员函数 `file dimension(int)`:

```
file fin=input("test.txt");
real[] A=fin.dimension(10);
```

这会将 10 个值读取到 A 中, 除非首先到达文件末尾 (或行模式下的行末)。尝试超出文件末尾读取将产生运行时错误消息。为整数限制指定值 0 等价于前面的示例: 一直读取直到遇到文件末尾 (或行模式下的行末)。

基本数据类型的二维和三维数组可以像这样读入:

```
file fin=input("test.txt");
real[] [] A=fin.dimension(2,3);
real[] [] [] B=fin.dimension(2,3,4);
```

有时, 数组维度作为整数字段与数据一起存储在数组的开头。这种一维、二维或三维数组可以分别使用虚成员函数 `read(1)`、`read(2)` 或 `read(3)` 读入:

```
file fin=input("test.txt");
real[] A=fin.read(1);
real[] [] B=fin.read(2);
real[] [] [] C=fin.read(3);
```

基本数据类型的一维、二维和三维数组可以分别使用函数 `write(file,T[])`、`write(file,T[][])` 和 `write(file,T[][][])` 输出。

6.13.1 切片

Asymptote 允许使用类似 Python 的语法将数组的一部分作为切片寻址。如果 A 是一个数组, 表达式 `A[m:n]` 返回一个新数组, 由索引从 m 开始直至 (但不包括) n 的 A 的元素组成。例如,

```
int[] x={0,1,2,3,4,5,6,7,8,9};
int[] y=x[2:6]; // y={2,3,4,5};
int[] z=x[5:10]; // z={5,6,7,8,9};
```

如果省略左索引, 则取为 0。如果省略右索引, 则取为数组的长度。如果两者都省略, 则切片将从数组的开头一直延伸到末尾, 生成该数组的非循环深拷贝。例如:

```
int[] x={0,1,2,3,4,5,6,7,8,9};
int[] y=x[:4]; // y={0,1,2,3}
int[] z=x[5:]; // z={5,6,7,8,9}
int[] w=x[:]; // w={0,1,2,3,4,5,6,7,8,9}, distinct from array x.
```

如果 A 是非循环数组，则任一索引使用负值都是不合法的。然而，如果索引超过数组长度，它们会被适度截断为该长度。

对于循环数组，切片 A[m:n] 仍然由索引在集合[m,n) 中的单元组成，但现在允许负值和超出数组长度的值。索引只是简单地循环折返。例如：

```
int[] x={0,1,2,3,4,5,6,7,8,9};
x.cyclic=true;
int[] y=x[8:15]; // y={8,9,0,1,2,3,4}.
int[] z=x[-5:5]; // z={5,6,7,8,9,0,1,2,3,4}
int[] w=x[-3:17]; // w={7,8,9,0,1,2,3,4,5,6,7,8,9,0,1,2,3,4,5,6}
```

注意，对于循环数组，在一个切片内可以多次包含原数组的同一个元素。无论原数组如何，由切片生成的数组总是非循环的。

如果切片的左索引和右索引相同，结果是一个空数组。如果被切片的数组为空，结果也是一个空数组。任何左索引大于右索引的切片都会引发错误。

切片也可以被赋值，从而改变原数组的值。如果被赋值给切片的数组与切片本身长度不同，将从数组中插入或删除元素以适应赋值。例如：

```
string[] toppings={"mayo", "salt", "ham", "lettuce"};
toppings[0:2]=new string[] {"mustard", "pepper"};
// Now toppings={"mustard", "pepper", "ham", "lettuce"}
toppings[2:3]=new string[] {"turkey", "bacon" };
// Now toppings={"mustard", "pepper", "turkey", "bacon", "lettuce"}
toppings[0:3]=new string[] {"tomato"};
// Now toppings={"tomato", "bacon", "lettuce"}
```

如果将一个数组赋值给它自己的切片，则会将原数组的副本赋值给该切片。也就是说，诸如 x[m:n]=x 的代码等价于 x[m:n]=copy(x)。可以使用简写 x[m:m]=y 将数组 y 的内容插入到数组 x 中紧接在 x[m] 之前的位置。

对于循环数组，如果切片寻址到的单元到达数组末尾，然后继续寻址数组开头的单元，则该切片是跨越式的 (bridging)。例如，如果 A 是长度为 10 的循环数组，A[8:12]、A[-3:1] 和 A[5:25] 是跨越式切片，而 A[3:7]、A[7:10]、A[-3:0] 和 A[103:107] 则不是。只有当切片中的元素数量与赋给它的元素数量完全相等时，跨越式切片才能被赋值。否则，没有明确的方法来决定哪个新条目应当是 A[0]，并会报告错误。非跨越式切片可以被赋予任意长度的数组。

对于循环数组 A，形式为 A[A.length:A.length] 的表达式等价于表达式 A[0:0]，因此给该切片赋值将在数组开头插入值。A.append() 可用于在数组末尾插入值。

对重复包含任何单元的循环数组切片进行赋值是不合法的。

6.14 类型转换

Asymptote 会隐式地将 int 转换为 real、int 转换为 pair、real 转换为 pair、pair 转换为 path、pair 转换为 guide、path 转换为 guide、guide 转换为 path、real 转换为 pen、pair[] 转换为 guide[]、pair[] 转换为 path[]、path 转换为 path[] 以及 guide 转换为 path[]，此外还有模块 three 中定义的各种三维类型转换。在赋值以及尝试将函数调用与可能的函数签名进行匹配时，会自动尝试隐式类型转换。可以通过在函数签名中将单个参数声明为 explicit 来禁止隐式转换，例如为了在以下示例中避免有歧义的函数调用（该示例输出 o）：

```
int f(pair a) {return 0;}
```

```
int f(explicit real x) {return 1;}
```

```
write(f(0));
```

其他转换, 例如 `real` 转换为 `int` 或 `real` 转换为 `string`, 则需要显式转换:

```
int i=(int) 2.5;
string s=(string) 2.5;
```

```
real[] a={2.5,-3.5};
int[] b=(int []) a;
write(stdout,b);      // Outputs 2,-3
```

在从字符串转换为类型 `T` 失败的情况下, 会返回一个未初始化的变量; 这一状况可以通过函数 `bool initialized(T)`; 检测到:

```
int i=(int) "2.5";
assert(initialized(i),"Invalid cast.");
```

```
real x=(real) "2.5a";
assert(initialized(x),"Invalid cast.");
```

也可以使用 `operator cast` 转换为用户自定义类型:

```
struct rpair {
    real radius;
    real angle;
}

pair operator cast(rpair x) {
    return (x.radius*cos(x.angle),x.radius*sin(x.angle));
}
```

```
rpair x;
x.radius=1;
x.angle=pi/6;
```

```
write(x);      // Outputs (0.866025403784439,0.5)
```

定义新的转换运算符时必须特别小心。假设在某些代码中, 希望所有整数都表示 100 的倍数。要将它们转换为 `real`, 首先会想要将它们乘以 100。然而, 这种直观的实现

```
real operator cast(int x) {return x*100;}
```

等同于无限递归, 因为结果 `x*100` 本身也需要从整数转换为 `real`。相反, 我们应该使用 `int` 到 `real` 的标准转换:

```
real convert(int x) {return x*100;}
real operator cast(int x)=convert;
```

显式转换的实现方式类似, 使用的是 `operator ecast`。

6.15 导入

虽然 `Asymptote` 默认提供了许多功能, 但某些应用需要包含在外部 `Asymptote` 模块中的专门功能。例如, 代码行

```
access graph;
graph.axes();
```

能在二维图表上绘制 x 轴与 y 轴。在这里，该命令在全局模块字典中按名称 `graph` 查找模块，并将其存入名为 `graph` 的新变量中。该模块是一个结构体，我们可以像平时使用结构体一样引用它的字段。

我们常常希望使用模块函数而无需指定模块名称。代码

```
from graph access axes;
```

会将 `graph` 的 `axes` 字段添加到本地命名空间中，从而随后只需写入 `axes()` 即可。如果给定的名称被重载，该名称对应的所有类型和变量都会被添加。要添加多个名称，只需使用以逗号分隔的列表：

```
from graph access axes, xaxis, yaxis;
```

可以使用通配符记号将模块的所有非私有字段和类型添加到本地命名空间中：

```
from graph access *;
```

类似地，可以使用 `unravel` 关键字将结构体的非私有字段和类型添加到本地环境中：

```
struct matrix {
    real a,b,c,d;
}
```

```
real det(matrix m) {
    unravel m;
    return a*d-b*c;
}
```

或者，可以解开(`unravel`)选定的字段：

```
real det(matrix m) {
    from m unravel a,b,c as C,d;
    return a*d-b*C;
}
```

命令

```
import graph;
```

是下述命令的便捷简写：

```
access graph;
unravel graph;
```

也就是说，`import graph` 首先将模块加载到名为 `graph` 的结构体中，然后将其非私有字段和类型添加到本地环境中。这样，如果某个成员变量（或函数）被本地变量（或具有相同签名的函数）覆盖，仍然可以通过模块名限定来访问原始的成员。

使用通配符导入在大多数情况下都能正常工作，但人们通常无法了解模块内部所有的类型和变量，而且随着模块编写者添加或更改模块功能，这些内容也可能发生改变。因此，谨慎的做法是在 `Asymptote` 文件的开头添加 `import` 命令，这样导入的名称就不会遮蔽(`shadow`)本地定义的函数。尽管如此，导入的名称仍可能根据导入顺序遮蔽其他导入的名称；如果导入的函数与随后定义的本地函数同名，还可能会引发重载决议(`overloading resolution`)问题。

要在将模块或字段添加到本地环境时对其进行重命名，请使用 `as`：

```
access graph as graph2d;
from graph access xaxis as xline, yaxis as yline;
```

命令

```
import graph as graph2d;
```

是下述命令的便捷简写：

```
access graph as graph2d;
unravel graph2d;
```

除少数内置模块（例如 `settings`）外，所有模块均实现为 `Asymptote` 文件。在查找尚未加载的模块时，`Asymptote` 会在标准搜索路径（见 2.5 节 [Search paths], 5 页）中搜索匹配的文件。与该名称对应的文件将被读取，其内部的代码会被解释为定义该模块的结构体主体。

如果文件包含在子目录中，则可以用以点分隔的名称指定目录结构。例如，命令

```
import directoryName.moduleName;
access directoryName.moduleName;
```

将把文件 `directoryName/moduleName.asy` 作为模块 `moduleName` 加载。同样地，

```
from directoryName.moduleName access functionName;
```

将访问 `directoryName/moduleName.asy` 中的 `functionName`。目录 `directoryName` 是相对于当前目录、`base` 目录或搜索路径（见 2.5 节 [Search paths], 5 页）中的任何目录的。只要目录名和模块名同时也是合法的变量名，就允许嵌套目录规范。否则（例如如果目录是绝对路径），文件名必须用引号括起来：

```
import '/usr/local/share/asymptote/graph.asy' as graph;
```

如果编译 `Asymptote` 时启用了对 `libcurl` 的支持，文件名甚至可以是一个 URL：import `'https://raw.githubusercontent.com/vectorgraphics/asymptote/HEAD/doc/axis3.asy'` as `axis3`;

模块自导入（或互相循环导入）属于错误。要导入的模块名称必须在编译时已知。

不过，你可以在运行时导入由字符串 `s` 决定的 `Asymptote` 模块，如下所示：

```
eval('import '+s,true);
```

要条件式地执行 `asy` 文件数组，请使用

```
void asy(string format, bool overwrite, ... string[] s);
```

仅当 `overwrite` 为 `true` 或输出文件缺失时，才会使用输出格式 `format` 来处理该文件。

可以通过以下函数求值包含在字符串 `s` 中的 `Asymptote` 表达式（但没有任何返回值）：

```
void eval(string s, bool embedded=false);
```

无需以分号终止字符串 `s`。如果 `embedded` 为 `true`，该字符串将在当前环境的顶层被求值。如果 `embedded` 为 `false`（默认值），该字符串将在独立环境中被求值，共享同一个 `settings` 模块（见 [settings], 185 页）。

可以使用以下命令对任意 `Asymptote` 代码（其中可能包含未转义的引号）求值：

```
void eval(code s, bool embedded=false);
```

这里的 `code` 是一种特殊类型，与 `quote {}` 配合使用来包裹 `Asymptote` code，如下所示：

```
real a=1;
code s=quote {
    write(a);
};
eval(s,true);           // Outputs 1
```

要逐字包含已有文件 `graph` 的内容（如同将该文件的内容直接插入该位置一样），请使用以下形式之一：

```
include graph;
include '/usr/local/share/asymptote/graph.asy';
```

要列出由字符串 `s` 的内容命名的模块中定义的所有全局函数和变量，请使用函数 `void list(string s, bool imports=false);`；如果 `imports` 为 `true`，则导入的全局函数和变量也会被列出。

6.15.1 模板化导入

在 `Asymptote` 中，类型是在导入模块时指定的。任何此类模块的首行可执行代码必须采用 `typedef import(<types>)` 的形式，其中 `<types>` 是所需的类型参数列表。例如，

```
typedef import(T, S, Number);
```

可以是一个需要三个类型参数的模块的第一行。模块中其余的代码随后就可以将 `T`、`S` 和 `Number` 当作类型使用。

要导入这样的模块，必须指定要使用的类型。例如，如果上述模块名为 `templatedModule`，可以使用如下导入命令针对类型 `string`、`int[]` 和 `real` 访问它：

```
access templatedModule(T=string, S=int[], Number=real)
    as templatedModule_string_int_real;
```

请注意，这实际上是一条 `access` 命令而非 `import` 命令，因此定义在 `templatedModule.asy` 中的类型、函数或变量 `A` 需要通过修饰名 `templatedModule_string_int_real.A` 来访问。

或者，可以通过如下命令导入该模块：

```
from templatedModule(T=string, S=int[], Number=real) access
    Wrapper_Number as Wrapper_real,
    operator ==;
```

该命令会自动将 `Wrapper_Number` 重命名为 `Wrapper_real`，并且还允许使用模块中定义的任何 `operator ==` 重载。

更多示例可以在 `Asymptote` 源码目录的 `tests/template` 子目录中找到。

已知问题：某些预期的运算符（例如 `operator ==`）可能仅适用于内置类型或定义在模块 `plain` 中的类型实参。

6.16 迭代器

如果结构体 `a` 拥有成员函数 `operator iter()`，代码

```
for(var i : a) {
    <statements>
}
```

是以下代码的语法糖：

```
for(var it=a.operator iter(); it.valid(); it.advance()) {
    var i=it.get();
    <statements>
}
```

因此，我们可以通过定义一个返回包含以下方法的对象的 `operator iter` 方法，来使结构体可迭代：

`T get()` 返回迭代器当前位置的值（不改变位置）。注意 `T` 可以是任何类型，内置类型或用户定义类型；

`void advance()`
将迭代器推进到下一个位置；

`bool valid()`
如果迭代器处于有效位置则返回 `true`，如果迭代器已推进超过最后一个元素则返回 `false`。

尽管理论上 `operator iter` 的返回类型可以是任何包含这三个方法的类型，但强烈建议使用在模板化模块 `collections.iter(T)` 中定义的 `Iter_T` 结构体，以便其他实用程序（见 6.20.2 节 [Iterators and utilities], 88 页）可以与该迭代器配合使用。此结构体的三个方法可以作为字段来设置。

作为一个示例，以下是用于字符串数组的偶数索引元素的迭代器：

```
from collections.iter(T=string) access
    Iter_T as Iter_string,
    Iterable_T as Iterable_string;
struct EvenStrings {
    string[] a;
    void operator init(string[] a) {
        this.a=a;
    }
    Iter_string operator iter() {
        int i = 0;
        Iter_string result;
        result.get = new string() { return a[i]; };
        result.advance = new void() { i += 2; };
        result.valid = new bool() { return i < a.length; };
        return result;
    }
    autounravel Iterable_string operator cast(EvenStrings es) {
        Iterable_string result;
        result.operator iter = es.operator iter;
        return result;
    }
}
string[] a = {'a', 'b', 'c', 'd', 'e', 'f'};
for (string s : EvenStrings(a)) {
    write(s);
}
```

运行此代码将打印出以下内容：

```
a
c
e
```

自动展开（见 6.18 节 [Autounravel], 85 页）隐式转换到 `Iterable_string` 并不是严格必需的，但它允许使用可迭代对象实用程序；见 6.20.2 节 [Iterators and utilities], 88 页。这使得能够编写如下代码，该代码使用 `enumerate` 实用程序将计数器附加到迭代的字符串上：

```
from collections.enumerate(T=string) access enumerate;
for (var kv : enumerate(EvenStrings(a))) {
    write(string(kv.k) + ": " + kv.v);
}
```

运行此代码将打印出以下内容：

```
0: a
1: c
2: e
```

6.16.1 range

以下函数产生整数上的迭代器：

```
Iterable_int range(int n)
    返回一个产生整数 0, ..., n-1 的可迭代对象；

Iterable_int range(int n, int m, int skip=1)
    如果 (m-n)/skip >= 0, 返回一个产生整数 n, n+skip, ..., m 的可迭代对象；否则不产生整数。
```

一般而言，`for (int i : range(n))` 等价于 `for (int i : sequence(n))`。后者使用 $O(n)$ 内存，但通常更快。前者使用 $O(1)$ 内存，但较慢。不过，如果明确需要迭代器，则 `range(n)` 比 `range(sequence(n))` 更快（且具有更高的内存效率）。

6.17 Static 静态

静态(static)限定符将变量的内存地址分配在更高的外围层级中。

对于函数体，变量分配在定义该函数的块中；因此在代码

```
struct s {
    int count() {
        static int c=0;
        ++c;
        return c;
    }
}
```

中，每个 `s` 对象（而不是每次调用 `count`）都存在变量 `c` 的一个实例。

类似地，在

```
int factorial(int n) {
    int helper(int k) {
        static int x=1;
        x *= k;
        return k == 1 ? x : helper(k-1);
    }
    return helper(n);
}
```

中，每次调用 `factorial`（而不是每次调用 `helper`）都存在一个 `x` 实例，因此这是一个正确但很丑陋的阶乘实现。

类似地，在结构体内部声明的静态变量在定义该结构体的块中分配。因此，

```
struct A {
    struct B {
        static pair z;
    }
}
```

会为创建的每个 `A` 类型对象创建一个 `z` 对象。

在这个例子中，

```
int pow(int n, int k) {
    struct A {
        static int x=1;
        void helper() {
            x *= n;
        }
    }
    for(int i=0; i < k; ++i) {
        A a;
        a.helper();
    }
    return A.x;
}
```

每次调用 `pow` 都存在一个 `x` 实例，因此这是一个很丑陋的求幂实现。

循环结构在每次迭代中都会分配一个新帧。这样高阶函数就可以引用循环特定迭代中的变量：

```
void f();
for(int i=0; i < 10; ++i) {
    int x=i;
    if(x==5) {
        f=new void() {write(x);};
    }
}
f();
```

在这里，循环的每次迭代都有自己的变量 `x`，因此 `f()` 将写入 5。如果在循环中将变量声明为静态(`static`)，则它将在定义外围函数或结构体的地方分配（就如同它在循环外部声明为静态一样）。例如，在：

```
void f() {
    static int x;
    for(int i=0; i < 10; ++i) {
        static int y;
    }
}
```

中，`x` 和 `y` 都将被分配在同一个地方，这也是 `f` 被分配的地方。

语句也可以声明为静态(static)，在这种情况下，它们会在定义外围函数或结构体的地方运行。未包围在函数或结构体定义中的声明或语句本身就已经处于顶层，因此静态修饰符没有任何意义。在这种情况下会给出警告。

由于结构体可以拥有静态字段，对于限定名称而言，限定符是变量还是类型并不总是清晰的。例如，在：

```
struct A {
    static int x;
}
pair A;

int y=A.x;
```

中，A.x 中的 A 是指结构体还是指数对(pair)变量。Asymptote 中的惯例是，如果存在与限定符同名的非函数变量，则限定符指该变量，而非类型。这与该变量实际拥有什么字段无关。

6.18 Autounravel 自动展开

autounravel (自动展开) 修饰符可用于自动解开 (unravel) 字段。在构建对结构体进行操作的关联函数库时，这很有用。例如，考虑在 rational.asy 中定义的 rational 结构体的简易实现：

```
struct rational {
    int p=0, q=1;
    void operator init(int p, int q) {
        this.p=p;
        this.q=q;
    }
}

rational operator +(rational a, rational b) {
    return rational(a.p*b.q+b.p*a.q, a.q*b.q);
}
```

为了允许将 rational 用作添加 rational 实例的模板化导入的类型参数，我们应该将 operator + 移入 rational 结构体的主体中，并添加 autounravel 修饰符：

```
struct rational {
    int p=0, q=1;
    void operator init(int p, int q) {
        this.p=p;
        this.q=q;
    }
    autounravel rational operator +(rational a, rational b) {
        return rational(a.p*b.q+b.p*a.q, a.q*b.q);
    }
}
```

这与之前的代码几乎等价，但现在只要 rational 结构体可访问，+ 运算符就将可访问。

目前，类型不能被自动展开。如果用户遇到认为此功能有用的情况，可以在 <https://github.com/vectorgraphics/asymptote/issues> 提交功能请求。

6.18.1 字段自动展开的位置

如果 `struct` 包含使用 `autounravel` 声明的字段（包括函数），这些字段将在以下位置从 `struct` 中解开：

- `struct` 定义的末尾；
- `typedef import` 语句，如果该 `struct` 是某个类型参数的实参；
- 从模块或外层 `struct` 解开该 `struct` 的 `unravel` 或 `access` 语句（例如，`from rational access rational`；将使 `rational.asy` 中定义的 `rational struct` 中的 `+` 运算符可用）；
- 重命名 `struct` 的 `typedef` 语句（如 `typedef rational.rational rat`；）。

6.18.2 autounravel 合法的位置

`autounravel` 修饰符隐含了 `static`，并且可以用于与 `static` 许多相同的场所。然而，在模块的顶层（在任何结构体或函数外部）指定 `autounravel` 是一项错误，而 `static` 仅给出一个警告。¹ 在 `struct` 定义或 `typedef` 语句的前面禁用 `autounravel`，因为类型不能被自动展开。虽然 `static static` 会导致错误，但 `static autounravel` 和 `autounravel static` 都是合法的，并且效果与单独使用 `autounravel` 完全相同。

6.19 哈希函数

要将对象用作哈希映射（见 6.20.3 节 [Maps], 91 页）中的键或哈希集合（见 6.20.5 节 [Sets], 94 页）中的成员，它必须具有一个返回非负整数的方法 `int hash()`。为了确保正确性，此方法对于根据 `==` 运算符相等的同类型两个对象必须返回相同的值。为了保证效率，它的行为应当类似于伪随机函数，使得不同对象的哈希值互不相关。还要注意，哈希函数在程序的不同运行之间不必保持一致；`a.hash() == a.hash()` 应当始终为 `true`，但每次运行程序时 `write(a.hash())` 可能会产生不同的输出。

在 `Asymptote` 中，为三种内置类型定义了 `hash` 方法：`int`、`real` 和 `string`。请注意，类似于 `3.hash()` 和 `'hello'.hash()` 的形式不是有效语法；这些形式的正确写法应当是 `(3).hash()` 和 `('hello').hash()`，即在字面量周围加上括号。如果想对 `real` 进行哈希计算，有两个需要注意的重点：第一，即使两个 `real` 非常接近，它们的哈希值也几乎肯定不会接近；第二，同一个数值在作为 `int` 与作为 `real` 进行哈希计算时，几乎肯定会具有不同的哈希值。

6.19.1 为用户定义结构体计算哈希

此外，还有一个函数 `hash(int[])`，在为用户定义的结构体构建哈希方法时非常有用：将所有字段转换为整数（可能通过对它们求哈希值），将这些整数放入一个数组中，然后对该数组求哈希值。

让我们回到来自 6.9 节 [Structures], 57 页, 的 `Person` 结构体示例，添加一个 `age` 字段使其更有趣。对于该结构体，我们有两种合理的方式为其定义哈希函数。第一种是基于结构体的所有字段创建一个哈希函数：

```
struct Person {
    string firstname;
    string lastname;
```

¹ 如果允许顶层的 `autounravel`，用户可能会错误地认为每当 `access` 该模块时，字段都会被解开。允许在顶层使用 `static` 修饰符是因为虽然它什么也不做，但它不会误导用户。

```

int age;
int hash() {
    return hash(new int[] {firstname.hash(), lastname.hash(), age});
}
autounravel bool operator ==(Person a, Person b) {
    if (alias(a, null)) return alias(b, null);
    if (alias(b, null)) return false;
    return a.firstname == b.firstname &&
           a.lastname == b.lastname &&
           a.age == b.age;
}
autounravel bool operator !=(Person a, Person b) {
    return !(a == b);
}
}

```

如果我们正在重定义 == 运算符，这种方法是必要的。如果两个 `Person` 对象具有相同的名字、姓氏和年龄，则它们相等。

用户定义结构体的默认 == 运算符认为两个对象如果是同一个对象则相等，这意味着对其中一个对象的更改将反映在另一个对象中。能与默认 == 运算符良好配合的哈希方法，即使字段发生改变，也必须保持不变。在这种情况下，我们可以添加一个永不改变的 `id` 字段，该字段唯一标识一个 `Person` 对象，并将其用于哈希计算：

```

struct Person {
    string firstname;
    string lastname;
    int age;
    private static int lastID=-1;
    restricted int id = ++lastID;
    int hash() {
        return id.hash();
    }
}

```

内置哈希函数旨在恰好应用一次时消除不良的规律性（相对于模算术而言）。这就解释了为什么第二个例子返回 `id.hash()` 而不是 `id`，而在第一个例子中 `age` 没有被独立求哈希值。有时将哈希函数的输出反馈回自身可能是最切合实际的做法，如上面的 `firstname` 和 `lastname` 那样，但这不会提高哈希质量；在理论上，如果我们能将 `Person` 对象序列化为字符串然后对字符串求哈希值，哈希质量还会更高。

6.20 集合容器(Collections)

`collections` 子目录通过一系列模板化模块提供了许多有用的容器与工具。

哈希映射(hashmap，有时也称为字典(dictionary)或关联数组(associative array))的相关信息参见见 6.20.3 节 [Maps], 91 页；可以说，它是 `collections` 模块中最有用的容器。

6.20.1 数对(Pairs)

Asymptote 内置了一种用于 real 数对的类型。要获得由任意两种类型组成的数对类型，可以使用 `collections.genericpair(K,V)` 模板化模块。其字段名为 `k` 和 `v`，而不是 `x` 和 `y`。该数对类型名为 `Pair_K_V`，建议用户根据所用类型对其重命名。此外还有一个函数 `makePair`，它由两个值创建一个数对；由于函数重载的存在，函数名无需指明这两个类型。

下面是一个例子：

```
from collections.genericpair(K=int, V=string) access
    Pair_K_V as Pair_int_string,
    makePair;
var p=makePair(3, 'hello');
write(p.k); // Outputs 3
write(p.v); // Outputs hello
var q = makePair(4, 'hello');
var s = makePair(3, 'world');
Pair_int_string t = makePair(3, 'hello');
assert(p != q);
assert(p != s);
assert(p == t);
```

注意，数对重载了 `==` 和 `!=` 运算符，重载依据是数对中两种类型各自的 `==` 运算符。由此带来的一个不幸后果是：该模块无法用于由数组构成的数对，因为数组的 `==` 运算符返回的是 `bool[]` 而非 `bool`。变通办法参见 6.20.8 节 [Wrapping arrays], 100 页。

`Pair_K_V` 类型还有一个初始化为 `null` 的 `int hash()` 方法。如果想把数对用作哈希映射中的键，就应当定义该方法，大致像这样：

```
from collections.genericpair(K=int, V=string) access
    Pair_K_V as Pair_int_string;
Pair_int_string p = makePair(3, 'hello');
p.hash = new int() {
    return hash(new int[] {p.k, p.v.hash()});
};
```

更多信息参见 6.19 节 [Hash functions], 86 页。

6.20.2 迭代器与工具

6.20.2.1 collections.iter(T)

`collections.iter(T)` 模板化模块提供了 `Iter_T` 和 `Iterable_T` 两个结构体，可以方便地用于定义迭代器(iterator)。此外，还有若干用于操作这些类型的迭代器的工具。严格说来，语法 `for(T x : a)` 并不依赖本模块的任何内容，但若干工具仅适用于 `Iterable_T` 类型的可迭代对象。

`Iter_T` 结构体具有以下字段，默认情况下它们的值都是 `null`：

- `T get()`
- `void advance()`
- `bool valid()`

`Iter_T` 旨在用作 `operator iter` 方法的返回类型；关于如何使用这些字段的更多信息，参见 6.16 节 [Iterators], 81 页。

`Iterable_T` 是一个泛型结构体，表示任何具有 `operator iter` 方法（返回 `Iter_T`）的对象。能够出现在基于范围的 `for` 循环内部的类型正是这一结构体，而不是 `Iter_T`。因此，下面的大多数工具操作的是 `Iterable_T` 的实例而非 `Iter_T`。

对于定义了 `operator iter` 方法的结构体，良好的做法是同时定义一个到 `Iterable_T` 的隐式转换，以便更好地兼容本模块中的工具。此外，`Iterable_T` 的实例 `a` 可以通过语法 `(T[]) a` 转换为 `T[]`。（这是显式转换，因此需要括号。）反过来，只要相应的 `Iterable_T` 类型已被 `access`（见 6.18 节 [Autounravel], 85 页），`T[]` 就可以隐式转换为 `Iterable_T`（无需括号）。这一转换也可以用下面列出的 `range` 函数显式完成。

`collections.iter` 模块提供以下函数：

`Iterable_T Iterable(Iter_T iter())`

创建一个 `Iterable_T`，其 `operator iter` 方法被设置为 `iter`（每当 `Iterable_T` 被 `access` 时都会自动展开）。本函数是构造函数 `Iterable_T(Iter_T iter())` 的别名。

`Iterable_T range(T[] items)`

创建一个遍历 `items` 中各元素的 `Iterable_T`。对 `operator iter` 的每次单独调用都会返回相互独立的迭代器。

`Iter_T Iter_T(T[] items)`

创建一个 `Iter_T`，它遍历 `items` 中的各个元素。

6.20.2.2 合并(zip)迭代器

`collections.zip2(K,V)` 模板化模块提供了一种在单个 `for` 循环中遍历两个类型可能不同的容器的方法，例如：

```
from collections.zip2(K=int, V=string) access zip, operator cast;
int[] x = {0, 1, 2, 3};
string[] y = {'0', '1', '2', '3'};
for (var ab : zip(x, y)) {
    assert(string(ab.k) == ab.v);
}
```

其中

`Iterable_Pair_K_V zip(Iterable_K a, Iterable_V b,
Pair_K_V keyword default=null)`

返回一个可迭代对象，它产生由 `a` 和 `b` 的元素组成的数对。若未提供 `default`，则当 `a` 或 `b` 二者之一耗尽元素时，该可迭代对象即停止。若提供了 `default`，返回的可迭代对象的长度将等于 `a` 与 `b` 中较长者的长度，任何缺失的元素都将使用默认值。

此外，该模块还导出以下类型，以及它们的所有 `autounravel`：

类型	来自
<code>Pair_K_V</code>	<code>collections.genericpair(K,V)</code>
<code>Iterable_K</code>	<code>collections.iter(T=K)</code>
<code>Iterable_V</code>	<code>collections.iter(T=V)</code>
<code>Iterable_Pair_K_V</code>	<code>collections.iter(T=Pair_K_V)</code>
<code>Iter_K</code>	<code>collections.iter(T=K)</code>
<code>Iter_V</code>	<code>collections.iter(T=V)</code>
<code>Iter_Pair_K_V</code>	<code>collections.iter(T=Pair_K_V)</code>

`collections.zip(T)` 模板化模块提供了一种在单个 `for` 循环中遍历多个容器的方法。它与 `collections.zip2(K,V)` 相似，但会把各个可迭代对象浓缩成对数组的遍历，而非对数对的遍历。这要求所有可迭代对象都遍历同一类型（例如，全部是字符串迭代器，或全部是整数迭代器）：

```
from collections.zip(T=int) access zip;
int[] x = {0, 1, 2, 3};
int[] y = {0, 1, 2};
int[] z = {0, 1, 2, 3, 4};
for (int[] abc : zip(default=-1, x, y, z)) {
    write(string(abc[0]) + '\t' + string(abc[1]) + '\t' + string(abc[2]));
}
```

其输出为

```
0      0      0
1      1      1
2      2      2
3      -1     3
-1     -1     4
```

`collections.zip(T)` 模块导出以下类型

类型	来自
<code>Iterable_array_T</code>	<code>collections.iter(T=T[])</code>
<code>Iter_array_T</code>	<code>collections.iter(T=T[])</code>
<code>Iterable_T</code>	<code>collections.iter(T)</code>
<code>Iter_T</code>	<code>collections.iter(T)</code>

并定义以下函数（它们都名为 `zip`）：

`Iterable_array_T zip(...Iterable_T[] iterables)`

把一个或多个由 `T` 组成的可迭代对象 `zip` 到一起。如果 `a` 是结果中的一个条目，那么 `a[0]` 来自第一个可迭代对象，`a[1]` 来自第二个，依此类推。如果这些可迭代对象长度不全相同，结果的长度将等于最短者的长度。

`Iterable_array_T zip(T keyword default ...Iterable_T[] iterables)`

把一个或多个由 `T` 组成的可迭代对象 `zip` 到一起。结果的长度将等于最长可迭代对象的长度，缺失的元素以默认值填充。

`T[][] zip(...T[][] arrays)`

把一个或多个数组 `zip` 到一起。如果这些数组长度不全相同，结果的长度将等于最短数组的长度。

`T[][] zip(T keyword default ...T[][] arrays)`

把一个或多个数组 `zip` 到一起。结果的长度将等于最长数组的长度，缺失的元素以默认值填充。这基本上等价于先应用 `transpose`，再将任何未初始化的元素替换为默认值。

6.20.2.3 为迭代器附加计数器

`collections.enumerate(T)` 模块提供了一种为迭代器附加计数器的方法。有两个函数，都名为 `enumerate`：

- `Iterable_Pair_int_T enumerate(Iterable_T iterable)`

- `Iterable_Pair_int_T enumerate(T[] array)`

下面是一个简单的例子：

```
from collections.enumerate(T=int) access enumerate;
int[] x = {1, 4, 9, 16};
for (var ab : enumerate(x))
    write('Element ' + string(ab.k) + ' is ' + string(ab.v));
```

其输出为

```
Element 0 is 1
Element 1 is 4
Element 2 is 9
Element 3 is 16
```

注意，对数组而言 `enumerate` 不一定值得使用，因为数组本身已经以整数为索引：

```
int[] x = {1, 4, 9, 16};
for (int i=0; i < x.length; ++i)
    write('Element ' + string(i) + ' is ' + string(x[i]));
```

对于没有内建整数索引的结构体（例如 `Map` 和 `Set`），`enumerate` 工具更为有用。

6.20.3 映射(Map)

许多现代编程语言都内置了一种数据结构，它有多种名称：映射(`map`)、字典(`dictionary`)或关联数组(`associative array`)。这一结构基于数学中函数的概念，但与此前讨论过的编程函数有所不同（见 6.12 节 [Functions], 64 页）：

- 编程函数为每个有效的输入计算出一个输出值。与数学函数不同，它可以带有副作用。
- 一个 `Map` 存储一组输入-输出对。它拥有一个有限的定义域，由那些输出已被显式添加的输入组成。通常，`Map` 的输入被称为键(`key`)，输出被称为值(`value`)。

下面这个 `Map` 示例将整数值赋给字符串键：

```
from collections.hashmap(K=string, V=int) access
    HashMap_K_V as HashMap_string_int;

// Create a Map called h.
HashMap_string_int h;

// Add some key-value pairs to h.
h['hello'] = 5;
h['world'] = 3;

// Iterate over the keys (the domain) of h.
for (string key : h) {

    // Compute the output for the given input.
    int value = h[key];
    write(key + ' ' + string(value));
}
```

产生的输出为

```
hello 5
```

world 3

6.20.3.1 初始化 Map

Map_K_V 结构本身定义于模板化模块 `collections.map(K,V)` 之中。要初始化一个 Map_K_V, 你需要使用来自 `collections.hashmap` 或 `collections.btreemap` 的函数:

```
HashMap_K_V()
```

```
HashMap_K_V(V keyword nullValue, bool keyword isNullValue(V v) =  
new bool(V v) {return v == nullValue; })
```

这两个函数用于创建哈希映射, 必须从 `collections.hashmap(K,V)` 模板化模块中访问。键类型必须具有 `hash` 方法, 见 6.19 节 [Hash functions], 86 页。

```
BTreeMap_K_V()
```

```
BTreeMap_K_V(V keyword nullValue, bool keyword isNullValue(V v) =  
new bool(V v) {return v == nullValue; })
```

这两个函数必须从 `collections.btreemap(K,V)` 模板化模块中访问。它们创建的是 b-tree 映射, 这种映射会按排序后的顺序存储键。键类型必须具有 `<` 运算符。

注意: `HashMap_K_V` 对大多数用途而言都是合适的选择。

如果未提供 `nullValue`, 查找一个缺失的键将抛出错误。`nullValue` 参数为映射提供了在查找不存在的键时可以返回的内容。这在预期会出现缺失的键、并希望优雅处理这种情形的场景中很有用。`isNullValue` 参数默认为一个函数, 该函数通过 `operator ==` 检查其参数是否等于 `nullValue`。这通常正是想要的行为, 但偶尔也有必要覆盖它; 例如, 如果你已在假定参数非空的前提下覆盖了 `operator ==`, 那么类似 `isNullValue=new bool(V v){return alias(v,null);}` 的写法可能是合适的。

`HashMap_K_V` 和 `BTreeMap_K_V` 都可以被隐式转换为 `Map_K_V`。即使不经过转换, 这两个类型也拥有与 `Map_K_V` 相同的方法。类似地, 可选参数 `nullValue` 和 `isNullValue` 在这两种类型上的行为也相同。见 6.20.3.2 节 [The Map API], 92 页。

理论上, 添加、删除或查找一个元素平均所需的时间, 对 `HashMap_K_V` 而言应为 $O(1)$, 对 `BTreeMap_K_V` 而言应为 $O(\log n)$ (其中 n 表示容器中元素的个数)。实践中, 假设条目数不超过几百万, `HashMap_K_V` 比 `BTreeMap_K_V` 快大约 2 倍。(这是一个非常粗略的估计, 它依赖于键类型 `K` 具有低廉的哈希、相等性判断和比较函数。)这些映射实现的相对性能可能会随着代码在未来得到优化而发生变化。

6.20.3.2 Map 的 API

Map_K_V 结构拥有以下方法。注意, 它的 \子类" `HashMap_K_V` 和 `BTreeMap_K_V` 各自都拥有相同的字段。

取决于所使用的构造函数, 一个 Map 可能拥有、也可能不拥有有效的 `nullValue` 和 `isNullValue(V)` 实例。如果其中一个是有有效的, 另一个也会有效。在下面的一些方法中, 除非能够返回 `nullValue` 来指示某个元素不存在, 否则将抛出错误。当为映射 `m` 定义了 `nullValue` 时, 表达式 `m.isNullValue(m[k])` 等价于 `m.contains(k)`。

```
int size()
```

返回映射中不同键的数目。

```
bool empty()
```

当且仅当 `size() == 0` 时返回 true。

```
bool contains(K key)
```

如果 `key` 作为键存在于映射中, 则返回 true。

V operator [] (K key)

返回与元素 `key` 相关联的值。如果 `key` 不存在，此方法要么返回 `nullValue`，要么在 `nullValue` 不存在的情况下抛出错误。见 6.10.4 节 [Bracket operators], 62 页。

void operator [=] (K key, V value)

将键值对 (`key`, `value`) 添加到映射中。如果已存在与 `key` 等价的键，其关联的值将被替换（键本身也会被替换，但由于旧键和新键是等价的，这一点通常无关紧要）。然而，如果 `isNullValue(value)` 为真，那么该赋值将转而从映射中移除 `key`（如果它本来在其中）。见 6.10.4 节 [Bracket operators], 62 页。

void delete(K key)

将 `key` 从映射中移除。如果 `key` 本不在映射中，则会抛出错误。（一个不会抛出错误的替代做法是将 `key` 的值设为 `nullValue`。）

Iter_K operator iter()

返回一个遍历映射中所有键的迭代器。它使映射可以被用于基于范围的 `for` 循环，例如

```
for (K key : m) {<statements>}
```

如果删除了某个键或插入了新的键，那么在该改动之前创建的任何迭代器都会失效，且行为可能不可预测。当前的实现会尝试在使用了已失效的迭代器时抛出错误，但这一行为并无保证，且出于性能原因可能在未来的版本中被移除。

Iterable_K_V pairs()

返回一个可遍历所有键值对的可迭代对象。例如，

```
for (Pair_K_V p : m.pairs()) {
    K key = p.k;
    V value = p.v;
    <statements>
}
```

K[] keys()

返回由所有键组成的数组。该数组是一个浅拷贝；对它的修改不会影响映射（反之亦然），但对这些键所指向条目的修改会破坏映射的完整性。

void add(Iterable_K_V other)

将来自 `other` 的键值对添加进来。运行 `m.add(o)`；应当等价于运行

```
for (var pr : o) { m[pr.k] = pr.v; }
```

除这些方法之外，还存在一个从 `Map_K_V` 到 `Iterable_K` 的自动展开(`autounraveled`)隐式转换。该转换允许将 `Map_K_V` 作为参数传递给诸如 `zip` 或 `enumerate` 之类的实用工具。

6.20.4 将映射用作集合

有时，你会想要一个由 `K` 类型对象组成的集合，它让你能够快速添加条目、删除条目，并判断某个条目是否存在。在 `Asymptote` 中，针对这样的集合，最直接的选择是一个从 `K` 类型键到 `bool` 值的 `Map`，并把 `nullValue` 设为 `false`。

我们需要用到 `Map_K_bool` 的某个版本。如果 `K` 是可哈希的，可以这样写：

```
from collections.hashmap(K=K,V=bool) access
    Map_K_V as Map_K_bool,
```

```
HashMap_K_V as HashMap_K_bool;
Map_K_bool mset = HashMap_K_bool(nullValue=false);
```

如果 K 不可哈希但确实拥有 `operator <`, 则可以改用类似这样的写法:

```
from collections.btreemap(K=K,V=bool) access
    Map_K_V as Map_K_bool,
    BTreeMap_K_V as BTreeMap_K_bool;
Map_K_bool mset = BTreeMap_K_bool(nullValue=false);
```

要将条目 k 添加到 `mset`, 可为它赋值 `true`:

```
mset[k] = true;
```

如果该条目已经存在于集合中, 这条语句将不产生任何效果。

要判断某个条目是否在集合中, 你可以使用 `contains` 方法, 也可以用方括号访问与 k 相关联的值:

```
if (mset.contains(k))
    write('membership verified');
```

或者, 等价地,

```
if (mset[k])
    write('membership verified');
```

要从集合中移除一个条目, 你可以使用 `delete` 方法 (若该条目本不存在于集合中, 它会抛出错误), 也可以将其值设为 `false` (若该条目本不存在, 则什么也不做):

```
mset.delete(k); // guaranteed to reduce mset.size() by 1 or throw an error
mset[k] = false; // would delete k if we had not already deleted it
```

一旦某个键的值被设为 `false`, 它就不会再出现在遍历该集合的迭代器中, 也不会计入该集合的 `size()`:

```
mset[k] = true;
int size1 = mset.size();
mset[k] = false;
assert(mset.size() == size1 - 1);
```

可以对该集合进行迭代, 或检查其大小:

```
int count = 0;
for (K k : mset) { ++count; }
assert(mset.size() == count);
```

6.20.5 集合

还有一个专门的 `Set.T` 数据结构, 它更高效一些 (不存储不需要的 `bool`), 并允许使用更精确的方法 (特别是在涉及有序集合时)。`Set.T` API 尝试遵循可逆性原则 (principle of reversibility), 即修改数据结构的方法理想情况下应当恰好返回撤销该修改所需的信息量。例如, 删除元素的方法应该返回该元素——除非调用者已知该元素 (例如作为参数传递), 在这种情况下返回一个布尔值指示删除是否成功就足够了。(相比之下, `Map_K_V` API 遵循的原则是: 方法要么返回值, 要么修改数据结构, 但不能两者兼有。)

6.20.5.1 初始化 Set

`Set.T` 数据结构本身定义在模板化模块 `collections.set(T)` 中。然而, `Set.T` 结构是抽象的, 其许多方法默认为 `null`。要获得可用的实现, 你需要 `HashSet.T` 或 `BTreeSet.T`:

HashSet.T()

```
HashSet.T(T nullT, bool equiv(T a, T b) = operator ==, bool
isNullT(T) = new bool(T t) { return equiv(t, nullT); } )
```

这两个用于创建哈希集合(hash set)的函数必须从 `collections.hashset(T)` 模板化模块中访问。类型 `T` 必须包含 `hash` 方法, 见 6.19 节 [Hash functions], 86 页。如果未定义 `isNullT`, 或者如果 `!isNullT(null)`, 则向大多数方法传递 `null` 都将抛出错误。

BTreeSet.T()

```
BTreeSet.T(T nullT, bool isNullT(T) = new bool(T t) { return t == nullT; } )
```

这两个函数必须从 `collections.btree(T)` 模块中访问。它们创建一个 B 树集合(b-tree set), 该集合根据 `<` 运算符按顺序存储其元素。`<` 运算符必须在类型 `T` 的所有元素 (满足 `isNullT` 的元素除外) 上定义严格弱序(strict weak ordering)。如果两个项目互不小于对方, 则认为它们是等价的。

访问 `collections.btree(T)` 模块要求为类型 `T` 定义 `operator <`; 如果没有定义, 可以使用 `collections.btreegeneral(T)` 模块中定义的构造函数, 下文将对此进行讨论。

```
BTreeSet.T(bool lessThan(T,T))
```

```
BTreeSet.T(bool lessThan(T,T), T nullT, bool isNullT(T) =
new bool(T t) { return t == nullT; } )
```

这两个函数必须从 `collections.btreegeneral(T)` 模块访问。它们创建一个 `BTreeSet`, 该集合根据 `lessThan` 函数按有序顺序存储其元素。`lessThan` 函数必须在类型 `T` 的所有项目 (满足 `isNullT` 的项目除外) 上定义严格弱序。如果两个项目互不小于对方, 则认为它们是等价的。

`HashSet.T` 和 `BTreeSet.T` 都可以隐式转换为 `Set.T`, 并包含 `Set.T` 的所有方法。类似地, `nullT` 和 `isNullT` 可选参数对这两种类型的行为完全相同。见 6.20.5.2 节 [The Set API], 95 页。

此外, `BTreeSet.T` 可以隐式转换为 `SortedSet.T` 并且拥有其所有方法, 这些方法提供了有序集合特有的附加功能 (例如检索最小元素或大于给定元素的下一个元素)。见 6.20.5.3 节 [The SortedSet API], 97 页。

6.20.5.2 Set API

取决于使用了哪种构造函数, `nullT` 和 `isNullT` 可能为 `Set` 定义, 也可能未定义。如果定义了其中之一, 则另一个也会被定义。在下面的某些方法中, 除非能够返回 `nullT` 来表示元素不存在, 否则将抛出错误。

`Set.T` 结构具有以下方法:

```
int size()
```

返回集合中不同元素的数量。

```
bool empty()
```

当且仅当 `size() == 0` 时返回 `true`。

```
bool contains(T item)
```

如果集合中存在 `item` (或等价项目), 则返回 `true`。如果已定义 `isNullT` 且 `isNullT(item)` 为 `true`, 该方法始终返回 `false`。

`bool add(T item)`

如果集合中尚未存在 `item`，则将其添加到集合中。如果 `isNullT(item)` 已定义且为 `true`，或者如果 `item` 或等价项目已存在，则此方法不执行任何操作。当且仅当 `item` 被成功添加时，返回值为 `true`。

`void add(Iterable_T other)`

将来自 `other` 的元素添加到集合中。该方法违背了可逆性原则。

`bool delete(T item)`

如果集合中存在 `item` 则将其移除，否则不执行任何操作。如果 `isNullT(item)` 已定义且为 `true`，则此方法不执行任何操作。当且仅当集合被修改时，返回值为 `true`。

`void delete(Iterable_T other)`

从集合中移除来自 `other` 的元素。`other` 中不在集合中的元素将被忽略。该方法违背了可逆性原则。

`Iter_T operator iter()`

返回集合元素的迭代器。它允许集合 `s` 用于基于范围的 `for` 循环：

```
for (T item : s) {<statements>}
```

如果删除或插入了新元素，则在此更改之前创建的任何迭代器都将失效，且行为可能不可预测。当前的实现会尝试在使用已失效的迭代器时抛出错误，但这种行为并不得到保证，并且可能出于性能原因在未来版本中被移除。

`Set_T newEmpty()`

返回一个新的空集合，该集合与当前集合具有相同的 `nullT`、`isNullT`、等价关系以及（如适用）顺序。新集合类型的实现者应当实现此方法，否则应该返回集合的二元运算符将改为抛出错误。

假设等价元素是可互换的，上述方法就足够了。然而，在极少数情况下，人们可能会关心两个等价元素中哪一个存在于集合中。例如，如果将集合配置为当两个有序数对具有相同的第一个元素时即被视为等价，那么上述方法将无法根据数对的第一个元素查找该数对。对于这种情况，`Set_T` API 提供了以下方法：

`T get(T item)`

返回集合中与 `item` 等价的项目。如果不存在此类项目，则返回 `nullT`；如果未定义 `nullT`，则抛出错误。

`T push(T item)`

将 `item` 插入集合中，即使它与现有项目等价。如果 `item` 与现有项目等价，则返回被替换的项目；否则返回 `nullT`（除非未定义 `nullT`，在这种情况下抛出错误）。如果 `isNullT(item)` 为 `true`，集合不会被修改，并返回 `nullT`。

`push` 方法可用于向集合添加新项目，但前提是必须定义了 `nullT`。

`T extract(T item)`

移除并返回集合中与 `item` 等价的项目。如果不存在此类项目，则返回 `nullT`；如果未定义 `nullT`，则抛出错误。如果 `isNullT(item)` 为 `true`，集合不会被修改，并返回 `nullT`。

除了上述方法之外，还有一些自动展开(`autounraveled`)的函数用于处理集合。

`Iterable_T operator cast(Set_T set)`

隐式地将集合转换为可迭代对象，以便它可以作为参数传递给像 `zip` 或 `enumerate` 这样的实用工具。

`bool operator <= (Set_T a, Set_T b)`
如果 a 是 b 的子集, 则返回 true。

`bool operator >= (Set_T a, Set_T b)`
如果 b 是 a 的子集, 则返回 true。

`bool operator == (Set_T a, Set_T b)`
如果 a 和 b 互为子集, 则返回 true。

`bool operator != (Set_T a, Set_T b)`
如果 a 或 b 中包含不在另一个集合中的元素, 则返回 true。

`bool sameElementsInOrder(Set_T a, Set_T b)`
如果在 a 和 b 上进行迭代按相同顺序产生相同的元素, 则返回 true。

以下运算符保持其两个参数不变; 结果通过调用第一个参数的 `newEmpty()` 方法进行初始化, 以生成与第一个参数具有相同底层实现的新集合。

`Set_T operator + (Set_T a, Iterable_T b)`
返回 a 和 b 的并集。

`Set_T operator - (Set_T a, Set_T b)`
返回一个新的集合, 包含属于 a 但不属于 b 的元素。

`Set_T operator & (Set_T a, Set_T b)`
返回 a 和 b 的交集。

`Set_T operator ^ (Set_T a, Set_T b)`
返回 a 和 b 的对称差(symmetric difference), 换句话说, 即恰好属于 a, b 其中之一的所有项目的集合。

注意: 虽然可以使用 `+=` 和 `-=` 运算符, 但它们的行为可能不如预期。具体来说, 这两个运算符将构造一个新的集合, 而不是原地修改集合。对于大多数用途, 推荐使用 `add(Iterable_T)` 和 `delete(Iterable_T)` 方法。

6.20.5.3 SortedSet API

`Set_T` 的 `BTreeSet_T` 实现 (见 6.20.5.1 节 [Initializing a Set], 94 页) 维护其元素的有序顺序。对于 `Set_T` API, 除了迭代顺序之外, 这没有任何其他效果。然而, `SortedSet_T` API 提供了额外的方法来利用这种顺序。`SortedSet_T` 结构定义在 `collections.sortedset(T)` 模块中。以下是 `BTreeSet_T` 实现的各种 API 之间关系的列表:

- `BTreeSet_T` 拥有 `Set_T` 的所有方法, 并且可以隐式转换为 `Set_T`。
- `BTreeSet_T` 拥有 `SortedSet_T` 的所有方法, 并且可以隐式转换为 `SortedSet_T`。
- `SortedSet_T` 拥有 `Set_T` 的所有方法, 并且可以隐式转换为 `Set_T`。
- 所有这些类型都可以隐式转换为 `Iterable_T`。

除了 `Set_T` 的方法之外, `SortedSet_T` 还具有以下方法:

`T min()` 返回集合中的最小元素, 如果集合为空则返回 `nullT`。如果集合为空且未定义 `nullT`, 则抛出错误。

`T max()` 返回集合中的最大元素, 如果集合为空则返回 `nullT`。如果集合为空且未定义 `nullT`, 则抛出错误。

- T popMin()**
移除并返回集合中的最小元素，或在集合为空时返回 `nullT`。如果集合为空且未定义 `nullT`，则会抛出错误。
- T popMax()**
移除并返回集合中的最大元素，或在集合为空时返回 `nullT`。如果集合为空且未定义 `nullT`，则会抛出错误。
- T after(T item)**
返回集合中严格大于 `item` 的最小元素。如果不存在此类元素，则返回 `nullT`；如果未定义 `nullT`，则抛出错误。
- T atOrAfter(T item)**
返回集合中大于或等于 `item` 的最小元素。如果不存在此类元素，则返回 `nullT`；如果未定义 `nullT`，则抛出错误。
- T before(T item)**
返回集合中严格小于 `item` 的最大元素。如果不存在此类元素，则返回 `nullT`；如果未定义 `nullT`，则抛出错误。
- T atOrBefore(T item)**
返回集合中小于或等于 `item` 的最大元素。如果不存在此类元素，则返回 `nullT`；如果未定义 `nullT`，则抛出错误。

作为示例，以下代码在介于 `a` (包含) 与 `b` (不包含) 之间的有序集合 `sset` 元素上进行迭代：

```
for (var x = sset.atOrAfter(a);
    !sset.isNullT(x) && x < b;
    x = sset.after(x)) {
    <statements using x>
}
```

当然，此代码假设为 `sset` 定义了 `nullT`。

注意，如果你想在有序集合中的大部分或全部元素上进行迭代，通常更高效的做法是使用基于范围的 `for` 循环，并跳过不在所需范围内的任何元素：

```
for (var x : sset) {
    if (x < a) continue;
    if (x >= b) break;
    <statements using x>
}
```

6.20.6 将集合用作映射

在数学中，映射有时被定义为有序对的集合。在 `Asymptote` 编程语言中，这个定义可以字面理解：`Pair_K_V` 对象的 `Set_T` 可以用作从 `K` 到 `V` 的映射，前提是等价、哈希和比较函数配置为仅查看数对的第一个元素。这对于有序集合特别有用，因为有序集合拥有诸如 `atOrAfter` 和 `after` 之类目前在 `Map` API 中尚不可用的功能。

我们通过实现一个分段线性函数(piecewise linear function)来说明这种方法（在这种情况下我们可以使用内置的 `pair` 类型而不是 `Pair_K_V`）。

```
from collections.btreegeneral(T=pair) access
SortedSet_T as SortedSet_pair,
```

```

    BTreeSet_T as BTreeSet_pair;

bool keysLT(pair a, pair b) { return a.x < b.x; }
bool isnan(pair a) { return isnan(a.y); }

struct PLFunction {
    SortedSet_pair pairs = BTreeSet_pair(
        lessThan=keysLT,
        nullT=(nan,nan),
        isNullT = isnan
    );
    void operator [=] (real x, real y) {
        pairs.push((x,y));
    }
    real operator [] (real x) {
        pair p = (x, 0);
        pair left = pairs.atOrBefore(p);
        pair right = pairs.atOrAfter(p);
        if (isnan(left.y) || isnan(right.y)) {
            return nan;
        }
        if (left.x == right.x) {
            return left.y;
        }
        real t = (x - left.x) / (right.x - left.x);
        return interp(left.y, right.y, t);
    }
}

PLFunction f;
f[0] = 17;
f[1] = 18;
f[2001] = 19;

for (real x : new real[] {-1, 0, 0.5, 1, 2, 2001, 2002}) {
    write('f[' + (string)x + '] = ' + (string)f[x]);
}

```

输出

```

f[-1] = nan
f[0] = 17
f[0.5] = 17.5
f[1] = 18
f[2] = 18.0005
f[2001] = 19
f[2002] = nan

```

6.20.7 队列

两种标准数据结构是栈(stack)和队列(queue)。栈是一种后进先出(LIFO)结构;在 `Asymptote` 中,数组可以通过 `push` 和 `pop` 方法用作栈。队列是一种先进先出(FIFO)结构; `collections.queue(T)` 模块提供了队列的实现。

`Queue_T makeQueue(T[] initialData)`

返回一个使用数组 `initialData` 的副本初始化的队列。需要参数 `initialData` 是为了指定类型 `T`, 以防当前作用域中存在多个 `Queue_T` 类型。如果不需要初始数据, 使用 `makeQueue(new T[])` 创建一个空数组作为参数。

`Queue_T` 结构具有以下方法:

`void push(T item)`

将 `item` 添加到队列末尾。

`T pop()` 移除并返回队列中的第一个项目。如果队列为空, 则抛出错误。

`T peek()` 返回队列中的第一个项目而不将其移除。如果队列为空, 则抛出错误。

`int size()`

返回队列中的项目数量。

`Iter_T operator iter()`

按项目将被 `pop()` 返回的顺序, 返回队列中项目的迭代器。

如果推入或弹出了元素, 则在此更改之前创建的任何迭代器都将失效, 且行为可能不可预测。当前的实现会尝试在使用已失效的迭代器时抛出错误, 但这种行为并不得到保证, 并且可能出于性能原因在未来版本中被移除。

除了这些方法之外, 还有从 `Queue_T` 到 `Iterable_T` 的自动展开隐式转换。这种转换允许将 `Queue_T` 作为参数传递给像 `zip` 或 `enumerate` 这样的实用工具。

6.20.8 包装数组

`Asymptote` 数组具有内置的 `==` 运算符, 可返回布尔数组而非单个布尔值。这在许多语境下都很有用, 但也阻止了它们在数对中或在哈希映射中作为值使用。 `collections.wraparray(T)` 模板化模块为数组提供了一个包装器(wrapper), 它具有更常规的 `==` 运算符并返回单个布尔值。它还提供了一个 `hash()` 方法, 如果经过初始化, 允许将数组用作哈希映射中的键。

在 `collections.wraparray(T)` 中的主要类型是 `Array_T`。虽然 `Array_T` 并不是 `T[]` 的直接替代品, 但它已尽可能接近。为方便起见, 函数 `Array_T wrap(T[] data)` 可用于将数组转换为 `Array_T`。此外, `Array_T` 具有与 `T[]` 之间的自动展开隐式转换, 因此它可以用在大多数需要 `T[]` 的地方 (反之亦然)。注意, 这些转换不会创建副本, 因此如果你修改原始数组, 更改将反映在 `Array_T` 中, 反之亦然。

`Array_T` 的方法和字段与 `T[]` 的方法和字段完全相同, 但有以下例外:

`int length()`

是一个方法 (后面需要括号), 返回数组的长度。

`void cyclic(bool b)`

设置被包装数组的 `cyclic` 字段。

`bool cyclic()`

返回被包装数组的 `cyclic` 字段。

```
int[] keys()
```

是一个方法（后面需要括号），返回被包装数组的 `keys` 字段。

```
T[] data
```

是保存被包装数组的字段。

此外，如果 `a` 是 `Array_T`，则支持迭代语法 `for (T x : a)`，以及元素访问 `a[i]`。但是，不支持切片。

要使 `Array_T` 可哈希，请向 `wrap` 函数传递第二个参数，该参数计算单个元素的整数哈希。例如，以下代码构造了字符串和整数的可哈希数组：

```
from collections.wraparray(T=string) access Array_T as Array_string;
from collections.wraparray(T=int) access Array_T as Array_int;
string[] s = {'hello', 'world'};
Array_string ss = wrap(s, new int(string s) { return s.hash(); });
int[] a = {1, 2, 3};
Array_int aa = wrap(a, new int(int i) { return i; });
```

生成的哈希函数通过使用提供的函数将数组的所有元素转换为整数，然后对生成的 `int[]` 进行哈希计算，见 6.19 节 [Hash functions], 86 页。

重要注意：如果将 `Array_T` 用作哈希映射中的键，你必须确保在数组作为哈希映射中的键期间不会被修改。

7 L^AT_EX 用法

Asymptote 自带一个方便的 L^AT_EX 样式文件 `asymptote.sty` (需要 v1.36 或更高版本), 它能让 L^AT_EX 识别 Asymptote。把 Asymptote 代码直接写到 L^AT_EX 源文件中需要它的地方, 可以让图形保持井然有序, 也免去了为每幅图形另起文件名的麻烦。只需在文件开头加上 `\usepackage{asymptote}` 这一行, 并把你的 Asymptote 代码放进 `\begin{asy}... \end{asy}` 环境中。与 L^AT_EX 的 `comment` 环境一样, `\end{asy}` 命令必须独占一行, 后面不得再跟命令/注释。`\begin{asy}` 之后不允许出现空行。

下面这个名为 `latexusage.tex` 的 L^AT_EX 示例文件可以这样运行:

```
latex latexusage
asy latexusage-*.asy
latex latexusage
```

或者

```
pdflatex latexusage
asy latexusage-*.asy
pdflatex latexusage
```

若要在 `latex` 与 `pdflatex` 之间切换以处理内联 Asymptote 代码, 可能需要先删除文件 `latexusage-*.tex`。

处理含有内嵌 Asymptote 代码的 L^AT_EX 文件, 还有一种更好的方法, 就是使用 `latexmk` 工具, 它来自

<http://mirror.ctan.org/support/latexmk/>

先把 <https://raw.githubusercontent.com/vectorgraphics/asymptote/HEAD/doc/latexmkrc> 的内容放进同一目录下名为 `latexmkrc` 的文件。命令:

```
latexmk -pdf latexusage
```

随后就会自动调用 Asymptote, 只重新编译发生变化的图形。由于每幅图形都在单独的系统进程中编译, 这种方式往往也更省内存。若要把图形存放到名为 `asy` 的单独目录中, 可以在 `latexusage.tex` 中定义:

```
\def\asydir{asy}
```

外部 Asymptote 代码可以用

```
\asyinclude[<options>]{<filename.asy>}
```

来包含, 这样 `latexmk` 就能识别代码何时发生变化。注意, `latexmk` 需要 `perl`, 可从 <https://www.perl.org/> 获取。

可以为 `asy` 和 `asyinclude` 环境指定 `keyval` 风格的选项: `width`、`height`、`keepAspect`、`viewportwidth`、`viewportheight` 和 `inline`。三维 PRC 文件既可以嵌入页面 (默认方式), 也可以作为带注释 (但仍可打印) 的附件附加, 这要用到 `attach` 选项以及 `attachfile2` (或较旧的 `attachfile`) L^AT_EX 宏包。`inline` 选项生成内联 L^AT_EX 代码, 而不是 EPS 或 PDF 文件。这使得 2D L^AT_EX 符号对 `\begin{asy}... \end{asy}` 环境可见。在此模式下, Asymptote 能正确对齐在 `\begin{asy}... \end{asy}` 之外定义的 2D L^AT_EX 符号, 但会把它们的尺寸当作零; 可以给 Label 提供可选的第二个字符串, 用来给出未知标签尺寸的估计值。

注意, 如果 `latex` L^AT_EX 引擎与 `inline` 选项一起使用, 标签可能无法显示在无法处理原始 PostScript 代码的 DVI 查看器中。可以用 `dvips/dvipdf` 生成 PostScript/PDF 输出(我们推荐使用 Asymptote patches 目录中修改过的 `dvipdf` 版本, 它支持 `dvips -z hyperdvi` 选项)。

下面就是 `latexusage.tex`:

```
\documentclass[12pt]{article}

% Use this form to include EPS (latex) or PDF (pdflatex) files:
%\usepackage{asymptote}

% Use this form with latex or pdflatex to include inline LaTeX code by default:
\usepackage[inline]{asymptote}

% Use this form with latex or pdflatex to create PDF attachments by default:
%\usepackage[attach]{asymptote}

% Enable this line to support the attach option:
%\usepackage[dvips]{attachfile2}

\begin{document}

% Optional subdirectory for latex files (no spaces):
\def\asylatexdir{}
% Optional subdirectory for asy files (no spaces):
\def\asydir{}

\begin{asydef}
// Global Asymptote definitions can be put here.
settings.prc=true;
import three;
usepackage("bm");
texpreamble("\def\V#1{\bm{#1}}");
// One can globally override the default toolbar settings here:
// settings.toolbar=true;
\end{asydef}
```

Here is a venn diagram produced with Asymptote, drawn to width 4cm:

```
\def\A{A}
\def\B{\V{B}}

%\begin{figure}
\begin{center}
\begin{asy}
size(4cm,0);
pen colour1=red;
pen colour2=green;

pair z0=(0,0);
pair z1=(-1,0);
```

```

pair z2=(1,0);
real r=1.5;
path c1=circle(z1,r);
path c2=circle(z2,r);
fill(c1,colour1);
fill(c2,colour2);

picture intersection=new picture;
fill(intersection,c1,colour1+colour2);
clip(intersection,c2);

add(intersection);

draw(c1);
draw(c2);

//draw("$A$",box,z1); // Requires [inline] package option.
//draw(Label("$B$", "$B$"),box,z2); // Requires [inline] package option.
draw("$A$",box,z1);
draw("$\V{B}$",box,z2);

pair z=(0,-2);
real m=3;
margin BigMargin=Margin(0,m*dot(unit(z1-z),unit(z0-z)));

draw(Label("$A\cap B$",0),conj(z)-z0,Arrow,BigMargin);
draw(Label("$A\cup B$",0),z-z0,Arrow,BigMargin);
draw(z-z1,Arrow,Margin(0,m));
draw(z-z2,Arrow,Margin(0,m));

shipout(bbox(0.25cm));
\end{asy}
%\caption{Venn diagram}\label{venn}
\end{center}
%\end{figure}

```

Each graph is drawn in its own environment. One can specify the width and height to $\text{\LaTeX}$ explicitly. This 3D example can be viewed interactively either with Adobe Reader or Asymptote's fast OpenGL-based renderer. To support $\text{\tt latexmk}$, 3D figures should specify $\text{\verb+inline=true+}$. It is sometimes desirable to embed 3D files as annotated attachments; this requires the $\text{\verb+attach=true+}$ option as well as the $\text{\verb+attachfile2+}$ $\text{\LaTeX}$ package.

```

\begin{center}
\begin{asy}[height=4cm,inline=true,attach=false,viewportwidth=\linewidth]
currentprojection=orthographic(5,4,2);
draw(unitcube,blue);

```

```
label("$V-E+F=2$", (0,1,0.5), 3Y, blue+fontsize(17pt));
\end{asy}
\end{center}
```

One can also scale the figure to the full line width:

```
\begin{center}
\begin{asy}[width=\the\linewidth,inline=true]
pair z0=(0,0);
pair z1=(2,0);
pair z2=(5,0);
pair zf=z1+0.75*(z2-z1);

draw(z1-z2);
dot(z1,red+0.15cm);
dot(z2,darkgreen+0.3cm);
label("$m$", z1, 1.2N, red);
label("$M$", z2, 1.5N, darkgreen);
label("$\hat{\phantom{x}}$", zf, 0.2*S, fontsize(24pt)+blue);

pair s=-0.2*I;
draw("$x$", z0+s-z1+s, N, red, Arrows, Bars, PenMargins);
s=-0.5*I;
draw("$\bar{x}$", z0+s-zf+s, blue, Arrows, Bars, PenMargins);
s=-0.95*I;
draw("$X$", z0+s-z2+s, darkgreen, Arrows, Bars, PenMargins);
\end{asy}
\end{center}
\end{document}
```

```

\begin{center}
\begin{asy}[height=4cm,inline=true,attach=false,viewportwidth=\linewidth]
currentprojection=orthographic(5,4,2);
draw(unitcube,blue);
label("$V-E+F=2$", (0,1,0.5),3Y,blue+fontsize(17pt));
\end{asy}
\end{center}

```

One can also scale the figure to the full line width:

```

\begin{center}
\begin{asy}[width=\the\linewidth,inline=true]
pair z0=(0,0);
pair z1=(2,0);
pair z2=(5,0);
pair zf=z1+0.75*(z2-z1);

draw(z1--z2);
dot(z1,red+0.15cm);
dot(z2,darkgreen+0.3cm);
label("$m$",z1,1.2N,red);
label("$M$",z2,1.5N,darkgreen);
label("$\hat{\ }$",zf,0.2*S,fontsize(24pt)+blue);

pair s=-0.2*I;
draw("$x$",z0+s--z1+s,N,red,Arrows,Bars,PenMargins);
s=-0.5*I;
draw("$\bar{x}$",z0+s--zf+s,blue,Arrows,Bars,PenMargins);
s=-0.95*I;
draw("$X$",z0+s--z2+s,darkgreen,Arrows,Bars,PenMargins);
\end{asy}
\end{center}
\end{document}

```

8 用户模块

这些是用于创建图形和可视化最常用的基础模块。

8.1 graph

该模块实现了二维线性与对数图表，包括自动缩放和刻度选择（并支持手动覆盖）。图表(graph)是一个 guide(可用 draw 命令绘制，并可选带图例)，由以下例程之一构建：

•

```
guide graph(picture pic=currentpicture, real f(real), real a, real b,
            int n=ngraph, real T(real)=identity,
            interpolate join=operator -);
guide[] graph(picture pic=currentpicture, real f(real), real a, real b,
              int n=ngraph, real T(real)=identity, bool3 cond(real),
              interpolate join=operator -);
```

返回一个图表，使用图片 pic 的缩放信息（见 [automatic scaling], 122 页）来绘制区间 $[T(a), T(b)]$ 上函数 f 的图像，采样点数为 n ，均匀分布于 $[a, b]$ ，并可选由区间 $[a, b]$ 上的 bool3 函数 cond 加以限制。若 cond 为：

- true, 该点将被添加到现有 guide 中；
- default, 该点将被添加到新的 guide 中；
- false, 该点被省略，并开始一个新的 guide。

这些点使用 join 指定的插值方式进行连接：

- operator -- (线性插值；也接受缩写 Straight)；
- operator .. (分段三次贝塞尔样条(spline)插值；也接受缩写 Spline)；
- linear (线性插值)，
- Hermite (使用边界条件 notaknot、natural、periodic、clamped(real slopea, real slopeb) 的标准三次样条插值)，或 monotonic。对于非周期数据，缩写 Hermite 等价于 Hermite(notaknot)；对于周期数据，等价于 Hermite(periodic)。

•

```
guide graph(picture pic=currentpicture, real x(real), real y(real),
            real a, real b, int n=ngraph, real T(real)=identity,
            interpolate join=operator -);
guide[] graph(picture pic=currentpicture, real x(real), real y(real),
              real a, real b, int n=ngraph, real T(real)=identity,
              bool3 cond(real), interpolate join=operator -);
```

返回一个图表，使用图片 pic 的缩放信息绘制参数化函数 $(x(t), y(t))$ （其中 t 位于区间 $[T(a), T(b)]$ 内），在 $[a, b]$ 上均匀采样 n 个点，可选地受区间 $[a, b]$ 上的 bool3 函数 cond 限制，并使用给定的插值类型。

•

```
guide graph(picture pic=currentpicture, pair z(real), real a, real b,
            int n=ngraph, real T(real)=identity,
            interpolate join=operator -);
guide[] graph(picture pic=currentpicture, pair z(real), real a, real b,
```

```
int n=ngraph, real T(real)=identity, bool3 cond(real),
interpolate join=operator -);
```

返回一个图表，使用图片 `pic` 的缩放信息绘制参数化函数 $z(t)$ （其中 t 位于区间 $[T(a), T(b)]$ 内），在 $[a, b]$ 上均匀采样 n 个点，可选地受区间 $[a, b]$ 上的 `bool3` 函数 `cond` 限制，并使用给定的插值类型。

•

```
guide graph(picture pic=currentpicture, pair[] z,
interpolate join=operator -);
guide[] graph(picture pic=currentpicture, pair[] z, bool3[] cond,
interpolate join=operator -);
```

返回一个图表，使用图片 `pic` 的缩放信息绘制数组 `z` 中的元素，可选地限制为布尔数组 `cond` 的元素为 `true` 的那些索引，并使用给定的插值类型。

•

```
guide graph(picture pic=currentpicture, real[] x, real[] y,
interpolate join=operator -);
guide[] graph(picture pic=currentpicture, real[] x, real[] y,
bool3[] cond, interpolate join=operator -);
```

返回一个图表，使用图片 `pic` 的缩放信息绘制数组对 (x, y) 的元素，可选地限制为布尔数组 `cond` 的元素为 `true` 的那些索引，并使用给定的插值类型。

•

```
guide polargraph(picture pic=currentpicture, real f(real), real a,
real b, int n=ngraph, interpolate join=operator -);
```

返回一个极坐标图表，使用图片 `pic` 的缩放信息绘制区间 $[a, b]$ 上函数 f 的图像，均匀采样 n 个点，并使用给定的插值类型。

•

```
guide polargraph(picture pic=currentpicture, real[] r, real[] theta,
interpolate join=operator -);
```

返回一个极坐标图表，使用图片 `pic` 的缩放信息绘制数组对 (r, θ) 的元素，并使用给定的插值类型。

可以使用以下命令之一在图片上绘制坐标轴：

•

```
void xaxis(picture pic=currentpicture, Label L="", axis axis=YZero,
real xmin=-infinity, real xmax=infinity, pen p=currentpen,
ticks ticks=NoTicks, arrowbar arrow=None, bool above=false);
```

在图片 `pic` 上绘制从 $x=xmin$ 到 $x=xmax$ 的 x 坐标轴，使用画笔 `p`，可选地带有标签 (Label) `L`。标签沿坐标轴的相对位置（区间 $[0, 1]$ 内的实数）默认值为 1（见 [Label], 19 页），因此标签会绘制在坐标轴的末端。`xmin` 或 `xmax` 为无穷大值指定对应的坐标轴极限将自动根据图片边界确定。可选的 `arrow` 参数取值与 `draw` 命令中相同（见 [arrows], 14 页）。除非 `above=true`，否则坐标轴会在 `pic` 中任何现有对象之前绘制。坐标轴的位置由以下 `axis` 类型之一决定：

```
YZero(bool extend=true)
```

请求在 $y=0$ （对数轴上为 $y=1$ ）处绘制 x 坐标轴，延伸至图片的完整尺寸，除非 `extend=false`。

`YEquals(real Y, bool extend=true)`
请求在 $y=Y$ 处绘制延伸至图片完整尺寸的 x 坐标轴, 除非 `extend=false`。

`Bottom(bool extend=false)`
请求底端坐标轴。

`Top(bool extend=false)`
请求顶端坐标轴。

`BottomTop(bool extend=false)`
请求底端与顶端坐标轴。

可以通过参考模块 `graph.asy` 中的示例来创建自定义坐标轴类型。也可以轻松覆盖标准坐标轴类型的默认值:

```
import graph;

YZero=new axis(bool extend=true) {
    return new void(picture pic, axisT axis) {
        real y=pic.scale.x.scale.logarithmic ? 1 : 0;
        axis.value=I*pic.scale.y.T(y);
        axis.position=1;
        axis.side=right;
        axis.align=2.5E;
        axis.value2=Infinity;
        axis.extend=extend;
    };
};
YZero=YZero();
```

默认刻度选项为 `NoTicks`。选项 `LeftTicks`、`RightTicks` 或 `Ticks` 可用于沿路径绘制方向的左侧、右侧或两侧绘制刻度线。这些刻度例程接受许多可选参数:

```
ticks LeftTicks(Label format="", ticklabel ticklabel=null,
                bool beginlabel=true, bool endlabel=true,
                int N=0, int n=0, real Step=0, real step=0,
                bool begin=true, bool end=true, tickmodifier modify=None,
                real Size=0, real size=0, bool extend=false,
                pen pTick=nullpen, pen ptick=nullpen);
```

如果省略其中任何参数, 将选用合理的默认值:

Label format

覆盖默认刻度标签格式 (`defaultformat`, 初始为 `"$%.4g$"`)、旋转、画笔和基于坐标轴的对齐方式 (例如 `LeftSide`、`Center` 或 `RightSide`)。要启用 $\text{\LaTeX}$ 数学模式字体, 格式字符串应以 `$` 开头和结尾 见 `[format]`, 29 页。如果格式字符串为 `trailingzero`, 将在刻度标签中添加末尾零; 如果格式字符串为 `"%"`, 则将抑制刻度标签;

ticklabel

是一个函数 `string(real x)`, 返回每个主刻度值 x 的标签 (默认为 `format(format.s,x)`);

```

bool beginlabel
    包含第一个标签;

bool endlabel
    包含最后一个标签;

int N      当启用自动缩放时（默认；见 [automatic scaling], 122 页），将线性坐标轴
            均匀分为这么多由主刻度分隔的区间；对于对数坐标轴，这是带标签刻度之
            间的数量级(decade)数；

int n      将每个区间划分为这么多由次刻度分隔的子区间；

real Step  主刻度之间的刻度值间距（若 N=0）；

real step  次刻度之间的刻度值间距（若 n=0）；

bool begin
    包含第一个主刻度；

bool end   包含最后一个主刻度；

tickmodifier modify;
    一个可选函数，接受并返回带有 real[] 成员 major 与 minor（由刻度值组成）
    的 tickvalue 结构（以允许修改自动生成的刻度值）；

real Size  主刻度的大小（以 PostScript 坐标表示）；

real size  次刻度的大小（以 PostScript 坐标表示）；

bool extend;
    在两条坐标轴之间延伸刻度线（对在图表上绘制网格很有用）；

pen pTick  用于绘制主刻度的可选画笔；

pen ptick  用于绘制次刻度的可选画笔。

```

为方便起见，预定义的 tickmodifier OmitTick(...real[] x)、OmitTickInterval(real a, real b) 与 OmitTickIntervals(real[] a, real[] b) 可用于移除指定的自动生成刻度及其标签。ticklabel OmitFormat(string s=defaultformat, ... real[] x) 可用于移除特定的刻度标签但保留对应的刻度。tickmodifier NoZero 是 OmitTick(0) 的缩写，而 ticklabel NoZeroFormat 是 OmitFormat(0) 的缩写。

还可以通过给 LeftTicks、RightTicks 和 Ticks 传递分别包含主刻度和（可选）次刻度位置的显式 real 数组 Ticks 与 ticks，来指定自定义刻度位置：

```

ticks LeftTicks(Label format="", ticklabel ticklabel=null,
                bool beginlabel=true, bool endlabel=true,
                real[] Ticks, real[] ticks=new real[],
                real Size=0, real size=0, bool extend=false,
                pen pTick=nullpen, pen ptick=nullpen)

```

•

```

void yaxis(picture pic=currentpicture, Label L="", axis axis=XZero,
           real ymin=-infinity, real ymax=infinity, pen p=currentpen,
           ticks ticks=NoTicks, arrowbar arrow=None, bool above=false,
           bool autorotate=true);

```

在图片 `pic` 上绘制从 $y=ymin$ 到 $y=ymax$ 的 y 坐标轴, 使用画笔 `p`, 可选地带有会自动旋转的标签 `L` (除非 `autorotate=false`)。标签的相对位置 (区间 $[0,1]$ 内的实数) 默认值为 1 (见 [Label], 19 页)。 `ymin` 或 `ymax` 为无穷大值指定对应的坐标轴极限将自动根据图片边界确定。可选的 `arrow` 参数取值与 `draw` 命令中相同 (见 [arrows], 14 页)。除非 `above=true`, 否则坐标轴会在 `pic` 中任何现有对象之前绘制。刻度类型由 `ticks` 指定, 坐标轴的位置由以下 `axis` 类型之一决定:

`XZero(bool extend=true)`
请求在 $x=0$ (对数轴上为 $x=1$) 处绘制 y 坐标轴, 延伸至图片的完整尺寸, 除非 `extend=false`。

`XEquals(real X, bool extend=true)`
请求在 $x=X$ 处绘制延伸至图片完整尺寸的 y 坐标轴, 除非 `extend=false`。

`Left(bool extend=false)`
请求左侧坐标轴。

`Right(bool extend=false)`
请求右侧坐标轴。

`LeftRight(bool extend=false)`
请求左侧与右侧坐标轴。

- 为方便起见, 函数

```
void xequals(picture pic=currentpicture, Label L="", real x,
             bool extend=false, real ymin=-infinity, real ymax=infinity,
             pen p=currentpen, ticks ticks=NoTicks, bool above=true,
             arrowbar arrow=None);
```

与

```
void yequals(picture pic=currentpicture, Label L="", real y,
             bool extend=false, real xmin=-infinity, real xmax=infinity,
             pen p=currentpen, ticks ticks=NoTicks, bool above=true,
             arrowbar arrow=None);
```

可分别用于通过适当的坐标轴类型 `XEquals(x,extend)` 和 `YEquals(y,extend)` 调用 `yaxis` 与 `xaxis`。这是在任意位置绘制垂直线或水平线以及坐标轴的推荐方式。

-

```
void axes(picture pic=currentpicture, Label xlabel="", Label ylabel="",
          bool extend=true,
          pair min=(-infinity,-infinity), pair max=(infinity,infinity),
          pen p=currentpen, arrowbar arrow=None, bool above=false);
```

该便捷例程在图片 `pic` 上绘制从 `min` 到 `max` 的 x 和 y 坐标轴, 带有可选的标签 `xlabel` 与 `ylabel` 以及 `arrow` 指定的箭头。只有当 `above=true` 时, 坐标轴才会绘制在 `pic` 中现有对象的上方。

-

```
void axis(picture pic=currentpicture, Label L="", path g,
          pen p=currentpen, ticks ticks, ticklocate locate,
          arrowbar arrow=None, int[] divisor=new int[],
          bool above=false, bool opposite=false);
```

该例程可用于在图片 `pic` 上基于任意路径 `g` 使用画笔 `p` 绘制通用坐标轴。可以可选地用 `Label L` 为坐标轴添加标签并添加箭头 `arrow`。刻度类型由 `ticks` 给定。可选的整数数组 `divisor` 指定在尝试生成不拥挤的刻度标签时尝试哪些刻度除数。标记 `opposite` 为 `true` 值代表未加标签的次要坐标轴（通常绘制在主要坐标轴的对面）。除非 `above=true`，否则坐标轴会在 `pic` 中任何现有对象之前绘制。刻度定位器 `ticklocate` 由以下例程构造：

```
ticklocate ticklocate(real a, real b, autoscaleT S=defaultS,
                      real tickmin=-infinity, real tickmax=infinity,
                      real time(real)=null, pair dir(real)=zero);
```

其中 `a` 与 `b` 分别指定 `point(g,0)` 和 `point(g,length(g))` 处的刻度值，`S` 指定自动缩放变换，函数 `real time(real v)` 返回对应于值 `v` 的 `time`，而 `pair dir(real t)` 返回作为 `t` 的函数的绝对刻度方向（零表示垂直于坐标轴绘制刻度）。

- 这些例程对于在坐标轴上手动放置刻度和标签很有用（如果将变量 `Label` 作为 `Label` 参数传入，则 `format` 参数将用于根据刻度位置格式化字符串）：

```
void xtick(picture pic=currentpicture, Label L="", explicit pair z,
           pair dir=N, string format="",
           real size=Ticksize, pen p=currentpen);
void xtick(picture pic=currentpicture, Label L="", real x,
           pair dir=N, string format="",
           real size=Ticksize, pen p=currentpen);
void ytick(picture pic=currentpicture, Label L="", explicit pair z,
           pair dir=E, string format="",
           real size=Ticksize, pen p=currentpen);
void ytick(picture pic=currentpicture, Label L="", real y,
           pair dir=E, string format="",
           real size=Ticksize, pen p=currentpen);
void tick(picture pic=currentpicture, pair z,
          pair dir, real size=Ticksize, pen p=currentpen);
void labelx(picture pic=currentpicture, Label L="", explicit pair z,
            align align=S, string format="", pen p=currentpen);
void labelx(picture pic=currentpicture, Label L="", real x,
            align align=S, string format="", pen p=currentpen);
void labelx(picture pic=currentpicture, Label L,
            string format="", explicit pen p=currentpen);
void labely(picture pic=currentpicture, Label L="", explicit pair z,
            align align=W, string format="", pen p=currentpen);
void labely(picture pic=currentpicture, Label L="", real y,
            align align=W, string format="", pen p=currentpen);
void labely(picture pic=currentpicture, Label L,
            string format="", explicit pen p=currentpen);
```

以下是一些二维图表的简单示例：

1. 此示例绘制了 $y = \exp(x)$ 的教科书风格图表，其中 y 轴始于 $y = 0$ ：

```
import graph;
size(150,0);

real f(real x) {return exp(x);}
```

```

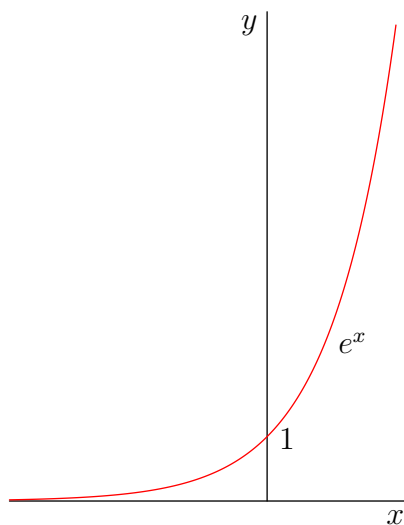
pair F(real x) {return (x,f(x));}

draw(graph(f,-4,2,operator ..),red);

xaxis("$x$");
yaxis("$y$",0);

labeled(1,E);
label("$e^x$",F(1),SE);

```



2. 下一个示例绘制了一个带图例的科研风格图表。图例的位置可以通过显式设置或使用图形用户界面（见 第 13 章 [GUI], 189 页）来调整。如果向 `add` 指定了 `UnFill(real xmargin=0, real ymargin=xmargin)` 或 `Fill(pen)` 选项，图例将遮挡任何下层对象。这里我们演示如何裁剪被标签覆盖的图片部分：

```

import graph;

size(400,200,IgnoreAspect);

real Sin(real t) {return sin(2pi*t);}
real Cos(real t) {return cos(2pi*t);}

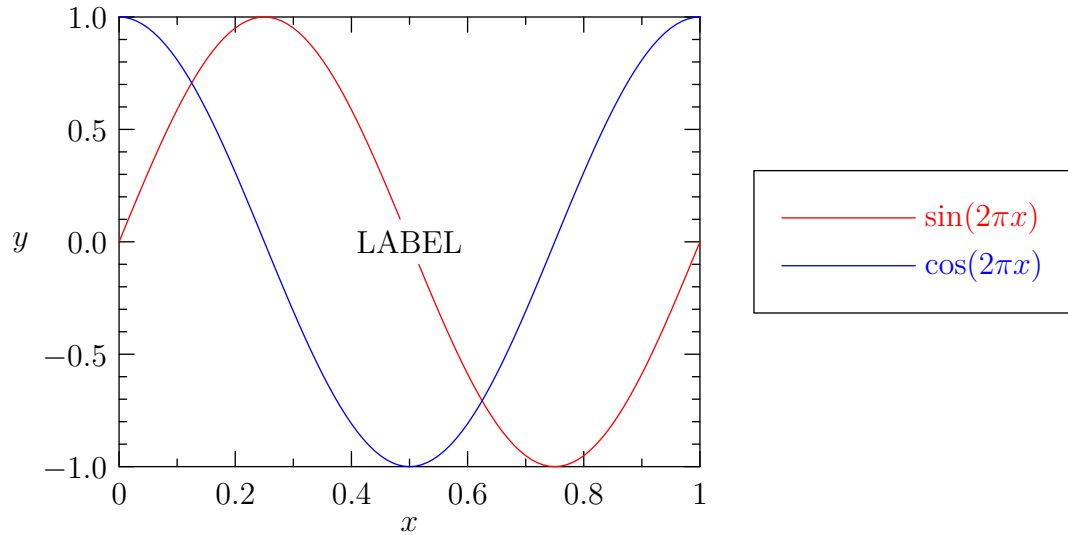
draw(graph(Sin,0,1),red,"$\sin(2\pi x)$");
draw(graph(Cos,0,1),blue,"$\cos(2\pi x)$");

xaxis("$x$",BottomTop,LeftTicks);
yaxis("$y$",LeftRight,RightTicks(trailingzero));

label("LABEL",point(0),UnFill(1mm));

```

```
add(legend(),point(E),20E,UnFill);
```



要为图表本身指定固定尺寸, 请使用 `attach`:

```
import graph;

size(250,200,IgnoreAspect);

real Sin(real t) {return sin(2pi*t);}
real Cos(real t) {return cos(2pi*t);}

draw(graph(Sin,0,1),red,"$\sin(2\pi x)$");
draw(graph(Cos,0,1),blue,"$\cos(2\pi x)$");

axis("$x$",BottomTop,LeftTicks);
yaxis("$y$",LeftRight,RightTicks(trailingzero));

label("LABEL",point(0),UnFill(1mm));

attach(legend(),truepoint(E),20E,UnFill);
```

图例每行可以有多个条目:

```
import graph;
size(8cm,6cm,IgnoreAspect);

typedef real realfcn(real);
realfcn F(real p) {
    return new real(real x) {return sin(p*x);};
}

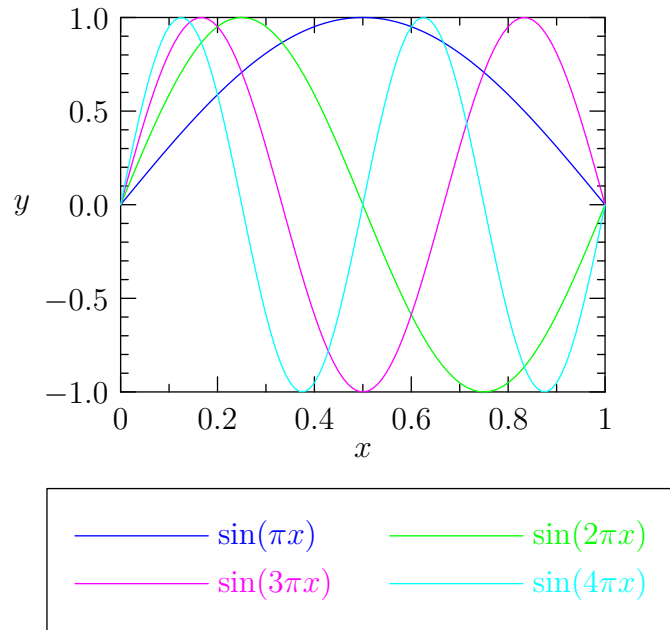
for(int i=1; i < 5; ++i)
```

```

draw(graph(F(i*pi),0,1),Pen(i),
      "$\sin("+i == 1 ? "" : (string) i)+"\pi x)$");
xaxis("$x$",BottomTop,LeftTicks);
yaxis("$y$",LeftRight,RightTicks(trailingzero));

attach(legend(2),(point(S).x,truepoint(S).y),10S,UnFill);

```



3. 此示例绘制了一个数组相对于另一个数组（两者尺寸相同）的图表，在 y 坐标轴上使用了自定义刻度位置以及更小的刻度标签字号。

```

import graph;

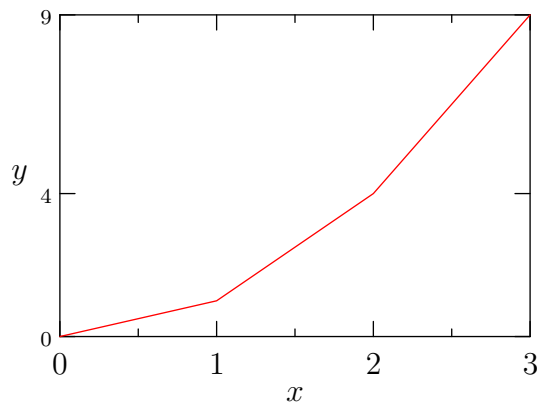
size(200,150,IgnoreAspect);

real[] x={0,1,2,3};
real[] y=x^2;

draw(graph(x,y),red);

xaxis("$x$",BottomTop,LeftTicks);
yaxis("$y$",LeftRight,
      RightTicks(Label(fontsize(8pt)),new real[]{0,4,9}));

```



4. 此示例展示了如何为从文件中读取的数据列绘制图表。

```
import graph;

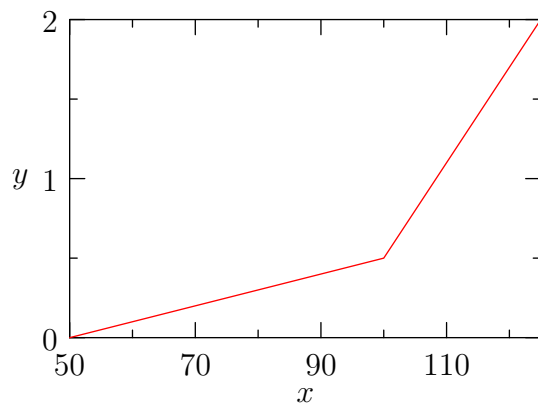
size(200,150,IgnoreAspect);

file in=input("filegraph.dat").line();
real[] [] a=in;
a=transpose(a);

real[] x=a[0];
real[] y=a[1];

draw(graph(x,y),red);

xaxis("$x$",BottomTop,LeftTicks);
yaxis("$y$",LeftRight,RightTicks);
```



5. 下一个示例绘制了坐标对数组的两个图表，使用了框(frame)对齐和数据标记(marker)。在左侧图表中，使用路径 unitcircle (见 [filltype], 49 页) 由例程

```
marker marker(path g, markroutine markroutine=marknodes,
               pen p=currentpen, filltype filltype=NoFill,
               bool above=true);
```

构造的标记绘制在每个节点下方。任何框(frame)都可以转换为标记, 使用例程

```
marker marker(frame f, markroutine markroutine=marknodes,
               bool above=true);
```

在右侧图表中, 单位 n 边正多边形 `polygon(int n)` 和单位 n 顶点循环十字 `cross(int n, bool round=true, real r=0)` (其中 r 是一个可选的“内”半径) 用于构建自定义标记框(frame)。此处 `markuniform(bool centered=false, int n, bool rotated=false)` 沿着路径弧长在 n 个均匀分布的点处添加此框, 可选地按路径局部切线的角度旋转 (若 `centered` 为 `true`, 框将在 n 个均匀间隔的弧长区间内居中)。或者, 可以使用 `markroutine marknodes` 要求将标记放置在路径的每个贝塞尔节点上, 或者使用 `markroutine markuniform(pair z(real t), real a, real b, int n)` 在 $[a, b]$ 内 t 的 n 个均匀间隔值处的点 $z(t)$ 放置标记。

以下标记是预定义的:

```
marker[] Mark={
    marker(scale(circlescale)*unitcircle),
    marker(polygon(3)),marker(polygon(4)),
    marker(polygon(5)),marker(invert*polygon(3)),
    marker(cross(4)),marker(cross(6)),marker(diamond),marker(plus);
};

marker[] MarkFill={
    marker(scale(circlescale)*unitcircle,Fill),marker(polygon(3),Fill),
    marker(polygon(4),Fill),marker(polygon(5),Fill),
    marker(invert*polygon(3),Fill),marker(diamond,Fill)
};
```

该示例还展示了 `errorbar` 例程:

```
void errorbars(picture pic=currentpicture, pair[] z, pair[] dp,
               pair[] dm={}, bool[] cond={}, pen p=currentpen,
               real size=0);

void errorbars(picture pic=currentpicture, real[] x, real[] y,
               real[] dpx, real[] dpy, real[] dmx={}, real[] dmy={},
               bool[] cond={}, pen p=currentpen, real size=0);
```

在此, 误差的正向和负向范围由数对数组 `dp` 和可选数对数组 `dm` 元素的绝对值给出。若未指定 `dm`, 则假定误差的正向和负向范围相等。

```
import graph;

picture pic;
real xsize=200, ysize=140;
size(pic,xsize,ysize,IgnoreAspect);

pair[] f={(5,5),(50,20),(90,90)};
pair[] df={(0,0),(5,7),(0,5)};

errorbars(pic,f,df,red);
draw(pic,graph(pic,f),"legend",
```

```

marker(scale(0.8mm)*unitcircle,red,FillDraw(blue),above=false));

scale(pic,true);

axis(pic,"$x$",BottomTop,LeftTicks);
axis(pic,"$y$",LeftRight,RightTicks);
add(pic,legend(pic),point(pic,NW),20SE,UnFill);

picture pic2;
size(pic2,xsize,ysize,IgnoreAspect);

frame mark;
filldraw(mark,scale(0.8mm)*polygon(6),green,green);
draw(mark,scale(0.8mm)*cross(6),blue);

draw(pic2,graph(pic2,f),marker(mark,markuniform(5)));

scale(pic2,true);

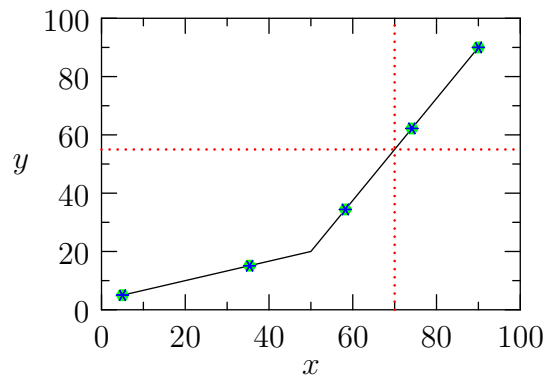
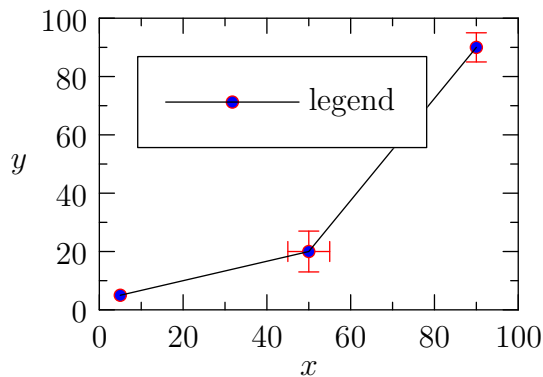
axis(pic2,"$x$",BottomTop,LeftTicks);
axis(pic2,"$y$",LeftRight,RightTicks);

yequals(pic2,55.0,red+Dotted);
xequals(pic2,70.0,red+Dotted);

// Fit pic to W of origin:
add(pic.fit(),(0,0),W);

// Fit pic2 to E of (5mm,0):
add(pic2.fit(),(5mm,0),E);

```



6. 还可以指定自定义标记例程:

```
import graph;
```

```

size(200,100,IgnoreAspect);

markroutine marks() {
    return new void(picture pic=currentpicture, frame f, path g) {
        path p=scale(1mm)*unitcircle;
        for(int i=0; i <= length(g); ++i) {
            pair z=point(g,i);
            frame f;
            if(i % 4 == 0) {
                fill(f,p);
                add(pic,f,z);
            } else {
                if(z.y > 50) {
                    pic.add(new void(frame F, transform t) {
                        path q=shift(t*z)*p;
                        unfill(F,q);
                        draw(F,q);
                    });
                } else {
                    draw(f,p);
                    add(pic,f,z);
                }
            }
        }
    };
}

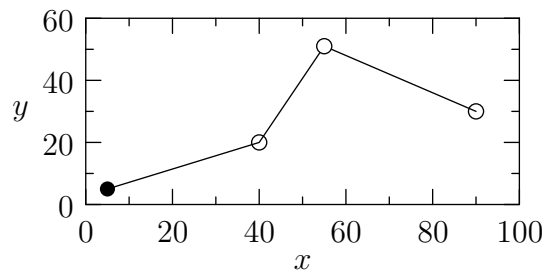
pair[] f={(5,5),(40,20),(55,51),(90,30)};

draw(graph(f),marker(marks()));

scale(true);

xaxis("$x$",BottomTop,LeftTicks);
yaxis("$y$",LeftRight,RightTicks);

```



7. 此示例展示了如何使用任意字符串标记坐标轴。

```
import graph;

size(400,150,IgnoreAspect);

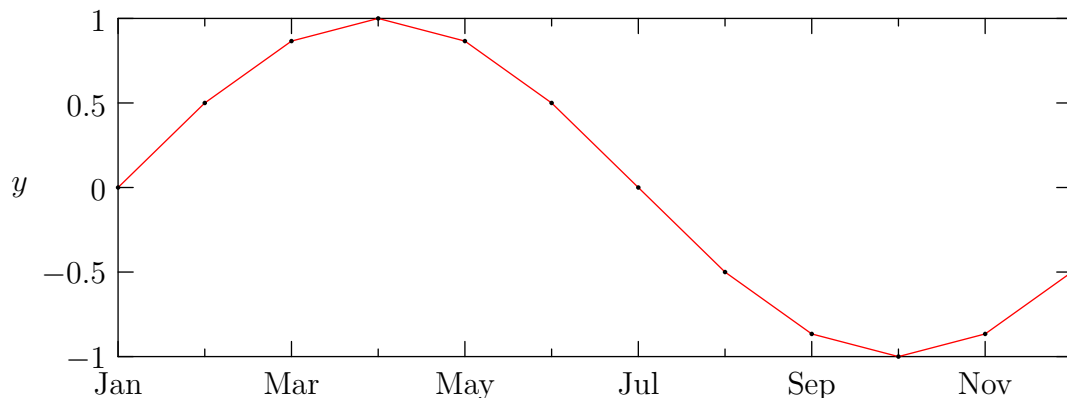
real[] x=sequence(12);
real[] y=sin(2pi*x/12);

scale(false);

string[] month={"Jan","Feb","Mar","Apr","May","Jun",
               "Jul","Aug","Sep","Oct","Nov","Dec"};

draw(graph(x,y),red,MarkFill[0]);

xaxis(BottomTop,LeftTicks(new string(real x) {
    return month[round(x % 12)];}));
yaxis("$y$",LeftRight,RightTicks(4));
```



8. 下一个示例绘制了参数曲线的图表。可以通过取消对

```
xlimits(picture pic=currentpicture, real min=-infinity,
        real max=infinity, bool crop=NoCrop);
```

以及类似函数 `ylimits` 调用的注释，将图片 `pic` 对应坐标轴的极限设置为指定的 `min` 与 `max` 值。或者，可以使用函数

```
void limits(picture pic=currentpicture, pair min, pair max, bool crop=NoCrop);
```

将坐标轴限制在对角顶点位于给定数对上的框内。若 `crop=Crop`，图片 `pic` 中的现有对象将被裁剪至给定极限范围内。函数 `crop(picture pic)` 可用于将图表裁剪至当前的图表极限。

```
import graph;
```

```
size(0,200);
```

```

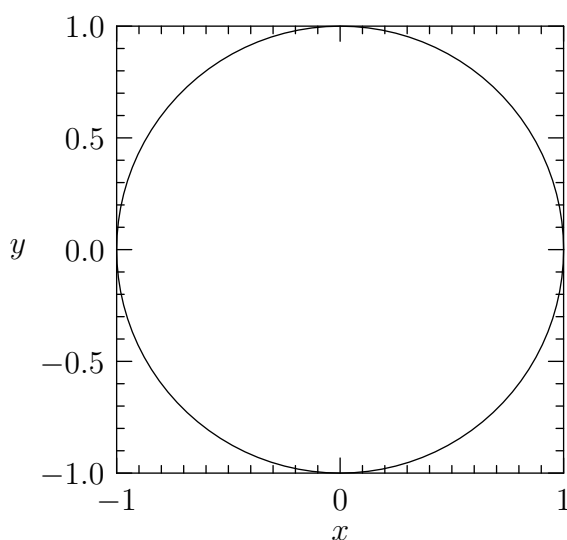
real x(real t) {return cos(2pi*t);}
real y(real t) {return sin(2pi*t);}

draw(graph(x,y,0,1));

//limits((0,-1),(1,0),Crop);

xaxis("$x$",BottomTop,LeftTicks);
yaxis("$y$",LeftRight,RightTicks(trailingzero));

```



函数

```

guide graphwithderiv(pair f(real), pair fprime(real), real a, real b,
    int n=ngraph#10);

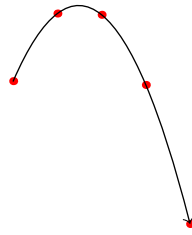
```

可用于构造区间 $[a,b]$ 上参数函数 f 的图表，其中 n 个贝塞尔线段的控制点由指定的导数 $fprime$ 决定：

```

unitsize(2cm);
import graph;
pair F(real t) {
    return (1.3*t,-4.5*t^2+3.0*t+1.0);
}
pair Fprime(real t) {
    return (1.3,-9.0*t+3.0);
}
path g=graphwithderiv(F,Fprime,0,0.9,4);
dot(g,red);
draw(g,arrow=Arrow(TeXHead));

```



下一个示例演示了如何提取通用的坐标轴缩放因子。

```
import graph;

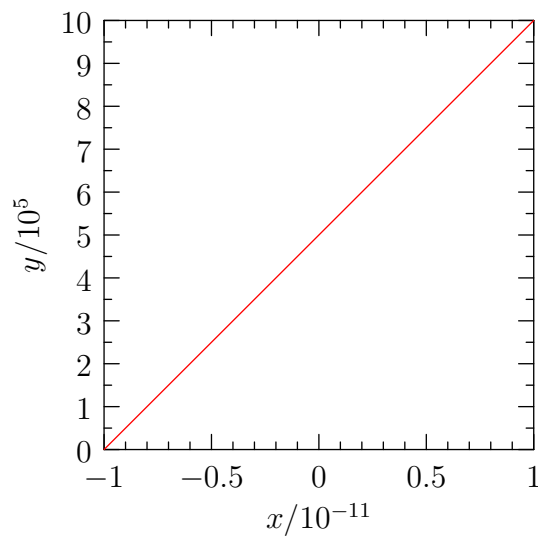
axiscoverage=0.9;
size(200,IgnoreAspect);

real[] x={-1e-11,1e-11};
real[] y={0,1e6};

real xscale=round(log10(max(x)));
real yscale=round(log10(max(y)))-1;

draw(graph(x*10^(-xscale),y*10^(-yscale)),red);

xaxis("$x/10^{ "+(string) xscale+" }$",BottomTop,LeftTicks);
yaxis("$y/10^{ "+(string) yscale+" }$",LeftRight,RightTicks(trailingzero));
```



可以通过以下 `scale` 例程之一请求坐标轴缩放，和/或禁止坐标轴极限的自动选择：

```
void scale(picture pic=currentpicture, scaleT x, scaleT y);

void scale(picture pic=currentpicture, bool xautoscale=true,
           bool yautoscale=xautoscale, bool zautoscale=yautoscale);
```

这会设置图片 `pic` 的缩放。`graph` 例程接受一个可选的 `picture` 参数以确定要使用的适当缩放；若未指定，则使用为 `currentpicture` 设置的缩放。

两个常用的缩放例程 `Linear` 和 `Log` 在 `graph` 中预定义。

所有图片坐标（包括路径中的坐标以及传给 `label` 和 `limits` 函数的坐标）总是被当作线性（缩放后）坐标处理。使用

```
pair Scale(picture pic=currentpicture, pair z);
```

将图表坐标转换为缩放后的图片坐标。

x 与 y 分量可以使用类似例程分别独立缩放

```
real ScaleX(picture pic=currentpicture, real x);
real ScaleY(picture pic=currentpicture, real y);
```

预定义的缩放例程可以传入两个可选的布尔参数：`automin=false` 与 `automax=automin`。它们默认为 `false`，但可分别设置为 `true` 以启用坐标轴最小值和最大值的“合适”自动选择。`Linear` 缩放还可以接受可选的最终参数：乘性缩放因子和截距（对于深度轴，`Linear(-1)` 请求坐标轴反转）。

例如，要绘制函数的双对数(log/log)图表，请使用 `scale(Log,Log)`：

```
import graph;

size(200,200,IgnoreAspect);

real f(real t) {return 1/t;}

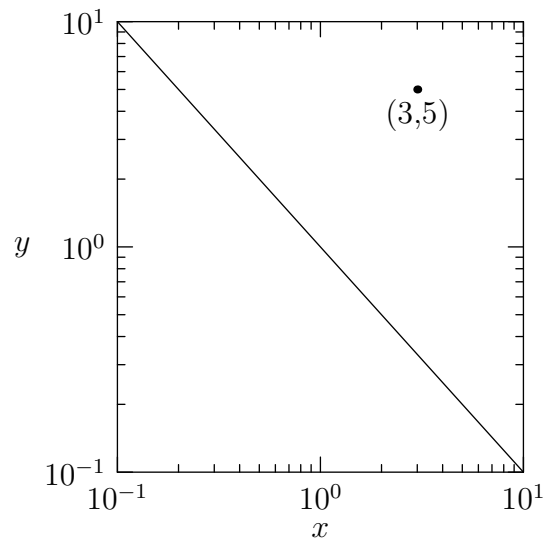
scale(Log,Log);

draw(graph(f,0.1,10));

//limits((1,0.1),(10,0.5),Crop);

dot(Label("(3,5)",align=S),Scale((3,5)));

xaxis("$x$",BottomTop,LeftTicks);
yaxis("$y$",LeftRight,RightTicks);
```

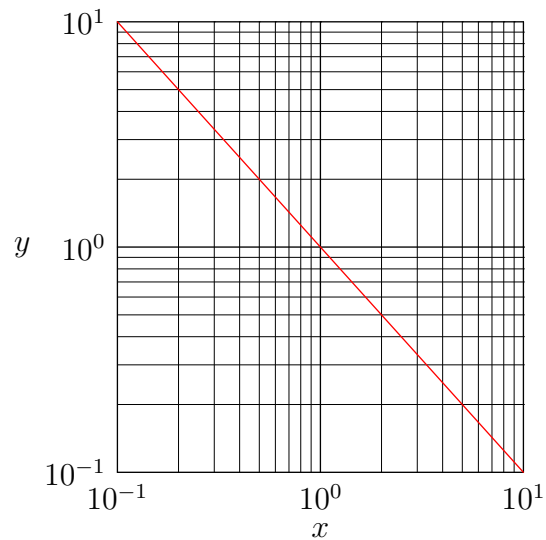


通过延伸刻度线，可以轻松生成对数网格：

```
import graph;
size(200,200,IgnoreAspect);

real f(real t) {return 1/t;}

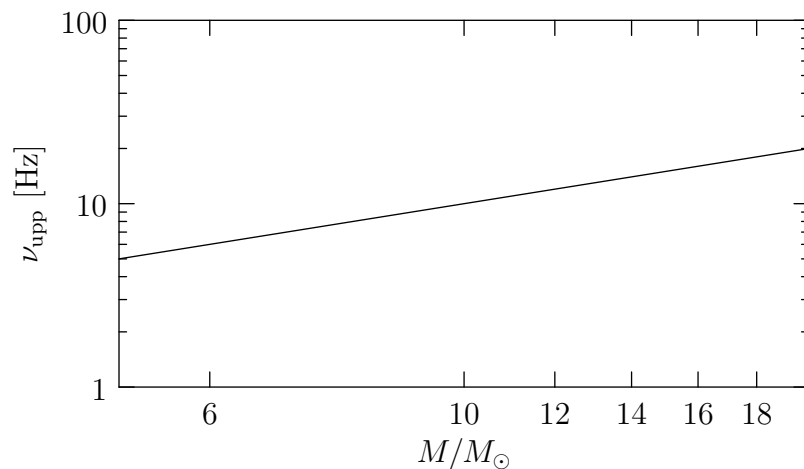
scale(Log,Log);
draw(graph(f,0.1,10),red);
pen thin=lineWidth(0.5*lineWidth());
xaxis("$x$",BottomTop,LeftTicks(begin=false,end=false,extend=true,
                                ptick=thin));
yaxis("$y$",LeftRight,RightTicks(begin=false,end=false,extend=true,
                                ptick=thin));
```



也可以为对数坐标轴指定自定义刻度位置和格式:

```
import graph;

size(300,175,IgnoreAspect);
scale(Log,Log);
draw(graph(identity,5,20));
xlimits(5,20);
ylimits(1,100);
xaxis("$M/M_\odot$",BottomTop,LeftTicks(DefaultFormat,
                                     new real[] {6,10,12,14,16,18}));
yaxis("$\nu_{\rm upp}$ [Hz]",LeftRight,RightTicks(DefaultFormat));
```



绘制以其他数为底的对数图表也很容易:

```
import graph;
size(200,IgnoreAspect);

// Base-2 logarithmic scale on y-axis:

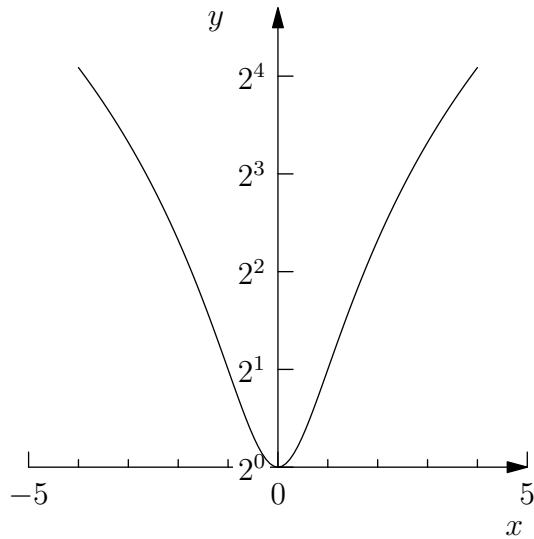
real log2(real x) {static real log2=log(2); return log(x)/log2;}
real pow2(real x) {return 2^x;}

scaleT yscale=scaleT(log2,pow2,logarithmic=true);
scale(Linear,yscale);

real f(real x) {return 1+x^2;}

draw(graph(f,-4,4));

yaxis("$y$",ymin=1,ymax=f(5),RightTicks(Label(Fill(white))),EndArrow);
xaxis("$x$",xmin=-5,xmax=5,LeftTicks,EndArrow);
```



这里是一个折断(broken)线性 x 轴与对数 y 轴的示例, 它们分别省略了区间 $[3,8]$ 和 $[100,1000]$ 。对于对数坐标轴, 折断端点会自动取整为底数的最近整数次幂。

```
import graph;

size(200,150,IgnoreAspect);

// Break the x axis at 3; restart at 8:
real a=3, b=8;

// Break the y axis at 100; restart at 1000:
real c=100, d=1000;

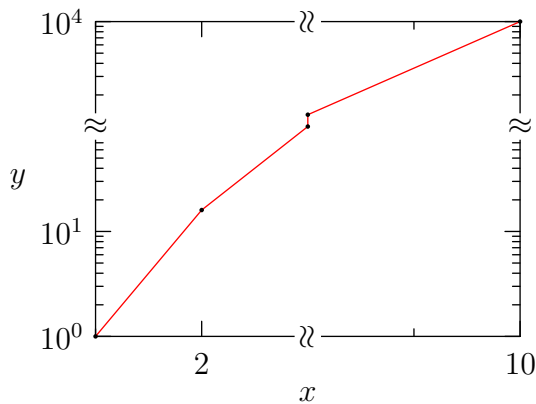
scale(Broken(a,b),BrokenLog(c,d));

real[] x={1,2,4,6,10};
real[] y=x^4;

draw(graph(x,y),red,MarkFill[0]);

xaxis("$x$",BottomTop,LeftTicks(Break(a,b)));
yaxis("$y$",LeftRight,RightTicks(Break(c,d)));

label(rotate(90)*Break,(a,point(S).y));
label(rotate(90)*Break,(a,point(N).y));
label(Break,(point(W).x,ScaleY(c)));
label(Break,(point(E).x,ScaleY(c)));
```



9. Asymptote 可以使用以下例程绘制次要坐标轴:

```
picture secondaryX(picture primary=currentpicture, void f(picture));
picture secondaryY(picture primary=currentpicture, void f(picture));
```

在此示例中, `secondaryY` 用于绘制相对于主要对数 y 轴的次要线性 y 轴:

```
import graph;
texpreamble("\def\Arg{\mathop {\rm Arg}\nolimits}");

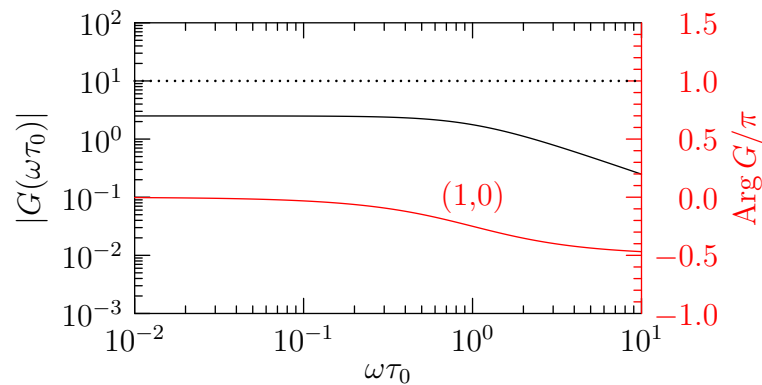
size(10cm,5cm,IgnoreAspect);

real ampl(real x) {return 2.5/sqrt(1+x^2);}
real phas(real x) {return -atan(x)/pi;}

scale(Log,Log);
draw(graph(ampl,0.01,10));
ylimits(0.001,100);

xaxis("$\omega\tau_0$",BottomTop,LeftTicks);
yaxis("$|G(\omega\tau_0)|$",Left,RightTicks);

picture q=secondaryY(new void(picture pic) {
    scale(pic,Log,Linear);
    draw(pic,graph(pic,phas,0.01,10),red);
    ylimits(pic,-1.0,1.5);
    yaxis(pic,"$\Arg G/\pi$",Right,red,
        LeftTicks("$\% \#.1f$",
            begin=false,end=false));
    yequal(pic,1,Dotted);
});
label(q,"(1,0)",Scale(q,(1,0)),red);
add(q);
```



可以像这样绘制次要对数 y 轴:

```
import graph;

size(9cm,6cm,IgnoreAspect);
string data="secondary.csv";

file in=input(data).line().csv();

string[] titlelabel=in;
string[] columnlabel=in;

real[] [] a=in;
a=transpose(a);
real[] t=a[0], susceptible=a[1], infectious=a[2], dead=a[3], larvae=a[4];
real[] susceptibleM=a[5], exposed=a[6], infectiousM=a[7];

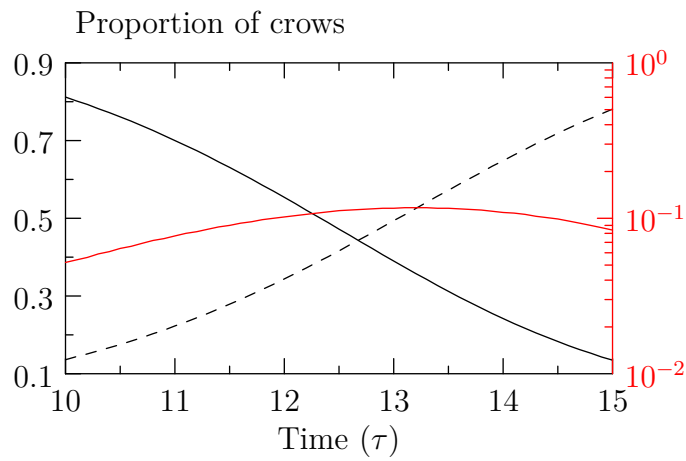
scale(true);

draw(graph(t,susceptible,t >= 10 & t <= 15));
draw(graph(t,dead,t >= 10 & t <= 15),dashed);

xaxis("Time ( $\tau$ )",BottomTop,LeftTicks);
yaxis(Left,RightTicks);

picture secondary=secondaryY(new void(picture pic) {
    scale(pic,Linear(true),Log(true));
    draw(pic,graph(pic,t,infectious,t >= 10 & t <= 15),red);
    yaxis(pic,Right,red,LeftTicks(begin=false,end=false));
});

add(secondary);
label(shift(5mm*N)*"Proportion of crows",point(NW),E);
```



10. 这是一个直方图示例，使用了 `stats` 模块。

```
import graph;
import stats;

size(400,200,IgnoreAspect);

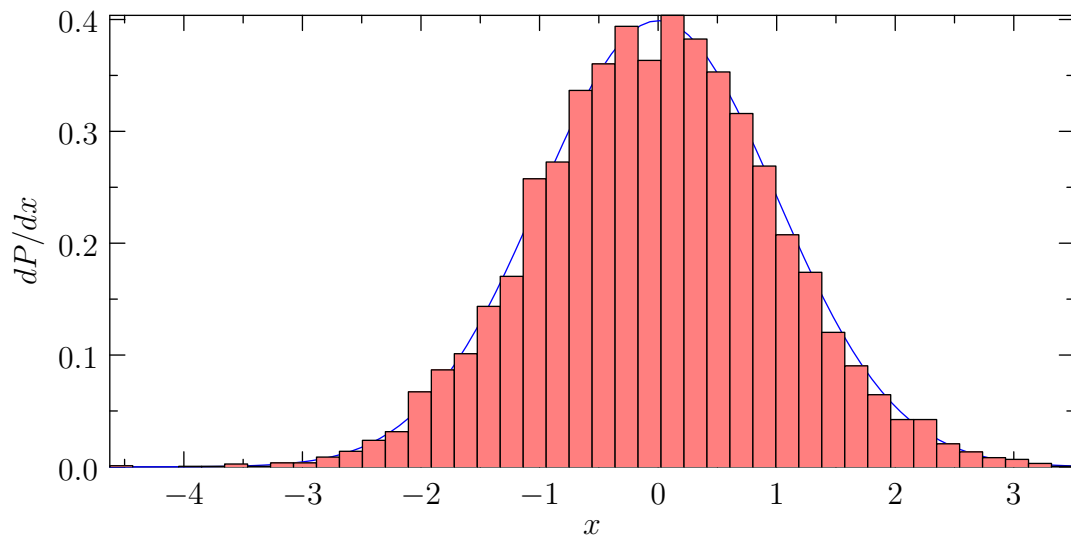
int n=10000;
real[] a=new real[n];
for(int i=0; i < n; ++i) a[i]=Gaussrand();

draw(graph(Gaussian,min(a),max(a)),blue);

// Optionally calculate "optimal" number of bins a la Shimazaki and Shinomoto.
int N=bins(a);

histogram(a,min(a),max(a),N,normalize=true,low=0,lightred,black,bars=true);

axis("$x$",BottomTop,LeftTicks);
yaxis("$dP/dx$",LeftRight,RightTicks(trailingzero));
```



11. 这是一个从文件中读取列数据并进行最小二乘拟合的示例，使用了 `stats` 模块。

```
size(400,200,IgnoreAspect);

import graph;
import stats;

file fin=input("leastquares.dat").line();

real[] [] a=fin;
a=transpose(a);

real[] t=a[0], rho=a[1];

// Read in parameters from the keyboard:
//real first=getreal("first");
//real step=getreal("step");
//real last=getreal("last");

real first=100;
real step=50;
real last=700;

// Remove negative or zero values of rho:
t=rho > 0 ? t : null;
rho=rho > 0 ? rho : null;

scale(Log(true),Linear(true));

int n=step > 0 ? ceil((last-first)/step) : 0;

real[] T,xi,dxi;
```

```

for(int i=0; i <= n; ++i) {
    real first=first+i*step;
    real[] logrho=(t >= first & t <= last) ? log(rho) : null;
    real[] logt=(t >= first & t <= last) ? -log(t) : null;

    if(logt.length < 2) break;

    // Fit to the line logt=L.m*logrho+L.b:
    linefit L=leastquares(logt,logrho);

    T.push(first);
    xi.push(L.m);
    dxi.push(L.dm);
}

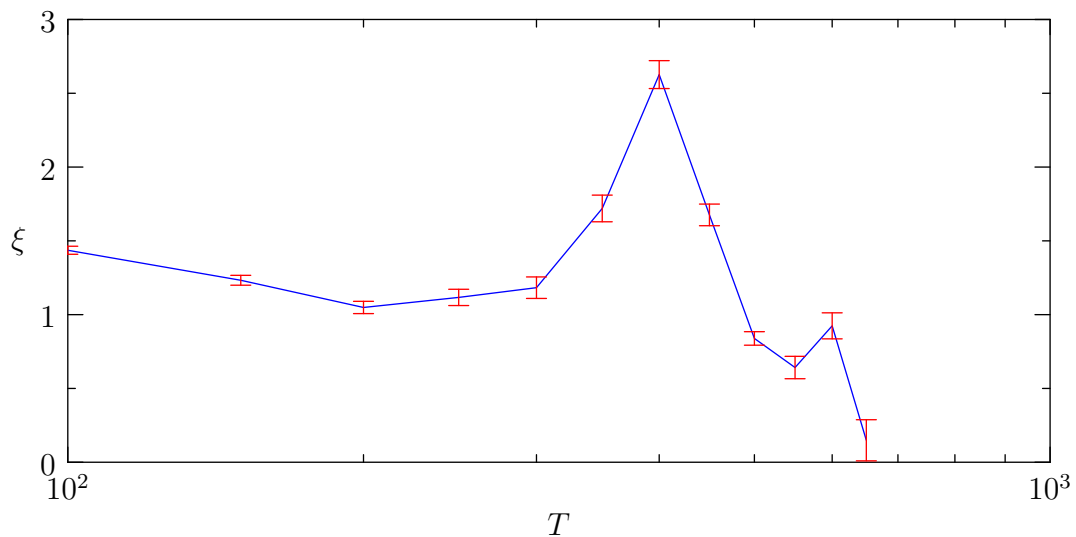
draw(graph(T,xi),blue);
errorbars(T,xi,dxi,red);

crop();

ylimits(0);

axis("$T$",BottomTop,LeftTicks);
axis("$\xi$",LeftRight,RightTicks);

```



12. 这是一个演示通用 axis 例程的示例。

```

import graph;
size(0,100);

```

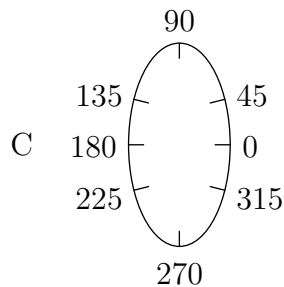
```

path g=ellipse((0,0),1,2);

scale(true);

axis(Label("C",align=10W),g,LeftTicks(endlabel=false,8,end=false),
      ticklocate(0,360,new real(real v) {
        path h=(0,0)-max(abs(max(g)),abs(min(g)))*dir(v);
        return intersect(g,h)[0];}))

```



13. 要沿着路径弧长绘制均匀间隔 n 个矢量的向量场(vector field), 请使用例程

```

picture vectorfield(path vector(real), path g, int n, bool truesize=false,
                    pen p=currentpen, arrowbar arrow=Arrow);

```

如该流场简单示例所示:

```

import graph;
defaultpen(1.0);

size(0,150,IgnoreAspect);

real arrowsize=4mm;
real arrowlength=2arrowsize;

typedef path vector(real);

// Return a vector interpolated linearly between a and b.
vector vector(pair a, pair b) {
  return new path(real x) {
    return (0,0)-arrowlength*interp(a,b,x);
  };
}

real f(real x) {return 1/x;}

real epsilon=0.5;
path g=graph(f,epsilon,1/epsilon);

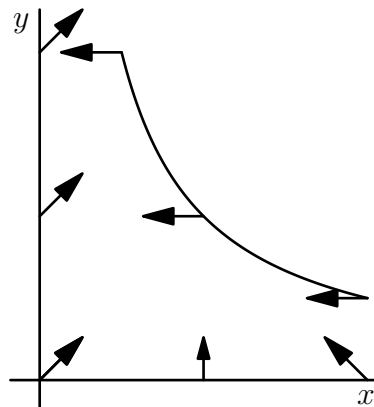
```

```

int n=3;
draw(g);
xaxis("$x$");
yaxis("$y$");

add(vectorfield(vector(W,W),g,n,true));
add(vectorfield(vector(NE,NW),(0,0)-(point(E).x,0),n,true));
add(vectorfield(vector(NE,NE),(0,0)-(0,point(N).y),n,true));

```



14. 要在 $\text{box}(a,b)$ 中绘制 $n_x \times n_y$ 个矢量的向量场, 请使用例程

```

picture vectorfield(path vector(pair), pair a, pair b,
    int nx=nmesh, int ny=nx, bool truesize=false,
    real maxlength=truesize ? 0 : maxlength(a,b,nx,ny),
    bool cond(pair z)=null, pen p=currentpen,
    arrowbar arrow=Arrow, margin margin=PenMargin)

```

如该示例所示:

```

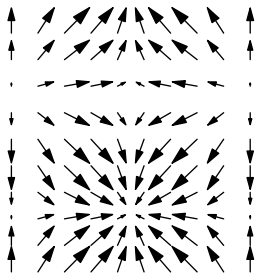
import graph;
size(100);

pair a=(0,0);
pair b=(2pi,2pi);

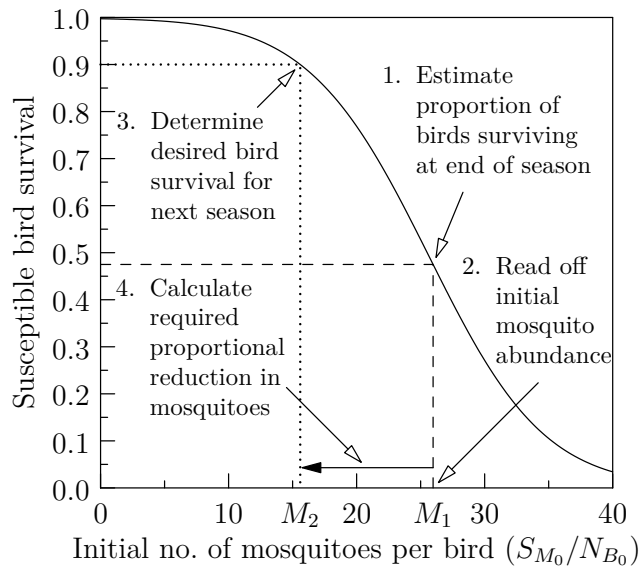
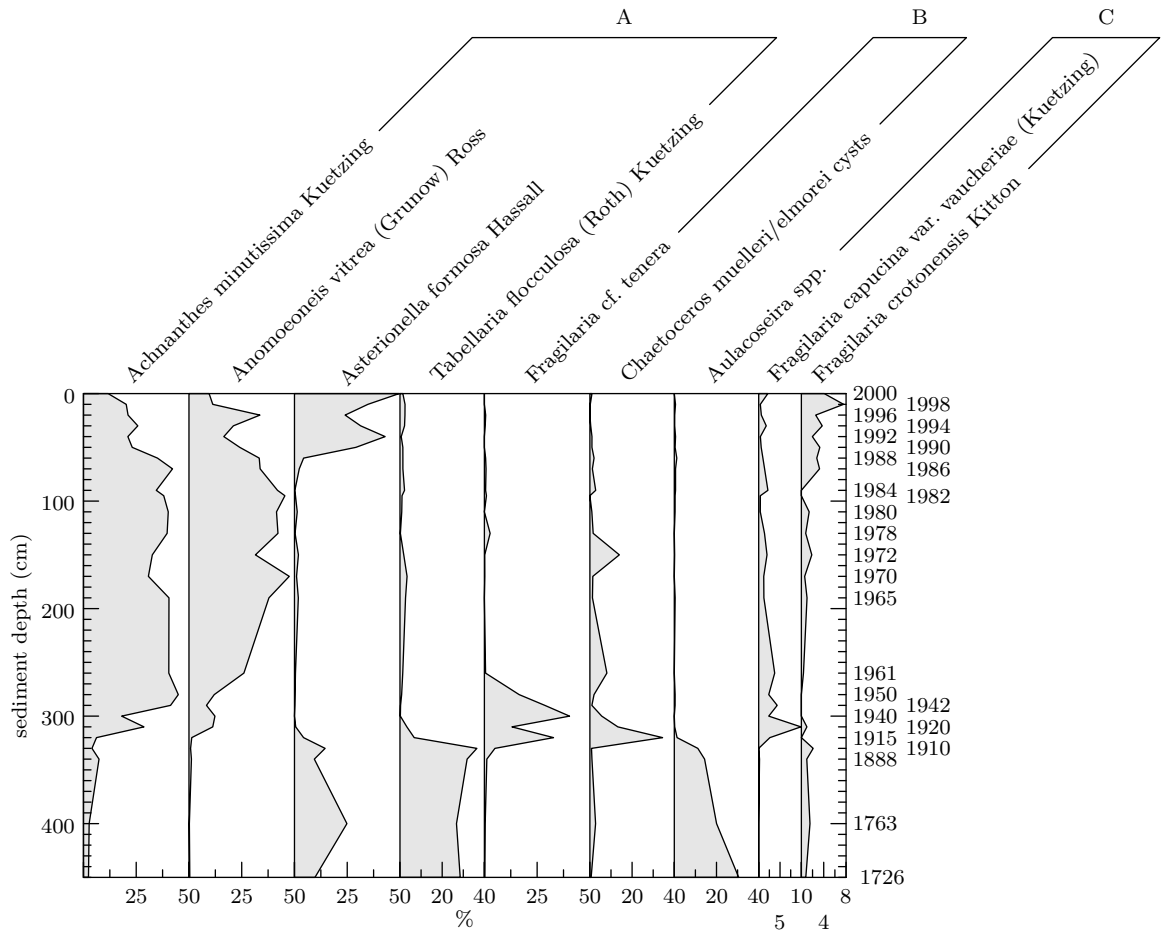
path vector(pair z) {return (sin(z.x),cos(z.y));}

add(vectorfield(vector,a,b));

```



15. 以下科学图表展示了 Asymptote 图形例程的许多特性，它们生成自示例 `diatom.asy` 与 `westnile.asy`，使用了 `diatom.csv` 与 `westnile.csv` 中的逗号分隔数据。



8.2 three

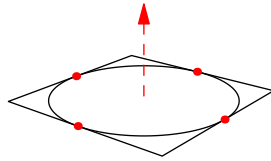
该模块将 Asymptote 中 `guide` (引导) 与 `path` (路径) 的概念完整扩展到了三维空间。它引入了新的类型 `guide3`、`path3` 以及 `surface`。三维空间中的 `guide` 使用与二维空间相同的语法进行指定, 区别仅在于节点和方向指定符中使用三元组 (x,y,z) 代替了数对 (x,y) 。这项对 John Hobby 样条(spline)算法的推广在三维旋转、缩放和平移变换下具有形状不变性, 且在平面情形下退化为 Asymptote、MetaPost 及 MetaFont 所使用的二维算法[参见 J. C. Bowman, *Proceedings in Applied Mathematics and Mechanics*, 7:1, 2010021-2010022 (2007)]。

例如, XY 平面上的单位圆可以像这样进行填充与绘制:

```
import three;

size(100);

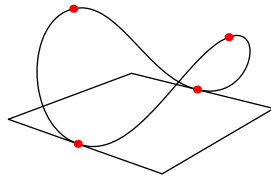
path3 g=(1,0,0)..(0,1,0)..(-1,0,0)..(0,-1,0)..cycle;
draw(g);
draw(O-Z,red+dashed,Arrow3);
draw(((1,-1,0)-(1,-1,0)-(1,1,0)-(-1,1,0)-cycle));
dot(g,red);
```



然后变形为一个鞍面:

```
import three;

size(100,0);
path3 g=(1,0,0)..(0,1,1)..(-1,0,0)..(0,-1,1)..cycle;
draw(g);
draw(((1,-1,0)-(1,-1,0)-(1,1,0)-(-1,1,0)-cycle));
dot(g,red);
```



模块 `three` 提供了用于在二维路径与三维路径之间互相转换的构造函数:

```
path3 path3(path p, triple plane(pair)=XYplane);
path path(path3 p, pair P(triple)=xyplane);
```

贝塞尔曲面(Bezier surface)是贝塞尔曲线在二维上的自然推广, 它在 `three_surface.asy` 中被定义为一个包含贝塞尔块(Bezier patch)数组的结构体。曲面可以使用以下例程之一绘制:

```

void draw(picture pic=currentpicture, surface s, int nu=1, int nv=1,
          material surfacepen=currentpen, pen meshpen=nullpen,
          light light=currentlight, light meshlight=nolight, string name="",
          render render=defaultrender);
void draw(picture pic=currentpicture, surface s, int nu=1, int nv=1,
          material[] surfacepen, pen meshpen,
          light light=currentlight, light meshlight=nolight, string name="",
          render render=defaultrender);
void draw(picture pic=currentpicture, surface s, int nu=1, int nv=1,
          material[] surfacepen, pen[] meshpen=nullpens,
          light light=currentlight, light meshlight=nolight, string name="",
          render render=defaultrender);

```

参数 `nu` 和 `nv` 指定为每个贝塞尔块绘制可选网格线时的细分次数（这些参数对贝塞尔三角形会被忽略）。可选参数 `name` 用作在 PRC 模型树中命名曲面块的前缀。这里的 `material` 是一个在 `three.light.asy` 中定义的结构体：

```

struct material {
    pen[] p; // diffusepen,emissivepen,specularpen
    real opacity;
    real shininess;
    real metallic;
    real fresnel0;
}

```

这些材质属性用于利用 `plain_prethree.asy` 与 `three.light.asy` 中定义的光照属性来实现基于物理的渲染(PBR)：

```

struct light {
    real[] [] diffuse;
    real[] [] specular;
    pen background=nullpen; // Background color of the canvas.
    real specularfactor;
    triple[] position; // Only directional lights are currently implemented.
}

```

```
light Viewport=light(specularfactor=3,(0.25,-0.25,1));
```

```
light White=light(new pen[] {rgb(0.38,0.38,0.45),rgb(0.6,0.6,0.67),
                             rgb(0.5,0.5,0.57)},specularfactor=3,
                  new triple[] {(-2,-1.5,-0.5),(2,1.1,-2.5),(-0.5,0,2)});
```

```
light Headlamp=light(gray(0.8),specular=gray(0.7),
                     specularfactor=3,dir(42,48));
```

```
currentlight=Headlamp;
```

```
light nolight;
```

`currentlight.background`(或指定 `light` 的 `background` 成员) 可用于设置二维(或三维)图像的背景颜色。默认背景在 HTML 图像中为白色, 而在所有其他格式中为透明。对于三维 WebGL 图像, 可以通过以下方式请求完全透明的背景: `currentlight.background=black+opacity(0.0)`;

`render`

可以将函数 `render()` 赋值给可选的 `render` 参数, 从而通过如下参数向曲面绘制例程传递专门的渲染选项:

```
bool tessellate;    // use tessellated mesh to store straight patches
real margin;        // shrink amount for rendered OpenGL viewport, in bp.
bool partnames;     // assign part name indices to compound objects
bool defaultnames;  // assign default names to unnamed objects
interaction interaction; // billboard interaction mode
```

以及 `three.asy` 中描述的旧版 PRC 格式渲染参数。

Asymptote 还通过设置 `settings.ibl=true` 支持基于图像的光照(image-based lighting)。这会使用由 `-imageDir` 指定目录(默认为 `ibl`)中的预渲染 EXR 图像, 或者对于 WebGL 渲染, 使用由 `-imageURL` 指定的 URL(默认为 <https://vectorgraphics.gitlab.io/asymptote/ibl>)。可以使用 Asymptote 源码目录的 `cudareflect` 子目录中的 `reflect` 程序在 NVIDIA GPU 上生成额外的渲染图像。

示例文件 `BezierSurface.asy`、`teapot.asy`、`teapotIBL.asy` 和 `parametricsurface.asy` 中包含了贝塞尔曲面示例。

结构体 `render` 包含专门的渲染选项, 详见模块 `three` 开头的说明。

示例 `elevation.asy` 说明了如何绘制带有依块颜色(patch-dependent colors)的曲面。示例 `sphericalharmonic.asy`、`vertexshading.asy` 和 `smoothelevation.asy` 说明了依顶点颜色(vertex-dependent colors), 它们受到 Asymptote 原生的 OpenGL/WebGL 渲染器、三维矢量输出格式 V3D 以及二维矢量输出(`settings.render=0`)的支持。由于旧版的 PRC 输出格式不支持贝塞尔曲面的顶点着色, 因此 PRC 曲面块使用四个顶点颜色的平均值进行着色。

可以通过构造函数从闭合的 `path3` 构造一个曲面:

```
surface surface(path3 external, triple[] internal=new triple[],
                pen[] colors=new pen[], bool3 planar=default);
```

然后填充:

```
draw(surface(unitsquare3,new triple[] {X,Y,Z,0}),red);
draw(surface(0-X{Y}..Y{-X}-cycle,new triple[] {Z}),red);
draw(surface(path3(polygon(5))),red,nolight);
draw(surface(unitcircle3),red,nolight);
draw(surface(unitcircle3,new pen[] {red,green,blue,black}),nolight);
```

第一个示例绘制一个贝塞尔块, 第二个示例绘制一个贝塞尔三角形。第三和第四个示例是平面曲面。最后一个示例构造一个带有特定顶点颜色的曲面块。平面 `plane` 中的三维平面曲面可以通过构造函数从二维闭合路径 `g` 构造:

```
surface surface(path p, triple plane(pair)=XYplane);
```

然后填充:

```
draw(surface((0,0)-E+2N-2E-E+N..0.2E..cycle),red);
```

平面贝塞尔曲面块是使用 Orest Shardt 的 `bezulate` 例程构造的, 该例程将由简单闭合路径(仅在端点相交)所包围的区域(根据 `zerowinding` 填充规则, 可能是非单连通的)分解为由长度不大于 4 的闭合路径包围的子区域。

还存在一个更高效的例程，用于绘制由许多三维三角形构成的剖分（网格），支持指定顶点以及可选的法线或顶点颜色：

```
void draw(picture pic=currentpicture, triple[] v, int[][] vi,
          triple[] n={}, int[][] ni=vi, material m=currentpen, pen[] p={},
          int[][] pi=vi, light light=currentlight);
```

在此，triple 数组 `v` 列出了（通常不同的）顶点，而数组 `vi` 包含长度为 3 的整数数组，其中包含构成每个三角形顶点的 `v` 中元素的索引。类似地，参数 `n` 和 `ni` 包含可选的法线数据，而 `p` 和 `pi` 包含可选的画笔顶点数据。如果为一个顶点指定了多个法线或画笔，则使用最后一个。该剖分功能的示例在 `triangles.asy` 中给出。

任意粗细的三维曲线和线帽（OpenGL 标准并不要求实现提供）可以通过以下函数构造：

```
tube tube(path3 p, real width, render render=defaultrender);
```

这将返回一个 tube 结构体，代表大致以 `g` 为中心、直径为 `width` 的管状体。tube 结构体由一个曲面 `s` 和实际的管中心 `path3 center` 组成。将粗线绘制为管状体渲染速度可能较慢，特别是在 Adobe Reader 渲染器中。可以使用设置 `thick=false` 来禁用此功能，并强制所有线都以 `linewidth(0)` 绘制（无论分辨率如何均为 1 像素宽）。默认情况下，三维中的网格线和等高线始终以细线绘制，除非在画笔参数中给出了显式的线宽，或者设置 `thin` 被设为 `false`。在 `plain-pens.asy` 中定义的画笔 `thin()` 和 `thick()` 也可以用于覆写特定绘图命令的这些默认值。

查看三维 Asymptote 输出有 6 种选择：

1. 使用 Asymptote 原生的基于 OpenGL 的自适应渲染器（带有命令行选项 `-v` 以及默认设置 `outformat=""` 和 `render=-1`）。在支持多重采样图形的 UNIX 系统上，采样宽度可以通过设置 `multisample` 控制。物理屏幕像素与逻辑屏幕像素的比例可以通过设置 `devicepixelratio` 指定。初始屏幕位置可以通过 `pair` 设置 `position` 指定，其中负值将被解释为相对于对应的最大屏幕尺寸。默认设置

```
import settings;
leftbutton=new string[] {"rotate","zoom","shift","pan"};
middlebutton=new string[] {""};
rightbutton=new string[] {"zoom","rotateX","rotateY","rotateZ"};
wheelup=new string[] {"zoomin"};
wheeldown=new string[] {"zoomout"};
```

如下绑定鼠标按键：

- 左键：旋转
- Shift+左键：缩放
- Ctrl+左键：平移视口
- Alt+左键：平移
- 滚轮向上：放大
- 滚轮向下：缩小
- 右键：缩放
- Shift+右键：绕 X 轴旋转
- Ctrl+右键：绕 Y 轴旋转
- Alt+右键：绕 Z 轴旋转

键盘绑定如下：

- h：复位视角(home)

- f: 切换适应屏幕(fitscreen)
 - x: 绕 X 轴自转
 - y: 绕 Y 轴自转
 - z: 绕 Z 轴自转
 - s: 停止自转
 - m: 渲染模式 (实体/块/网格, solid/patch/mesh)
 - e: 导出
 - c: 显示相机参数
 - p: 播放动画
 - r: 反向播放动画
 - 空格: 单步执行动画
 - +: 放大
 - =: 放大
 - >: 放大
 - -: 缩小
 - _: 缩小
 - <: 缩小
 - q: 退出
 - Ctrl-q: 退出
2. 使用命令行选项 `-f html` (或设置 `outformat="html"`) 生成 WebGL 交互式矢量图形输出。生成的 3D HTML 文件可以直接在任何现代桌面或移动端浏览器中查看, 甚至可以嵌入到另一个网页中:

```
<iframe src="logo3.html" width="561" height="321" frameborder="0">
</iframe>
```

通常, 由 Asymptote 生成的 WebGL 文件会自动动态重新剖分网格以适应浏览器窗口尺寸。然而, 可以使用设置 `absolute=true` 来强制图像按照其设计尺寸进行渲染 (考虑到每个 css 像素包含多个设备像素)。

对于特殊应用, 设置 `keys=true` 可用于在为每个生成对象输出 WebGL 代码前立即生成一个标识键(key)。默认的键是顶层源代码中相关函数调用的 `"line:column"` (行:列), 可以通过在函数调用的第一个参数中添加 `KEY="x"` 来覆盖, 其中 `x` 代表用户提供的文本。

Asymptote 生成的交互式 WebGL 文件使用与 OpenGL 渲染器相同的默认鼠标键绑定 (以及许多相同的键盘绑定)。嵌入在其他页面中的 WebGL 图像通过鼠标滚轮进行缩放的功能处于禁用状态, 直到通过点击或触摸事件激活该图像为止, 激活后将保持启用状态, 直到按下 ESC 键。

默认情况下, 查看由 Asymptote 生成的 3D HTML 文件需要网络访问以下载 AsyGL 渲染库, 该库通常由浏览器缓存以备将来使用。然而, 可以使用设置 `offline=true` 将这个微小 (约 48kB) 的库嵌入到一个独立的 HTML 文件中, 以便离线查看。

3. 按照设置 `render=n` 的指定, 以每 bp 单位 `n` 像素的分辨率将场景渲染为指定的栅格化格式 `outformat`。render 的默认值为 2。默认情况下, 场景在内部以指定分辨率的两倍进行渲染; 这可以通过设置 `antialias=1` 来禁用。高分辨率渲染是通过分块(tiling)拼贴图像完成的。如果显卡支持, 可以通过将最大块尺寸 `maxtile` 增加到屏幕尺寸 (表示为

`maxtile=(0,0)`) 来提高渲染效率。如果显卡在输出中产生了不希望出现的黑色条纹, 请尝试将 `maxtiles` 的水平和垂直分量设置为小于屏幕尺寸的值。分块尺寸还受到设置 `maxviewport` 的限制, 该设置约束了视口的最大宽度和高度。某些显卡驱动程序支持在最小化(图标化)窗口中进行批处理模式(`-noV`)渲染; 这可以通过设置 `iconify=true` 来启用。

4. 将 3D 旧版 PRC 格式嵌入到 PDF 文件中, 并使用 9.0 或更高版本的 Adobe Reader 查看生成的 PDF 文件。这需要 `settings.outformat="pdf"` 与 `settings.prc=true`, 可以通过命令行选项 `-f pdf` 和 `-f prc` 指定, 也可以放入 Asymptote 配置文件中(见 [configuration file], 185 页), 或者在导入模块 `three` (或 `graph3`) 之前在脚本中指定。还需要 `media9` LaTeX 宏包(见 9.10 节 [embed], 175 页)。示例 `100d.asy` 说明了如何生成预定义视图列表(参见 `100d.views`)。可以通过设置 `render=n` 嵌入分辨率为每 bp 单位 `n` 像素的静态预览图像; 这允许使用其他 PDF 查看器查看该文件。另外, 文件 `externalprc.tex` 说明了如何提取生成的 PRC 及渲染图像文件并在单独的 $\text{\LaTeX}$ 文件中进行处理。不过, 请参见第 7 章 [LaTeX usage], 102 页, 以获取在 $\text{\LaTeX}$ 中嵌入三维 Asymptote 图片的更简便方法。对于仅需要原始 PRC 文件的特殊应用, 请指定 `settings.outformat="prc"`。PRC 规范可以从 https://web.archive.org/web/20081204104459/http://livedocs.adobe.com/acrobat_sdk/9/Acrobat9_HTMLHelp/API_References/PRCReference/PRC_Format_Specification/ 获取。
5. 使用 `settings.outformat="v3d"` 输出 V3D 便携式压缩矢量图形文件, 该文件可以使用外部查看器查看, 或使用 Python `pyv3d` 库转换为其他 3D 格式。使用选项 `settings.outformat="pdf"` 和 `settings.v3d=true` 可以将 V3D 内容自动嵌入到 PDF 文件中。或者, 可以使用 `media9` $\text{\LaTeX}$ 宏包手动将 V3D 文件 `file.v3d` 嵌入到 PDF 文件中:

```
\includemedia[noplaybutton,width=100pt,height=200pt]{}{file.v3d}%
```

基于 Javascript 的在线 V3D 兼容 PDF 查看器可在 <https://github.com/vectorgraphics/pdfv3dReader> 获取。

V3D 规范和 `pyv3d` 库可在 <https://github.com/vectorgraphics/v3d> 获取。可以通过在命令行指定 `file.v3d`:

```
asy -V file.v3d
```

或者在交互模式下(或在 Asymptote 文件中)使用 `v3d` 模块和 `importv3d` 函数来由 Asymptote 导入并查看 V3D 文件 `file.v3d`:

```
import v3d;
importv3d("file.v3d");
```

6. 通过 `render=0` 将场景投影为二维矢量(EPS 或 PDF)格式。这种方法仅对消隐(hidden surface removal)、光照和透明度提供有限的支持(见 [PostScript3D], 147 页)。

三维空间中自动控制图片尺寸是通过双重延迟绘制(double deferred drawing)实现的。可以使用如下例程选择性地指定场景在三个维度中各自期望的最大尺寸:

```
void size3(picture pic=currentpicture, real x, real y=x, real z=y,
          bool keepAspect=pic.keepAspect);
```

接着求解一个单纯形线性规划问题, 以生成 3D 版本的 `frame` (框, 实际上实现为 3D picture)。然后将该结果再次应用延迟绘制, 拟合至与常规二维图片 `size` 参数相对应的视口尺寸。全局数对 `viewportmargin` 可用于向视口尺寸添加水平和垂直边距。或者, 可以指定最小的 `viewportsize`。通过调用

```
frame pic.fit3(projection P=currentprojection);
```

可以将 3D picture `pic` 显式拟合为 3D frame, 然后使用以下函数将其在 `position` 处添加到 picture `dest` 中:

```
void add(picture dest=currentpicture, frame src, triple position=(0,0,0));
```

为了方便起见, `three` 模块定义了 $O=(0,0,0)$ 、 $X=(1,0,0)$ 、 $Y=(0,1,0)$ 以及 $Z=(0,0,1)$, 以及 XY 平面上的单位圆:

```
path3 unitcircle3=X..Y..-X..-Y..cycle;
```

可以使用例程画一个垂直于方向 `normal` 的通用 (近似) 圆:

```
path3 circle(triple c, real r, triple normal=Z);
```

当 $\theta_2 > \theta_1$, 或者当 $\theta_2 == \theta_1$ 且 $\phi_2 \geq \phi_1$ 时, 相对于法向量 $\text{cross}(\text{dir}(\theta_1, \phi_1), \text{dir}(\theta_2, \phi_2))$ 逆时针绘制的、以 c 为中心、半径为 r 、从 $c+r*\text{dir}(\theta_1, \phi_1)$ 到 $c+r*\text{dir}(\theta_2, \phi_2)$ 的圆弧, 可以使用以下例程构造:

```
path3 arc(triple c, real r, real theta1, real phi1, real theta2, real phi2,
          triple normal=0);
```

如果 c 与端点共线, 则必须显式指定法向量。如果 $r < 0$, 则构造半径为 $|r|$ 的补弧。为了方便起见, 以 c 为中心、在 CCW (逆时针) 或 CW (顺时针) 方向上从三元组 v_1 到 v_2 的圆弧 (假设 $|v_2 - c| = |v_1 - c|$) 也可以通过以下例程构造:

```
path3 arc(triple c, triple v1, triple v2, triple normal=0,
          bool direction=CCW);
```

当需要高精度时, 可以使用 `graph3` 中定义的例程 `Circle` 和 `Arc` 来代替。有关三维圆弧的示例, 请参见 [GaussianSurface], 152 页。

穿过点 O 且法向量为 $\text{cross}(u, v)$ 的平面的表示 `0--0+u--0+u+v--0+v--cycle` 由以下例程返回:

```
path3 plane(triple u, triple v, triple O=0);
```

对角顶点位于三元组 v_1 与 v_2 的三维盒子可以使用函数绘制:

```
path3[] box(triple v1, triple v2);
```

例如, 单位盒子预定义为:

```
path3[] unitbox=box(0,(1,1,1));
```

`Asymptote` 还为三维路径 `unitsquare3` 和 `unitcircle3`, 以及曲面 `unitdisk`、`unitplane`、`unitcube`、`unitcylinder`、`unitcone`、`unitsolidcone`、`unitfrustum` (`real t1, real t2`)、`unitsphere` 和 `unithemisphere` 提供了优化后的定义。

预定义了以下投影至二维的函数:

`oblique`

`oblique(real angle)`

点 (x, y, z) 被投影为 $(x - 0.5z, y - 0.5z)$ 。如果给出了可选的实数参数, 负 z 轴将以此角度 (度数) 绘制。投影 `obliqueZ` 是 `oblique` 的同义词。

`obliqueX`

`obliqueX(real angle)`

点 (x, y, z) 被投影为 $(y - 0.5x, z - 0.5x)$ 。如果给出了可选的实数参数, 负 x 轴将以此角度 (度数) 绘制。

`obliqueY`

`obliqueY(real angle)`

点 (x, y, z) 被投影为 $(x+0.5y, z+0.5y)$ 。如果给出了可选的实数参数, 正 y 轴将以此角度 (度数) 绘制。

`orthographic(triple camera, triple up=Z, triple target=0, real
zoom=1, pair viewportshift=0, bool showtarget=true, bool
center=true)`

该函数使用从方向 `unit(camera)` 的无穷远处看到的视角将三维投影至二维, 并调整相机方向使向量 `up` 尽可能向上。平行线被投影为平行线。若 `showtarget=true`, 包围体(bounding volume)将被扩展以包含 `target`。若 `center=true`, 目标点(target)将被调整至包围体的中心。

`orthographic(real x, real y, real z, triple up=Z, triple target=0,
real zoom=1, pair viewportshift=0, bool showtarget=true, bool
center=true)`

这等价于

`orthographic((x,y,z),up,target,zoom,viewportshift,showtarget,center)`

例程

`triple camera(real alpha, real beta);`

可用于计算相机位置, 其中 x 轴位于水平下方角度 α 处, y 轴位于水平下方角度 β 处, 而 z 轴朝上。

`perspective(triple camera, triple up=Z, triple target=0, real
zoom=1, real angle=0, pair viewportshift=0, bool showtarget=true,
bool autoadjust=true, bool center=autoadjust)`

该函数考虑透视感, 从位置 `camera` 观察 `target`, 将三维投影至二维, 并调整相机方向使向量 `up` 尽可能向上。若 `autoadjust=true`, 相机将被自动调整, 使其在绕 `target` 进行所有可能的交互旋转时均位于包围体之外。若 `center=true`, 目标点将被调整至包围体的中心。

`perspective(real x, real y, real z, triple up=Z, triple target=0,
real zoom=1, real angle=0, pair viewportshift=0, bool
showtarget=true, bool autoadjust=true, bool center=autoadjust)`

这等价于

`perspective((x,y,z),up,target,zoom,angle,viewportshift,showtarget,
autoadjust,center)`

默认投影 `currentprojection` 初始设置为 `perspective(5,4,2)`。

我们还定义了工程制图中使用的标准正交视图:

```
projection LeftView=orthographic(-X,showtarget=true);
projection RightView=orthographic(X,showtarget=true);
projection FrontView=orthographic(-Y,showtarget=true);
projection BackView=orthographic(Y,showtarget=true);
projection BottomView=orthographic(-Z,showtarget=true);
projection TopView=orthographic(Z,showtarget=true);
```

函数

```
void addViews(picture dest=currentpicture, picture src,
              projection[] [] views=SixViewsUS,
```

```
bool group=true, filltype filltype=NoFill);
```

使用布局 `projection[][] views` 向图片 `dest` 中添加图片 `src` 的一组视图数组。默认布局 `SixViewsUS` 将投影 `FrontView` 对齐在 `TopView` 下方、`BottomView` 上方，位于 `LeftView` 的右侧以及 `RightView` 和 `BackView` 的左侧。预定义的布局有：

```
projection[] [] ThreeViewsUS={{TopView},
                               {FrontView,RightView}};
```

```
projection[] [] SixViewsUS={{null,TopView},
                             {LeftView,FrontView,RightView,BackView},
                             {null,BottomView}};
```

```
projection[] [] ThreeViewsFR={{RightView,FrontView},
                              {null,TopView}};
```

```
projection[] [] SixViewsFR={{null,BottomView},
                             {RightView,FrontView,LeftView,BackView},
                             {null,TopView}};
```

```
projection[] [] ThreeViews={{FrontView,TopView,RightView}};
```

```
projection[] [] SixViews={{FrontView,TopView,RightView},
                          {BackView,BottomView,LeftView}};
```

使用 `project(triple, projection P=currentprojection)` 或 `project(path3, projection P=currentprojection)` 可以将三元组或 `path3` 投影为数对或路径。

有时反转投影会很有用，即将投影平面上的数对 `z` 映射到穿过点 `point` 且垂直于 `normal` 的平面上：

```
triple invert(pair z, triple normal, triple point,
              projection P=currentprojection);
```

投影平面上的数对 `z` 可以通过以下例程反转为三元组：

```
triple invert(pair z, projection P=currentprojection);
```

投影平面上的数对方向 `dir` 可以通过以下例程反转为相对于点 `v` 的三元组方向：

```
triple invert(pair dir, triple v, projection P=currentprojection).
```

三维对象可以使用以下内置 `transform3` 类型之一进行变换（恒等变换为 `identity4`）：

```
shift(triple v)
    按三元组 v 平移；
```

```
xscale3(real x)
    在 x 方向上按 x 缩放；
```

```
yscale3(real y)
    在 y 方向上按 y 缩放；
```

```
zscale3(real z)
    在 z 方向上按 z 缩放；
```

```

scale3(real s)
    在  $x$ 、 $y$  和  $z$  方向上均按  $s$  缩放;

scale(real x, real y, real z)
    在  $x$  方向按  $x$  缩放, 在  $y$  方向按  $y$  缩放, 在  $z$  方向按  $z$  缩放;

rotate(real angle, triple v)
    绕轴  $0 \rightarrow v$  旋转  $\text{angle}$  (度数);

rotate(real angle, triple u, triple v)
    绕轴  $u \rightarrow v$  旋转  $\text{angle}$  (度数);

reflect(triple u, triple v, triple w)
    关于穿过  $u$ 、 $v$  和  $w$  的平面镜像(reflect)。

```

当未在左侧乘以 `transform3` 时, 三维 $\text{T}_\text{E}\text{X}$ Label (标签) 直接在投影平面上绘制为贝塞尔曲面:

```

void label(picture pic=currentpicture, Label L, triple position,
    align align=NoAlign, pen p=currentpen,
    light light=nolight, string name="",
    render render=defaultrender, interaction interaction=
    settings.autobillboard ? Billboard : Embedded)

```

可选参数 `name` 用作在 PRC 模型树中命名标签块的前缀。默认交互模式为 `Billboard`, 这意味着标签会交互式地旋转, 始终保持面向相机。交互模式 `Embedded` 意味着标签作为普通 3D 曲面参与交互, 正如示例 `billboard.asy` 中所示。或者, 可以使用显式的 `transform3` 从 XY 平面变换标签, 或者使用预定义的 `transform3` 类型 XY、YZ、ZX、YX、ZY、ZX 将其映射到指定的二维平面。这些变换还有接受可选参数 `projection P=currentprojection` 的修改版本, 用于旋转和/或翻转标签, 使其从初始视角看更易于阅读。

沿方向 `dir` 投影到穿过点 0 且法向量为 n 的平面的 `transform3` 可以由以下例程返回:

```

transform3 planeproject(triple n, triple O=0, triple dir=n);

```

可以使用

```

triple normal(path3 p);

```

来求平面三维路径 p 的单位法向量。如示例 `planeproject.asy` 中所示, 沿方向 `dir` 投影到由平面路径 p 定义的平面的 `transform3` 由以下例程返回:

```

transform3 planeproject(path3 p, triple dir=normal(p));

```

函数

```

surface extrude(path p, triple axis=Z);
surface extrude(Label L, triple axis=Z);

```

返回沿 `axis` 挤出(extrude)路径 p 或 Label L 得到的曲面。

还定义了路径函数 `length`、`size`、`point`、`dir`、`accel`、`radius`、`precontrol`、`postcontrol`、`arclength`、以及 `straight` 的三维版本。

例程

```

real[] intersect(path3 p, surface s, real fuzz=-1);

```

返回一个包含路径 p 与曲面 s 相交时间 (如果有的话) 的长度为 3 的 `real` 数组。例程

```

real[][] intersections(path3 p, surface s, real fuzz=-1);

```

将路径 p 与曲面 s 的所有（除非有无数个）相交时间作为按顺序排列的长度为 3 的 real 数组的数组返回，而

```
triple[] intersectionpoints(path3 p, surface s, real fuzz=-1);
```

返回相应的交点。在此，计算按照 $fuzz$ 指定的绝对误差进行，若 $fuzz < 0$ ，则精确到机器精度。例程

```
real orient(triple a, triple b, triple c, triple d);
```

是 $\text{dot}(\text{cross}(a-d, b-d), c-d)$ 的数值稳健计算，即行列式：

```
|a.x a.y a.z 1|
|b.x b.y b.z 1|
|c.x c.y c.z 1|
|d.x d.y d.z 1|
```

如果从 d 观察时 a 、 b 、 c 按逆时针（顺时针）顺序出现，则结果为负（正）；如果所有四个点共面，则结果为零。

例程

```
real insphere(triple a, triple b, triple c, triple d, triple e);
```

当穿过点 a, b, c, d 且方向使 $\text{dot}(\text{cross}(a-d, b-d), c-d)$ 为正的球体内包含（不包含） e 时，返回正值（负值）；如果所有五个点共球，则返回零。返回的值为行列式：

```
|a.x a.y a.z a.x^2+a.y^2+a.z^2 1|
|b.x b.y b.z b.x^2+b.y^2+b.z^2 1|
|c.x c.y c.z c.x^2+c.y^2+c.z^2 1|
|d.x d.y d.z d.x^2+d.y^2+d.z^2 1|
|e.x e.y e.z e.x^2+e.y^2+e.z^2 1|
```

这里是一个展示所有五种 guide3 连接符的示例：

```
import graph3;
```

```
size(200);
```

```
currentprojection=orthographic(500,-500,500);
```

```
triple[] z=new triple[10];
```

```
z[0]=(0,100,0); z[1]=(50,0,0); z[2]=(180,0,0);
```

```
for(int n=3; n <= 9; ++n)
```

```
    z[n]=z[n-3]+(200,0,0);
```

```
path3 p=z[0]..z[1]-z[2>::{Y}z[3]
```

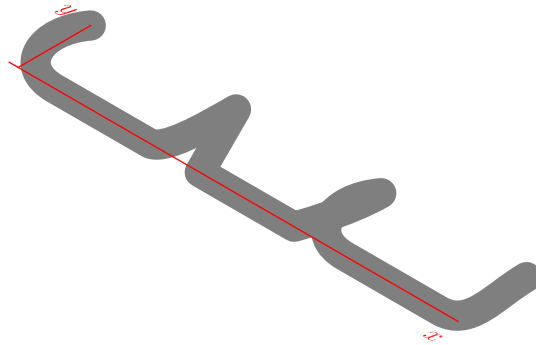
```
&z[3]..z[4]-z[5>::{Y}z[6]
```

```
&z[6>::z[7]-z[8]..{Y}z[9];
```

```
draw(p, grey+linewidth(4mm), currentlight);
```

```
xaxis3(Label(XY())*"$x$", align=-3Y, red, above=true);
```

```
yaxis3(Label(XY())*"$y$", align=-3X, red, above=true);
```



可以使用限定符 `None`、`Blank`、`BeginBar3`、`EndBar3` (或等价地 `Bar3`)、`Bars3`、`BeginArrow3`、`MidArrow3`、`EndArrow3` (或等价地 `Arrow3`)、`Arrows3`、`BeginArcArrow3`、`EndArcArrow3` (或等价地 `ArcArrow3`)、`MidArcArrow3` 以及 `ArcArrows3` 之一来绘制三维版本的端点棒(bar)或箭头(arrow)。三维 bar 接受可选参数 (`real size=0`, `triple dir=0`)。如果 `size=0`, 则使用默认棒长; 如果 `dir=0`, 则棒垂直于路径和初始观察方向绘制。预定义的三维箭头样式为 `DefaultHead3`、`HookHead3` 和 `TeXHead3`。还定义了提升到三维空间并根据初始视角 (或可选指定的 `normal` 向量) 对齐的二维箭头样式版本: `DefaultHead2(triple normal=0)`、`HookHead2(triple normal=0)` 和 `TeXHead2(triple normal=0)`。这些在示例 `arrows3.asy` 中展示。

模块 `three` 还定义了三维边距(`margin`) `NoMargin3`、`BeginMargin3`、`EndMargin3`、`Margin3`、`Margins3`、`BeginMargin3` 以及 `TrueMargin3`。

例程

```
void pixel(picture pic=currentpicture, triple v, pen p=currentpen,
           real width=1);
```

可用于使用画笔 `p` 在位置 `v` 处的图片 `pic` 上绘制宽度为 `width` 的像素。

更多三维示例在文件 `near_earth.asy`、`coniccurv.asy` 以及 (位于 `animations` 子目录中的) `cube.asy` 中提供。

通过设置 `render=0` 提供对投影矢量图形 (实际上是未渲染的三维 PostScript) 的有限支持。这目前仅适用于分段平面曲面, 例如由 `graph3` 模块中的参数化 `surface` 例程生成的曲面。如果参数 `nslices` 足够大, 由 `solids` 模块生成的曲面也将得到正确渲染。

在 `bsp` 模块中, 平面图片的消隐(hidden surface removal)是通过二叉空间分割(binary space partition)和图片裁剪实现的。平面路径首先被转换为派生自 `picture` 的结构体 `face`。可以向任何二维绘制例程传入 `face` 来代替 `picture` 参数。然后可以绘制此类 `face` 的数组并进行消隐(hidden surface removal):

```
void add(picture pic=currentpicture, face[] faces,
         projection P=currentprojection);
```

`Label` 可以使用投影 `P` 投影到二维, 通过在左侧乘以变换, 投影到穿过点 `0` 且法向量为 `cross(u,v)` 的平面上:

```
transform transform(triple u, triple v, triple 0=0,
                   projection P=currentprojection);
```

这里是一个展示如何利用二叉空间分割绘制三个正交相交平面的二维矢量图形投影的示例:

```
size(6cm,0);
```

```
import bsp;
```

```
real u=2.5;
```

```

real v=1;

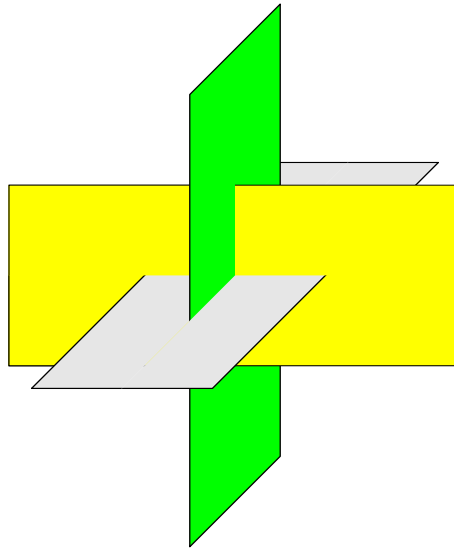
currentprojection=oblique;

path3 y=plane((2u,0,0),(0,2v,0),(-u,-v,0));
path3 l=rotate(90,Z)*rotate(90,Y)*y;
path3 g=rotate(90,X)*rotate(90,Y)*y;

face[] faces;
filldraw(faces.push(y),project(y),yellow);
filldraw(faces.push(l),project(l),lightgrey);
filldraw(faces.push(g),project(g),green);

add(faces);

```



8.3 graph3

本模块实现了 `graph.asy` 中各函数的三维版本。要在三维空间中绘制 x 坐标轴，可使用例程

```

void xaxis3(picture pic=currentpicture, Label L="", axis axis=YZZero,
    real xmin=-infinity, real xmax=infinity, pen p=currentpen,
    ticks3 ticks=NoTicks3, arrowbar3 arrow=None,
    margin3 margin=NoMargin3, bool above=false,
    projection P=currentprojection);

```

类似地，例程 `yaxis` 和 `zaxis` 可用于在三维空间中绘制 y 轴与 z 轴。此外还有一个绘制全部三条坐标轴的例程：

```

void axes3(picture pic=currentpicture,
    Label xlabel="", Label ylabel="", Label zlabel="",
    bool extend=false,

```

```

triple min=(-infinity,-infinity,-infinity),
triple max=(infinity,infinity,infinity),
pen p=currentpen, pen py=p, pen pz=p,
arrowbar3 arrow=None, margin3 margin=NoMargin3,
projection P=currentprojection);

```

预定义的三维坐标轴类型为

```

axis YZEquals(real y, real z, triple align=0, bool extend=false);
axis XZEquals(real x, real z, triple align=0, bool extend=false);
axis XZEEquals(real x, real y, triple align=0, bool extend=false);
axis YZZero(triple align=0, bool extend=false);
axis XZZero(triple align=0, bool extend=false);
axis XYZZero(triple align=0, bool extend=false);
axis Bounds(int type=Both, int type2=Both, triple align=0, bool extend=false);

```

这些例程的可选参数 `align` 可用于指定坐标轴及刻度标签的默认对齐方式。Bounds 坐标轴接受两个类型参数，每个都必须取 Min、Max 或 Both 之一。这些参数指定四条可能的三维边界框棱边中应绘制哪些。

三维刻度选项为 NoTicks3、InTicks、OutTicks 与 InOutTicks。它们为 Bounds 坐标轴类型指定刻度方向；其他坐标轴类型则继承 Bounds(Min,Min) 坐标轴将会使用的方向。

下面是一个螺旋线与带刻度和坐标轴标签的边界框坐标轴的例子，采用正交投影：

```

import graph3;

size(0,200);
size3(200,IgnoreAspect);

currentprojection=orthographic(4,6,3);

real x(real t) {return cos(2pi*t);}
real y(real t) {return sin(2pi*t);}
real z(real t) {return t;}

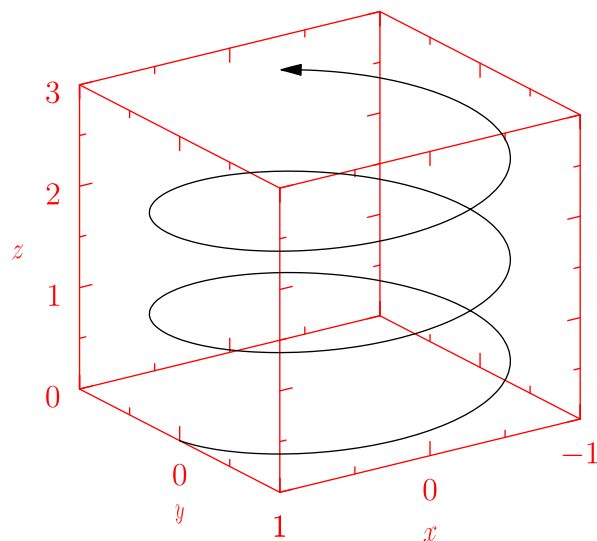
path3 p=graph(x,y,z,0,2.7,operator ..);

draw(p,Arrow3);

scale(true);

xaxis3(XZ()*"$x$",Bounds,red,InTicks(Label,2,2));
yaxis3(YZ()*"$y$",Bounds,red,InTicks(beginlabel=false,Label,2,2));
zaxis3(XZ()*"$z$",Bounds,red,InTicks);

```



下一个例子展示三维 x 、 y 与 z 坐标轴，且不对坐标轴范围做自动缩放：

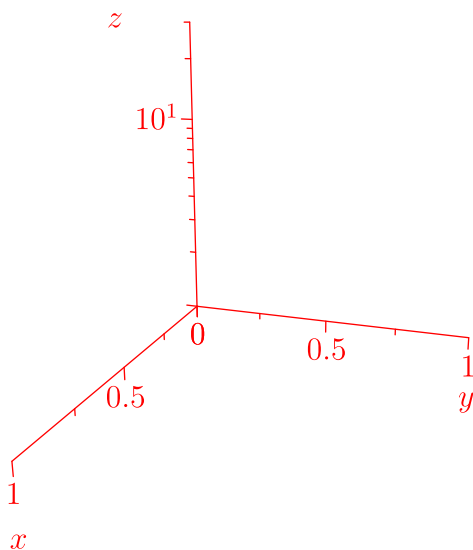
```
import graph3;

size(0,200);
size3(200,IgnoreAspect);

currentprojection=perspective(dir(75,20));

scale(Linear,Linear,Log);

xaxis3("$x$",0,1,red,OutTicks(2,2));
yaxis3("$y$",0,1,red,OutTicks(2,2));
zaxis3("$z$",1,30,red,OutTicks(beginlabel=false));
```



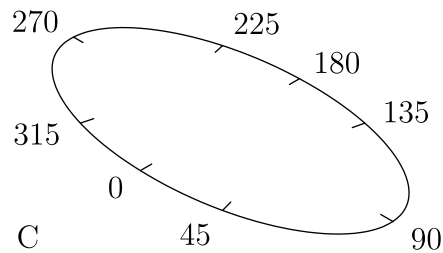
也可以沿一条一般的三维坐标轴放置刻度:

```
import graph3;

size(0,100);

path3 g=yscale3(2)*unitcircle3;
currentprojection=perspective(10,10,10);

axis(Label("C",position=0,align=15X),g,InTicks(endlabel=false,8,end=false),
      ticklocate(0,360,new real(real v) {
        path3 h=0-max(abs(max(g)),abs(min(g)))*dir(90,v);
        return intersect(g,h)[0];},
        new triple(real t) {return cross(dir(g,t),Z);}));
```



矩阵与函数在 XY 平面上区域 $\text{box}(a,b)$ 上的曲面图也已实现:

```
surface surface(real[][] f, pair a, pair b, bool[][] cond={});
surface surface(real[][] f, pair a, pair b, splinetype xsplintype,
                splinetype ysplintype=xsplintype, bool[][] cond={});
surface surface(real[][] f, real[] x, real[] y,
                splinetype xsplintype=null, splinetype ysplintype=xsplintype,
                bool[][] cond={});
surface surface(triple[][] f, bool[][] cond={});
surface surface(real f(pair z), pair a, pair b, int nx=nmesh, int ny=nx,
                bool cond(pair z)=null);
surface surface(real f(pair z), pair a, pair b, int nx=nmesh, int ny=nx,
                splinetype xsplintype, splinetype ysplintype=xsplintype,
                bool cond(pair z)=null);
surface surface(triple f(pair z), real[] u, real[] v,
                splinetype[] usplintype, splinetype[] vsplintype=Spline,
                bool cond(pair z)=null);
surface surface(triple f(pair z), pair a, pair b, int nu=nmesh, int nv=nu,
                bool cond(pair z)=null);
surface surface(triple f(pair z), pair a, pair b, int nu=nmesh, int nv=nu,
                splinetype[] usplintype, splinetype[] vsplintype=Spline,
                bool cond(pair z)=null);
```

最后两个版本在参数空间 $\text{box}(a,b)$ 上为函数 $f(u,v)$ 绘制参数曲面(parametric surface), 如示例 `parametricsurface.asy` 所示。可以指定可选的样条类型 `Spline`。布尔数组或函数 `cond` 可用于控制实际绘制哪些曲面网格单元(默认绘制 $\text{box}(a,b)$ 上的全部网格单元)。

还可以构造由路径 g 绕直线 $c--c+\text{axis}$ 在 angle1 到 angle2 (单位为度) 之间采样 n 次旋转所生成的曲面:

```
surface surface(triple c, path3 g, triple axis, int n=nslice,
               real angle1=0, real angle2=360, pen color(int i, real j)=null);
```

可选参数 `color(int i, real j)` 可用于覆盖将顶点 i 旋转角度 j (单位为度) 后所得点处的曲面颜色。

曲面光照如示例文件 `parametricsurface.asy` 和 `sinc.asy` 所示。设置 `light=nolight` 可禁用光照, 如下面这个高斯曲面的例子:

```
import graph3;

size(200,0);

currentprojection=perspective(10,8,4);

real f(pair z) {return 0.5+exp(-abs(z)^2);}

draw((-1,-1,0)-(1,-1,0)-(1,1,0)-(-1,1,0)-cycle);

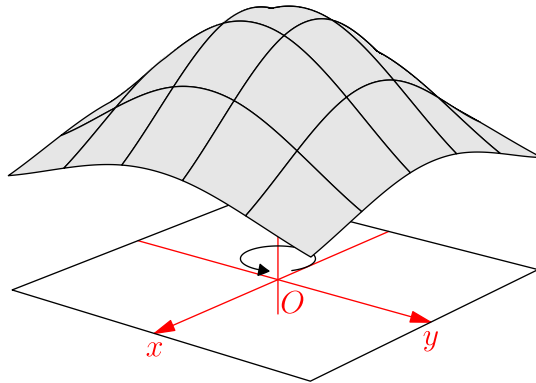
draw(arc(0.12Z,0.2,90,60,90,25),ArcArrow3);

surface s=surface(f,(-1,-1),(1,1),nx=5,Spline);

xaxis3(Label("$x$"),red,Arrow3);
yaxis3(Label("$y$"),red,Arrow3);
zaxis3(XYZZero(extend=true),red,Arrow3);

draw(s,lightgray,meshpen=black+thick(),nolight,render(merge=true));

label("$0$",0,-Z+Y,red);
```



将 `surfacepen` 指定为 `nullpen` 即可只绘制网格而不填充曲面。

参数曲面 `f` 在 `box(a,b)` 上由 `nu×nv` 个箭头组成的向量场可用例程绘制

```
picture vectorfield(path3 vector(pair v), triple f(pair z), pair a, pair b,
                    int nu=nmesh, int nv=nu, bool truesize=false,
                    real maxlength=truesize ? 0 : maxlength(f,a,b,nu,nv),
                    bool cond(pair z)=null, pen p=currentpen,
                    arrowbar3 arrow=Arrow3, margin3 margin=PenMargin3)
```

如示例 `vectorfield3.asy` 和 `vectorfieldsphere.asy` 所示。

8.4 palette

Asymptote 还能生成颜色密度图像与调色板。以下调色板已在 `palette.asy` 中预定义：

```
pen[] Grayscale(int NColors=256)
```

灰度调色板；

```
pen[] Rainbow(int NColors=32766)
```

彩虹光谱调色板；

```
pen[] BWRainbow(int NColors=32761)
```

两端渐隐为黑/白的彩虹光谱调色板；

```
pen[] BWRainbow2(int NColors=32761)
```

两端渐隐为黑/白的双彩虹调色板，强度按线性方式缩放。

```
pen[] Wheel(int NColors=32766)
```

完整色轮调色板；

```
pen[] Gradient(int NColors=256, ... pen[] p)
```

在指定的画笔数组上线性变化的调色板，每个插值区间使用 `NColors` 种颜色；

函数 `cmyk(pen[] Palette)` 可用于将这些调色板中的任何一个转换为 CMYK 色彩空间。

使用调色板 `palette` 的颜色密度图可由函数 `f(x,y)` 生成并添加到图片 `pic`：

```
bounds image(picture pic=currentpicture, real f(real, real),
              range range=Full, pair initial, pair final,
              int nx=ngraph, int ny=nx, pen[] palette, int divs=0,
              bool antialias=false)
```

函数 `f` 将在由点 `initial` 与 `final` 定义的矩形上按 `nx` 与 `ny` 个等距点采样，并遵循 `pic` 当前的图形缩放。色彩空间按照 z 轴缩放进行缩放（见 [automatic scaling], 122 页）。若 `divs > 1`，调色板被量化为 `divs - 1` 个值。返回函数值的 `bounds` 结构：

```
struct bounds {
    real min;
    real max;
    // Possible tick intervals:
    int[] divisor;
}
```

这一信息可用于生成可选的调色板条。调色板色彩空间对应于由参数 `range` 指定的取值范围，后者可以是 `Full`、`Automatic` 或显式范围 `Range(real min, real max)`。（类型 `range` 定义于 `palette.asy`。）这里 `Full` 指定一个在采样区间上从函数最小值变化到最大值的范围，而

Automatic 则选择“美观”的界限。示例 `fillcontour.asy` 与 `imagecontour.asy` 展示了如何在颜色密度图上绘制等值集（等值线）（见 8.9 节 [contour], 159 页）。

颜色密度图也可以由显式的 `real[][]` 数组 `data` 生成：

```
bounds image(picture pic=currentpicture, real[][] f, range range=Full,
             pair initial, pair final, pen[] palette, int divs=0,
             bool transpose=(initial.x < final.x && initial.y < final.y),
             bool copy=true, bool antialias=false);
```

若初始点位于终止点的左侧和下方，默认情况下数组索引按笛卡尔惯例解释（第一个索引： x ，第二个索引： y ），而非通常的矩阵惯例（第一个索引： $-y$ ，第二个索引： x ）。

要由不规则分布的点数组及这些点上的值数组 `f` 构造图像，可使用下列例程之一

```
bounds image(picture pic=currentpicture, pair[] z, real[] f,
             range range=Full, pen[] palette)
bounds image(picture pic=currentpicture, real[] x, real[] y, real[] f,
             range range=Full, pen[] palette)
```

带可选标签的调色板条可由例程生成

```
void palette(picture pic=currentpicture, Label L="", bounds bounds,
            pair initial, pair final, axis axis=Right, pen[] palette,
            pen p=currentpen, paletteticks ticks=PaletteTicks,
            bool copy=true, bool antialias=false);
```

`palette` 的色彩空间取为界限 `bounds` 上的范围，缩放由 `pic` 的 z 缩放给出。调色板的方向由 `axis` 指定，它可以是 `Right`、`Left`、`Top` 或 `Bottom` 之一。色条绘制在从 `initial` 到 `final` 的矩形上。参数 `paletteticks` 是一种特殊的刻度类型（见 [ticks], 109 页），它接受以下参数：

```
paletteticks PaletteTicks(Label format="", ticklabel ticklabel=null,
                          bool beginlabel=true, bool endlable=true,
                          int N=0, int n=0, real Step=0, real step=0,
                          pen pTick=nullpen, pen ptick=nullpen);
```

图像与调色板条可装入一个框(frame)并添加、且可选择性地对齐到图片上的期望位置：

```
size(12cm,12cm);

import graph;
import palette;

int n=256;
real ninv=2pi/n;
real[][] v=new real[n][n];

for(int i=0; i < n; ++i)
  for(int j=0; j < n; ++j)
    v[i][j]=sin(i*ninv)*cos(j*ninv);

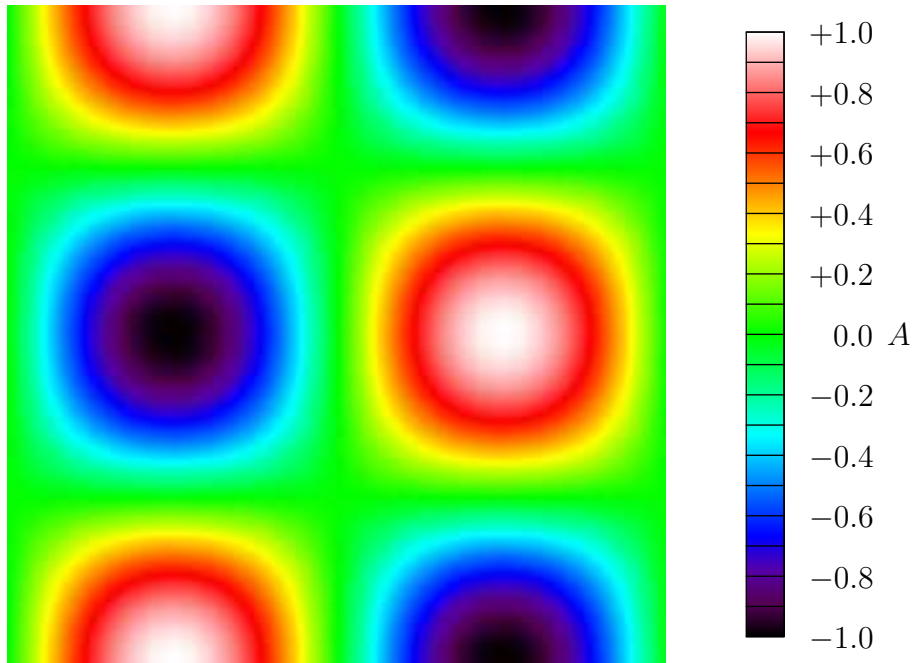
pen[] Palette=BWRainbow();

picture bar;
```

```

bounds range=image(v,(0,0),(1,1),Palette);
palette(bar,"$A$",range,(0,0),(0.5cm,8cm),Right,Palette,
        PaletteTicks("$%+#.1f$"));
add(bar.fit(),point(E),30E);

```



下面是一个对函数值使用对数缩放的例子：

```

import graph;
import palette;

size(10cm,10cm,IgnoreAspect);

real f(real x, real y) {
    return 0.9*pow10(2*sin(x/5+2*y^0.25)) + 0.1*(1+cos(10*log(y)));
}

scale(Linear,Log,Log);

pen[] Palette=BWRainbow();

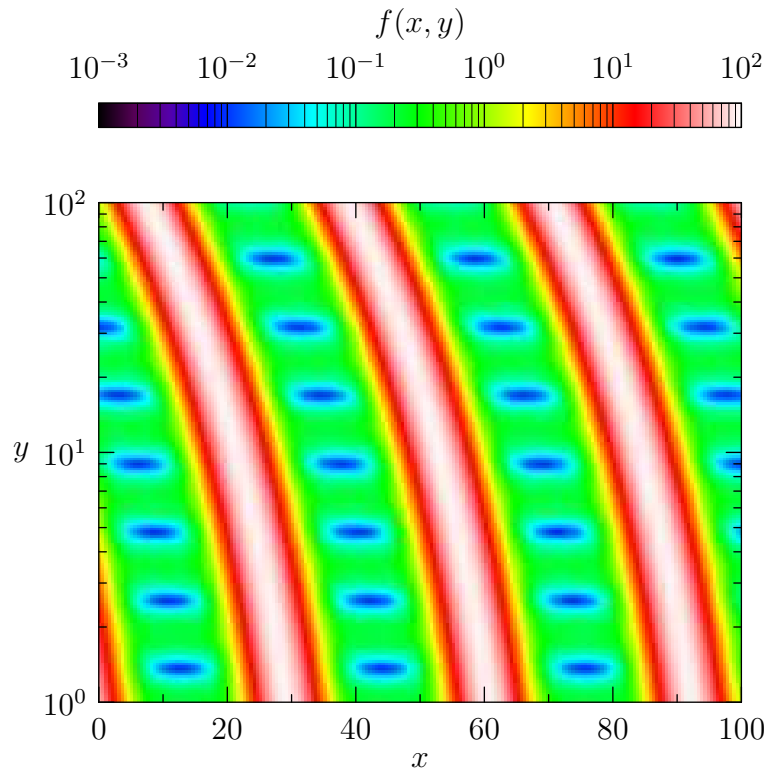
bounds range=image(f,Automatic,(0,1),(100,100),nx=200,Palette);

axis("$x$",BottomTop,LeftTicks,above=true);
axis("$y$",LeftRight,RightTicks,above=true);

palette("$f(x,y)$",range,(0,200),(100,250),Top,Palette,

```

```
PaletteTicks(ptick=linewidth(0.5*linewidth())));
```



还可以直接由二维画笔数组或函数 `pen f(int, int)` 绘制图像:

```
void image(picture pic=currentpicture, pen[] [] data,
           pair initial, pair final,
           bool transpose=(initial.x < final.x && initial.y < final.y),
           bool copy=true, bool antialias=false);
void image(picture pic=currentpicture, pen f(int, int), int width, int height,
           pair initial, pair final,
           bool transpose=(initial.x < final.x && initial.y < final.y),
           bool antialias=false);
```

如下例所示:

```
size(200);

import palette;

int n=256;
real ninv=2pi/n;
pen[] [] v=new pen[n][n];

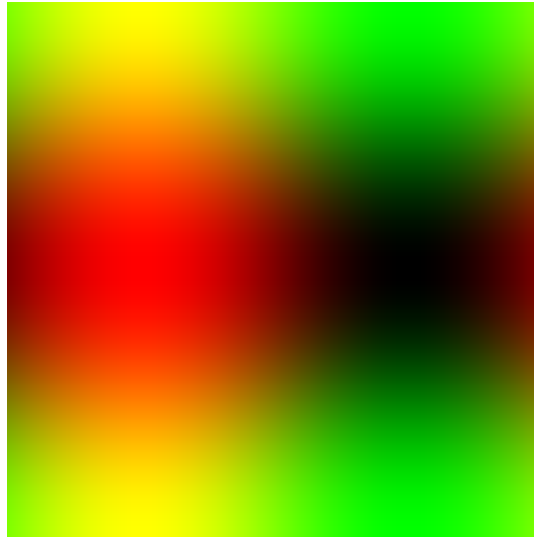
for(int i=0; i < n; ++i)
```

```

for(int j=0; j < n; ++j)
    v[i][j]=rgb(0.5*(1+sin(i*ninv)),0.5*(1+cos(j*ninv)),0);

image(v,(0,0),(1,1));

```



```

import palette;

size(200);

real fracpart(real x) {return (x-floor(x));}

pair pws(pair z) {
    pair w=(z+exp(pi*I/5)/0.9)/(1+z/0.9*exp(-pi*I/5));
    return exp(w)*(w^3-0.5*I);
}

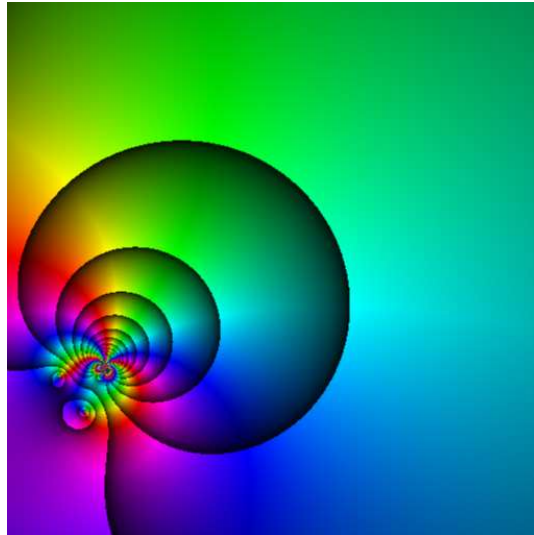
int N=512;

pair a=(-1,-1);
pair b=(0.5,0.5);
real dx=(b-a).x/N;
real dy=(b-a).y/N;

pen f(int u, int v) {
    pair z=a+(u*dx,v*dy);
    pair w=pws(z);
    real phase=degrees(w,warn=false);
    real modulus=w == 0 ? 0: fracpart(log(abs(w)));
    return hsv(phase,1,sqrt(modulus));
}

```

```
}
image(f,N,N,(0,0),(300,300),antialias=true);
```



为方便起见，模块 `palette` 还定义了一些函数，可用于由给定的函数和调色板构造画笔数组：

```
pen[] palette(real[] f, pen[] palette);
pen[][] palette(real[][] f, pen[] palette);
```

8.5 colormap

本模块实现了从 Python 库 `matplotlib` 转换而来的一组色彩映射(`colormap`)。这些色彩映射可用 `palette()` 方法生成调色板。例如：

```
import colormap;
pen[] Palette = wistia.palette();
```

调色板分两类：分段调色板（接受颜色数与 `gamma` 参数）和列表调色板（返回一组固定的颜色）。可用色彩映射的完整列表见 <https://matplotlib.org/tutorials/colors/colormaps.html>。

8.6 texcolors

本模块将标准的 68 种 CMYK $\text{\TeX}$ 颜色定义为具名画笔。可通过如下方式导入：

```
import texcolors;
```

这些颜色包括 `GreenYellow`、`Goldenrod`、`Melon`、`Mahogany` 等名称，以及标准 $\text{\TeX}$ 颜色集中定义的许多其他颜色。

8.7 x11colors

本模块将标准的 140 种 RGB X11 颜色定义为具名画笔。可通过如下方式导入：

```
import x11colors;
```

这些颜色包括 AliceBlue、Aquamarine、CadetBlue、Chartreuse 等名称, 以及标准 X11 颜色集中的许多其他颜色。

注意 TeX 与 X11 颜色标准之间有一定重叠, 而某些颜色 (如 Green) 的定义实际上并不一致。

8.8 fontsize

本模块通过加载 TeX 宏包 type1cm 提供对非标准字号的支持。在文件开头导入本模块, 即可在 fontsize() 中使用任意字号:

```
import fontsize;
defaultpen(fontsize(10pt));
```

8.9 contour

本模块绘制等值线。要为 box(a,b) 上的函数 f 构造对应于实数数组 c 中各值的等值线, 可使用例程

```
guide[] [] contour(real f(real, real), pair a, pair b,
                    real[] c, int nx=ngraph, int ny=nx,
                    interpolate join=operator -, int subsample=1);
```

整数 nx 与 ny 定义分辨率。默认分辨率 ngraph x ngraph (此处 ngraph 默认为 100) 可增大以提高精度。默认插值算子为 operator -- (线性)。样条插值 (operator ..) 可能产生更光滑的等值线, 但也可能导致过冲。参数 subsample 指明在每个 1 x 1 方格内采样等值线所用的内部点数; 默认值 1 通常已足够。

要为 box(a,b) 上均匀二维格点上的数据值数组构造等值线, 使用

```
guide[] [] contour(real[] [] f, pair a, pair b, real[] c,
                    interpolate join=operator -, int subsample=1);
```

要为由二维数组 z 指定的互不重叠的规则网格上的数据值数组构造等值线,

```
guide[] [] contour(pair[] [] z, real[] [] f, real[] c,
                    interpolate join=operator -, int subsample=1);
```

要为在位置不规则的点 z 上给定的值数组 f 构造等值线, 可使用例程

```
guide[] [] contour(pair[] z, real[] f, real[] c, interpolate join=operator -);
```

等值线本身可用下列例程之一绘制

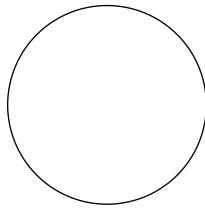
```
void draw(picture pic=currentpicture, Label[] L=new Label[],
          guide[] [] g, pen p=currentpen);
```

```
void draw(picture pic=currentpicture, Label[] L=new Label[],
          guide[] [] g, pen[] p);
```

下面的简单例子绘制函数 $z = x^2 + y^2$ 在值 1 处的等值线, 即一个单位圆:

```
import contour;
size(75);

real f(real a, real b) {return a^2+b^2;}
draw(contour(f,(-1,-1),(1,1),new real[] {1}));
```



下一个例子以分辨率 100 x 100 绘制并标注函数 $z = x^2 - y^2$ 的多条等值线，负值等值线用虚线画笔，正值（及零值）等值线用实线画笔：

```
import contour;

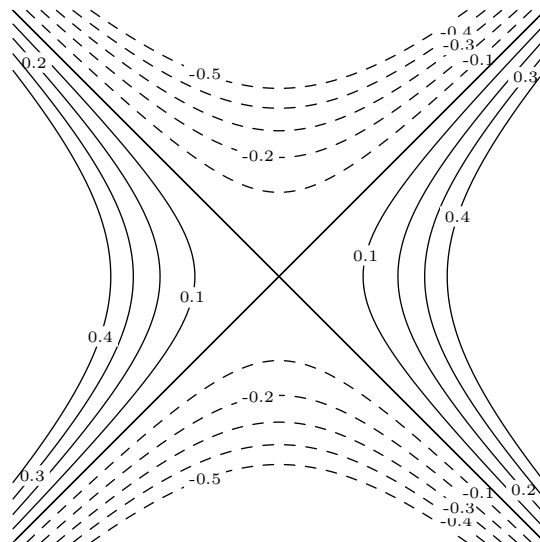
size(200);

real f(real x, real y) {return x^2-y^2;}
int n=10;
real[] c=new real[n];
for(int i=0; i < n; ++i) c[i]=(i-n/2)/n;

pen[] p=sequence(new pen(int i) {
    return (c[i] >= 0 ? solid : dashed)+fontsize(6pt);
},c.length);

Label[] Labels=sequence(new Label(int i) {
    return Label(c[i] != 0 ? (string) c[i] : "",Relative(unitrand()),(0,0),
        UnFill(1bp));
},c.length);

draw(Labels,contour(f,(-1,-1),(1,1),c),p);
```



随后的例子展示如何在颜色密度图像上绘制等值线，分别演示了带与不带调色板量化的情形：

```
import graph;
import palette;
import contour;

size(10cm,10cm);

pair a=(0,0);
pair b=(2pi,2pi);

real f(real x, real y) {return cos(x)*sin(y);}

int N=200;
int Divs=10;
int divs=1;
int n=Divs*divs;

defaultpen(1bp);
pen Tickpen=black;
pen tickpen=gray+0.5*linewidth(currentpen);
pen[] Palette=quantize(BWRainbow(),n);

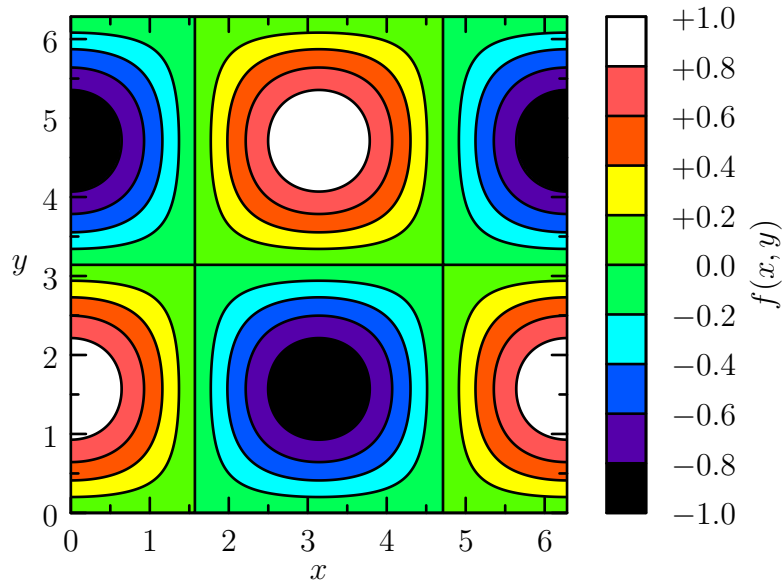
bounds range=image(f,Automatic,a,b,3N,Palette,n);

// Major contours
real[] Cvals=uniform(range.min,range.max,Divs);
draw(contour(f,a,b,Cvals,N,operator -),Tickpen+squarecap+beveljoin);

// Minor contours (if divs > 1)
real[] cvals;
for(int i=0; i < Cvals.length-1; ++i)
    cvals.append(uniform(Cvals[i],Cvals[i+1],divs)[1:divs]);
draw(contour(f,a,b,cvals,N,operator -),tickpen);

xaxis("$x$",BottomTop,LeftTicks,above=true);
yaxis("$y$",LeftRight,RightTicks,above=true);

palette("$f(x,y)$",range,point(SE)+(0.5,0),point(NE)+(1,0),Right,Palette,
    PaletteTicks("%+#0.1f$",N=Divs,n=divs,Tickpen,tickpen));
```



```

import graph;
import palette;
import contour;

size(10cm,10cm);

pair a=(0,0);
pair b=(2pi,2pi);

real f(real x, real y) {return cos(x)*sin(y);}

int N=200;
int Divs=10;
int divs=1;

defaultpen(1bp);
pen Tickpen=black;
pen tickpen=gray+0.5*linewidth(currentpen);
pen[] Palette=BWRainbow();

bounds range=image(f,Automatic,a,b,N,Palette);

// Major contours
real[] Cvals=uniform(range.min,range.max,Divs);
draw(contour(f,a,b,Cvals,N,operator -),Tickpen+squarecap+beveljoin);

// Minor contours (if divs > 1)
real[] cvals;
for(int i=0; i < Cvals.length-1; ++i)
    cvals.append(uniform(Cvals[i],Cvals[i+1],divs)[1:divs]);

```

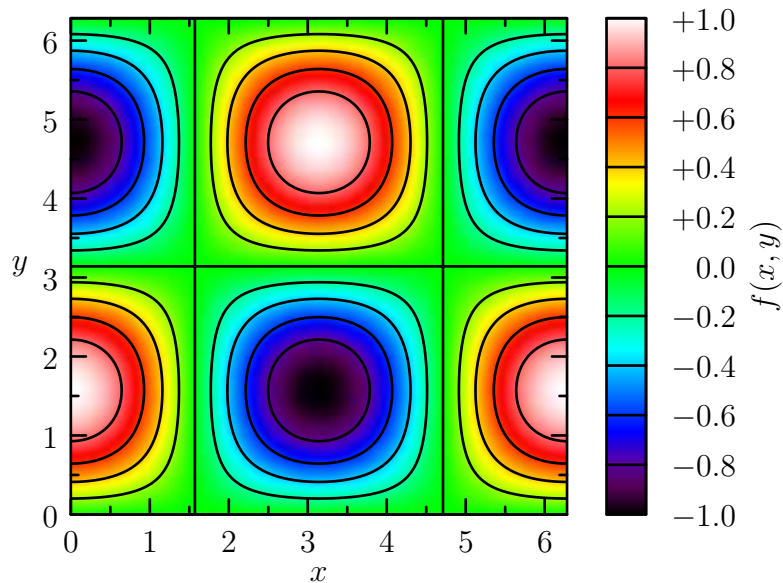
```

draw(contour(f,a,b,cvals,N,operator -),tickpen+squarecap+beveljoin);

xaxis("$x$",BottomTop,LeftTicks,above=true);
yaxis("$y$",LeftRight,RightTicks,above=true);

palette("$f(x,y)$",range,point(SE)+(0.5,0),point(NE)+(1,0),Right,Palette,
      PaletteTicks("%"+#0.1f$",N=Divs,n=divs,Tickpen,tickpen));

```



最后这个例子展示如何由不规则分布的数据构造等值线:

```

import contour;

size(200);

int n=100;

real f(real a, real b) {return a^2+b^2;}

srand(1);

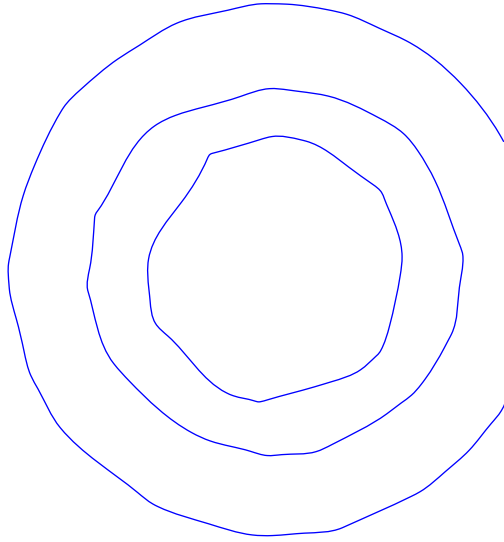
real r() {return 1.1*(rand()/randMax*2-1);}

pair[] points=new pair[n];
real[] values=new real[n];

for(int i=0; i < n; ++i) {
    points[i]=(r(),r());
    values[i]=f(points[i].x,points[i].y);
}

```

```
draw(contour(points,values,new real[]{0.25,0.5,1},operator ..),blue);
```



在上面的例子中，不规则分布数据的等值线是这样构造的：先由数对数组 z 创建一个三角网格：

```
int[][] triangulate(pair[] z);

size(200);
int np=100;
pair[] points;

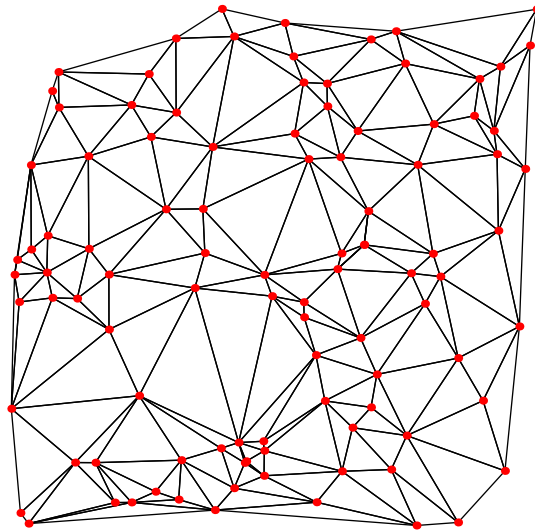
real r() {return 1.2*(rand()/randMax*2-1);}

for(int i=0; i < np; ++i)
    points.push((r(),r()));

int[][] trn=triangulate(points);

for(int i=0; i < trn.length; ++i) {
    draw(points[trn[i][0]]-points[trn[i][1]]);
    draw(points[trn[i][1]]-points[trn[i][2]]);
    draw(points[trn[i][2]]-points[trn[i][0]]);
}

for(int i=0; i < np; ++i)
    dot(points[i],red);
```



示例 `Gouraudcontour.asy` 展示了如何在此类不规则三角网格上生成颜色密度图像。Asymptote 使用 Paul Bourke 的 Delaunay 三角剖分算法的一个稳健版本，它基于 Jonathan Shewchuk 编写的公有领域精确算术谓词。

8.10 geometry

本模块由 Philippe Ivaldi 编写，提供了一套内容广泛的几何例程，包括 `perpendicular` 符号和 `triangle` 结构。geometry 模块文档的链接发布于：<https://asymptote.sourceforge.io/links.html>，其中包括大量示例 <https://web.archive.org/web/20201130113133/http://www.piprime.fr/files/asymptote/geometry/>，以及一个索引：

<https://web.archive.org/web/20201130113133/http://www.piprime.fr/files/asymptote/geometry/modules/geometry.asy.index.type.html>

8.11 grid3

本模块由 Philippe Ivaldi 贡献，可用于绘制三维网格。下面是一个例子（更多例子见 `grid3.asy` 与 <https://web.archive.org/web/20201130113133/http://www.piprime.fr/files/asymptote/grid3/>）：

```
import grid3;

size(8cm,0,IgnoreAspect);
currentprojection=orthographic(0.5,1,0.5);

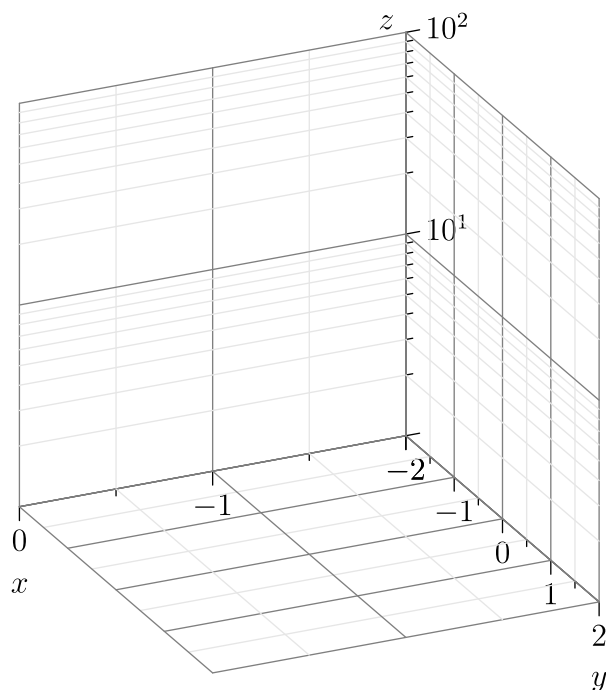
scale(Linear, Linear, Log);

limits((-2,-2,1),(0,2,100));

grid3(XYZgrid);

xaxis3(Label("$x$",position=EndPoint,align=S),Bounds(Min,Min),
        OutTicks());
yaxis3(Label("$y$",position=EndPoint,align=S),Bounds(Min,Min),OutTicks());
```

```
zaxis3(Label("$z$"),position=EndPoint,align=(-1,0.5)),Bounds(Min,Min),
      OutTicks(beginlabel=false));
```



8.12 interpolate

本模块在 Asymptote 中实现了 Lagrange、Hermite 与标准三次样条(cubic spline)插值，如示例 interpolate1.asy 所示。

8.13 markers

本模块实现了用于标记路径和角度的专用例程。本模块提供的主要标记例程为

```
markroutine markinterval(int n=1, frame f, bool rotated=false);
```

它将框 f 的 n 份副本居中放置在沿路径按弧长均匀分布的区间内，并可选择按局部切线的角度旋转。

marker (见 [marker], 117 页) 例程可用于从这些预定义的框构造新的标记：

```
frame stickframe(int n=1, real size=0, pair space=0, real angle=0,
      pair offset=0, pen p=currentpen);
```

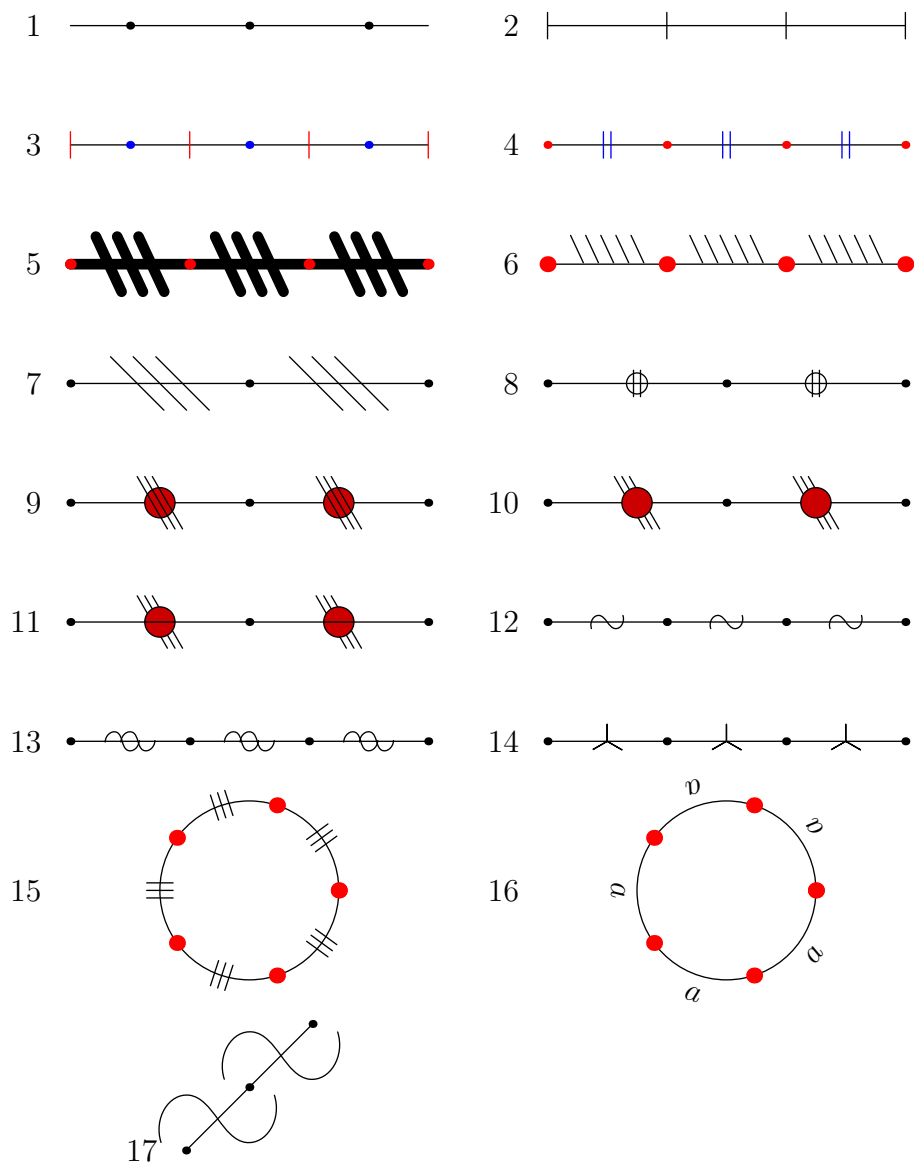
```
frame circlebarframe(int n=1, real barsize=0,
      real radius=0, real angle=0,
      pair offset=0, pen p=currentpen,
      filltype filltype=NoFill, bool above=false);
```

```
frame crossframe(int n=3, real size=0, pair space=0,
      real angle=0, pair offset=0, pen p=currentpen);
```

```
frame tildeframe(int n=1, real size=0, pair space=0,
```

```
real angle=0, pair offset=0, pen p=currentpen);
```

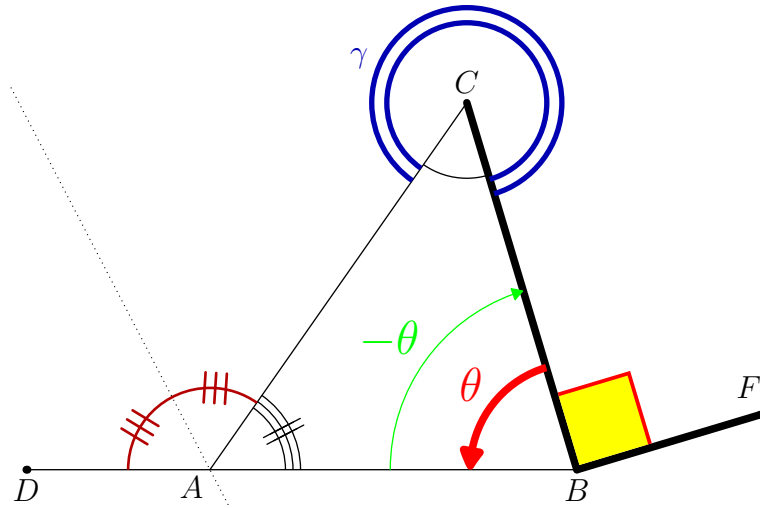
为方便起见, 本模块还由上述框构造出标记 `StickIntervalMarker`、`CrossIntervalMarker`、`CircleBarIntervalMarker` 与 `TildeIntervalMarker`。示例 `markers1.asy` 展示了这些标记的用法:



本模块还提供了一个标记角 AOB 的例程:

```
void markangle(picture pic=currentpicture, Label L="",
    int n=1, real radius=0, real space=0,
    pair A, pair O, pair B, arrowbar arrow=None,
    pen p=currentpen, margin margin=NoMargin,
    marker marker=nomarker);
```

如示例 `markers2.asy` 所示。



8.14 math

本模块通过一些有用的函数扩展了 Asymptote 的数学能力，例如

`void drawline(picture pic=currentpicture`
 绘制经过 P 与 Q 的（无限）直线的可见部分，而不改变图片 pic 的尺寸，使用画笔 p。

`real intersect(triple P, triple Q, triple n, triple Z);`
 返回线段 PQ 的延长线与垂直于 n 且经过 Z 的平面的相交时间(intersection time)。

`triple intersectionpoint(triple n0, triple P0, triple n1, triple P1);`
 返回分别以 n0 与 n1 为法向量、分别经过点 P0 与 P1 的两平面交线上的任一点。
 若两平面平行，则返回 (infinity,infinity,infinity)。

`pair[] quarticroots(real a, real b, real c, real d, real e);`
 返回四次方程 $ax^4 + bx^3 + cx^2 + dx + e = 0$ 的四个复根。

`real time(path g, real x, int n=0, real fuzz=-1)`
 返回路径 g 与经过 x 的竖直线的第 n 次相交时间。

`real time(path g, explicit pair z, int n=0, real fuzz=-1)`
 返回路径 g 与经过 (0,z.y) 的水平线的第 n 次相交时间。

`real value(path g, real x, int n=0, real fuzz=-1)`
 返回 g 在 x 处的第 n 个 y 值。

`real value(path g, explicit pair z, int n=0, real fuzz=-1)`
 返回 g 在 y=z.y 处的第 n 个 x 值。

`real slope(path g, real x, int n=0, real fuzz=-1)`
 返回 g 在 x 处的第 n 个斜率。

`real slope(path g, explicit pair z, int n=0, real fuzz=-1)`
 返回 g 在 y=z.y 处的第 n 个斜率。

`int[] segment(bool[] b)` 返回 bool[] b 中连续真值段的索引。

`real[] partialsum(real[] a)`
 返回实数数组 a 的部分和。

```

real[] partialsum(real[] a, real[] dx)
    返回实数数组 a 的 dx 加权部分和。

bool increasing(real[] a, bool strict=false)
    当 strict=false 时, 返回  $i > j$  是否蕴含  $a[i] \geq a[j]$ ; 当 strict=true 时,
    返回  $i > j$  是否蕴含  $a[i] > a[j]$ 。

int unique(real[] a, real x)
    若已排序的数组 a 不包含 x, 则按序插入之, 并返回 x 在结果数组中的索引。

bool lexorder(pair a, pair b)
    返回 a 与 b 的严格字典序偏序关系。

bool lexorder(triple a, triple b)
    返回 a 与 b 的严格字典序偏序关系。

```

8.15 patterns

本模块实现了 PostScript 平铺图案, 并包含若干便捷的图案生成例程。

8.16 roundedpath

本模块由 Stefan Knorr 贡献, 可用于将路径的尖锐拐角变圆, 如示例文件 `roundpath.asy` 所示。

8.17 slopefield

要绘制微分方程 $dy/dx = f(x, y)$ (或 $dy/dx = f(x)$) 的斜率场, 使用:

```

picture slopefield(real f(real,real), pair a, pair b,
    int nx=nmesh, int ny=nx,
    real tickfactor=0.5, pen p=currentpen,
    arrowbar arrow=None);

```

这里, 点 a 与 b 是绘制斜率场的矩形的左下角和右上角, nx 与 ny 分别是 x 与 y 方向上的刻度数, tickfactor 是绘制刻度时所用的最小单元尺寸的分数的, 而 p 是绘制斜率场所用的画笔。返回值是一个图片, 可通过 `add(picture)` 命令添加到 `currentpicture`。

函数

```

path curve(pair c, real f(real,real), pair a, pair b);

```

接受一个点 (c) 和一个定义斜率场的函数 f, 并以路径的形式返回经过该点的曲线。点 a 与 b 表示对曲线进行插值的矩形边界。

`slopefield` 与 `curve` 也可以改而接受一个仅依赖于 x 的函数 `real f(real)`, 如下例所示:

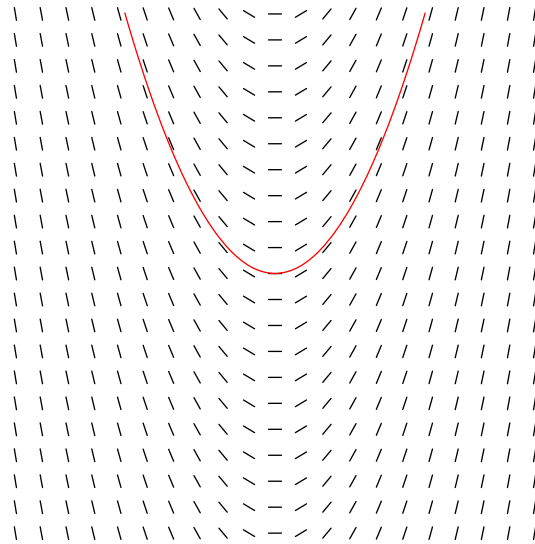
```

import slopefield;

size(200);

real func(real x) {return 2x;}
add(slopefield(func, (-3,-3), (3,3), 20));
draw(curve((0,0), func, (-3,-3), (3,3)), red);

```



8.18 smoothcontour3

本模块由 Charles Staats 编写，用于绘制外观光滑的隐式定义曲面。本模块的用途与 `contour3` 类似：给定实值函数 $f(x, y, z)$ ，构造由方程 $f(x, y, z) = 0$ 描述的曲面。`smoothcontour3` 模块通常比 `contour3` 产生更好的结果，但编译时间更长。此外，该算法假定函数与曲面都是光滑的；若并非如此，则 `contour3` 可能是更好的选择。

要构造函数 $f(\text{triple})$ 或 $ff(\text{real}, \text{real}, \text{real})$ 在 $\text{box}(a, b)$ 上的零曲面，可使用例程

```
surface implicitsurface(real f(triple)=null,
                        real ff(real,real,real)=null,
                        triple a,
                        triple b,
                        int n=nmesh,
                        bool keyword overlapedges=false,
                        int keyword nx=n,
                        int keyword ny=n,
                        int keyword nz=n,
                        int keyword maxdepth=8,
                        bool usetriangles=true);
```

可选参数 `overlapedges` 试图补偿一种可能导致渲染器“看穿”曲面片之间边界的伪影。虽然其默认值为 `false`，但通常应将其设为 `true`。示例 `genustwo.asy` 展示了此函数的用法。更多示例以及对本模块用法与陷阱的更深入说明，见 <https://github.com/charlesstaats/smoothcontour3>。

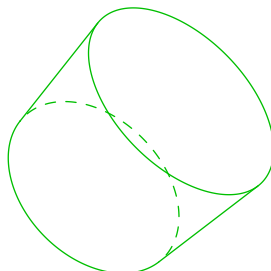
8.19 solids

本立体几何模块定义了一个结构 `revolution`，可用于填充和绘制旋转曲面。下面的例子用它以透视投影显示一个半径为 1、轴向为 $0 \rightarrow 1.5 \text{unit}(Y+Z)$ 的圆柱面轮廓：

```
import solids;
```

```
size(0,100);
```

```
revolution r=cylinder(0,1,1.5,Y+Z);
draw(r,heavygreen);
```



更多图示见示例文件 `cylinder.asy`、`cones.asy`、`hyperboloid.asy` 与 `torus.asy`。

结构 `skeleton` 包含用于可视化旋转体的三维线框：

```
struct skeleton {
    struct curve {
        path3[] front;
        path3[] back;
    }
    // transverse skeleton (perpendicular to axis of revolution)
    curve transverse;
    // longitudinal skeleton (parallel to axis of revolution)
    curve longitudinal;
}
```

8.20 stats

本模块实现了一个高斯随机数生成器和一组统计例程，包括 `histogram` 与 `leastquares`。

8.21 tube

本模块将 `three_arrows.asy` 中构造的 `tube` 曲面推广到任意截面、颜色与脊线变换。例程

```
surface tube(path3 g, coloredpath section,
             transform T(real)=new transform(real t) {return identity();},
             real corner=1, real relstep=0);
```

在 `point(g,t)` 处施加变换 `T(t)` 后，沿 `g` 绘制截面为 `section` 的管体(`tube`)。参数 `corner` 控制 `g` 的角点处基本管体的数目。`relstep` 的非零值指定一个固定的相对时间步长（按 `relpoint(g,t)` 的意义），用于沿 `g` 构造基本管体。类型 `coloredpath` 是 `path` 的推广，`path` 可被强制转换(`cast`)为它：

```
struct coloredpath
{
    path p;
    pen[] pens(real);
    int colortype=coloredSegments;
}
```

这里 `p` 定义截面，方法 `pens(real t)` 返回一个画笔数组（按循环数组解释），用于为 `relpoint(g,t)` 处的管体曲面片着色。若 `colortype=coloredSegments`，则管体曲面片的填充方式如同截面的每一段都用 `pens(t)` 返回的画笔着色；若 `colortype=coloredNodes`，则管体各组件按顶点着色，如同截面的各节点都被着色一样。

`coloredpath` 可用下列例程之一构造：

```
coloredpath coloredpath(path p, pen[] pens(real),
                        int colortype=coloredSegments);
coloredpath coloredpath(path p, pen[] pens=new pen[] {currentpen},
                        int colortype=coloredSegments);
coloredpath coloredpath(path p, pen pen(real));
```

在第二种情形中，画笔与相对时间无关。在第三种情形中，画笔数组只含一个画笔，它依赖于相对时间。

将 `path` 强制转换为 `coloredpath` 使得可以用 `path` 代替 `coloredpath`；此时着色行为即为曲面的默认着色行为。

`tube` 的一个例子见文件 `trefoilknot.asy`。更多例子见 <https://web.archive.org/web/20201130113133/http://www.piprime.fr/files/asymptote/tube>。

9 专用模块

这些模块用于专用应用。

9.1 animation

该模块允许生成动画, 正如示例目录的 `animations` 子目录中的文件 `wheel.asy`、`wavepacket.asy` 以及 `cube.asy` 所展示的那样。这些动画使用 ImageMagick 的 `magick` 程序将多张图像合并为 GIF 或 MPEG 动画电影。

派生自 `animation` 模块的相关 `animate` 模块可生成更高质量、带有可选控制按钮的可点击便携式 PDF 动画电影。这需要在本地 $\text{\LaTeX}$ 目录 (例如 `/usr/local/share/texmf/tex/latex/animate`) 的新目录 `animate` 中安装宏包

<http://mirror.ctan.org/macros/latex/contrib/animate/animate.sty>

(2007/11/30 或更高版本)。在 UNIX 系统上, 随后必须执行命令 `texhash`。

`animations` 目录中的示例 `pdfmovie.asy`, 以及幻灯片演示文稿 `slidemovies.asy` 与 `intro`, 展示了嵌入式 PDF 动画电影的使用。示例 `inlinemovie.tex` 和 `inlinemovie3.tex` 展示了如何直接在 $\text{\LaTeX}$ 文件中生成并嵌入 PDF 动画电影 (见第 7 章 [LaTeX usage], 102 页)。 `animate` 结构体的成员函数

```
string pdf(fit fit=NoBox, real delay=animationdelay, string options="",
           bool keep=settings.keep, bool multipage=true);
```

接受在此处描述的任何 `animate.sty` 选项:

<http://mirror.ctan.org/macros/latex/contrib/animate/doc/animate.pdf>

9.2 animate

该模块在导入 `animation` 模块之前加载 $\text{\TeX}$ `animate` 宏包, 以生成带有可选控制按钮的可点击便携式 PDF 动画电影。

9.3 annotate

该模块支持用于在 Adobe Reader 中查看的 PDF 注释(annotation), 通过如下函数实现:

```
void annotate(picture pic=currentpicture, string title, string text,
              pair position);
```

示例文件 `annotation.asy` 展示了注释的用法。目前, 注释仅针对 `latex` (默认) 和 `tex` 这两个 $\text{\TeX}$ 引擎实现。

9.4 babel

该模块在 `Asymptote` 中实现了 $\text{\LaTeX}$ 的 `babel` 宏包。例如:

```
import babel;
babel("german");
```

9.5 binarytree

该模块可用于绘制任意二叉树，并包含用于二叉搜索树特殊情况的输入例程，正如示例 `binarytreetest.asy` 所展示的那样：

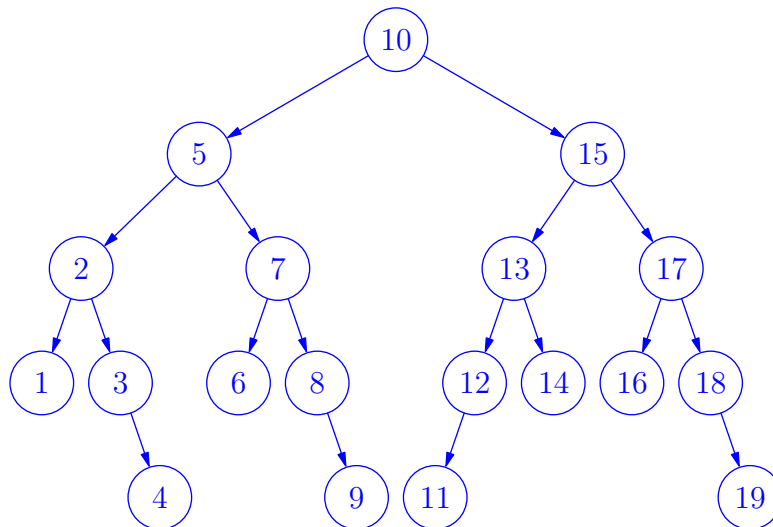
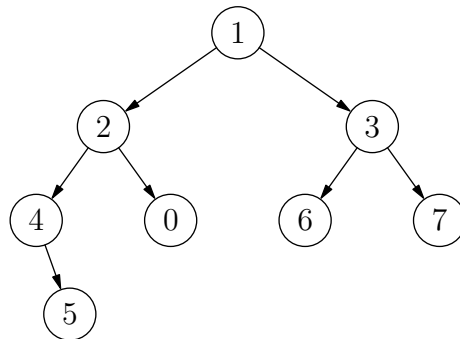
```
import binarytree;

picture pic,pic2;

binarytree bt=binarytree(1,2,4,nil,5,nil,nil,0,nil,nil,3,6,nil,nil,7);
draw(pic,bt,condensed=false);

binarytree st=searchtree(10,5,2,1,3,4,7,6,8,9,15,13,12,11,14,17,16,18,19);
draw(pic2,st,blue,condensed=true);

add(pic.fit(),(0,0),10N);
add(pic2.fit(),(0,0),10S);
```



9.6 bsp

该模块使用二叉空间分割 (binary space partition, BSP) 和图片裁剪来实现平面图片的消隐(hidden surface removal)。它用于将三维场景投影到带有恰当遮挡关系的二维空间。

9.7 CAD

该模块由 Mark Henning 贡献, 为符合 DIN 15 标准的简单二维 CAD 图纸提供基础画笔定义和测量函数。它在独立的文档文件 CAD.pdf 中进行了说明。

9.8 contour3

该模块用于绘制描述为 (x, y, z) 实值函数的零空间或 `real[][][]` 矩阵的曲面。示例文件 magnetic.asy 展示了其用法。

9.9 drawtree

这是一个简单的树绘制模块, 由示例 treetest.asy 所使用。

9.10 embed

该模块提供了调用 $\LaTeX$ 宏包 (随 MikTeX 附带)

<http://mirror.ctan.org/macros/latex/contrib/media9>

的接口, 用于在 PDF 文档中嵌入视频、声音和 3D 对象。

使用 external 模块替代 embed 模块提供了一种更便携的视频文件嵌入方法, 该方法应当可在任何平台上工作且不需要 media9 宏包。

上述两个接口的示例见示例目录的 animations 子目录中的文件 embeddedmovie.asy, 以及 externalmovie.asy。对于直接由 Asymptote 生成的高质量嵌入式视频, 可使用 animate 模块配合 animate.sty 宏包来嵌入便携式 PDF 动画 (见 9.2 节 [animate], 173 页)。

嵌入 U3D 代码的示例见文件 embeddedu3d。

9.11 external

该模块提供了一种在 PDF 文档中嵌入视频文件的便携方法, 不需要 media9 宏包。它是 embed 模块的替代方案, 具备更好的跨平台兼容性。

9.12 feynman

该模块由 Martin Wiebusch 贡献, 用于绘制费曼图(Feynman diagram), 正如示例 eetomumu.asy 和 fermi.asy 所展示的那样。

9.13 flowchart

该模块提供用于绘制流程图的例程。主要结构体为 block, 代表流程图中的单个区块(block)。在给定图片变换 t 时, 以下八个函数返回区块相应边缘上的位置:

```
pair block.top(transform t=identity());
pair block.left(transform t=identity());
pair block.right(transform t=identity());
```

```

pair block.bottom(transform t=identity());
pair block.topleft(transform t=identity());
pair block.topright(transform t=identity());
pair block.bottomleft(transform t=identity());
pair block.bottomright(transform t=identity());

```

要以用户坐标获取沿区块边界的任意位置, 请使用:

```

pair block.position(real x, transform t=identity());

```

用户坐标下的区块中心存储在 `block.center` 中, 而 PostScript 坐标下的区块尺寸由 `block.size` 给出。

包含该区块的框(frame)由以下函数返回:

```

frame block.draw(pen p=currentpen);

```

以下区块生成例程接受 Label、string 或 frame 作为其 object 参数:

带可选标题栏 (以及标题栏和主体周围填充 dx) 的矩形区块:

```

block rectangle(object header, object body, pair center=(0,0),
                pen headerpen=mediumgray, pen bodypen=invisible,
                pen drawpen=currentpen,
                real dx=3, real minheaderwidth=minblockwidth,
                real minheaderheight=minblockwidth,
                real minbodywidth=minblockheight,
                real minbodyheight=minblockheight);
block rectangle(object body, pair center=(0,0),
                pen fillpen=invisible, pen drawpen=currentpen,
                real dx=3, real minwidth=minblockwidth,
                real minheight=minblockheight);

```

平行四边形区块:

```

block parallelogram(object body, pair center=(0,0),
                    pen fillpen=invisible, pen drawpen=currentpen,
                    real dx=3, real slope=2,
                    real minwidth=minblockwidth,
                    real minheight=minblockheight);

```

菱形区块:

```

block diamond(object body, pair center=(0,0),
              pen fillpen=invisible, pen drawpen=currentpen,
              real ds=5, real dw=1,
              real height=20, real minwidth=minblockwidth,
              real minheight=minblockheight);

```

圆形区块:

```

block circle(object body, pair center=(0,0), pen fillpen=invisible,
             pen drawpen=currentpen, real dr=3,
             real mindiameter=mincirclediameter);

```

圆角矩形区块:

```

block roundrectangle(object body, pair center=(0,0),
                    pen fillpen=invisible, pen drawpen=currentpen,

```

```
real ds=5, real dw=0, real minwidth=minblockwidth,
real minheight=minblockheight);
```

斜角矩形区块:

```
block bevel(object body, pair center=(0,0), pen fillpen=invisible,
pen drawpen=currentpen, real dh=5, real dw=5,
real minwidth=minblockwidth, real minheight=minblockheight);
```

要用直角折线连接 point 中的各数对来绘制路径, 请使用例程:

```
path path(pair point[], ... flowdir dir[]);
```

dir 中的条目指定 point 所给出的各数对之间的连续线段应当在 Horizontal (水平) 还是 Vertical (垂直) 方向绘制。

这是一个简单的流程图示例 (另请参阅示例 controlsystem.asy):

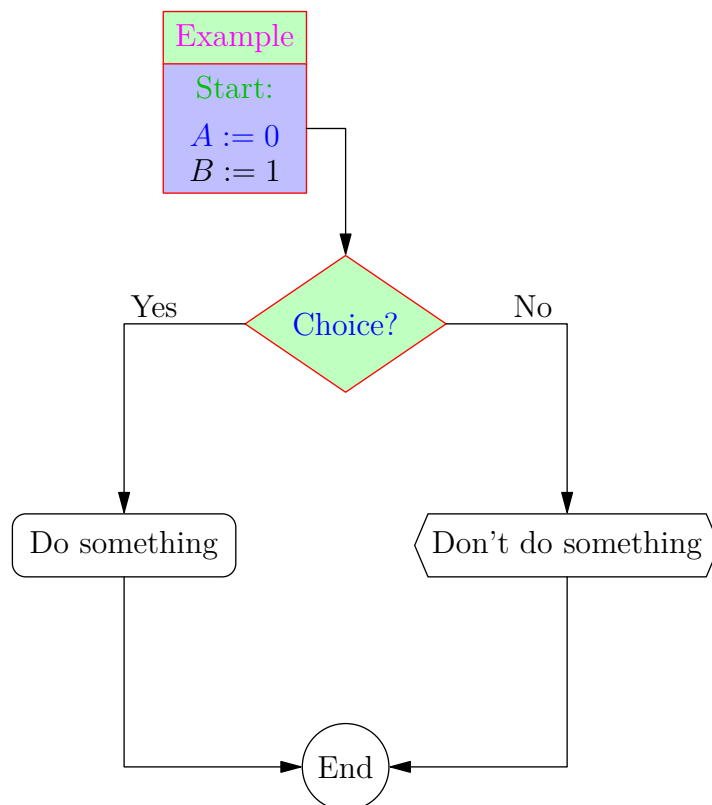
```
size(0,300);

import flowchart;

block block1=rectangle(Label("Example",magenta),
pack(Label("Start:",heavygreen),"",Label("$A:=0$",blue),
"$B:=1$"),(-0.5,3),palegreen,paleblue,red);
block block2=diamond(Label("Choice?",blue),(0,2),palegreen,red);
block block3=roundrectangle("Do something",(-1,1));
block block4=bevel("Don't do something",(1,1));
block block5=circle("End",(0,0));

draw(block1);
draw(block2);
draw(block3);
draw(block4);
draw(block5);

add(new void(picture pic, transform t) {
blockconnector operator -=blockconnector(pic,t);
// draw(pic,block1.right(t)-block2.top(t));
block1-Right-Down-Arrow-block2;
block2-Label("Yes",0.5,NW)-Left-Down-Arrow-block3;
block2-Right-Label("No",0.5,NE)-Down-Arrow-block4;
block4-Down-Left-Arrow-block5;
block3-Down-Right-Arrow-block5;
});
```



9.14 labelpath

该模块使用 PSTricks 的 `pstextpath` 宏将标签贴合在路径上绘制（具有恰当的字距调整，正如示例文件 `curvedlabel.asy` 所展示的那样），使用命令：

```
void labelpath(picture pic=currentpicture, Label L, path g,
               string justify=Centered, pen p=currentpen);
```

这里的 `justify` 是 `LeftJustified`、`Centered` 或 `RightJustified` 之一。应用到 `Label` 的平移变换的 x 分量被解释为沿着曲线的平移，而 y 分量被解释为偏离曲线的平移。所有其他 `Label` 变换均被忽略。该模块需要 `latex tex` 引擎，并且继承了 PSTricks `\pstextpath` 宏的限制。

9.15 labelpath3

该模块由 Jens Schwaiger 贡献，实现了无需 PSTricks 宏包的 3D 版本 `labelpath`。示例见 `curvedlabel3.asy`。

9.16 lmfit

该模块为数据分析提供最小二乘拟合例程。它可用于将数学模型拟合到实验数据。

9.17 MetaPost

该模块提供了一些有用的例程，帮助 MetaPost 用户将旧的 MetaPost 代码迁移到 Asymptote。欢迎在此做出进一步贡献。

与 MetaPost 不同，Asymptote 不会隐式求解线性方程组，因此没有 `whatever` 未知数的概念。例程 `extension`（见 [extension], 34 页）为 `whatever` 的常见用法提供了一个有用的替

代：求解通过 P 、 Q 和 p 、 q 的直线的交点。对于 `whatever` 不太常见的用法，可以使用内置的显式线性方程组求解器 `solve` 代替。

9.18 obj

该模块允许从简单的 `obj` 文件构建曲面，正如示例文件 `galleon.asy` 与 `triceratops.asy` 所展示的那样。

9.19 ode

正如示例 `odetest.asy` 所展示的那样，`ode` 模块实现了若干用于常微分方程(ordinary differential equation)的显式数值积分方案。

9.20 pstoeedit

该模块通过带 Asymptote 后端的 `pstoeedit` 工具，提供将 PostScript 文件转换为 Asymptote 代码的支持。

9.21 rational

该模块实现了有理数算术，用于无浮点误差的精确计算。它提供支持基本算术运算的 `rational` 类型。

9.22 rationalSimplex

该模块提供利用有理数算术求解线性规划的单纯形法(simplex method)。它是 `simplex` 模块的精确版本，避免了浮点精度问题。

9.23 simplex

该模块使用单纯形法求解通用的线性规划问题。

9.24 size10

该模块提供了一种简单的方法来使用对应于 `\documentclass[10pt]{article}` 的具名 $\TeX$ 字体大小：

```
import size10;
pen small=fontcommand("\small");
label("M",0,W,red+small);
pen normal=fontsize(9pt);
label("M",0,E,normal);
```

9.25 size11

该模块提供了一种简单的方法来使用对应于 `\documentclass[11pt]{article}` 的具名 $\TeX$ 字体大小。用法与 `size10` 类似。

9.26 slide

该模块提供了一个简单而高质量的制作演示文稿幻灯片的功能，包括便携式嵌入 PDF 动画（参见文件 `slidemovies.asy`）。简易示例见 `slidedemo.asy`。

9.27 syzygy

该模块自动绘制编织(braid)、关系(relation)和齐次关系(syzygy)及其对应的方程, 正如示例 `knots.asy` 所展示的那样。

9.28 tree

该模块实现了动态二叉搜索树的示例。

9.29 trembling

该模块由 Philippe Ivaldi 编写, 正如示例 `floatingdisk.asy` 所展示的那样, 允许绘制波浪线, 如同手绘一般。

10 开发者模块

这些模块面向 Asymptote 开发者。

10.1 plain

这是默认的 Asymptote 基础文件(base file) 绘图语言（例如 picture 结构）。

默认情况下，在翻译一个文件之前，以及在交互模式下给出第一条命令之前，都会隐式执行一条 `private import plain;`。在翻译用于模块定义的文件时情况也是如此（当然，翻译 plain 本身时除外）。这意味着在 plain 中定义的类型和函数几乎可以在所有 Asymptote 代码中访问。使用 `-noautoplain` 命令行选项可以禁用这一特性。

10.2 bezulate

本模块提供用于细分曲面(subdivision surface)生成的 bezulate 例程。它将（可能是非单连通的）网格分解为一组适合渲染的贝塞尔曲面片(Bezier patch)。该模块由 Asymptote 在内部使用，用于从网格数据创建光滑曲面。

10.3 simplex2

本模块求解二变量线性规划问题的一个特例，模块 plain 用它来实现图片的自动定尺寸(automatic sizing) (见 [deferred drawing], 52 页)。

10.4 v3d

本模块提供对 V3D 三维图形格式的支持，该格式可以使用 Python 的 pyv3d 库查看，也可以嵌入到 PDF 文档中。通过设置 `settings.v3d=true` 可以自动生成 V3D 内容，或者使用 `importv3d` 函数导入已有的 V3D 文件：

```
import v3d;
importv3d("file.v3d");
```

11 命令行选项

输入 `asy -h` 可以查看 Asymptote 所支持的完整命令行选项列表:

Usage: `../asy [options] [file ...]`

Options (negate boolean options by replacing `-` with `-no`):

<code>-GPUblockSize n</code>	Compute shader block size [8]
<code>-GPUcompress</code>	Compress GPU transparent fragment counts [false]
<code>-GPUindexing</code>	Compute indexing partial sums on GPU [true]
<code>-GPUinterlock</code>	Use fragment shader interlock [true]
<code>-GPUlocalSize n</code>	Compute shader local size [256]
<code>-V,-View</code>	View output; command-line only
<code>-absolute</code>	Use absolute WebGL dimensions [false]
<code>-a,-align C B T Z</code>	Center, Bottom, Top, or Zero page alignment [C]
<code>-aligndir pair</code>	Directional page alignment (overrides align) [(0,0)]
<code>-animating</code>	[false]
<code>-antialias n</code>	Antialiasing width for rasterized output [2]
<code>-auto3D</code>	Automatically activate 3D scene [true]
<code>-autobillboard</code>	3D labels always face viewer by default [true]
<code>-autoimport str</code>	Module to automatically import
<code>-autoplain</code>	Enable automatic importing of plain [true]
<code>-autoplay</code>	Autoplay 3D WebGL animations [true]
<code>-autorotate</code>	Enable automatic PDF page rotation [false]
<code>-axes3</code>	Show 3D axes in PDF output [true]
<code>-batchMask</code>	Mask fpv exceptions in batch mode [false]
<code>-batchView</code>	View output in batch mode [false]
<code>-bw</code>	Convert all colors to black and white false
<code>-cd directory</code>	Set current directory; command-line only
<code>-cmyk</code>	Convert rgb colors to cmyk false
<code>-c,-command str</code>	Command to autoexecute
<code>-compact</code>	Conserve memory at the expense of speed false
<code>-compress</code>	Compress images in PDF output [true]
<code>-convertOptions str</code>	[]
<code>-curlAfterRead</code>	Allow libcurl after reading local files via input() false
<code>-d,-debug</code>	Enable debugging messages and traceback false
<code>-device n</code>	Set Vulkan device [-1]
<code>-devicepixelratio n</code>	Ratio of physical to logical pixels [0]
<code>-digits n</code>	Default output file precision [7]
<code>-divisor n</code>	Garbage collect using purge(divisor=n) [2]
<code>-dvipsOptions str</code>	[]
<code>-dvisvgmMultipleFiles</code>	dvisvgm supports multiple files [true]
<code>-dvisvgmOptions str</code>	[-optimize]
<code>-embed</code>	Embed rendered preview image [true]
<code>-e,-environment</code>	Show summary of environment settings; command-line only
<code>-exitonEOF</code>	Exit interactive mode on EOF [true]
<code>-fitscreen</code>	Fit rendered image to screen [true]

-framerate frames/s	Animation speed [30]
-fxaa	Enable FXAA. Multisampling is turned off if FXAA is enabled [false]
-glOptions str	[]
-globalread	Allow read from other directory true
-globalwrite	Allow write to other directory false
-gray	Convert all colors to grayscale false
-gsOptions str	[]
-h,-help	Show summary of options; command-line only
-historylines n	Retain n lines of history [1000]
-htmlviewerOptions str	[]
-hyperrefOptions str	[setpagesize=false,unicode,pdfborder=0 0 0]
-ibl	Enable environment map image-based lighting [false]
-iconify	[false]
-image str	Environment image name [snowyField]
-imageDir str	Environment image library directory [ibl]
-imageURL str	Environment image library URL [https://vectorgraphics.gitlab.io/asym]
-inlineimage	Generate inline embedded image [false]
-inlinetex	Generate inline TeX code [false]
-inpipe n	Input pipe [-1]
-interactiveMask	Mask fpu exceptions in interactive mode [true]
-interactiveView	View output in interactive mode [true]
-interactiveWrite	Write expressions entered at the prompt to stdout [true]
-interrupt	[false]
-k,-keep	Keep intermediate files [false]
-keepaux	Keep intermediate LaTeX .aux files [false]
-keys	Generate WebGL keys false
-level n	Postscript level [3]
-licenses	Show license and third-party attribution; command-line only
-l,-listvariables	List available global functions and variables [false]
-localhistory	Use a local interactive history file [false]
-loop	Loop 3D animations [false]
-lossy	Use single precision for V3D reals [false]
-lsp	Interactive mode for the Language Server Protocol [false]
-m,-mask	Mask fpu exceptions; command-line only
-maxFramesInFlight n	Maximum frames queued to the GPU [3]
-maxtile pair	Maximum rendering tile size [(1024,768)]
-maxviewport pair	Maximum viewport size [(0,0)]
-multiline	Input code over multiple lines at the prompt [false]
-multipleView	View output from multiple batch-mode files [false]
-multisample n	Multisampling width for screen images [4]
-offline	Produce offline html files [false]
-offscreen	Use offscreen rendering [false]
-O,-offset pair	PostScript offset [(0,0)]
-f,-outformat format	Convert each output file to specified format
-o,-outname name	Alternative output directory/file prefix
-outpipe n	Output pipe [-1]

<code>-paperheight bp</code>	Default page height [0]
<code>-paperwidth bp</code>	Default page width [0]
<code>-p,-parseonly</code>	Parse file [false]
<code>-pdfreload</code>	Automatically reload document in pdfviewer [false]
<code>-pdfreloadOptions str</code>	[]
<code>-pdfreloaddelay usec</code>	Delay before attempting initial pdf reload [750000]
<code>-pdfviewerOptions str</code>	[]
<code>-position pair</code>	Initial 3D rendering screen position [(0,0)]
<code>-prc</code>	Embed 3D PRC graphics in PDF output [false]
<code>-prerender resolution</code>	Prerender V3D objects (0 implies vector output) [0]
<code>-prompt str</code>	Prompt [>]
<code>-prompt2 str</code>	Continuation prompt for multiline input [...]
<code>-psviewerOptions str</code>	[]
<code>-q,-quiet</code>	Suppress welcome text and noninteractive stdout [false]
<code>-render n</code>	Render 3D graphics using n pixels per bp [2]
<code>-resizestep step</code>	Resize step [1.2]
<code>-reverse</code>	reverse 3D animations [false]
<code>-rgb</code>	Convert cmYk colors to rgb false
<code>-safe</code>	Disable system call true
<code>-scroll n</code>	Scroll standard output n lines at a time [0]
<code>-shiftHoldDistance n</code>	WebGL touch screen distance limit for shift mode [20]
<code>-shiftWaitTime ms</code>	WebGL touch screen shift mode delay [200]
<code>-spinstep deg/s</code>	Spin speed [60]
<code>-srgb</code>	Render 3D images in sRGB space [false]
<code>-svgemulation</code>	Emulate unimplemented SVG shading [true]
<code>-tabcompletion</code>	Interactive prompt auto-completion [true]
<code>-tex engine</code>	latex pdflatex xelatex lualatex tex pdfTeX luatex context none [late
<code>-thick</code>	Render thick 3D lines [true]
<code>-thin</code>	Render thin 3D lines [true]
<code>-threads</code>	Use POSIX threads for 3D rendering [true]
<code>-toolbar</code>	Show 3D toolbar in PDF output [true]
<code>-s,-translate</code>	Show translated virtual machine code [false]
<code>-twice</code>	Run LaTeX twice (to resolve references) [false]
<code>-twosided</code>	Use two-sided 3D lighting model for rendering [true]
<code>-u,-user str</code>	General purpose user string
<code>-v3d</code>	Embed 3D V3D graphics in PDF output [false]
<code>-v,-verbose</code>	Increase verbosity level (can specify multiple times) 0
<code>-version</code>	Show version; command-line only
<code>-vibrateTime ms</code>	WebGL shift mode vibrate duration [25]
<code>-viewportmargin pair</code>	Horizontal and vertical 3D viewport margin [(0.5,0.5)]
<code>-vsync</code>	Vertically synchronize with monitor [false]
<code>-vulkan</code>	Use Vulkan renderer if available [true]
<code>-wait</code>	Wait for child processes to finish before exiting [false]
<code>-warn str</code>	Enable warning; command-line only
<code>-webgl2</code>	Use webgl2 if available [false]
<code>-where</code>	Show where listed variables are declared [false]
<code>-wsl</code>	Run asy under the Windows Subsystem for Linux [false]

```

-xasy                    Interactive mode for xasy false
-zoomPinchCap limit      WebGL maximum zoom pinch [100]
-zoomPinchFactor n       WebGL zoom pinch sensitivity [10]
-zoomThreshold threshold
                        Zoom remesh threshold [0.02]
-zoomfactor factor       Zoom step factor [1.05]
-zoomstep step           Mouse motion zoom step [0.1]

```

所有布尔选项都可以通过在选项名前加上 `no` 来取反。

如果没有给出参数, `Asymptote` 将以交互模式运行(见 第 12 章 [Interactive mode], 187 页)。在这种情况下, 默认的输出文件为 `out.eps`。

如果文件参数给出的是 `-`, `Asymptote` 将从标准输入读取。

如果指定了多个文件, 它们将被视为各自独立的 `Asymptote` 运行。

如果字符串 `autoimport` 非空, 则在每次运行中, 都会作为加载模块 `plain` 的最后一步自动导入一个以该名称命名的模块。

默认选项值可以作为 `Asymptote` 代码输入到名为 `config.asy` 的配置文件中(或由环境变量 `ASYMPTOTE_CONFIG` 或 `-config` 选项指定的文件)。`Asymptote` 会在其通常的搜索路径中查找该文件(见 2.5 节 [Search paths], 5 页)。通常, 配置文件放在用户主目录下的 `.asy` 目录中(在 MSDOS 下为 `%USERPROFILE%\asy`)。配置变量通过选项名的长形式来访问:

```

import settings;
outformat="pdf";
batchView=false;
interactiveView=true;
batchMask=false;
interactiveMask=true;

```

命令行选项会覆盖这些默认值。大多数配置变量也可以在运行时更改。高级配置变量 `dvipsOptions`、`hyperrefOptions`、`convertOptions`、`gsOptions`、`htmlviewerOptions`、`psviewerOptions` 和 `dvisvgmOptions` 允许以字符串的形式将专用选项传递给相应的应用程序或库。`hyperrefOptions` 的默认值为 `setpagesize=false,unicode,pdfborder=0 0 0`。

如果你将

```

import plain;
settings.autoplain=true;

```

插入到配置文件的开头, 那么该配置文件就可以包含任意的 `Asymptote` 代码。

对于(默认的) `latex` 和 `tex` TeX 引擎, 默认输出格式为 EPS; 对于 `pdflatex`、`xelatex`、`context`、`luatex` 和 `lualatex` TeX 引擎, 默认输出格式为 PDF。可以使用 `-f` 选项(或 `outformat` 设置)生成其他输出格式。

要生成 SVG 输出, 你需要来自 <https://dvisvgm.de> 的 `dvisvgm` (版本 3.2.1 或更高), 它可以为通过 `graphic()` 函数嵌入的 EPS、PDF、PNG 和 JPEG 图像显示 SVG 输出(由 `xasy` 编辑器使用)。在默认设置 `settings.dvisvgmOptions="--optimize"` 下, 生成的输出会得到优化。

`Asymptote` 还可以生成 `ImageMagick` 的 `magick` 程序(版本 7 或更高)所支持的任何输出格式。可选设置 `-render n` 请求每个 `bp n` 像素的输出分辨率。抗锯齿(antialiasing)由参数 `antialias` 控制, 默认情况下它指定 2 像素的采样宽度。要向 `magick` 传递其他选项, 可以

使用 `convertOptions` 设置, 或者手动调用 `magick convert`。下面的示例模拟了 `Asymptote` 如何以每个 bp 一像素的分辨率生成抗锯齿的 tiff 输出:

```
asy -o - venn | magick convert -alpha Off -density 144x144 -geometry 50%x eps:- venn.tiff
```

如果给出了 `-nosafe` 选项, `Asymptote` 将以非安全(unsafe)模式运行。这会启用 `int system(string s)` 和 `int system(string[] s)` 调用, 允许执行任意 shell 命令。默认模式 `-safe` 会禁用此调用。

可以使用 `-O` 选项将 PostScript 偏移(offset)指定为一个数对 (单位为 bp):

```
asy -O 0,0 file
```

默认偏移为零。数对 `aligndir` 指定页面边界 (映射为矩形 $[-1,1] \times [-1,1]$) 上的一个可选方向, 图片应对齐到该方向; 默认值 (0,0) 表示居中对齐。

`-c (command)` 选项可用于在命令行上以字符串形式执行任意 `Asymptote` 代码。字符串不必以分号结尾。多个 `-c` 选项按给出的顺序执行。例如

```
asy -c 2+2 -c "sin(1)" -c "size(100); draw(unitsquare)"
```

会产生输出

```
4
```

```
0.841470984807897
```

并绘制一个大小为 100 的 `unitsquare`。

`-u (user)` 选项可用于在命令行上以字符串形式指定任意 `Asymptote` 设置。字符串不必以分号结尾。多个 `-u` 选项按给出的顺序执行。形如 `-u x=sqrt(2)` 的命令行代码可以像这样在模块内执行:

```
real x;
usersetting();
write(x);
```

当 `-l (listvariables)` 选项与文件参数一起使用时, 只会列出在指定文件中定义的全局函数和变量。

每增加一个 `-v` 选项, 就会产生更多的调试输出:

- `-v` 显示顶层模块名和最终输出文件名。
- `-vv` 还显示导入和包含的模块名, 以及最终的 `TeX` 和 `dvips` 处理信息。
- `-vvv` 还输出 `TeX` 双向管道诊断信息。
- `-vvvv` 还输出结(knot) guide 解算器诊断信息。
- `-vvvvv` 还输出 `Asymptote` 回溯(traceback)诊断信息。

输入 `asy -licenses` 可以查看 `Asymptote` 及其捆绑组件的许可证和第三方署名信息。输入 `asy -licenses=full` 或 `asy --licenses=full` 可以打印完整的许可证文本 (读取自己安装的 `licenses/` 目录)。

12 交互模式

在不指定文件参数的情况下执行命令 `asy` 即可进入交互模式(interactive mode)。当禁用 `-multiline` 选项（默认状态）时，每行必须是一条完整的 `Asymptote` 语句（除非使用末尾的反斜杠字符 `\` 显式续行）；无需以分号终止输入行。如果设置 `settings.multiline=true`，则可以跨多行输入交互代码；在此模式下，自动为交互输入行添加分号终止符的功能会被禁用。多行模式对于将 `Asymptote` 代码直接剪切粘贴到交互输入缓冲区非常有用。

交互模式可以方便地用作计算器：在交互提示符下输入的表达式（前提是存在对应的 `write` 函数）会自动求值并写入 `stdout`。如果表达式不可写，则会打印其类型签名。在任何一种情况下，都可以在提示符下的下一行输入中使用符号 `%` 来引用该表达式。例如：

```
> 2+3
5
> %*4
20
> 1/%
0.05
> sin(%)
0.0499791692706783
> currentpicture
<picture currentpicture>
> %.size(200,0)
>
```

`%` 符号用作变量时，是标识符 `operator answer` 的简写，提示符在每次表达式求值并输出后对其进行设置。

以下特殊命令仅在交互模式下受支持，且必须紧跟在提示符之后输入：

<code>help</code>	查看手册；
<code>erase</code>	擦除 <code>currentpicture</code> ；
<code>reset</code>	将 <code>Asymptote</code> 环境重置为其初始状态，但 <code>settings</code> 模块的更改（见 [settings], 185 页）、当前目录（见 [cd], 54 页）以及断点（见 第 18 章 [Debugger], 194 页）除外；
<code>input FILE</code>	执行交互式重置，随后执行命令 <code>include FILE</code> 。如果文件名 <code>FILE</code> 包含非字母数字字符，请用双引号将其括起来。可以在同一行输入一个尾随分号以及后续的可选 <code>Asymptote</code> 命令。
<code>quit</code>	退出交互模式（ <code>exit</code> 为同义词；缩写 <code>q</code> 也被接受，除非存在名为 <code>q</code> 的顶层变量）。最近 1000 条（此数量可通过 <code>historylines</code> 配置变量更改）历史命令将保留在用户主目录下的文件 <code>.asy/history</code> 中（除非指定了命令行选项 <code>-localhistory</code> ，此时历史命令将存储在当前目录下的文件 <code>.asy_history</code> 中）。

键入 `ctrl-C` 会中断 `Asymptote` 代码的执行，并将控制权返回给交互提示符。

交互模式采用 GNU `readline` 库实现，具备命令历史记录与自动补全功能。如需自定义键绑定，请参阅：<https://tiswww.case.edu/php/chet/readline/readline.html>

Asymptote 系统目录中的文件 `asymptote.py` 提供了另一种交互式输入 Asymptote 命令的方法, 并结合了 Python 的完整功能。将此文件复制到您的 Python path, 然后在 Python 3 中执行以下命令

```
from asymptote import *
g=asy()
g.size(200)
g.draw("unitcircle")
g.send("draw(unitsquare)")
g.fill("unitsquare, blue")
g.clip("unitcircle")
g.label("\$0$", (0,0), SW)
```

13 图形用户界面

如果需要对最终图形进行调整, Asymptote 附带的初步图形用户界面(GUI) xasy 允许您移动图形对象并绘制新对象。然后可以将修改后的图形保存为普通的 Asymptote 文件。

13.1 GUI 安装

由于 xasy 是用交互式脚本语言 Python/Qt 编写的, 因此需要 Python (<https://www.python.org>), 以及 Python 软件包 PySide6、cson 和 numpy:

```
pip3 install cson numpy PySide6
```

图片会被解构为 SVG 图像格式。由于 Qt5 不支持 SVG 裁剪, 您需要 rsvg-convert 实用工具, 它在 UNIX 系统上是 librsvg2-tools 软件包的一部分, 在 macOS X 上是 librsvg 软件包的一部分; 在 Microsoft Windows 下, 可从以下网址获取:

<https://sourceforge.net/projects/tumagcc/files/rsvg-convert-2.40.20.7z>

请确保 rsvg-convert 在 PATH 中可用, 或者在 xasy 的设置选项 rsvgConverterPath 中指定 rsvg-convert 的位置。

使用 $\text{\LaTeX}$ TeX 引擎时, 将图片解构成其组成部分的速度最快。默认设置 dvisvgmMultipleFiles=true 可以加快在 PDF TeX 引擎下的解构速度。

13.2 GUI 使用

方向键 (或鼠标滚轮) 便于在 xasy 内临时调高和调低对象, 以便选择对象。按方向键时, 按住 Shift 键将进行平移, 按住 Control 键将进行缩放。使用鼠标滚轮时, 按住 Alt 或 Shift 键将进行平移, 按住 Control 键将进行缩放。在平移模式下, 可用鼠标粗略拖动对象, 也可在按住鼠标按钮的同时用方向键精确定位对象。

复合对象 (例如箭头) 可以通过将其括在以下命令中来防止被解构:

```
void begingroup(picture pic=currentpicture);
void endgroup(picture pic=currentpicture);
```

默认情况下, 图片或框(frame)的元素在添加到图片中时会被组合在一起。然而, 添加到另一个框中的框元素默认不会组合在一起: 它们的元素将被单独解构 (见 [add], 50 页)。

14 命令行界面

Asymptote 代码可以直接从命令行发送到 <https://asymptote.ualberta.ca> 服务器:

- SVG 输出:
`curl -- data-binary 'import venn;' 'asymptote.ualberta.ca:10007? f=svg' | display -`
- HTML 输出:
`curl -- data-binary @/usr/local/share/doc/asymptote/examples/Klein.asy 'asymptote.ualberta.ca:10007' -o Klein.html`
- V3D 输出:
`curl --data-binary 'import teapot;' 'asymptote.ualberta.ca:10007?f=v3d' -o teapot.v3d`
- 渲染位图为每 bp 2 像素的 PDF 输出:
`curl --data-binary 'import teapot;' 'asymptote.ualberta.ca:10007?f=pdf' -o teapot.pdf`
- 渲染位图为每 bp 4 像素的 PDF 输出:
`curl -- data-binary 'import teapot;' 'asymptote.ualberta.ca:10007? f=pdf&render=4' -o teapot.pdf`
- PRC 输出:
`curl -- data-binary 'import teapot;' 'asymptote.ualberta.ca:10007? f=pdf&prc' -o teapot.pdf`
- 渲染预览位图为每 bp 4 像素的 PRC 输出:
`curl -- data-binary 'import teapot;' 'asymptote.ualberta.ca:10007? f=pdf&prc&render=4' -o teapot.pdf`

命令行界面的源代码可从 <https://github.com/vectorgraphics/asymptote-http-server> 获取。

15 语言服务器协议

在 UNIX 和 macOS X 下, Asymptote 支持 语言服务器协议(LSP) (https://en.wikipedia.org/wiki/Language_Server_Protocol)的功能, 包括函数签名和变量匹配。在 MSWindows 下, Asymptote 目前仅在 Windows Subsystem for Linux 中编译时才支持 LSP。

Emacs 用户可以通过按以下步骤安装 lsp-mode 来启用 Asymptote 语言服务器协议:

- 添加到 .emacs 初始化文件中:

```
(require 'package)
(add-to-list 'package-archives '("melpa" . "https://melpa.org/packages/") t)
(package-initialize)
```

- 启动 emacs 并执行

M-x package-refresh-contents

M-x package-install

然后选择 lsp-mode。

- 添加到 .emacs 初始化文件中:

```
(require 'lsp-mode)
(add-to-list 'lsp-language-id-configuration '(asy-mode . "asymptote"))

(lsp-register-client
  (make-lsp-client :new-connection (lsp-stdio-connection '("asy" "-lsp"))
                  :activation-fn (lsp-activate-on "asymptote")
                  :major-modes '(asy-mode)
                  :server-id 'asyls
                  )
)
```

- 启动 emacs 并执行

M-x lsp

16 PostScript 转 Asymptote

优秀的 PostScript 编辑器 `pstoedit` (3.50 或更高版本; 可从 <https://sourceforge.net/projects/pstoedit/> 获取) 可以将 PostScript 转换为 Asymptote 代码。该驱动包含原生裁剪、奇偶填充规则(even-odd fill rule)、PostScript 子路径以及完整的图像支持。示例如下:

```
asy -V /usr/local/share/doc/asymptote/examples/venn.asy
```

```
pstoedit -f asy venn.eps test.asy
```

```
asy -V test
```

如果线宽不太正确, 请尝试向 `pstoedit` 提供 `-dis` 选项。如果字体排版不正确, 请尝试向 `pstoedit` 提供 `-dt` 选项。

17 帮助

常见问题解答列表(FAQ)维护于

<https://asymptote.sourceforge.io/FAQ>

有关安装和使用 Asymptote 但未在 FAQ 中解答的问题，应发送至 Asymptote 论坛：

<https://sourceforge.net/p/asymptote/discussion/409349>

附带一个说明您尝试实现的目标的示例，有助于您获得有用的反馈。 $\LaTeX$ 问题通常可以通过 `-vv` 或 `-vvv` 命令行选项进行诊断。以补丁或 Asymptote 模块形式提供的贡献可以发布在这里：

<https://sourceforge.net/p/asymptote/patches>

若要接收即将发布的版本的通知，请在以下网址订阅 Asymptote：

<https://sourceforge.net/projects/asymptote/>

如果您在 Asymptote 中发现了缺陷(bug)，请在提交缺陷报告之前（如果可能）检查该缺陷是否仍然存在于最新 git 开发代码中（见 2.8 节 [Git], 7 页）。新缺陷可以在以下网址报告：

<https://github.com/vectorgraphics/asymptote/issues>

要查看缺陷是否已被修复，请检查状态为 Closed 的缺陷以及以下文件中的最新行：

<https://asymptote.sourceforge.io/ChangeLog>

Asymptote 可以配置可选的 GNU 库 `libsigsegv`（可从 <https://www.gnu.org/software/libsigsegv/> 获取），该库允许区分用户生成的 Asymptote 栈溢出（见 [stack overflow], 65 页）与真正的段错误（由内部 C++ 编程错误引起；请将生成此类段错误的 Asymptote 代码连同缺陷报告一并提交）。

18 调试器

Asymptote 现在包含一个基于行（而非基于代码）的调试器，可帮助用户跟踪流程控制。要在文件 `file` 的第 `line` 行设置断点，请使用命令：

```
void stop(string file, int line, code s=quote{ });
```

可选参数 `s` 可用于有条件地将 `plain_debugger.asy` 中的变量 `ignore` 设置为 `true`。例如，该断点的前 10 次触发将被忽略（变量 `int count=0` 定义在 `plain_debugger.asy` 中）：

```
stop("test",2,quote{ignore=(++count <= 10);});
```

要在文件 `file` 中包含字符串 `text` 的第一行设置断点，请使用：

```
void stop(string file, string text, code s=quote{ });
```

要列出所有断点，请使用：

```
void breakpoints();
```

要清除某个断点，请使用：

```
void clear(string file, int line);
```

要清除所有断点，请使用：

```
void clear();
```

可在调试提示符下输入以下命令：

<code>h</code>	帮助；
<code>c</code>	继续执行；
<code>i</code>	单步执行到下一条指令；
<code>s</code>	单步执行到下一个可执行行；
<code>n</code>	单步执行到当前文件中的下一个可执行行；
<code>f</code>	单步执行到下一个文件；
<code>r</code>	返回到与最近的断点相关联的文件；
<code>t</code>	切换追踪（-vvvvv）模式；
<code>q</code>	退出调试并结束执行；
<code>x</code>	退出调试器并运行至完成。

也可以在调试提示符下输入任意 Asymptote 代码；但是，由于调试器是使用 `eval` 实现的，目前只能显示或修改顶层（全局）变量。

可以通过以下调用手动进入调试提示符：

```
void breakpoint(code s=quote{ });
```

19 致谢

对 Asymptote 开发的资助由加拿大自然科学与工程研究理事会(Natural Sciences and Engineering Research Council of Canada)、太平洋数学科学研究所(Pacific Institute for Mathematical Sciences)以及阿尔伯塔大学理学院(University of Alberta Faculty of Science)慷慨提供。

我们还要感谢 John D. Hobby 先前的贡献，他是启发了 Asymptote 开发的 MetaPost 程序作者；以及 $\text{\TeX}$ 与 MetaFont (MetaPost 基于此构建) 的作者 Donald E. Knuth。

Asymptote 的作者是 Andy Hammerlindl、John Bowman 和 Tom Prince。Sean Healy 设计了 Asymptote 标志。其他贡献者包括 Orest Shardt、Jesse Frohlich、Michail Vidiassov、Charles Staats、Philippe Ivaldi、Olivier Guibé、Radoslav Marinov、Jeff Samuelson、Chris Savage、Jacques Pienaar、Mark Henning、Steve Melenchuk、Martin Wiebusch、Stefan Knorr、Supakorn “Jamie” Rassameemasuang、Jacob Skitsko、Joseph Chaumont 和 Oliver Cheng。Pedram Emami 开发了托管在 <https://asymptote.ualberta.ca> 的 Asymptote Web Application:

<https://github.com/vectorgraphics/asymptoteWebApplication>

总索引

!

! 61
!= 58, 61

#

..... 60

%

% 60, 187
%= 61

&

& 23, 61
&& 61

*

* 37, 60
** 61
*= 61

+

+ 37, 60
++ 61
+= 61

-

- 60
-- 12, 61
--- 23
-= 61

.

.. (two dots) 12
.asy 5

/

/ 60
/= 61

2

2D graphs 107

3

3D graphs 148
3D grids 165
3D PostScript 147

:

: 61
:: 23

<

< 61
<= 61

=

== 58, 61

>

> 61
>= 61

?

? 61

[

[=] 62
[] 62
Map.operator [=] 93
Map.operator [] 92
string.operator [] 28, 62

^

^ 60, 61
^= 61
^^ 12

A

- Ai 69
- Ai_deriv 69
- Airy 69
- Align 19
- Allow 46
- AND 61
- Arc 31
- ArcArrow 14
- ArcArrow3 147
- ArcArrows 14
- ArcArrows3 147
- Arrow 14
- Arrow3 147
- Arrows 14
- Arrows3 147
- ASYMPTOTE_CONFIG 185
- Aspect 48
- Asymptote Web Application 1
- a4 4
- abort 30
- abs 25, 26, 69
- abs2 25, 26
- absolute 4, 140
- aCos 68
- accel 32, 33, 145
- access 78, 79
- acknowledgments 195
- acos 68
- acosh 68
- add 50, 142
 - Map.add 93
 - Set.add 95, 96
- addViews 143
- adjust 40
- advance 82, 88
- after
 - SortedSet.after 98
- alias 58, 72
- aligndir 186
- all 75
- and 22
- angle 25
- animate 4, 56, 173
- animation 173
- annotate 173
- antialias 140, 185
- append 54, 70
- arc 31, 142
- arclength 33, 145
- arcpoint 33
- arctime 33, 145
- arguments 66
- arithmetic operators 60
- array 29, 71
- array iteration 24
- arrays 70
- arrow 14, 20
- arrow keys 9, 189
- arrowbar 14
- arrowhead 15
- arrows 14
- aSin 68
- as 79
- ascii 29
- asin 68
- asinh 68
- assert 30
- assignment 24
- associative array 见 Map_K_V
- asy 30, 80
- asy-mode 6
- asy.vim 7
- asygl 5
- asyinclude 102
- asymptote.sty 102
- asymptote.xml 7
- aTan 68
- atan 68
- atan2 68
- atanh 68
- atleast 22
- atOrAfter
 - SortedSet.atOrAfter 98
- atOrBefore
 - SortedSet.atOrBefore 98
- attach 51, 102, 114
- attribution 186
- autoadjust 143
- autoimport 185
- automatic scaling 122
- autounravel 85
- axialshade 17
- axis 129, 131, 150, 151
- azimuth 26

B

- BackView 143
- Bar 14
- Bar3 147
- Bars 14
- Bars3 147
- BeginArcArrow 14
- BeginArcArrow3 147
- BeginArrow 14
- BeginArrow3 147
- BeginBar 14
- BeginBar3 147
- BeginDotMargin 15
- BeginDotMargin3 147
- BeginMargin 15
- BeginMargin3 147
- BeginPenMargin 15
- BeginPenMargin2 147
- BeginPenMargin3 147

BeginPoint 19
 Bessel 69
 Bezier curves 22
 Bezier patch 138
 Bezier triangle 138
 Bi 69
 Bi_deriv 69
 Billboard 145
 Blank 14
 Bottom 109
 BottomTop 109
 BottomView 143
 Bounds 149
 BTreeMap_K_V 92
 BTreeSet_T 97
 Button-1 189
 Button-2 189
 BWRainbow 153
 BWRainbow2 153
 babel 173
 background 137
 background color 50
 barsize 14
 basealign 41
 baseline 20
 batch mode 9
 beep 30
 before
 SortedSet.before 98
 bevel 177
 beveljoin 40
 bezulate 138, 181
 binary 55
 binary format 55
 binary operators 60
 binarytree 174
 black stripes 140
 block.bottom 176
 block.bottomleft 176
 block.bottomright 176
 block.center 176
 block.draw 176
 block.left 176
 block.position 176
 block.right 176
 block.top 176
 block.topleft 176
 block.topright 176
 bool 24
 bool3 24
 boolean operators 60
 bounding box 50
 bounds 153
 box 49, 142
 bp 9
 brace 31
 bracket operators 62
 break 24

breakpoints 194
 brick 44
 broken axis 126
 bsp 175
 bug reports 193
 buildcycle 35

C

C string 27
 CAD 175
 Center 19
 Chinese 43
 Ci 69
 Circle 31
 CJK 43
 CLZ 61
 Compiling from UNIX source 6
 Coons shading 18
 Cos 68
 CTZ 61
 Cyrillic 43
 camera 143
 casts 77
 cbirt 68
 cd 54
 ceil 68
 center 143
 checker 44
 choose 69
 circle 30, 142, 176
 circlebarframe 166
 clamped 107
 clang 6
 clear 55, 194
 clip 18
 cm 10
 cmd 4
 cmyk 38
 colatitude 26
 collections 87
 collections.btreemap 92
 collections.enumerate 90
 collections.genericpair 88
 collections.hashmap 92
 collections.iter 82, 88
 collections.map 92
 collections.zip 89
 collections.zip2 89
 color 38, 152
 coloredNodes 172
 coloredpath 171
 coloredSegments 172
 colorless 38
 colormap 158
 colors 38
 comma 54
 comma-separated-value mode 76

command-line interface 190
 command-line options 5, 182
 comment character 54
 compass directions 11
 complement 72
 concat 72
 conditional 24, 61
 config 5, 185
 configuration file 4, 185
 configuring 4
 conj 25
 constructors 58
 containers 87
 contains
 Map.contains 92
 Set.contains 95
 context 185
 continue 24, 194
 contour 159
 contour3 175
 controlSpecifier 37
 controls 22, 136
 convert 56
 convertOptions 185
 copy 72
 cos 68
 cosh 68
 cputime 69
 crop 120
 cropping graphs 120
 cross 26, 27, 116
 crossframe 166
 crosshatch 44
 csv 75, 76
 cubicroots 75
 curl 23, 136
 curlSpecifier 37
 currentlight 137
 currentpen 37
 currentprojection 143
 curve 169
 custom axis types 109
 custom mark routine 118
 custom tick locations 110
 cut 34
 cycle 10, 12, 136
 cyclic 32, 36, 70, 145

D

Debian 3
 DefaultHead 15
 DefaultHead3 147
 Degrees 68
 DOSendl 54
 DOSnewl 54
 DotMargin 15
 DotMargin3 147

DotMargins 15
 DotMargins3 147
 Draw 49
 dashdotted 39
 dashed 39
 data types 24
 date 29
 debugger 194
 declaration 24
 deconstruct 189
 default arguments 66
 defaultformat 109
 defaultpen 38, 40, 42, 46
 defaultrender 136
 deferred drawing 52, 181
 degrees 25, 68
 delete 56, 70
 Map.delete 93
 Set.delete 96
 description 1
 developer modules 181
 devicepixelratio 139
 diagonal 75
 diamond 176
 dictionary 见 Map_K_V
 diffuse 137
 diffusepen 137
 dimension 75, 76
 dir 5, 25, 27, 32, 145
 direction specifier 22
 directory 54
 dirSpecifier 37
 dirtime 33
 display 4
 do 24
 dot 16, 26, 27, 74
 dotted 39
 double deferred drawing 141
 double precision 55
 draw 14, 16, 139
 drawer 53
 drawing commands 14
 drawline 168
 drawtree 175
 dvips 5
 dvipsOptions 185
 dvisvgm 5, 185
 dvisvgmMultipleFiles 189
 dvisvgmOptions 185

E

E 11, 69
 Editing modes 6
 Ei 69
 Embedded 145
 EndArcArrow 14
 EndArcArrow3 147
 EndArrow 14
 EndArrow3 147
 EndBar 14
 EndBar3 147
 EndDotMargin 15
 EndDotMargin3 147
 EndMargin 15
 EndMargin3 147
 EndPenMargin 15
 EndPenMargin2 147
 EndPenMargin3 147
 EndPoint 19
 EPS 20, 185
 ellipse 31
 elliptic functions 69
 else 24
 emacs 6
 embed 175
 emissivepen 137
 empty 47
 Map.empty 92
 Set.empty 95
 endl 54
 enumerate 90
 envelope 20
 environment variables 5
 eof 55, 75
 eol 55, 75
 erase 9, 28, 47, 52
 erf 68
 erfc 68
 error 54, 55
 error bars 117
 errorbars 116
 eval 80
 evenodd 12, 40
 exit 30, 187, 194
 exp 68
 expi 25, 27
 explicit 77
 explicit casts 78
 expm1 68
 exponential integral 69
 extendcap 40
 extension 34, 178
 external 175
 extract
 Set.extract 96
 extrude 145

F

F 69
 Fedora 3
 FFTW 6
 Fill 49
 FillDraw 49
 FrontView 143
 Function shading 18
 fabs 68
 face 147
 factorial 69
 feynman 175
 fft 74
 file 54, 194
 fill 16, 17
 filldraw 16
 filloutside 17
 fillrule 40
 filltype 49
 find 28, 72
 findall 72
 firstcut 34
 fit3 141
 fixedscaling 48
 floor 68
 flowchart 175
 flush 54, 55
 fmod 68
 font 42, 43
 font encoding 43
 fontcommand 43
 fontsize 42, 159
 for
 range-based 81
 format 29, 185
 forum 193
 frame 47
 freshnel0 137
 from 79
 function declarations 65
 function shading 18
 functions 64, 68
 functionsshade 18

G

Gaussrand 69
 GNU Scientific Library 69
 Gradient 153
 Grayscale 153
 GSL 6
 GUI 189
 GUI installation 189
 GUI usage 189
 gamma 68
 geometry 165
 get 82, 88
 Set.get 96

getc..... 54
 getint..... 55
 getpair..... 55
 getreal..... 55
 getstring..... 55
 gettriple..... 55
 git..... 7
 glOptions..... 140, 185
 globalwrite..... 54, 56
 gouraudshade..... 17
 gradient shading..... 17
 graph..... 107
 graph3..... 148
 graphic..... 20, 185
 graphical user interface..... 189
 graphwithderiv..... 121
 gray..... 38
 grayscale..... 38
 grid..... 44, 124
 grid3..... 165
 gs..... 4
 gsl..... 69
 gsOptions..... 185
 guide..... 35
 guide3..... 136

H

HashMap_K_V..... 92
 Headlamp..... 137
 Hermite..... 107
 Hermite(splintype splintype)..... 107
 HookHead..... 15
 HookHead3..... 147
 Horizontal..... 177
 HTML5..... 140
 hash..... 86
 hash function..... 86
 hatch..... 44
 height..... 102
 help..... 187, 193, 194
 hex..... 29, 38
 hexadecimal..... 29, 38
 hidden surface removal..... 147
 histogram..... 69
 history..... 56, 187
 historylines..... 187
 htmlviewer..... 4
 htmlviewerOptions..... 185
 hyperrefOptions..... 185
 hypot..... 68

I

I..... 69
 IgnoreAspect..... 48
 ImageMagick..... 5, 173, 185
 InOutTicks..... 149
 InTicks..... 149
 Iter_array_T..... 90
 Iter_T..... 82, 88, 89
 Iterable..... 89
 Iterable_array_T..... 90
 Iterable_T..... 88
 i_scaled..... 69
 ibl..... 138
 iconify..... 140
 identity..... 46, 68, 75
 identity4..... 144
 if..... 24
 image..... 153, 154
 image-based lighting..... 138
 images..... 153
 implicit casts..... 77
 implicit linear solver..... 178
 implicit scaling..... 63
 implicitsurface..... 170
 import..... 79
 importv3d..... 141
 inches..... 10
 incircle..... 26
 include..... 80
 including images..... 20
 increasing..... 169
 inf..... 25
 inheritance..... 59
 initialized..... 70
 initializers..... 56
 inline..... 102
 input..... 54, 187
 input encoding..... 43
 insert..... 28, 70
 inside..... 35
 insphere..... 146
 inst..... 194
 installation..... 3
 int..... 25
 integer division..... 60
 integral..... 69
 integrate..... 69
 interactive mode..... 9, 187
 interior..... 35
 interp..... 61
 interpolate..... 166
 intersect..... 34, 145, 168
 intersection
 Set..... 97
 intersectionpoint..... 34, 145, 168
 intersectionpoints..... 34, 145, 146
 intersections..... 34, 145
 intMax..... 25

intMin..... 25
 inverse..... 46, 75
 invert..... 144
 invisible..... 38
 isNullValue..... 92
 isnan..... 25
 iterators..... 81

J

J..... 68, 69
 Japanese..... 43

K

K..... 69
 Kate..... 7
 KDE editor..... 7
 Korean..... 43
 k_scaled..... 69
 keepAspect..... 48, 102
 keyboard bindings:..... 139
 keys..... 70
 Map.keys..... 93
 keyword..... 66
 keyword-only..... 66
 keywords..... 66

L

Label..... 16, 19, 112
 Landscape..... 48
 L^AT_EX NFSS fonts..... 42
 L^AT_EX usage..... 102
 Left..... 111
 LeftRight..... 111
 LeftSide..... 19
 LeftTicks..... 109, 110
 LeftView..... 143
 Legendre..... 69
 Linear..... 122
 Log..... 122
 LSP..... 191
 label..... 11, 18, 145
 labelmargin..... 18
 labelpath..... 178
 labelpath3..... 178
 labelx..... 112
 labely..... 112
 language context..... 43
 language server protocol..... 191
 lastcut..... 35
 lasy-mode..... 6
 latex..... 185
 latexmk..... 102
 latitude..... 27
 latticeshade..... 17
 layer..... 14

leastquares..... 130, 171
 legend..... 14, 15, 114
 length..... 25, 26, 28, 32, 36, 70, 145
 letter..... 4
 lexorder..... 169
 libcurl..... 80
 libm routines..... 68
 libsigsegv..... 65, 193
 licenses..... 186
 light..... 137
 limits..... 120
 line..... 75
 line mode..... 75
 linear..... 107
 linecap..... 40
 linejoin..... 40
 lineskip..... 42
 linetype..... 40
 linewidth..... 40
 lmfit..... 178
 locale..... 29
 log..... 68
 log-log graph..... 123
 log10..... 68
 log1p..... 68
 log2 graph..... 125
 logarithmic graph..... 123
 logical operators..... 60
 longdashdotted..... 39
 longdashed..... 39
 longitude..... 27
 loop..... 24
 luatex..... 185
 luatex..... 185

M

Map..... 91
 Map_K_V..... 91, 92
 .add..... 93
 .contains..... 92
 .delete..... 93
 .empty..... 92
 .keys..... 93
 .operator [=]..... 93
 .operator []..... 92
 .operator iter..... 93
 .pairs..... 93
 .size..... 92
 cast to Iterable..... 93
 iterating over..... 93
 Margin..... 15
 Margin3..... 147
 Margins..... 15
 Margins3..... 147
 MetaPost..... 178
 MetaPost..... 23
 MetaPost cutafter..... 35

MetaPost cutbefore..... 35
 MetaPost pickup..... 37
 MetaPost whatever..... 178
 Microsoft Windows..... 3
 MidArcArrow..... 14
 MidArcArrow3..... 147
 MidArrow..... 14
 MidArrow3..... 147
 MidPoint..... 19
 Move..... 46
 MoveQuiet..... 46
 macOS X binary distributions..... 3
 macOS X configuration..... 6
 magick..... 5, 56, 173, 185
 makePair..... 88
 makepen..... 45
 makeQueue..... 100
 map..... 72
 margins..... 141
 mark..... 116
 markangle..... 167
 marker..... 116
 markers..... 166
 marknodes..... 116
 markuniform..... 116
 mask..... 25
 material..... 137
 math..... 168
 mathematical functions..... 68
 max..... 35, 47, 73, 145
 SortedSet.max..... 97
 maxbound..... 26, 27
 maxtile..... 140
 maxtimes..... 34
 maxviewport..... 140
 metallic..... 137
 midpoint..... 33
 min..... 35, 47, 73, 145
 SortedSet.min..... 97
 minbound..... 26, 27
 minipage..... 20
 mintimes..... 34
 miterjoin..... 40
 miterlimit..... 40
 mktemp..... 54
 mm..... 10
 mobile browser..... 140
 mode..... 55
 monotonic..... 107
 mouse..... 189
 mouse bindings..... 139
 mouse wheel..... 189
 multisample..... 139

N

N..... 11
 NOT..... 61
 NoFill..... 49
 NoMargin..... 15
 NoMargin3..... 147
 None..... 14
 NoTicks..... 109
 NoTicks3..... 149
 name..... 55
 named arguments..... 66
 nan..... 25
 natural..... 107
 new..... 57, 71
 newEmpty
 Set.newEmpty..... 96
 newframe..... 47
 newl..... 54
 newpage..... 14
 newton..... 69
 next..... 194
 nobasealign..... 41
 noglobalread..... 54
 nolight..... 137
 none..... 54
 normal..... 145
 nosafe..... 186
 notaknot..... 107
 null..... 57
 nullpen..... 19, 49
 nullValue..... 92

O

O..... 142
 OmitTick..... 110
 OmitTickInterval..... 110
 OmitTickIntervals..... 110
 OpenGL..... 139
 OR..... 61
 OutTicks..... 149
 obj..... 179
 object..... 20
 oblique..... 142
 obliqueX..... 142
 obliqueY..... 143
 obliqueZ..... 142
 ode..... 179
 offset..... 40, 186
 opacity..... 43, 137
 open..... 54
 operator..... 62
 operator +(...string[] a)..... 29
 operator --..... 107
 operator 107
 operator ==..... 87
 operator [=]..... 62
 Map.operator [=]..... 93

operator [] 62
 Map.operator [] 92
 string.operator [] 28, 62
 operator answer 187
 operator cast 78
 operator ecast 78
 operator init 56, 59
 operator iter 81, 88
 Map.operator iter 93
 Queue.T.operator iter 100
 Set.operator iter 96
 operators 60
 options 182
 orient 26, 146
 orientation 48
 orthographic 143
 outformat 139
 outprefix 48
 output 54, 185
 overloading functions 65
 overwrite 46

P

P 69
 Pair_K_V 88
 PBR 137
 PDF 185
 PenMargin 15
 PenMargin2 147
 PenMargin3 147
 PenMargins 15
 PenMargins2 147
 PenMargins3 147
 Pl 69
 Portrait 48
 PostScript 52
 PostScript fonts 43
 PostScript subpath 12
 Python usage 187
 pack 20
 packing 67
 pad 50
 pair 10, 25
 pairs 73
 Map.pairs 93
 paperheight 4
 papertype 4
 paperwidth 4
 parallelogram 176
 parametric surface 151
 parametrized curve 120
 partialsum 168
 patch-dependent colors 138
 path 11, 30, 136, 177
 path markers 116
 path3 136
 path[] 12

patterns 43, 169
 pdflatex 185
 pdfreloadOptions 185
 pdfviewer 4
 pdfviewerOptions 185
 peek
 Queue.T.peek 100
 pen 37
 periodic 107
 perl 102
 perpendicular 165
 perspective 143
 physically based rendering 137
 picture 47
 picture alignment 50
 picture.add 53
 picture.addPoint 53
 picture.calculateTransform 49
 picture.fit 48
 picture.scale 49
 piecewisestraight 32
 pixel 147
 plain 181
 planar 138
 plane 142
 planeproject 145
 point 32, 37, 145
 polar 26
 polargraph 108
 polygon 116
 pop 70
 Queue.T.pop 100
 popMax
 SortedSet.popMax 98
 popMin
 SortedSet.popMin 97
 position 137, 139
 postcontrol 33, 145
 postfix operators 61
 pow10 68
 prc 141
 precision 55
 precontrol 33, 145
 prefix operators 61
 private 57
 programming 24
 pstoeedit 179, 192
 psviewer 4
 psviewerOptions 185
 pt 10
 public 57
 push 70
 Queue.T.push 100
 Set.push 96

Q

Queue_T	
.operator iter	100
.peek	100
.pop	100
.push	100
.size	100
cast to Iterable	100
iterating over	100
quadraticroots	75
quarticroots	168
quick reference	2
quit	9, 187, 194
quote	80
quotient	60

R

RadialShade	49
RadialShadeDraw	49
Rainbow	153
Relative	19
Riemann zeta function	69
Right	111
RightSide	19
RightTicks	109, 110
RightView	143
Rotate	19
Rotate(pair z)	19
RPM	3
Russian	43
radialshade	17
radians	68
radius	33, 145
rand	69
randMax	69
randString	69
range	83, 89
rational	179
rationalSimplex	179
read	75, 76
reading	54
reading string arrays	76
readline	55
real	25
realDigits	25
realEpsilon	25
realMax	25
realMin	25
realmult	26
realschur	74
rectangle	176
recursion	65
reference	2
reflect	47
relpoint	33
reltime	33
remainder	68

rename	56
render	136, 139, 185
replace	28
resetdefaultpen	46
rest arguments	67
restore	52
restricted	57
return	194
reverse	28, 33, 37, 72, 145
rewind	55
rfind	28
rgb	38
rotate	47, 145
round	68
roundcap	40
roundedpath	169
roundjoin	40
roundrectangle	176
runtime imports	80

S

S	11
Scale	19, 123
Seascape	48
Set	
!=	97
&	97
+	97
-	97
.add	95, 96
.contains	95
.delete	96
.empty	95
.extract	96
.get	96
.newEmpty	96
.operator iter	96
.push	96
.size	95
<=	96
==	97
>=	97
^	97
difference	97
intersection	97
iterating over	96
.sameElementsInOrder	97
.symmetric difference	97
.union	97
Shift	19
Si	69
SimpleHead	15
Sin	68
SixViews	144
SixViewsFR	144
SixViewsUS	144
Slant	19

SortedSet	97	Set.size	95
.after	98	size10	179
.atOrAfter	98	size11	179
.atOrBefore	98	size3	141
.before	98	slant	47
.max	97	sleep	30
.min	97	slice	34
.popMax	98	slices	76
.popMin	97	slide	179
iterating over	98	slope	168
SortedSet_T	97	slopefield	169
Spline	107, 151	smoothcontour3	170
Straight	107	sncndn	69
Suppress	46	solid	39
SuppressQuiet	46	solids	170
SVG	185	solve	74, 75
safe	186	sort	73
sameElementsInOrder	97	specialty modules	173
save	52	specular	137
saveline	55	specularfactor	137
scale	40, 47, 122, 145	specularpen	137
scale3	145	split	28
scaled graph	122	sqrt	68
schur	74	squarecap	40
scientific graph	113	srand	69
scroll	55	stack overflow	65, 193
search	72	static	83
search paths	5	stats	171
secondary axis	127	stdin	54
secondaryX	127	stdout	54
secondaryY	127	step	194
seconds	29	stickframe	166
seek	55	stop	194
seekeof	55	straight	32, 145
segment	168	strftime	29, 30
segmentation fault	193	string	27, 29
self operators	61	stroke	17, 18
sequence	71	strokepath	35
settings	4, 185	strptime	29
sgn	68	struct	57
shading	17	structures	57
shift	47, 144	subpath	33, 145
shiftless	47	subpictures	48
shininess	137	substr	28
shipout	48	sum	73
showtarget	143	superpath	12
signedint	55	surface	136, 138, 151, 152
simplex	179	symmetric difference	97
simplex2	181	system	30, 186
simpson	69	syzygy	180
sin	68		
single precision	55		
singleint	55		
singlereal	55		
sinh	68		
size	10, 32, 36, 48, 145, 185		
Map.size	92		
Queue_T.size	100		

T

Tan 68
 T_EX fonts 42
 T_EX string 27
 TeXHead 15
 TeXHead3 147
 ThreeViews 144
 ThreeViewsFR 144
 ThreeViewsUS 144
 Ticks 109, 110
 Top 109
 TopView 143
 TrueMargin 15
 TrueMargin3 147
 tab 54
 tab completion 9
 tan 68
 tanh 68
 target 143
 tell 55
 template 81
 tension 22, 136
 tensionSpecifier 37
 tensor product shading 18
 tensorshade 18
 tessellation 139
 tex 52, 185
 texcolors 158
 texcommand 5
 texpath 5, 20
 texpreamble 52
 texreset 52
 textbook graph 112
 tgz 3
 thick 139
 thin 139
 third-party licenses 186
 this 57
 three 136
 tick 112
 ticks 109
 tildeframe 166
 tile 44
 tilings 43
 time 29, 30, 168
 times 34
 trace 194
 trailingzero 109
 transform 46, 145
 transform3 144
 transparency 43
 transparent 137
 transpose 73
 tree 180
 trembling 180
 triangle 165
 triangles 139
 triangulate 164

tridiagonal 74
 trigonometric integrals 69
 triple 26
 tube 139, 171
 tutorial 9
 type1cm 42
 typedef 30, 64

U

U3D 175
 UNIX binary distributions 3
 UnFill 49
 UpsideDown 48
 URL 80
 undefined 35
 unfill 18
 uniform 72
 uninstall 8
 union 97
 unique 169
 unit 25, 27
 unitbox 12, 142
 unitcircle 12, 142
 unitrand 69
 unitsize 11, 48
 unpacking 67
 unravel 79
 up 143
 update 54
 usepackage 52
 user coordinates 11
 user modules 107
 user-defined operators 62
 using 30, 64
 usleep 30

V

Vertical 177
 Viewport 137
 v3d 141, 181
 valid 82, 88
 value 168
 var 57
 variable initializers 56
 vectorfield 132, 133
 vectorfield3 153
 vectorization 75
 verbatim 52
 vertex-dependent colors 138
 viewportheight 102
 viewportmargin 141
 viewportsize 141
 viewportwidth 102
 views 141
 vim 7
 virtual functions 59

void..... 24

W

W..... 11
 WebGL..... 140
 Wheel..... 153
 White..... 137
 warn..... 5
 whatever..... 34
 wheel mouse..... 189
 while..... 24
 white-space string delimiter mode..... 76
 width..... 102
 windingnumber..... 35
 word..... 75, 76
 write..... 54, 76

X

X..... 142
 XEquals..... 111
 XOR..... 61
 XY..... 145
 XEquals..... 149
 XYZZero..... 149
 XZEquals..... 149
 XZero..... 111
 XZZero..... 149
 x11colors..... 158
 xasy..... 189
 xaxis3..... 148
 xdr..... 55
 xelatex..... 185
 xequals..... 111
 xlimits..... 120
 xpart..... 25, 27
 xscale..... 47
 xscale3..... 144
 xtick..... 112

Y

Y..... 68, 69, 142
 YEquals..... 108
 YX..... 145
 YZ..... 145
 YZEquals..... 149
 YZero..... 108
 YZZero..... 149
 yaxis3..... 148
 yequals..... 111
 ylimits..... 120
 ypart..... 25, 27
 yscale..... 47
 yscale3..... 144
 ytick..... 112

Z

Z..... 142
 ZX..... 145
 ZY..... 145
 zaxis3..... 148
 zero_Ai..... 69
 zero_Ai_deriv..... 69
 zero_Bi..... 69
 zero_Bi_deriv..... 69
 zero_J..... 69
 zeroTransform..... 47
 zerowinding..... 40
 zeta..... 69
 zip..... 89, 90
 zpart..... 27
 zscale3..... 144

|

|..... 61

||..... 61